



DISCLAIMER

Il materiale contenuto nel drive è stato raccolto e richiesto tramite autorizzazione ai ragazzi frequentanti il corso di studi di Informatica dell'Università degli Studi di Salerno. Gli appunti e gli esercizi nascono da un uso e consumo degli autori che li hanno creati e risistemati per tanto non ci assumiamo la responsabilità di eventuali mancanze o difetti all'interno del materiale pubblicato.

Il materiale sarà modificato aggiungendo il logo dell'associazione, in tal caso questo possa recare problemi ad alcuni autori di materiale pubblicato, tale persona può contattarci in privato ed elimineremo o modificheremo il materiale in base alle sue preferenze.

Ringraziamo eventuali segnalazioni di errori così da poter modificare e fornire il miglior materiale possibile a supporto degli studenti.



CoScienze
Associazione

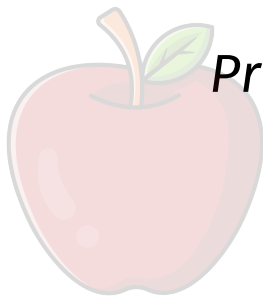


Università degli Studi di Salerno

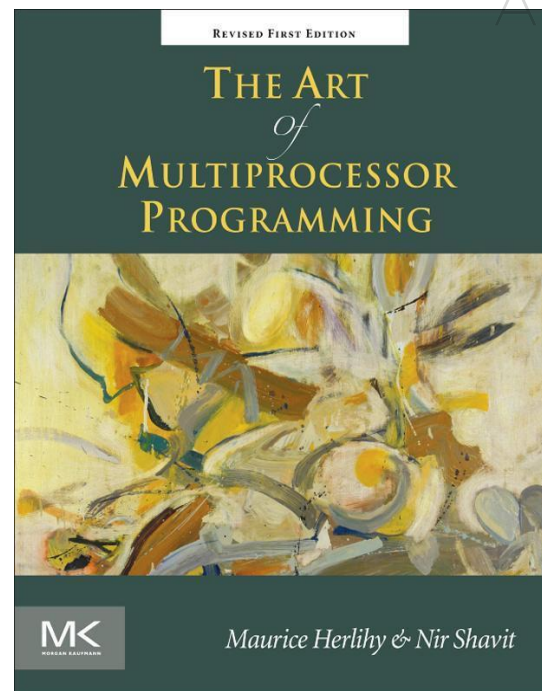
Dipartimento di Informatica
Corso di laurea Magistrale in Informatica

Appunti

Andrea Sorrentino – Francesco Parisi



Programmazione Concorrente, Parallela e su Cloud



Anno Accademico 2020-2021

Indice

1. Introduzione.....	6
1.1 Motivazioni.....	6
1.2 Le Sfide	8
1.3 Architetture Multicore.....	9
1.4 Processori e Core	10
1.5 Le Interconnessioni.....	11
1.6 La Memoria.....	11
1.7 La Cache.....	12
1.8 Protocollo MESI	13
1.9 Spinning.....	15
1.10 False Sharing	17
1.11 Architetture Multicore.....	17
1.12 Relaxed Consistency	18
Questionario di Fine Lezione	18
2. Introduzione alla Programmazione Concorrente.....	21
2.1 La Parallelizzazione.....	21
2.1.1 La Legge di Amdahl	21
2.1.2 Le Sfide della Programmazione Parallela	22
2.1.3 Le Scenari con Metriche Diverse.....	23
2.2 Java e la Concorrenza	26
2.2.1 Thread e Monitor.....	26
2.2.2 Thread-local Objects	28
2.2.3 La Java Virtual Machine e i Thread	29
3. Mutua Esclusione	31
3.1 Tempo	31
3.2 Sezione Critica	32
3.2.1 Proprietà: Rephrasing	34
3.3 Sezione Critica per 2 Thread	35
3.3.1 La Classe LockOne	35
3.3.2 La Classe LockTwo.....	37
3.3.3 Il Lock di Peterson.....	38
3.4 Sezione critica per N thread	40

3.4.1	L'Algoritmo del Fornaio di Lamport	40
3.5	Bounded Timestamps	41
3.6	Un Lower Bound sulla memoria	45
3.6.1	L'Introduzione.....	45
3.6.2	La Dimostrazione per 3 Thread	48
4.	Oggetti Concorrenti	54
4.1	Introduzione	54
4.1.1	Il Singleton	54
4.1.2	Concorrenza e Correttezza	56
4.1.3	Oggetti Sequenziali	59
4.2	Condizioni di Correttezza	59
4.2.1	Consistenza Quiescente	59
4.2.2	Consistenza Sequenziale	61
4.2.3	Linearizzabilità	62
4.3	Condizioni di Progresso	63
4.3.1	Proprietà di un Lock	63
4.3.2	Commenti alle Condizioni di Progresso	64
4.4	Il Modello di Memoria di Java	64
5.	Spinlocks.....	68
5.1	Il Mondo Reale	68
5.1.1	Test-and-Set.....	68
5.1.2	TTAS con Backoff	69
5.1.3	La Classe BackoffClass.....	70
5.2	Queue Locks	71
5.2.1	Lock di Anderson	73
5.2.2	CLH Locks	74
5.2.3	MCS Locks	77
5.3	Altri Locks	79
5.3.1	Locks con Timeout	79
5.3.2	Composite Locks	83
5.3.3	Hierarchical Locks	84
6.	Monitor.....	88
6.1	Introduzione	88
6.2	Monitor Lock e Condition	89
6.2.1	Definizione	89

6.2.2	Condition	90
6.2.3	Lost Wakeup	93
6.2.4	I Semafori.....	94
6.3	Readers-Writers Lock.....	95
6.3.1	I Readers Writers	95
6.3.2	Una Semplice Soluzione.....	96
6.3.3	Una Soluzione Equa tra Readers-Writers.....	98
7.	Liste	102
7.1	Introduzione	102
7.1.1	Tipi di Sincronizzazione	102
7.1.2	Struttura Dati per Insiemi	103
7.1.3	Problematiche della Concorrenza	104
7.2	Coarse-Grain Synchronization	106
7.3	Fine-Grain Synchronization.....	108
7.3.1	La Classe Finelist	108
7.3.2	Analisi della Soluzione	112
7.4	Optimistic Synchronization	114
7.5	Lazy Synchronization	118
7.5.1	La Classe LazyList	120
7.6	Non-Blocking Synchronization	122
7.6.1	Il tipo AtomicMarkableReference<T>	123
7.6.2	La Classe LockFreeList	125
7.6.3	Optimistic vs. Lazy Synchronization	130
8.	Code.....	132
8.1	Introduzione alle Code.....	132
8.1.1	Diversi tipi di Pool	135
8.2	Implementazioni Concorrenti di Code	136
8.2.1	Coda Bounded Partial	136
8.2.2	Coda Unbounded Total	141
8.2.3	Coda Unbounded Lock-Free	143
8.2.4	Code con Rendezvous	147
8.3	Gestione della Memoria e Problema ABA	148
8.3.1	Il Problema di ABA	149
9.	Scheduling.....	151
9.1	Pool di Thread.....	151

9.1.1	Operazioni Concorrenti tra Matrici	151
9.1.2	Thread pool per Operazioni su Matrici	152
9.2	Scheduling	156
9.2.1	Analisi del Parallelismo	156
9.2.2	Scheduling su Multiprocessore	159
9.3	Distribuzione del Lavoro	160
9.3.1	Work Stealing	161
9.3.2	Work Balancing.....	162



1. Introduzione

Questa parte del corso tratta della Programmazione Concorrente, cioè della programmazione dove i processori (core) condividono memoria, cioè la maniera in cui collaborano nella risoluzione comune è scambiandosi i dati attraverso la memoria.

All'interno delle elaborazioni contemporanee tratteremo quella che è la computazione condividendo memoria.

- Il vantaggio di utilizzare la memoria è che non si devono imparare nuovi costrutti;
- Lo svantaggio è che qualsiasi cosa nel nostro programma può essere comunicazione ad un altro thread, ed attualmente noi siamo ignari se questa comunicazione avviene;

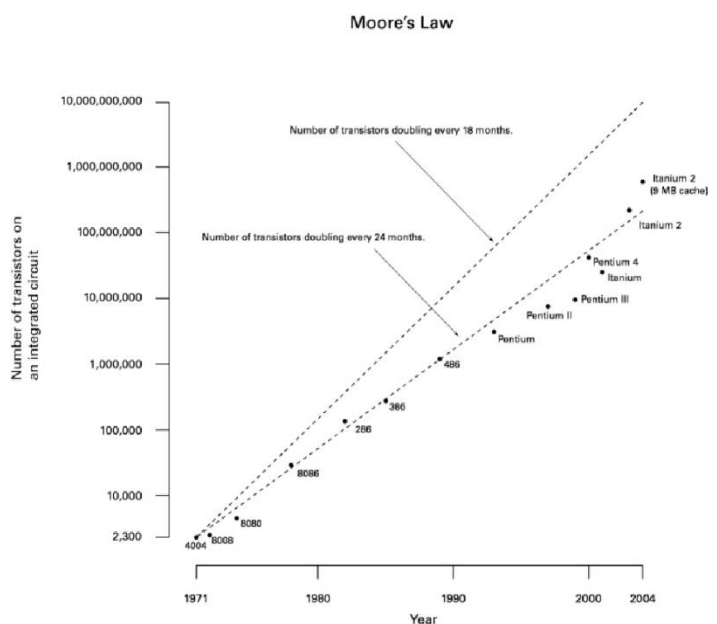
La strutturazione del programma non è suddivisa in fasi (momenti in cui si comunica e momenti in cui si computa), quindi il lavoro del programmatore è abbastanza complesso, ma ci sono degli strumenti che permettono di poter eseguire questa parte del programma in totale isolamento (sei l'unico che sta eseguendo questa parte del programma, quindi non stai facendo comunicazione, stai facendo solo computazione, gli altri non stanno leggendo le cose che si stanno scrivendo).

1.1 Motivazioni

Lo sviluppo dei microprocessori è rivolto alla produzione di CPU multicore, dove ogni core comunica con gli altri core attraverso una cache condivisa.

La programmazione concorrente diventerà lo standard di programmazione, non più un caso particolare, in quanto le architetture multiprocessore stanno avendo una notevole diffusione. La programmazione parallela sta quindi diventando sempre più accessibile e vicina a tanti problemi.

Una delle leggi più citate dell'informatica è quella di Moore: si dice legge, dato che il suo funzionamento si è interrotto altrimenti sarebbe stato un teorema, infatti la legge di Moore oggi è morta, dato che i livelli di miniaturizzazione non possono arrivare oltre un certo punto.



Legge di Moore - il numero di chip per transistor tende a raddoppiare ogni due anni.

Il problema del rallentamento della crescita del numero di transistor è dovuto al problema del *Thermal Noise*, ossia un effetto della termodinamica che crea numerosi problemi di surriscaldamento e che disturba quindi la crescita del numero dei transistor definita da Moore.

Questi due aspetti rendono quindi il Thermal Noise un problema:

- ❖ Alimentazione a basso voltaggio per limitare i problemi di raffreddamento;
- ❖ Aumento della velocità del clock;

Il rallentamento di incremento del numero di transistor inizia ad avere effetto con la tecnologia al di sotto di 40nm (attualmente il core i9-9900K usa 14nm mentre AMD con Ryzen 9 5950X usa 7nm), ed attualmente il limite di mercato è fermo a 7nm oltre questo limite non si ancora sceso.

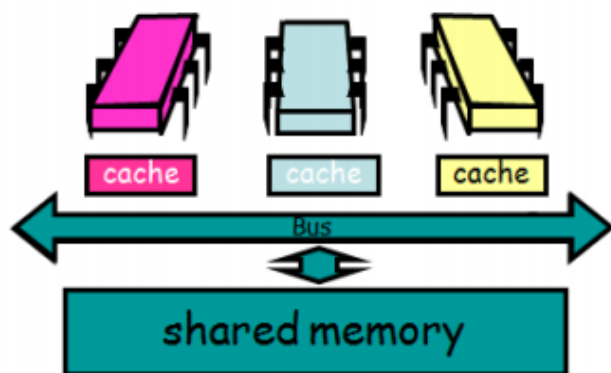
Quindi il Thermal Noise ci impedisce di avere tutto dalla vita: In pratica non si possono avere tanti transistor su un processore, che siano anche “facili da raffreddare” e che siano “veloci”. Si deve quindi rinunciare ad una di queste caratteristiche. Questo ha portato a un radicale cambiamento per quanto riguarda la progettazione di CPU, passando da un’architettura a Single Core ad una a Multicore.

Il punto cruciale a cui si vuole arrivare è che la velocità può essere ottenuta in maniera diversa. Questo ha cambiato il modo in cui i programmatori intendessero il loro lavoro.

Fino a poco tempo fa, il progresso tecnologico comportava infatti un automatico miglioramento delle prestazioni software (significava un incremento della velocità del clock dei processori che si traduceva direttamente in un incremento della velocità di esecuzione del software).

Con l’introduzione delle CPU multicore, il progresso tecnologico significa incrementare il parallelismo e non la velocità del clock, ovvero più transistor ma organizzati in core multipli.

Oggi invece un programma per essere più efficiente deve essere in grado di sfruttare al massimo il parallelismo delle applicazioni.



Un desktop con più core

L'idea è di avere diversi processori (core), un processore P_1 , P_2 , P_3 ... P_n ciascuno con la propria cache (la memoria cache tradizionale appartiene al singolo core) che tramite un bus condiviso da tutti i core, accedono a quella che è la memoria condivisa.

Il punto che risolve con più processori il problema del Thermal Noise, dove il problema essenzialmente è questo:

Nel Single core la comunicazione doveva avvenire tra ogni transistor all'interno del chip, quindi essenzialmente l'energia necessaria per poter muovere un messaggio in due punti distinti era tanta mentre in un processore Multicore lo stesso chip di silicio viene suddiviso funzionalmente in 4 core

identici dove la comunicazione avviene solamente all'interno dei singoli core, quindi ognuno di questi utilizza una singola porzione dell'area di silicio per implementare un chip completo, questo significa che riusciamo ad utilizzare meno energia per muovere un messaggio, quindi il consumo di Watt scende, riusciamo così a ottenere chip indipendenti e ad avere la velocità limitata a questi singoli core. Il processore Multicore nasce dall'esigenza di combattere il fenomeno del Thermal Noise suddividendo i processori e limitando la quantità di energia necessaria per comunicare, facendo in modo che si possa comunicare efficientemente con poca energia all'interno di uno stesso core dell'intero processore e questo meccanismo è importante.

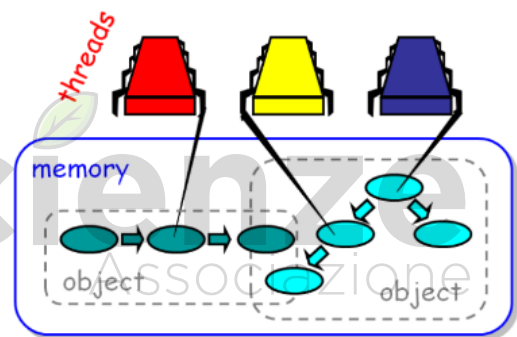
Free Lunch is over

- ❖ Fino a poco tempo fa, i miglioramenti della tecnologia comportavano un automatico miglioramento delle prestazioni software: CPU con clock maggiore eseguivano il codice con più velocità;
- ❖ Adesso il miglioramento consiste in più transistor organizzati in core multipli che per essere usati efficientemente hanno bisogno di software in grado di sfruttare il parallelismo delle applicazioni;

1.2 Le Sfide

Le sfide chiaramente sono dovute all'uso della memoria condivisa e questo è il grosso punto.

Quando avevamo un singolo thread che accedeva alla memoria strutturata in oggetti, era abbastanza semplice la gestione, ma la situazione si complica notevolmente se si usano più thread che devono accedere alle stesse aree di memoria.



Questo paradigma di programmazione ci porta inevitabilmente a scontrarci con sfide di natura gestionale, in quanto avendo più processori che accedono e lavorano contemporaneamente con una singola memoria condivisa si vanno a creare tutta una serie di problemi di concorrenza e di sincronia.

Quando si scrivono programmi per un Single core si ignorano i dettagli architetturali della macchina. Sfortunatamente, quando i programmi sono destinati per macchine dotate di Multicore, bisogna tener presente le caratteristiche dell'architettura e come si possono ottenere vantaggi dall'utilizzo dei thread, che in ogni caso comporta problemi dovuti all'asincronia. Si è soggetti a ritardi imprevedibili:

- Cache misses (breve);
- Page faults (lungo);
- Context-switch dallo scheduler (molto lungo);

Il trend del presente/futuro è chiaramente in direzione Multicore, l'ipotesi sul futuro è che i core saranno:

- 128 sui desktop;
- 512 sui server;
- 4096 su sistemi embedded;

Si ipotizza un corollario alla legge di Moore: I core raddoppieranno ogni 18 mesi.

1.3 Architetture Multicore

Quali sono i vantaggi delle architetture multicore? I vantaggi sono quelli di eseguire task diversi. Un esempio è dato da due thread che condividono una risorsa che deve essere usata in mutua esclusione, ovvero che se un thread sta usando una risorsa, allora nessun altro thread può usarla. La terminologia che si usa è quella classica, cioè fare il lock della risorsa prima di usarla e fare l'unlock dopo averla usata.

Il lock è una serratura, cioè qualcosa che viene acquisito dal thread in maniera esclusiva e che previene all'altro thread o agli altri di acquisire lo stesso lock se intanto il thread che lo aveva acquisito non l'ha rilasciato attraverso un unlock.

Assumiamo che il lock sia una semplice variabile booleana e assumiamo che abbiamo a disposizione un metodo `getAndset(v)`, che in maniera atomica, cioè non interrompibile, fa lo swap del parametro con lo stato di questa variabile, ovvero `getAndset()` restituisce tale variabile booleana. Quando si usa il metodo `getAndset()` con `v`, `v` viene scritto e il valore precedente viene restituito.

Questo significa fare lo swap atomico dei valori e ciò risulta utile perché un solo thread può completare la `getAndset()` che viene compiuta e portata a termine senza nessuna interruzione. Ciò può essere usato in maniera molto semplice: nel lock c'è un valore che ignoriamo, poi quando andiamo ad eseguire mettiamo `true`, e il valore che si trovava all'interno viene restituito. Le possibilità sono due:

- Se il lock valeva `false` vuol dire che la variabile valeva `false`, è stato messo a `true` e quindi il lock viene effettivamente acquisito;
- Se il lock valeva `true` vuol dire che il lock indicava che c'era già qualcuno nella zona di mutua esclusione, quindi è stato sostituito a `true`, cioè con lo stesso valore;

Test-And-Set: Una classe che implementa una semplice interfaccia che offre i metodi di lock e unlock. L'implementazione obbliga a implementare i metodi `lock` e `unlock`. La variabile `state` è la variabile che viene utilizzata per implementare lo stato booleano.

Il lock è estremamente semplice. C'è un `while` e una chiamata del metodo `getAndset(true)` sulla variabile `state`. Questo metodo `getAndset(true)` non fa altro che andare ripetutamente a mettere `true` all'interno della variabile; quindi, il risultato dell'esecuzione di questo metodo viene valutato come condizione della continuazione del `while`, ciò significa che se il lock è occupato allora `getAndset(true)` restituirà `true`, quindi il `while` sarà `while(true)` e ritornerà a eseguire il

corpo del `while`, in tal caso vuoto. Questo non è altro che una esecuzione ripetuta di `getAndset()` fino a quando capita che l'esecuzione mette `true` e restituisce `false`, cioè si è acquisito il lock. Quando si ritorna dall'esecuzione di `lock`, allora è possibile entrare in sezione critica.

```
public class TASLock implements Lock {
    ...
    public void lock() {
        while (state.getAndSet(true)) {}
    }
    ...
}

public class TTASLock implements Lock {
    ...
    public void lock() {
        while (true) {
            while (state.get()) {};
            if (!state.getAndSet(true))
                return;
        }
    }
    ...
}
```

Test-And-(Test-And-Set): A prima vista il codice non è migliore rispetto al Test-And-Set. C'è innanzitutto un `while(true)`, dove al suo interno c'è un ciclo di attesa, ovvero non il metodo `getAndset()` ma il metodo `get()`. In pratica tale ciclo fa in modo di continuare a ciclare mentre il valore della variabile `state` rimane a `true`. Il primo passo è attendere che la variabile diventi `false`, quindi quando si passa al controllo successivo, `state` è stato letto a `false`. A questo punto si sa che abbiamo letto `state` a `false`, da non confondere con `state` a `false`, questo perché a tal punto sappiamo che `state` è `false` ma per poter acquisire il lock bisogna metterlo a `true` attraverso l'operazione di `getAndset(true)`. Se questo restituisce `false` allora si può fare la `return`, ovvero acquisire il lock. Nel caso in cui la `getAndset()` non restituisce `false`, si ritorna al `while(true)` e si attende perché qualcuno ha già acquisito il lock. Quindi tra la `get()` e la `getAndset()`, la variabile `state` potrebbe cambiare valore.

TASLock e TTASLock fanno la stessa cosa, quindi sono equivalenti?

Sembra di sì anche se TASLock sembra molto più semplice mentre TTASLock sembra fare operazioni inutili. Secondo l'esperimento di Anderson il risultato è sorprendente. L'esperimento esegue tale piccola sezione critica per un milione di volte e il risultato è che all'aumentare del numero di thread TASLock funziona molto peggio di TTASLock, il quale risulta essere molto più efficiente e funziona decisamente meglio. La perdita del tempo c'è rispetto a quello che potrebbe essere un lock ideale che, indipendentemente dal numero di volte che viene eseguito, non porta a nessun overhead.

1.4 Processori e Core

I processori sono componenti hardware che eseguono thread. Esistono più thread che processori, e ogni processore ha uno scheduler che gestisce l'alternanza di esecuzione dei vari thread (**Context**

Switch). Quando un thread viene de-scheduled, può essere eseguito da un altro processore. Esiste un mezzo che gestisce l'interconnessione tra i processori e la memoria.

L'interconnessione è il mezzo di comunicazione che collega processori tra loro e i processori alla memoria. La memoria è una gerarchia di componenti che memorizzano dati, da o più livelli di cache che permettono l'accesso fino alla memoria principale. Esistono dei principi di design dei processori che dettano il fatto che il processore per accedere alla memoria richiede del tempo.

Un multiprocessore è composto da una serie di processori hardware, ognuno dei quali esegue codice sequenziale. L'unità base di tempo è chiamato *ciclo*: il tempo che occorre ad un processore per processare ed eseguire una singola istruzione. Mentre i tempi di esecuzione di un ciclo macchina sono cambiati nel tempo (da decine di milioni a tre miliardi al secondo), il costo di accesso alla memoria non è cambiato se espresso in termini di cicli macchina.

1.5 Le Interconnessioni

L'interconnessione tra i vari processori e la memoria avviene tramite un mezzo di comunicazione, chiamato *Bus*. Sia il processore che il controller della memoria possono trasmettere dati sul bus, ma un solo processore per volta può trasmettere, mentre tutti gli altri possono ascoltare.

Le interconnessioni possono essere essenzialmente di due tipi:

- **SMP (Symmetric Multiprocessing)**: i processori e la memoria sono collegati da un bus, che permette di accedere alla memoria. Vi è un bus controller tra i processori e la memoria centrale che controlla gli invii e le letture sul bus (quest'ultime chiamate *Snooping*). Sia sui processori che sulla memoria ci sono dei bus controller che effettuano il controllo del bus e permettono il Bus Snooping. Fare *Bus Snooping* significa ascoltare i messaggi che sono sul bus anche se non sono diretti a noi. SMP è un'architettura molto utilizzata perché è molto semplice da realizzare ma è poco scalabile per un elevato numero di processori, perché in quel caso il bus diverrebbe sovraccarico;
- **NUMA (Non Uniform Memory Access)**: i processori hanno ognuno la propria memoria e sono collegati punto-punto, cioè interconnessi gli uni agli altri. Quindi ogni nodo ha uno o più processori, la propria memoria locale e può accedere alla memoria degli altri processori. L'accesso alla memoria è non uniforme, quindi il tempo per l'accesso alla memoria locale è più veloce. Questa architettura risulta essere più scalabile in relazione all'aumentare del numero di processori. Sia quando si parla di bus sia quando si parla di connessione, l'interconnessione tra i core è una risorsa finita e questo risulta critico, perché è necessario utilizzare un certo tipo di risorsa per poter accedere alla memoria e si è costretti a utilizzare bus o reti di interconnessione che potrebbero influire sull'esecuzione del programma;

1.6 La Memoria

I processori condividono una memoria principale, vista come un array di word indicizzate da un indirizzo. La maniera in cui viene effettuata la lettura della memoria consiste nell'inviare un

messaggio dal processo alla memoria (con l'indirizzo del parametro), a cui la memoria risponde inviando il valore.

La scrittura in memoria è un messaggio inviato dal processore alla memoria con indirizzo / nuovo valore come parametri, a cui la memoria risponde con un ACK, cioè uno scambio di messaggi all'interno del processore. Processori e memoria sono lontani, infatti ci possono voler anche centinaia di cicli macchina per un accesso alla locazione di memoria.

1.7 La Cache

Sfortunatamente, l'accesso alla memoria centrale non è un'operazione molto veloce ma richiede centinaia di cicli di clock. Continui accessi fanno sprecare tempo alla CPU. Per risolvere in parte questo problema sono state introdotte una o più cache, cioè memorie di piccola dimensione situate tra i processori e la memoria centrale, quindi più veloci di quest'ultima. Quando un processore aspetta di leggere un valore da un determinato indirizzo di memoria, per prima cosa controlla se il valore è già presente in cache e se ha successo legge con velocità (**Hit**) dalla cache e non effettua la chiamata alla memoria, altrimenti deve leggere dalla memoria principale (**Miss**).

La cache miss ha il vantaggio di prendere il valore e metterlo nella cache, permettendo qualora il principio di località, il quale dice che le locazioni accedute in un certo passo del programma sono accedute più frequentemente subito dopo, se tale locazione venisse acceduta si sarebbe ritrovato il valore della cache.

La Hit Ratio è il rapporto tra i successi e i numeri totali di richieste. Se l'hit ratio è 1 significa che il programma mette tutto in cache e usa solo quelle variabili, e questo è un grande vantaggio.

Le cache sono efficaci se il programma ha *locality*, ovvero se una locazione acceduta nel passato ha buona probabilità di essere acceduta di nuovo. Le cache sono quindi strutturate in maniera tale da avere una cache line che comprende diverse word. Se si ha necessità di accedere ad una locazione, tutte le word adiacenti per la dimensione della cache line vengono accedute e caricate in memoria, questo perché se si scorre un array ed è un array di interi memorizzati ognuno in una word, se si ha effettuato l'accesso all'array $A[i]$ probabilmente verrà incrementato i e così si accede al valore successivo. Se si è fatta una cache miss per caricare l'intera cache line, ci sarà già il valore caricato in cache senza accedere alla memoria principale. Nella pratica molti processori hanno due livelli di cache:

- **L1 cache:** Sullo stesso chip del processore (1-2 cicli per l'accesso);
- **L2 cache:** On oppure Off-chip, ma richiede decine di cicli per l'accesso;

Entrambe sono molto più veloci di centinaia di cicli necessari per l'accesso alla memoria centrale. Le cache sono notevolmente più piccole, in quanto costose da costruire. Si rende necessario quindi un algoritmo di replacement per aggiornare le word memorizzate nella cache.

Un altro aspetto molto importante è appunto il meccanismo di replacement, ovvero in che maniera vengono sostituiti gli elementi presenti nella cache con i nuovi elementi che devono entrare. Per il rimpiazzo ci sono diverse politiche:

- **Fully Associative:** Tutte le linee sono candidate all'eviction;
- **Direct Mapped:** Solo una linea può essere evicted. È un meccanismo che richiede sforzo computazionale. L'algoritmo è molto semplice ma non si può fare una gestione intelligente della cache eliminando le linee importanti;

Quando un processore legge/scrive un indirizzo che è in cache da un altro processore si ha la *Memory Contention*. Il problema è quello di avere lo stesso valore in due cache diverse, creando la contesa della memoria. In caso di lettura non ci sono problemi, mentre in caso di scrittura è un problema non banale. La copia in cache dell'altro processore va invalidata, nel senso che se un processore scrive, quel valore precedente non è più valido. I protocolli della cache devono essere in grado di gestire queste situazioni.

Esistono diversi protocolli che assicurano la coerenza della cache e sono protocolli che vengono stabiliti all'interno del processore. Il protocollo per gestire la coerenza della cache è il Protocollo MESI.

1.8 Protocollo MESI

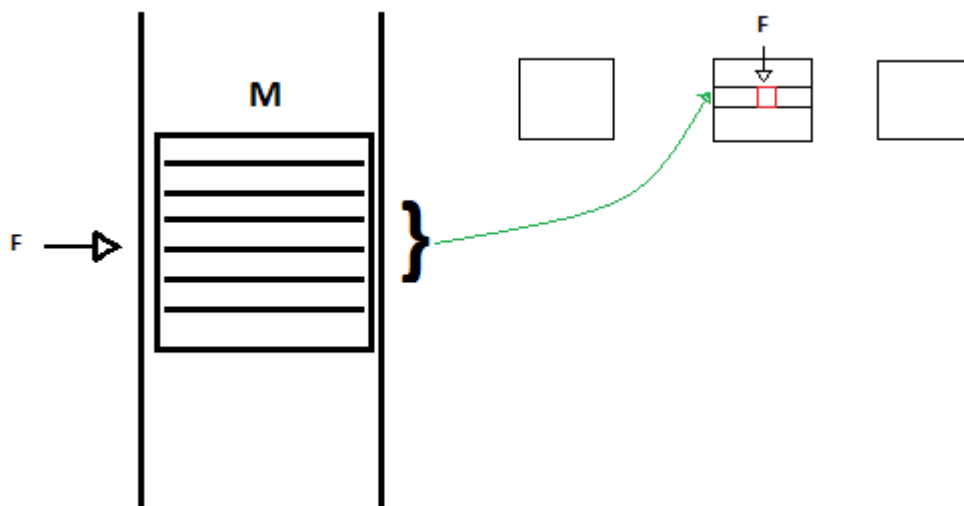
Il protocollo MESI viene fuori da quattro stati in cui può essere una linea di cache:

- ❖ **Modified:** La linea è stata modificata in cache, cioè è stata già portata, poi viene modificata e deve essere scritta (prima o poi) nella memoria principale;

Esempio:

Ho a disposizione un insieme di locazioni di memoria, ho le varie cache.

La parte in rosso è stata modificata, cioè è stato fatto qualcosa, ossia in una word della cache line è stato fatto qualcosa, quindi, questo è il punto chiave. C'è un valore in cache che è diverso dal valore in memoria, perché è stato scritto dal processore nella propria cache. Questa è una situazione transiente, una situazione che deve essere accomodata, cioè deve essere prima o poi scritta all'interno della memoria principale.



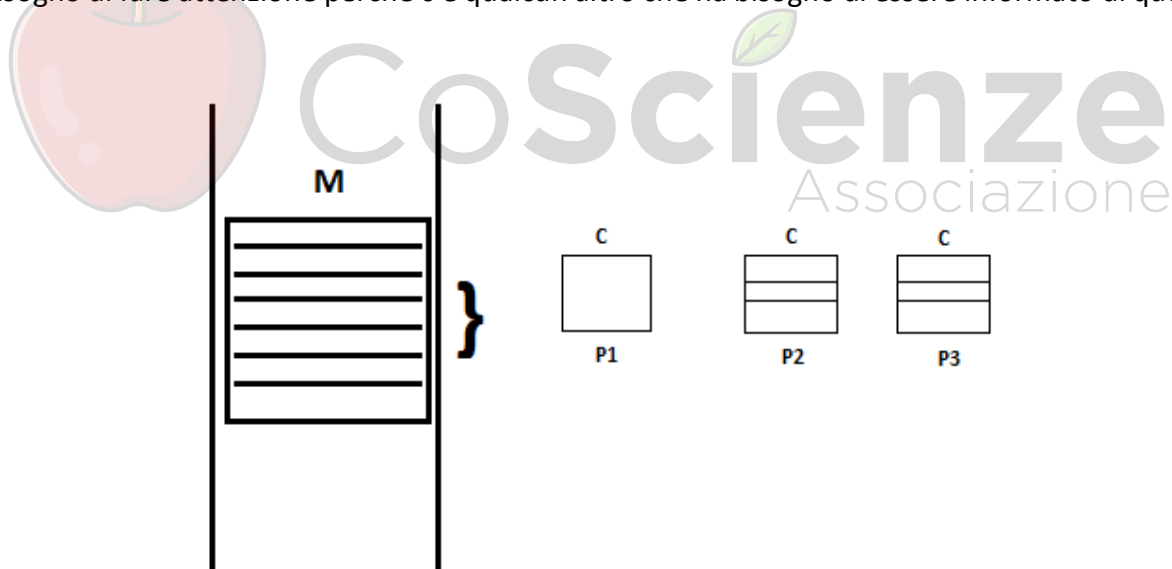
La caratteristica importante è che questa linea è esclusivamente all'interno della cache del processore dove è stata effettuata la modifica, ciò significa che nessun altro processore ha la stessa linea all'interno della propria cache.

- ❖ **Exclusive:** La linea di cache è stata portata all'interno della cache ma nessun altro processore la ha in cache, possiamo dire che la Exclusive è lo stato prima della Modified. Quindi è stato portato un insieme di locazioni di memoria all'interno della cache e non è stato modificato, però il processore è l'unico che ha a disposizione questo;
- ❖ **Shared:** la linea di cache non è stata modificata, ed altri processori la hanno in cache;

Esempio:

Ho sempre la mia solita memoria. Ho il mio blocco di word che rappresentano la cache line, ho un insieme di cache line, e questa linea di memoria è stata portata sia nella cache di P_2 che di P_3 , non è stata modificata, però sono a conoscenza entrambi (la linea è marcata Shared) del fatto che qualcun altro la ha.

Dato che per esempio il processore P_3 non sa chi altro ha questa cache line, questo è un aspetto critico. Dato che non saprebbe dove memorizzarlo, siamo ad un livello così basso che noi non possiamo realizzare un algoritmo, perché stiamo gestendo la memoria; quindi, in questo momento l'unica cosa che conosce è che questa è una cache line da trattare con attenzione dato che c'è qualcun altro che la ha all'interno della sua cache e quindi se ci dovesse scrivere (quindi modificarla) c'è bisogno di fare attenzione perché c'è qualcun altro che ha bisogno di essere informato di questa cosa.

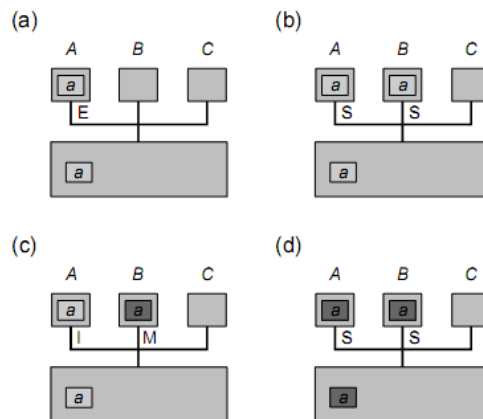


- ❖ **Invalid:** La linea di cache non contiene dati validi, cioè la linea di cache in pratica contiene dei dati che sono vecchi;

Esempio:

Quando qualcun altro ha scritto dei dati nella propria linea di cache e la mia linea di cache non contiene più dati validi; quindi, è qualcosa che non è più corretto, che non si deve andare ad usare.

Come Funziona il Protocollo MESI ?



- Quando il processore A legge il valore all'indirizzo a, lo memorizza nella sua cache nello stato Exclusive. Essendo Exclusive, il processore A è l'unico ad avere questo valore in memoria.
- Quando il processore B vuole leggere da a (quindi decide di accedere allo stesso valore), scrive sul Bus la richiesta di leggere il valore a dalla memoria. Quello che succede è che la scrittura di B viene fatta sul bus, ma sul bus A si sta facendo Snooping (Prendere controllo quindi, accesso per modificare) cioè è il processore A che risponde alla richiesta della variabile a, il processore A riconosce il conflitto e risponde con il dato associato e lo setta con Shared. Ora il valore a è cached sia in A che in B con lo stato Shared.
- Se B scrive all'indirizzo di a, questo significa che in questo caso il processore B è l'unico che ha a disposizione il valore reale della variabile a, allora cambia il suo stato in Modified e manda in broadcast (a tutti) un messaggio di warning per cambiare loro lo stato in Invalid.
- Se A legge da a, manda in broadcast una richiesta e B risponde inviando i dati modificati sia ad A che alla memoria principale, lasciando le copie nello stato Shared.

1.9 Spinning

Il processo che usa **TAS (Test And Set)** è il processo noto come *Spinning*. Lo Spinning è quando un processore controlla ripetutamente una word in memoria, in attesa che qualche altro processore ne cambi il valore (Attenzione, "cambi il valore" inteso che ci sono tutte le cache per mezzo, è il grosso punto). Su una architettura SMP (Symmetric Multiprocessing) senza cache, lo spinning è davvero una pessima idea essendo molto inefficiente, questo perché:

- Senza cache significa che i processori sono direttamente collegati ad un Bus che permette di accedere alla memoria, quindi non c'è cache;

- Per ogni operazione il processore deve andare a leggere nella memoria e scrivere sul bus per ottenere il valore. Fare Spinning qui è molto inefficiente, perché ogni volta si deve andare ad accedere al Bus e ogni volta che si accede si consuma banda del bus senza realizzare alcun lavoro utile, perché sto semplicemente facendo Spinning, quindi attendendo centinaia di istruzioni avrei potuto fare per centinaia di cicli, il risultato di questa operazione, che poi non serve a niente, è che intanto sto usando anche la banda del Bus, cioè sto impedendo ad altri processori di poter accedere alla memoria, dato che non possono farlo perché un altro processore sta consumando tutta la banda di accesso del Bus;

Insomma, ogni volta che il processore legge la memoria, consuma banda dal Bus senza realizzare alcun lavoro utile, non permettendo a processori che hanno lavoro utile da compiere di utilizzare il Bus.

Su un'architettura NUMA senza cache, lo spinning può essere accettabile se la variabile è nella memoria locale del processore, dato che facendo Spinning si accede alla memoria locale non andando a consumare banda esterna.

Su una architettura SMP/NUMA con cache, lo spinning consuma meno risorse:

- Alla prima lettura, si ha una cache miss e la variabile viene caricata in cache;
- Se la variabile non è modificata, allora si continua a leggere dalla cache (senza usare risorse, siano esse il bus condiviso oppure la comunicazione con un altro processore);
- Appena la variabile risulta modificata si ha un cache miss, si carica il valore e si ferma lo spinning.

Ora il comportamento bizzarro tra **TASLock** e **TTASLock** risulta più chiaro.

Il primo Spinning fa anche set, quindi l'operazione `getAndSet(true)` non è un'operazione che passa inosservata al protocollo di coerenza della cache.

Ad ogni `getAndSet(true)` di **TASLock** viene generato traffico sulla interconnessione fino a saturare il bus condiviso, ritardando gli altri thread.

Ad ogni `getAndSet(true)` tutti i processori che stanno facendo Spinning su quella variabile cercano di scrivere e leggere il valore che c'era prima in quella variabile, cioè stanno usando il meccanismo di Invalid in continuazione.

```
public class TASLock implements Lock {
    //...
    public void lock() {
        while (state.getAndSet(true)) {}
    }
    //...
}
```

```
public class TTASLock implements Lock {
    //...
    public void lock() {
        while (true) {
            while (state.get()) {}
            if (!state.getAndSet(true))
                return;
        } // end while (true)
    }
    //...
}
```

Figure 1: Nelle parti cerchiare si sta effettuando Spinning.

Lo spinning di **TTASLock** legge da una copia in cache e non produce carico sul bus.

Comunque, **TTAS** non è ideale dato che un poco di overhead lo ha:

Quando il lock viene rilasciato, la variabile viene messa a false, tutte le copie cached sono invalidate e tutti i thread che erano in attesa chiamano `getAndSet(true)`, aumentando il carico del Bus, più di TASLock, ma meno significativamente.

1.10 False Sharing

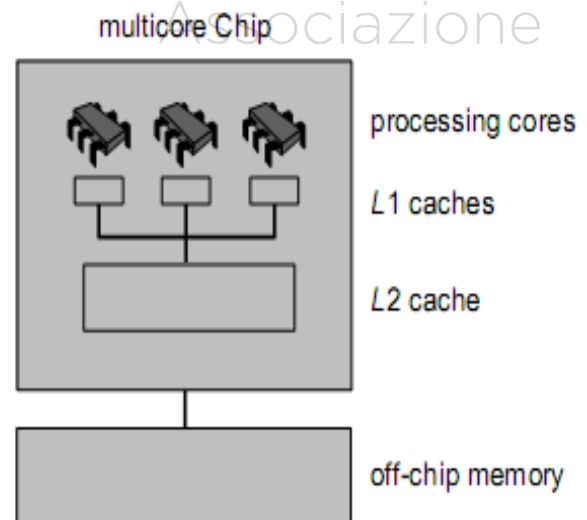
Il False sharing si verifica quando i processori dovrebbero accedere a dati logicamente distinti, ma che generano un conflitto perché le locazioni giacciono sulla stessa cache line (quando si chiede una word dalla memoria centrale, si ottengono un gruppo di word vicine, dette appunto *Cache Line*). È necessario trovare un trade-off in quanto le cache line ampie aumentano la possibilità di sfruttare la locality, quindi aumentano anche la possibilità di false sharing.

Per evitare false sharing si dovrebbe poter avere un controllo a grana fine sui dati (possibile in C/C++, di meno in Java):

- Allineare i dati (con padding) alla cache line (ogni dato stia da solo sulla propria cache line, inserendo dati fittizi in modo che ogni dato sia indipendente in una cache line separata);
- Separare i dati read-only da quelli che cambiano frequentemente;
- Separare gli oggetti in oggetti locali al thread, cioè avere la possibilità di poter usare un array di contatori, uno per thread, ognuno su una linea di cache diversa, in maniera tale che non si invalidino l'uno con l'altro;
- Tenere lock e risorsa su cache line separate, quindi se uso una variabile per fare il lock la risorsa poi la metto abbastanza lontana in memoria;

1.11 Architetture Multicore

- Un processore multicore contiene diversi core sullo stesso chip.
- Ogni core ha la sua L1-cache, mentre tutti i core condividono la L2-cache.
- La comunicazione tra core avviene attraverso la L2-cache, senza passare per la memoria.
- I processori moderni combinano multicore con multi-thread: ogni core è in grado di eseguire due o più thread, usando parallelismo interno.
- Multi-thread su core nasconde la latenza dell'accesso alla memoria: un thread in attesa della memoria viene sostituito con un altro thread pronto.



1.12 Relaxed Consistency

Quando un valore viene scritto in memoria, il valore è mantenuto in cache e segnato come *dirty*, cioè deve essere ancora scritto in memoria. Le richieste di scrittura in memoria non vengono eseguite subito, ma messe in un write buffer. Questa tecnica produce due vantaggi:

- **Batching delle Richieste:** Ammortizza l'overhead del messaggio;
- **Write Obsorption:** Se arriva una successiva write, la precedente non deve essere fatta;
L'uso di un write buffer comporta un'importante conseguenza: l'ordine con cui vengono richieste read/write in memoria non è l'ordine con cui realmente avvengono. I compilatori complicano ancora di più la situazione. Un compilatore può ottimizzare il codice effettuando un reordering delle read/write in memoria. Per esempio, può accadere che un thread riempia un buffer e poi setti un flag per indicare che è pieno. Il reordering può causare che gli altri thread vedano il flag che indica il buffer pieno quando ancora non lo è, e leggano i dati vecchi dal buffer. Esistono delle tecniche ad alto livello per il programmatore, cioè delle tecniche che permettono di fare il flush dei buffer di write buffer. Tra queste abbiamo:
- **Memory Barriers:** Effettua il flush dei buffer. Spesso vengono inserite automaticamente ed in modo trasparente per le operazioni atomiche o per le sezioni critiche. Sono costose in termini di tempo (anche centinaia di cicli macchina) ma necessarie per liberarsi dai bug. In Java, le istruzioni di read/write che sono fuori da un blocco synchronized possono essere riordinate, mentre le read/write su una variabile volatile non sono riordinate. Quindi nessuna garanzia è data se le istruzioni vengono eseguite secondo un certo ordine, cioè se le istruzioni non hanno un blocco sincronizzato, appena viene introdotta la concorrenza non è detto che funzionino correttamente;

Questionario di Fine Lezione

➤ Quali sono le motivazioni al Calcolo Concorrente?

- **Thermal Noise:** sicuramente è una delle motivazioni, dato che racchiude la motivazione di tipo tecnologica, cioè non è possibile ottenere singoli processori con tanti transistor facili da raffreddare e questo crea dei problemi, quindi è necessario utilizzare core diversi;
- **Problemi sempre più complessi:** i problemi complessi vanno affrontati attraverso il parallelismo;
- **Problemi di Tecnologia:** simile al Thermal Noise;

➤ Cos'è il Protocollo MESI?

MESI è un protocollo di coerenza di cache che stabilisce la visibilità della memoria e gestisce lo stato dei processori attraverso quattro stati.

➤ In una cache, qual è la strategia più flessibile per il Replacement?

La strategia migliore è quella Fully Associative, perché permette di mettere la linea di cache dappertutto, quindi è quella più flessibile. Direct Mapped invece non va bene perché non è flessibile poiché obbliga a mettere la linea di cache in una sola parte.

- Una cache L1 è più vicina al processore rispetto alla cache L2, ma non è più grande e più lontana dal processore rispetto alla L2 e soprattutto è meno efficiente in termini di tempo.
- **Qual è il motivo dell'efficienza della Write Obsorption?**
Il motivo della sua efficienza sta nel fatto che la scrittura in memoria viene effettuata meno volte, scrivendo solo il dato finale effettivamente rilevante, quindi non si crea overhead.



Domande di Riepilogo

- Perché, mentre l'efficienza del software scalava tipicamente insieme al progresso tecnologico nei processori (fino a quando sono stati single-core) questo non avviene più nei processori multicore?
- Perché è importante la programmazione concorrente, alla luce del trend tecnologico di processori multicore?
- Perché TASLock risulta essere meno efficiente di TTASLock?
- Perché TASLock è equivalente a TTASLock?
- Perché in TTASLock, dopo aver fatto spinning in lettura con la get, si deve fare una `getAndSet(true)` invece di usare semplicemente una `set(true)`?
- Descrivere la architettura SMP
- Descrivere la architettura NUMA
- Confrontare pro e contro delle architetture SMP e NUMA (ad esempio, efficienza, costi, scalabilità, ...)
- Per quali motivi le cache line dovrebbero essere più grandi possibile?
- Per quali motivi le cache line dovrebbero essere più piccole possibile?
- Perché le cache line non possono essere più piccole di una word?
- Definire cache hit, cache miss, hit ratio.
- Cosa è la locality di un programma?
- Descrivere le tecniche di rimpiazzo della cache
- Quali sono le differenze tra la cache L1 e L2 nei moderni multiprocessori (in termini architetturali, ma anche di funzionalità)
- Descrivere il protocollo MESI
- Provare a cercare di ridurre i 4 stati di MESI a 3 soli, eliminandone uno... e verificare i problemi che sorgono in ciascuna configurazione a 3 stati
- Descrivere lo spinning
- Descrivere la efficienza dello spinning per SMP senza cache, per NUMA con cache e per SMP/NUMA con cache
- Descrivere il False Sharing e le tecniche utilizzabili per limitarlo
- Cos'è la Relaxed Consistency?
- Come funziona e che vantaggi offre il write buffer? E che svantaggi porta?
- Cosa è la write Obsorption?
- Perché è più efficiente fare il Batching delle richieste di write?
- Cosa è il reordering effettuato dai compilatori?
- Perché i compilatori effettuano il reordering?
- Per quale motivo ogni singolo core effettua il multi-thread? Cosa succederebbe se progettassimo un'architettura tale che ogni core fosse single-thread?
- In che maniera in Java si può controllare la effettiva scrittura in memoria dei dati? E perché, visto che esistono questi metodi, essi non vengono usati in ogni parte del codice?

2. Introduzione alla Programmazione Concorrente

2.1 La Parallelizzazione

Partiamo innanzitutto con un esempio:

Ci sono 5 amici che vogliono dipingere una casa con 5 stanze. Se le 5 stanze sono uguali in dimensione (ed anche gli amici sono ugualmente capaci e produttivi) allora finiscono in $1/5$ del tempo che ci avrebbe impiegato una sola persona.

Lo Speedup S ottenuto è pari a 5, ovvero pari al numero degli amici. Se una stanza è grande il doppio il risultato è diverso. Il tempo per concludere la stanza più grande domina sul tempo delle altre.

2.1.1 La Legge di Amdahl

Lo Speedup S di un programma X è il rapporto tra il tempo impiegato da un processore per eseguire X rispetto al tempo impiegato da n processori per eseguire X . All'interno di un programma X è possibile distinguere due porzioni:

- Una parte p del programma che è possibile parallelizzare;
- Una parte $1 - p$ che non si può parallelizzare e quindi è necessario eseguire in tempo sequenziale;

È possibile supporre che la parte parallela possa essere eseguita da tutti i processori insieme, quindi il tempo necessario è p/n , mentre la parte sequenziale viene eseguita da un solo processore.

Lo Speedup è dato dalla Legge di Amdahl e si ottiene eseguendo il programma X su n processori, dove p è la parte di X che si può parallelizzare:

$$S = \frac{1}{1 - p + \frac{p}{n}}$$

La Legge di Amdahl è un'indicazione qualitativa del fatto che la parte sequenziale di un programma è quella che rallenta significativamente qualsiasi speedup che possiamo pensare di ottenere. Per velocizzare un programma non basta investire sull'hardware, ma è assolutamente necessario e molto più cost-effective impegnarsi a rendere la parte parallela predominante rispetto alla parte sequenziale.

```

1 Counter counter = new Counter(1); // shared by all threads
2 void primePrint {
3     long i = 0;
4     long limit = power(10, 10);
5     while (i < limit) {           // loop until all numbers taken
6         i = counter.getAndIncrement(); // take next untaken number
7         if (isPrime(i))
8             print(i);
9     }
10 }

```

Questo è un esempio di un contatore condiviso. C'è una variabile contatore. Viene calcolato il limite 10^{10} , usiamo delle operazioni di incremento atomico, cioè `getAndIncrement()` è un'operazione che in maniera atomica, ovvero non interrompibile da ogni altro thread,

permette ad un thread di ottenere il valore e di incrementarne quest'ultimo tutto in una sola operazione. Questa è una maniera abbastanza semplice per distribuire un carico di lavoro. Supponiamo di calcolare i numeri primi. Ad ogni thread capita uno degli interi da 1 a 10^{10} da controllare se sono o meno primi. La parti `isPrime(i)` e `print(i)` sono eseguite in parallelo mentre `i` è eseguito in maniera sequenziale. È chiaro che una soluzione del genere, che vede ogni processore in coda davanti ad un distributore di interi, ed ognuno riceve il proprio intero da controllare la primalità, è estremamente efficiente. Alcune linee guida sono minimizzare la granularità del codice sequenziale e rendere efficienti la comunicazione ed il coordinamento sul contatore condiviso.

2.1.2 Le Sfide della Programmazione Parallela

Le sfide della programmazione parallela sono essenzialmente problematiche relative alla complessità di utilizzare strumenti complessi per coordinarne l'esecuzione. Il coordinamento è quindi il punto critico. Ci sono anche problematiche di tipo tecnologico (come si valutano le prestazioni?) e serve soprattutto un approccio culturale diverso, sia per gli applicativi che per i teorici.

Alcuni problemi sono *Embarassingly Parallel*, ovvero problemi dove è estremamente facile renderli parallelizzabili. Qualsiasi sincronizzazione avviene nella fase iniziale e/o in quella finale. Altri invece hanno solo una parte parallelizzabile con facilità. La programmazione su multiprocessore offre molte sfide che vanno da problemi teorici a trucchi pratici, cioè si mette insieme l'algoritmo e la conoscenza della strategia di replacement della cache. L'approccio sarà quello di partire dalla formulazione teorica per arrivare a modelli più applicativi. Ci sono problemi per misurare le prestazioni, tra cui:

- Tempo di esecuzione;
- Scalabilità;
- Efficienza parallela;
- Requisiti di memori;
- Throughput;
- Latenza;
- I/O Data rate;
- Network throughput;
- Costi di progettazione, implementazione, verifica, manutenzione;
- Potenziale per riutilizzo e portabilità;
- Requisiti di hardware;

Tutte queste metriche sono introdotte dalla complessità del calcolo concorrente e parallelo. Ogni scenario ha metriche diverse, ognuna valida in scenari diversi.

2.1.3 Le Scenari con Metriche Diverse

Le prestazioni vanno interpretate in maniera diversa a seconda degli scenari nei quali vengono collocate:

- Sistemi di previsioni del tempo: Vincoli temporali stretti
 - Tempo di esecuzione;
 - Affidabilità;
 - Scalabilità e portabilità;

Se stiamo elaborando un sistema molto complesso che prevede il tempo in tre giorni, è chiaro che il tempo di computazione dovrebbe essere inferiore a tre giorni, perché se il tempo di computazione è superiore o uguale a tre giorni ovviamente si fa prima ad aprire la finestra e vedere che tempo fa, piuttosto che elaborare computazioni così complesse.

- Ricerca Parallela in database
 - Limite su tempo di implementazione;
 - Qualsiasi velocizzazione in tempo è ok;
 - Portabilità su Database design;
- Image-Processing (pipeline)
 - Throughput di immagini per secondo;
 - Latenza di ogni immagine nella pipeline;

La cosa importante è capire che l'approccio alle prestazioni per il calcolo parallelo deve essere ragionato, non è puramente aritmetico. La nostra implementazione sulla macchina parallela X ottiene uno speedup di 10.8 su 12 processori, con dimensione del problema $N = 100$. Sembra buono ma ci sono delle cose a cui bisogna stare attenti, e in particolar modo:

In questo caso il problema è che, oltre a cambiare la dimensione del problema, possono cambiare anche il numero di processori, cioè una soluzione che è particolarmente efficiente e che magari scala con N , con pochi processori potrebbe di colpo diventare estremamente inefficiente quando il numero di processori comincia ad aumentare.

Quindi se da una macchina parallela, che ha un certo numero di processori o un certo tipo di efficienza, si decide di passare ad una macchina parallela più grande, più ampia, con più numero di processori, allora ci si aspettano prestazioni incredibilmente alte, ma queste prestazioni non ci sono. Questo è un problema dovuto al fatto che il tempo è funzione non solo di N (dimensione del problema) ma anche di P (numero di processori).

Il risultato è che ci troviamo di fronte a diverse funzioni:

- Parte onerosa divisa tra processori, ma parte lineare(sequenziale) N non parallelizzabile, il carico (la parte onerosa) N^2 può essere diviso fra i processori, perciò N^2/P .

$$T_1 = N + \frac{N^2}{P}$$

- La parte onerosa in questo caso ($N + N^2$) è completamente parallelizzabile, abbiamo un overhead costante, indipendentemente della dimensione dei dati c'è un tempo costante (100) che deve essere impiegato.

$$T_2 = \frac{(N + N^2)}{P} + 100$$

- Per quanto riguarda la parte parallelizzabile, essa ha la stessa complessità della funzione precedente ma c'è una differenza: l'overhead è proporzionale a P^2 (quindi al numero di processori, ossia le risorse che stiamo impiegando) piuttosto che essere costante rispetto alla funzione precedente.

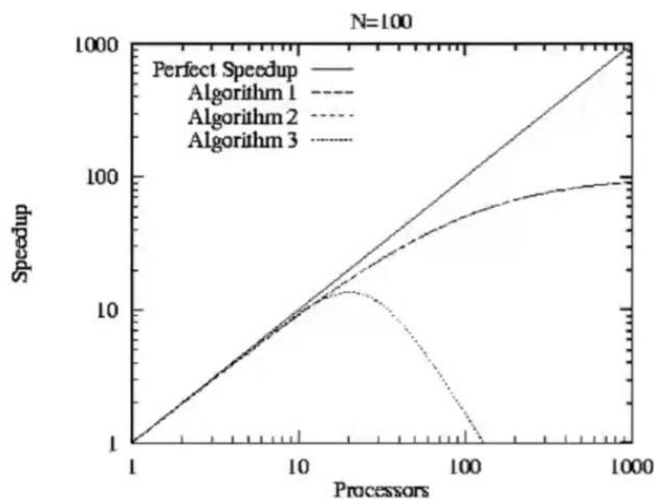
$$T_3 = \frac{(N + N^2)}{P} + 0.6P^2$$

N.B.

N: La dimensione del problema cresce molto e possiamo farla crescere in maniera significativa.

P: È indicativo del numero di processori, costa molto e cresce di meno.

PER $N = 100$



Questi tre algoritmi T_1 , T_2 , T_3 presentano delle problematiche non banali. In particolar modo colpisce molto il terzo algoritmo T_3 , dove al crescere del numero di processori, abbiamo che il peso è dovuto dall'aumentare di tempo. Il numero di processori del terzo algoritmo, quindi, aumenta il tempo (della versione parallela del programma) in modo proporzionale al quadrato del numero di processori, portando lo Speedup molto giù, quindi il programma basato su T_3 , che fino ad un certo punto era molto simile allo Speedup degli altri due algoritmi, di colpo diventa molto diverso a causa del numero di risorse (processori) utilizzate.

Questo è un problema non banale ed è un comportamento che si vede anche per una dimensione più grande.

In questo diagramma, rispetto a quello precedente si riescono a distinguere gli algoritmi T_1 e T_2 , dove all'aumento del numero di processori si ha che lo Speedup varia sensibilmente da un algoritmo ad un altro.

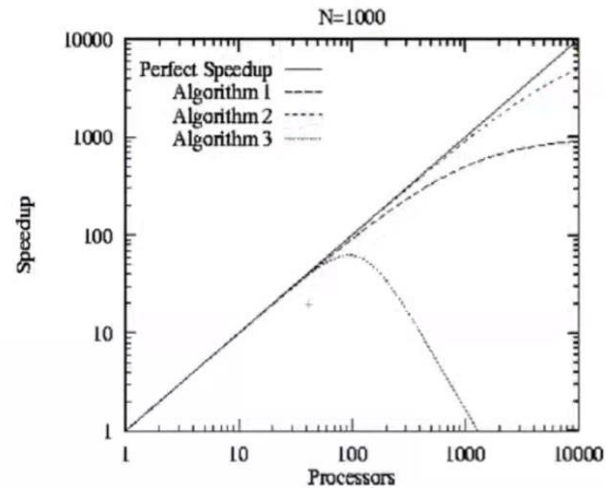
Esempio:

Questo è un problema assolutamente non banale. Si immagina di aver scritto un programma per 100 nodi, con uno Speedup vicino a 100, dove con un investimento di 3M € si compra per esempio una macchina da 256 nodi. Invece di avere uno Speedup di 256, quello che probabilmente succede è

che il programma va man mano peggio quando si va ad aggiungere hardware, quindi man mano che si vanno ad aggiungere processori il tempo aumenta. È necessario comprendere esattamente cosa viene nascosto dai modelli e dalle valutazioni algoritmiche, dato che programmi performanti per alcuni modelli quando vengono implementati su architetture reali si ha che non sono altrettanto efficienti.

La notazione "O grande" va usata con cura perché le costanti importano, l'algoritmo richiede tempo $O(N \log N)$ su N processori.

PER $N = 1000$



Ci sono situazioni nelle quali la notazione O grande nasconde delle prestazioni:

$$T(N) = 10N + N \log N: \text{se } N < 1024, 10N \text{ ha il suo peso}$$

$$T_1(N) = 1000N \log N \text{ e } T_2 = 10N^2$$

Per $N < 996$ l'algoritmo con T_2 è migliore

2.2 Java e la Concorrenza

2.2.1 Thread e Monitor

Un Thread esegue un singolo programma sequenziale. In java è generalmente una sottoclasse di `java.lang.Thread`.

```
public class HelloWorld implements Runnable {
    String message;
    public HelloWorld(String m) {
        message = m;
    }
    public void run() {
        System.out.println(message);
    }
}
```

Un oggetto Runnable può essere eseguito in un thread chiamando il costruttore della classe Thread che prende un oggetto Runnable come argomento.

```
String m = "Hello World from Thread" + i;
Thread thread = new Thread(new HelloWorld(m));
```

Un altro modo è quello di chiamare una classe interna anonima:

```
final String m = "Hello world from thread" + i;
thread = new Thread(new Runnable() {
    public void run() {
        System.out.println(m);
    }
});
```

Per eseguire un thread bisogna chiamare `thread.start()`. Se invece si vuole che il chiamante aspetti che il thread finisca, bisogna chiamare il metodo `thread.join()`.

```
1  public static void main(String[] args) {
2      Thread[] thread = new Thread[8];
3      for (int i = 0; i < thread.length; i++) {
4          final String message = "Hello world from thread" + i;
5          thread[i] = new Thread(new Runnable() {
6              public void run() {
7                  System.out.println(message);
8              }
9          });
10     }
11     for (int i = 0; i < thread.length; i++) {
12         thread[i].start();
13     }
14     for (int i = 0; i < thread.length; i++) {
15         thread[i].join();
16     }
17 }
```

I monitor in java rappresentano il modo per sincronizzare l'accesso ai dati condivisi ed è necessaria la mutua esclusione sulle invocazioni. Facciamo un esempio su una coda (codice didattico):

```
1  class CallQueue {
2      final static int QSIZE = 100; // arbitrary size
3      int head = 0; // next item to dequeue
4      int tail = 0; // next empty slot
5      Call[] calls = new Call[QSIZE];
6      public enq(Call x) { // called by switchboard
7          calls[(tail++) % QSIZE] = x;
8      }
9      public Call deq() { // called by operators
10         return calls[(head++) % QSIZE]
11     }
12 }
```

Questa classe non funziona correttamente se due operazioni provano a chiamare deque allo stesso istante. Per rendere la coda corretta, bisogna assicurare che solo un'operazione per volta possa chiamare la deque.

Ogni oggetto ha un lock implicito. Se un thread acquisisce il lock di un oggetto A, nessun altro thread può acquisirlo fino a quando non è stato rilasciato. Per acquisire il lock, ad esempio, basta dichiarare un metodo synchronized ed in questo modo nessun altro metodo dell'oggetto può essere invocato fino al rilascio del lock.

```
public synchronized T deq() {
    while (head == tail) {}; // spin while empty
    call[(head++) % QSIZE];
}
```

Fa spinning fin quando la coda è vuota. Problemi creati: Deadlock! Nessun altro metodo (compreso enq(!)) può essere eseguito.

```
public synchronized T deq() {
    while (head == tail) {wait()};
    call[(head++) % QSIZE];
}
```

La wait() rilascia prima il lock, e poi sospende il thread. Quando viene risvegliato (da una notify() o notifyAll()) effettua il lock di nuovo prima di ritornare.

Figura 3: Possibile soluzione (abbastanza semplice e non scala)

Il metodo yield() permette ad un thread di rilasciare il lock e lasciare allo scheduler la decisione su chi (e se) schedulare al posto suo.

Il metodo sleep(t) istruisce lo scheduler a non eseguire il thread *almeno* per t secondi (questa è l'istruzione più sbagliata dai programmatori, sleep(t) dove t indica i secondi, non significa che il thread dorme per t secondi, ma significa che dopo t secondi il thread diventa di nuovo eseguibile).

Recap per i monitor:

- Necessaria la mutua esclusione sulle invocazioni;

- Ogni oggetto ha un lock implicito;
- Se un thread X acquisisce il lock di un oggetto A, nessun altro thread Y può acquisirlo fino a quando non è stato rilasciato da X;
- Per acquisire il lock, ad esempio basta dichiarare un metodo *synchronized* ed in questo modo nessun altro metodo dell'oggetto può essere invocato fino al rilascio del lock.

2.2.2 Thread-local Objects

Le variabili thread-local sono variabili che appartengono al thread:

- Indipendentemente inizializzate;
- Vanno in memoria centrale (condivisa) ma con accesso separato (ThreadLocalMap);
- Non richiedono sincronizzazione;
- Non generano traffico per mantenere la coerenza nella cache (ci lavora solo quel thread, solo lui la tiene in cache).

ThreadLocal <T> ha metodi per:

- L'inizializzazione `initialValue()` (serve per inizializzare);
- Accesso in lettura `get()` ;
- Accesso in scrittura `set()` .

```

1  public class ThreadID {
2      private static volatile int nextID=0;
3
4      private static class ThreadLocalID extends ThreadLocal<Integer>{
5          protected synchronized Integer initialValue(){
6              return nextID++;
7          }
8      } // fine classe ThreadLocalID
9      private static ThreadLocalID threadID=new ThreadLocalID();
10     public static int get(){
11         return threadID.get();
12     }
13
14     public static void set (int index){
15         threadID.set(index);
16     }
17 }

```

Figure 4: Identificatore unico per-thread

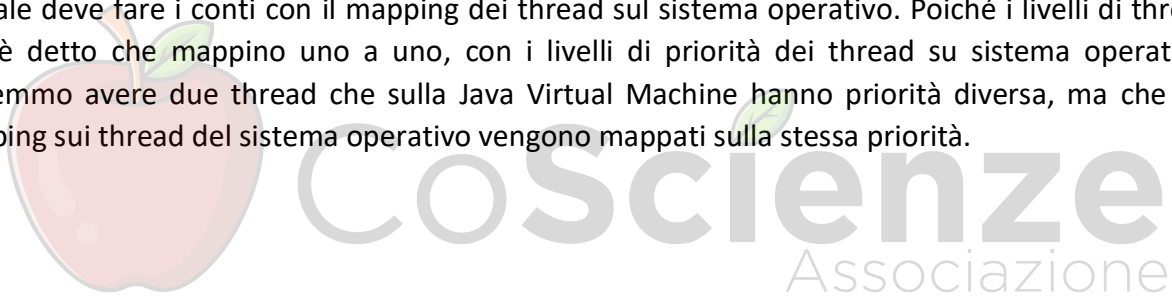
Questa classe mantiene un identificatore unico del thread che è accessibile senza sincronizzazione, proprio perché è riservato al thread. Quello che si fa è definire una classe interna (Riga 4). Questa classe interna estende `ThreadLocal<Integer>`, cioè è semplicemente un intero (per fare un intero non condiviso bisogna fare 10/15 linee di codice).

Ridefinizione del metodo `initialValue()` (Riga 5) metodo di inizializzazione. Quello che fa è restituire un integer, cioè il valore appena messo all'interno, e inserisce all'interno il valore di `nextID++` (Riga 6) della `public class ThreadID`, quindi il valore viene aumentato. L'idea è

che dentro `ThreadLocal` la prima volta che verrà istanziato un oggetto conterrà 0, la seconda volta 1. Questo valore viene scritto in una locazione di memoria specifica `ThreadLocal`, specifica per ogni thread. Con `ThreadLocalID()` (Riga 9) si definisce `threadID` come un new `ThreadLocalID` e si definiscono i metodi `get()` e `set()` come la restituzione dei metodi `get()` e `set()` di `ThreadLocalID`.

2.2.3 La Java Virtual Machine e i Thread

È il principio fondamentale che l'informatica usa per l'isolamento tra l'astrazione dei livelli superiori e quelli inferiori. La macchina virtuale serve per eseguire in maniera indipendente da quello che è l'hardware al di sotto, questo significa che esiste un thread nella macchina virtuale che non necessariamente corrisponde ai thread del processore/sistema operativo sottostante. Nelle prime macchine virtuali si usava una simulazione software, cioè thread software nella macchina virtuale. Adesso le macchine virtuali sono altamente ottimizzate per usare le capacità multi-thread dei sistemi operativi su cui si poggiano. L'idea è che i thread della macchina virtuale vengono mappati sui thread del sistema operativo, che automaticamente usa questi thread su qualsiasi macchina multiprocessore che abbia al di sotto. La priorità dei thread che vengono settati nella macchina virtuale deve fare i conti con il mapping dei thread sul sistema operativo. Poiché i livelli di thread non è detto che mappino uno a uno, con i livelli di priorità dei thread su sistema operativo, potremmo avere due thread che sulla Java Virtual Machine hanno priorità diversa, ma che nel mapping sui thread del sistema operativo vengono mappati sulla stessa priorità.



Domande di Riepilogo

- Cosa dice la legge di Amdahl?
- Perché questo ha impatto su dove si sincronizza in un programma (granularità della sincronizzazione)?
- Cos'è un problema Embarassingly Parallel?
- A cosa servono i metodi: start(), join(), wait(), notify(), yield(), sleep() ?
- Cosa sono i Thread-local objects? Che caratteristiche hanno?



3. Mutua Esclusione

La mutua esclusione è una delle forme più importanti di coordinamento nella programmazione concorrente. Questo capitolo mostra come gli algoritmi lavorano nella lettura e scrittura di memoria condivisa. Anche se questi algoritmi non vengono usati nella pratica, vengono studiati per dare un'introduzione ai tipi di algoritmi e alle problematiche sulla correttezza che si presentano nella sincronizzazione.

3.1 Tempo

Il tempo è un fattore fondamentale per il calcolo concorrente. A volte non vogliamo pensare che qualcosa accada simultaneamente, altre invece vogliamo che accadano in tempi differenti.

Ordini Totali e Parziali

Un Ordine Totale $\leq$ su un insieme X è una relazione binaria che è:

- **Antisimmetrica:** Se $a \leq b$ e $b \leq a$, allora $a = b$;
- **Transitiva:** Se $a \leq b$ e $b \leq c$, allora $a \leq c$;
- **Totale:** Se per ogni coppia di $a, b \in X$ vale che $a \leq b$ o $b \leq a$;
- **Riflessiva:** $a \leq a$;

Un Ordine Parziale su un insieme X è una relazione binaria che è antisimmetrica, transitiva e riflessiva (non totale). Questo significa che presi due elementi a e b dell'insieme, si potrebbe non essere in grado di dire se l'elemento a viene prima di b o viceversa. La relazione potrebbe essere una relazione che esprime una relazione con il tempo.

Programmazione concorrente significa parlare del tempo come:

- **Eventi istantanei:** In un singolo istante di tempo;
- **Eventi mai simultanei:** Se necessario scegliamo un qualsiasi ordine tra eventi molto vicini nel tempo.

Un thread A produce una serie di eventi $a_0, a_1, \dots$ dato che può contenere cicli, una singola istruzione di un programma può produrre diversi eventi. Indichiamo la j -esima occorrenza di un evento a_i attraverso a_i^j .

Un evento a precede un altro evento b ($a \rightarrow b$) se a si verifica prima. La precedente relazione $\rightarrow$ è un ordine totale sugli eventi.

Sia $a_0 \rightarrow a_i^j$. Un intervallo (a_0, a_1) è la durata di tempo tra a_0 e a_1 . L'intervallo $IA = (a_0, a_1)$ precede l'intervallo $IB = (b_0, b_1)$, scritto $IA \rightarrow IB$, se $a_1 \rightarrow b_0$, ossia che l'evento finale di IA precede quello iniziale di IB . Quindi non siamo sempre in grado di dire che A precede B , infatti $IA \rightarrow IB$ è un ordine parziale sugli intervalli.

Se per due intervalli I e J vale che $I \nrightarrow J$ e che $J \nrightarrow I$, allora I e J sono concorrenti.

Questo significa che in tale periodo di tempo, gli intervalli rappresentano un'esecuzione concorrente. Con un evento è possibile dire che è istantaneo ed è sempre possibile fare un ordine parziale. Un intervallo può essere invece concorrente.

3.2 Sezione Critica

L'esempio su Counter funziona bene su un sistema con un single thread, ma si comporta male quando viene utilizzato da due o più thread. Il problema si verifica quando i thread leggono il valore del campo dalla linea 8 alla 9.

```
1 class Counter {
2     private int value;
3     public Counter(int c) {    // constructor
4         value = c;
5     }
6     // increment and return prior value
7     public int getAndIncrement() {
8         int temp = value;      // start of danger zone
9         value = temp + 1;      // end of danger zone
10        return temp;
11    }
12 }
```

La variabile `value` è lo stato del contatore. C'è una inizializzazione e un'operazione di `getAndIncrement()`, che presuppone essere qualcosa in cui si può prendere il valore, incrementarlo e andarlo a mettere all'interno. Non è una soluzione perché in caso di interfogliamento tra thread questo crea dei problemi.

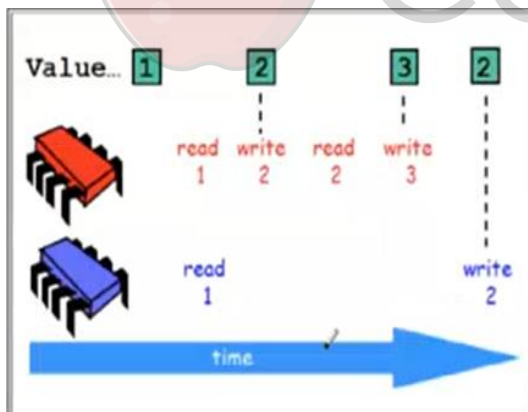


Figure 5 Esempio esecuzione Counter

Il valore del contatore inizialmente è 1, il processore rosso legge 1 e scrive 2, legge 2 e scrive 3, il processore blu nel frattempo legge 1 e scrive 2. Tutto quello letto e scritto dal processore rosso viene schedato dopo e quindi tutto quello che è successo tra la sua lettura e scrittura viene sovrascritto.

L'idea è quella di trasformare la zona di pericolo in una sezione critica, cioè una sezione che prevede l'accesso di un thread alla volta, in mutua esclusione. Il metodo standard è quello di utilizzare un lock, cioè un thread acquisisce il diritto ad entrare bloccando il lock e lo

rilascia quando esce. Un thread che trova il lock bloccato attende che si sblocchi.

Il modo migliore per sviluppare questo concetto è attraverso un oggetto Lock come descritto dall'interfaccia mostrata di seguito:

```
1 public interface Lock {
2     public void lock();    // before entering critical section
3     public void unlock(); // before leaving critical section
4 }
```

Si usa un'interfaccia Lock. Verranno definiti degli oggetti che implementano tale interfaccia. Essa implementa un metodo

`lock()` che verrà chiamato quando un thread acquisisce un lock, e quest'ultimo verrà rilasciato quando viene effettuata una chiamata al metodo `unlock()`.

```
1 public class Counter {
2     private long value;
3     private Lock lock;           // to protect critical section
4
5     public long getAndIncrement() {
6         lock.lock();             // enter critical section
7         try {
8             long temp = value;    // in critical section
9             value = temp + 1;     // in critical section
10        } finally {
11            lock.unlock();         // leave critical section
12        }
13        return temp;
14    }
15 }
```

L'idea è quella di fare `lock.lock()`. Da questo metodo viene restituito il controllo esclusivamente se viene acquisito il lock.

A tal punto si è già all'interno del `try`, vengono effettuate le operazioni e poi l'operazione `finally` garantisce l'esecuzione del

blocco senza interessarsi di eccezioni ed errori. Alla fine, si rilascia il lock e si fa il `return` del `temp`. Secondo l'interpretazione qualitativa della legge di Amdahl, l'operazione di `return temp` poteva essere posta prima di `finally`, ma non era cruciale. È meglio portarla fuori e rilasciare prima il lock, ovvero diminuire al massimo la sezione critica, che è quella sequenziale, in maniera tale da permettere l'esecuzione concorrente.

Come facciamo a dimostrare la correttezza della Mutua Esclusione?

Innanzitutto, bisogna dire che cosa significa che un thread è *well-formed*. Ci sono buone prassi riassunte dalla sua definizione che ci guidano nell'evitare errori banali.

Un thread che usa un lock è *well-formed* se:

- Ogni sezione critica è associata con un unico oggetto `Lock`;
- Il thread chiama il metodo `lock()` dell'oggetto quando si cerca di entrare nella sezione critica;
- Il thread chiama il metodo `unlock()` quando esce fuori dalla sezione critica.

Se si usa un lock che è *well-formed*, tutta la responsabilità della correttezza viene messa sulla correttezza del lock, cioè è l'implementazione del lock che è importante. Per poter garantire il corretto funzionamento della sezione critica è necessario formalizzare il comportamento per dimostrarne il corretto funzionamento.

Ora formalizziamo le proprietà che un buon algoritmo di Lock deve soddisfare. Indichiamo con CS^A_j l'intervallo in cui A esegue la sezione critica per la j-ma volta. Assumiamo per semplicità che ogni thread acquisisce e rilascia il lock un numero infinito di volte:

- **Mutua Esclusione:** Sezioni critiche di thread diversi non si sovrappongono. Per ogni thread A e B con gli interi j e k, vale che $CS^A_k \rightarrow CS^j_B$ oppure $CS^j_B \rightarrow CS^A_k$;
- **Deadlock-free:** Se qualche thread cerca di acquisire il lock, qualcuno ha successo. Se un thread chiama `lock()` ma non ottiene mai il lock, allora altri thread devono riuscire a completare un numero infinito di volte la sezione critica. Se un thread A è bloccato, allora lo è perché tutti gli altri thread stanno lavorando. Deadlock-free non significa che non si blocca un thread, ma bensì se si blocca, allora si blocca perché si sta facendo una quantità infinita di lavoro con gli altri

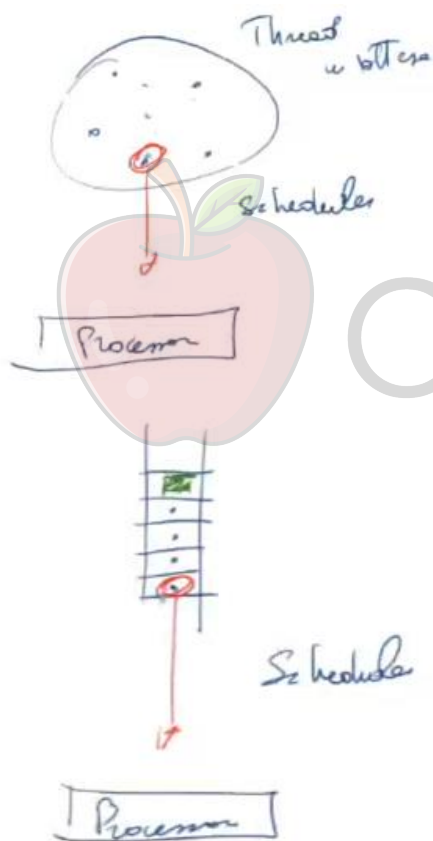
thread. Deadlock-free non impedisce quindi la possibilità che un thread venga bloccato, ma dice che se un thread viene bloccato in maniera infinita lo fa perché ci sono thread che accedono un infinito numero di volte, questo perché non è detto che ci sia un'equa distribuzione di risorse.

- **Starvation-free:** Ogni thread che cerca di acquisire il lock, ha successo alla fine. Ogni chiamata a `lock()` ritorna al metodo chiamante;

La Deadlock-free garantisce che la risorsa è usata sempre al massimo, se c'è qualcuno che è bloccato è perché gli altri accedono infinite volte. La Starvation-free invece garantisce che la risorsa è usata in maniera equa. Si è in attesa di entrare, e prima o poi si accederà.

3.2.1 Proprietà: Rephrasing

- **Deadlock-free:** se un processo cerca di entrare nella sezione critica, qualcuno entrerà;
- **Starvation-free:** se un processo cerca di entrare nella sezione critica, alla fine ci riuscirà;



Con il Deadlock-free in qualche maniera ci sono tutti i thread in attesa, e c'è lo scheduler che ne prende uno e lo esegue sul processore. Giocando su questo, l'avversario può mettere in crisi il thread e dire che può ritardarlo, schedulando all'infinito l'accesso a tutti gli altri.

Nella Starvation-free è più un meccanismo per esibire la priorità di un thread. Ci sono tutti i thread ma c'è una sorta di precedenza di ingresso. Lo scheduler prima o poi andrà ad eseguire il thread (in rosso) e prima o poi il thread entrerà.

Quindi la Starvation-free implica la Deadlock-free. Non c'è garanzia però che la Starvation-free garantisca quanto tempo debba aspettare un thread in attesa.

Separiamo contrattualmente l'esecuzione di A del metodo `lock()` in due componenti:

- **Doorway:** La cui esecuzione da parte di A prende un intervallo D_A di durata limitata superiormente. La Doorway è qualcosa che termina sempre, quindi l'ingresso in sezione critica;
- **Waiting:** La cui esecuzione da parte di A prende un intervallo W_A di durata non limitata;

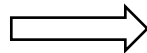
First-come-first-served: Se il thread finisce la sua doorway prima che B inizi la sua doorway, allora A non può essere sorpassato da B : se $D^j_A \rightarrow D^k_B$ allora $CS^j_A \rightarrow CS^k_B$.

Chi termina la doorway, ovvero nel momento in cui termina la doorway, acquisisce priorità rispetto a tutti quelli che termineranno la doorway dopo di lui.

3.3 Sezione Critica per 2 Thread

L'Algoritmo di Peterson è la prima soluzione per quanto riguarda la sezione critica per due Thread senza supporto hardware particolare. Per motivi didattici si presenta questo algoritmo come la fusione di due algoritmi: LockOne e LockTwo. Questi due algoritmi sono entrambi in grado di assicurare la mutua esclusione, ossia per ogni thread A e B, per ogni j e k vale che la sezione critica di A j-esima precede la sezione critica di B k-esima oppure la sezione critica di B k-esima precede la sezione critica di A j-esima.

$\forall A, B \text{ Thread}$



$CS^j_A \rightarrow CS^k_B \text{ oppure } CS^k_B \rightarrow CS^j_A$

$\forall i \in T$

Ma ciascuno di loro non è deadlock-free, quindi non è assicurato che qualcuno fa progresso. Si assicura che non entrano insieme, però ci sono delle condizioni in cui questi algoritmi essenzialmente non permettono l'accesso a nessuno e questo qui è il punto cruciale.

Ciascuno in due situazioni diverse (quando i task o sono interfogliati o quando vengono eseguiti uno dopo l'altro) ma messi insieme con il lock di Peterson, che quindi unisce le tecniche dei due algoritmi, soddisfa sia la mutua esclusione che la condizione di deadlock-free.

3.3.1 La Classe LockOne

Nell'algoritmo 2-thread i thread hanno id 0 ed 1: Uno si chiama i, l'altro j=1-i. Ogni thread ottiene il suo indice attraverso la chiamata `ThreadId.get()` e poi con `j = 1 - i` si calcola l'indice dell'altro.

La Riga 7 rappresenta l'interesse ad entrare in sezione critica. La Riga 8, si aspetta fin quando l'altro è interessato, fin quando l'altro è presente all'interno, che quindi esca. Quindi quando `flag[j] = false` si esce dal `while`, si esce dal metodo `lock` e quindi si è acquisito il lock.

```

1  class LockOne implements Lock {
2      private boolean[] flag = new boolean[2];
3      // thread-local index, 0 or 1
4      public void lock() {
5          int i = ThreadID.get();
6          int j = 1 - i;
7          flag[i] = true;
8          while (flag[j]) {}           // wait
9      }
10     public void unlock() {
11         int i = ThreadID.get();
12         flag[i] = false;
13     }
14 }

```

Riga 2: `private volatile boolean[] flag = new boolean[2];`

La variabile booleana `flag` deve essere dichiarata volatile, perché qualsiasi modifica a questo array fatta da un thread deve essere visibile all'altro thread.

La Riga 12 è il metodo dell'unlock, che prevede quindi lo sbloccaggio della situazione, si mette $flag[i] = false$ e questo potrebbe sbloccare l'altro thread che è in attesa di arrivare.

Indichiamo con $wA(x=v)$ l'evento in cui A assegna il valore v a x. ($w \rightarrow Write$).

Indichiamo con $rA(x=v)$ l'evento in cui A legge dalla variabile x un valore v ($r \rightarrow Read$).

Lemma. L'algoritmo LockOne soddisfa la mutua esclusione.

Dimostrazione. Per assurdo, supponiamo che vi siano due interi j e k t.c. $CS_A^j \rightarrow CS_B^k$ e $CS_B^k \rightarrow CS_A^j$. Considerando l'ultima esecuzione del metodo `lock()` di ogni thread prima di entrare nella k-esima (j-esima) sezione critica. Guardando il codice, possiamo vedere che:

$write_A(flag[A] = true) \rightarrow read_A(flag[B]) == false \rightarrow CS_A$

$write_B(flag[B] = true) \rightarrow read_B(flag[A]) == false \rightarrow CS_B$

Poiché quando $flag[B]$ è messo a true da B, nessuno lo cambia (siamo nel conflitto della CS), allora:

$read_A(flag[B] == false) \rightarrow write_B(flag[B] = true)$

Quindi $flag[B]$ è settato a true e rimane a true. Ne deriva che l'equazione 2.3.3 è vera, altrimenti il thread A non avrebbe potuto leggere $flag[B]$ come falsa. L'equazione 2.3.4 segue le equazioni 2.3.1 e 2.3.3 e la transitività dell'ordine precedente.

$write_A(flag[A] = true) \rightarrow read_A(flag[B]) == false \rightarrow write_B(flag[B] = true) \rightarrow read_B(flag[A]) == false$

Significa che $write_A(flag[A] = true) \rightarrow read_B(flag[A] == true)$ senza interventi scrive nell'array $flag[]$, una contraddizione.

Questo algoritmo è inadeguato perché si verifica deadlock se l'esecuzione dei thread sono intervallate. Se gli eventi $write_A(flag[A] = true)$ e $write_B(flag[B] = true)$ avvengono contemporaneamente prima degli eventi $read_A(flag[B])$ e $read_B(flag[A])$, i thread aspetteranno per sempre. Ciò nonostante, LockOne ha un'interessante proprietà: se un thread viene eseguito prima dell'altro, non accade deadlock e tutto andrà bene.

3.3.2 La Classe LockTwo

Victim, significa “io sono la vittima”, ossia indica il thread che deve aspettare.

La Riga 5 quindi indica chi deve aspettare.

La Riga 6, il ciclo viene eseguito mentre la variabile victim è messa al valore i.

```
1 class LockTwo implements Lock {
2     private volatile int victim;
3     public void lock() {
4         int i = ThreadID.get();
5         victim = i;           // let the other go first
6         while (victim == i) {} // wait
7     }
8     public void unlock() {}
9 }
```

La Riga 8, l’unlock indica, non fare niente perché essenzialmente l’idea di questo algoritmo è che “io provo ad entrare e che quando qualcuno prova ad entrare sblocca a me”.

Lemma. L’algoritmo soddisfa la mutua esclusione.

Per assurdo, supponiamo che vi siano gli interi j e k t.c. $CS_A^j \nrightarrow CS_B^k$ e $CS_B^k \nrightarrow CS_A^j$ (le due sezioni critiche sono eseguite in concorrenza).

Considerando che l’ultima esecuzione del metodo lock() di ogni thread avvenga prima che entri nella k -esima (j -esima) sezione critica. Guardando il codice, possiamo vedere:

$write_A(victim = A) \rightarrow read_A(victim == B) \rightarrow CS_A$

$write_B(victim = B) \rightarrow read_B(victim == A) \rightarrow CS_B$

Il thread B deve assegnare B al campo victim tra i due eventi $write_A (victim=A)$ e $read_A(victim=B)$ (Eq. 2.3.5). Poiché questo assegnamento è l’ultimo, abbiamo:

$write_A(victim = A) \rightarrow write_B(victim = B) \rightarrow read_A(victim == B)$.

Una volta che il campo victim è settato a B, esso non cambia, così qualsiasi sotto sequenza di lettura ritorna B, contraddicendo la 2.3.6.

Questa classe è inadeguata perché può causare deadlock se un thread viene eseguito completamente prima degli altri. Tale classe però ha un’interessante proprietà: se i thread vengono eseguiti correntemente, il metodo lock() ha successo.

Le classi LockOne e LockTwo si completano a vicenda.

3.3.3 Il Lock di Peterson

Ora, fondiamo gli algoritmi LockOne e LockTwo per costruire un algoritmo di Lock privo di starvation, dove si va ad utilizzare l'&& delle due condizioni.

```
1  class Peterson implements Lock {
2      // thread-local index, 0 or 1
3      private volatile boolean[] flag = new boolean[2];
4      private volatile int victim;
5      public void lock() {
6          int i = ThreadID.get();
7          int j = 1 - i;
8          flag[i] = true;           // I'm interested
9          victim = i;               // you go first
10         while (flag[j] && victim == i) {}; // wait
11     }
12     public void unlock() {
13         int i = ThreadID.get();
14         flag[i] = false;          // I'm not interested
15     }
16 }
```

È un modo più elegante di descrivere l'algoritmo di mutua esclusione ed è conosciuto come algoritmo di Peterson, dal nome del suo inventore.

Lemma. L'algoritmo soddisfa la mutua esclusione.

Dimostrazione. Per assurdo si supponga di no. Come prima, consideriamo l'ultima esecuzione del metodo lock() da parte dei thread A e B. Ispezionando il codice, possiamo vedere:

$\text{write}_A(\text{flag}[A] = \text{true}) \rightarrow \text{write}_A(\text{victim} = A) \rightarrow \text{read}_A(\text{flag}[B]) \rightarrow \text{read}_A(\text{victim}) \rightarrow \text{CS}_A$

$\text{write}_B(\text{flag}[B] = \text{true}) \rightarrow \text{write}_B(\text{victim} = B) \rightarrow \text{read}_B(\text{flag}[A]) \rightarrow \text{read}_B(\text{victim}) \rightarrow \text{CS}_B$

Assumiamo che A sia l'ultimo thread che scrive il campo victim:

$\text{write}_B(\text{victim} = B) \rightarrow \text{write}_A(\text{victim} = A)$

L'equazione sopra riportata implica che A rileva che il campo victim vale A nell'equazione 2.3.8.

Siccome A, ciò nonostante, entra nella sezione critica, deve aver osservato flag[B] falso, ottenendo:

$\text{write}_A(\text{victim} = A) \rightarrow \text{read}_A(\text{flag}[B] == \text{false})$

Le equazioni 2.3.9 e 2.3.11, insieme con la transitività di $\rightarrow$, implicano:

$\text{write}_B(\text{flag}[B] = \text{true}) \rightarrow \text{write}_B(\text{victim} = B) \rightarrow \text{write}_A(\text{victim} = A) \rightarrow \text{read}_A(\text{flag}[B] == \text{false})$

Significa che $\text{write}_B(\text{flag}[B] = \text{true}) \rightarrow \text{read}_A(\text{flag}[B] == \text{false})$.

Questa osservazione fornisce una contraddizione perché nessun altro scrive nel $\text{flag}[B]$ prima dell'esecuzione della sezione critica.

Lemma. Il Lock di Peterson è starvation-free.

Dimostrazione. Supponiamo per assurdo che A esegue sempre il metodo $\text{lock}()$. Deve essere eseguita l'istruzione $\text{while}()$, aspettando finché $\text{flag}[B]$ diventi falsa o victim sia settato a B.

Cosa sta facendo B mentre A non riesce ad andare avanti?

Forse B entra ed esce ripetutamente dalla sezione critica. Se è così, B setta victim a B appena rientra nella sezione critica. Una volta che victim è settato a B, non cambia e A deve eventualmente uscire dal metodo $\text{lock}()$, una contraddizione. Quindi deve essere che anche B è bloccato nella sua chiamata al metodo $\text{lock}()$, aspettando finché il valore di $\text{flag}[A]$ non diventi falso o victim sia settato ad A. Ma victim non può essere contemporaneamente A e B, una contraddizione.

Corollario. Il Lock di Peterson è deadlock-free. (Se è starvation free è anche deadlock free)

Correttezza. La proprietà starvation-freedom garantisce che ogni thread che chiama $\text{lock}()$ eventualmente entra nella sezione critica ma non garantisce per quanto tempo lo farà. Idealmente, se A chiama $\text{lock}()$ prima di B, allora A dovrebbe entrare nella sezione critica prima di B. Sfortunatamente non è possibile determinare quale thread chiama il $\text{lock}()$ per primo. Per garantire ciò, dividiamo il metodo $\text{lock}()$ in due sezioni di codice:

1. Una sezione doorway, in cui l'intervallo di esecuzione D_A consiste in un limitato numero di step;
2. Una sezione waiting, in cui l'intervallo di esecuzione W_A può impiegare un numero di step illimitato.

La necessità che la sezione doorway debba sempre finire in un limitato numero di step è un requisito molto forte. Chiameremo questo requisito bounded wait-free.

3.4 Sezione critica per N thread

3.4.1 L'Algoritmo del Fornaio di Lamport

Definizione. Un lock è first-come-first-served se, ogni volta che un thread A finisce la sua parte doorway prima che il thread B inizi la sua parte doorway, allora A non può essere superato da B.

$$\text{if } D_A^j \rightarrow D_B^k, \text{ then } CS_A^{jk} \rightarrow CS_B^k$$

Ogni thread prende un numero all'entrata e aspetta finché nessun thread con un numero più piccolo aspetti di entrare.

```
1 class Bakery implements Lock {
2     boolean[] flag;
3     Label[] label;
4     public Bakery (int n) {
5         flag = new boolean[n];
6         label = new Label[n];
7         for (int i = 0; i < n; i++) {
8             flag[i] = false; label[i] = 0;
9         }
10    }
11    public void lock() {
12        int i = ThreadID.get();
13        flag[i] = true;
14        label[i] = max(label[0], ..., label[n-1]) + 1;
15        while ((∃k != i)(flag[k] && (label[k],k) << (label[i],i))) {}
16    }
17    public void unlock() {
18        flag[ThreadID.get()] = false;
19    }
20 }
```

Nel lock del fornaio, $flag[A]$ è un flag booleano che indica se A vuole entrare nella sezione critica, e $label[A]$ è un intero che indica l'ordine relativo ai thread quando entrano nel panettiere, per ogni thread A. Ogni volta che un thread acquisisce un lock, genera un nuovo array $label[]$ in due step:

- Per primo, legge in qualsiasi ordine le label degli altri thread.
- Secondo, legge tutte le label degli altri thread uno dopo l'altro e genera una label più grande di quella massima letta.

Le Righe 13 e 14 rappresentano la parte doorway. Se due thread eseguono la loro doorway concorrentemente, possono leggere la stessa etichetta massima e scegliere la stessa label. Per rompere questa simmetria, l'algoritmo usa un ordine lessicografico (Riga 15) su una coppia di $label[]$ e id dei thread:

$$(label[i], i) << (label[j], j)$$

If and only if

$$label[i] < label[j] \text{ or } label[i] = label[j] \text{ and } i < j$$

Nella Riga 15 (sezione di waiting), un thread ripetutamente rilegge le etichette una dopo l'altra in un ordine arbitrario prima di determinare che nessun thread abbia una coppia label/id più piccola.

Siccome rilasciare un lock non resetta la $label[]$, è facile vedere che le label dei thread sono rigorosamente crescenti. Sia nella sezione doorway che waiting, i thread leggono le etichette

asincronamente e in ordine arbitrario, così che l'insieme delle etichette visto prima della scelta della nuova può non essere mai esistito in memoria. In tal caso l'algoritmo lavora bene.

Lemma. L'algoritmo del fornaio è esente da deadlock (Deadlock free).

Dimostrazione. Qualche thread aspetta che A ha l'unica coppia più piccola ($label[A], A$), e questo thread non aspetta mai un altro thread.

Lemma. L'algoritmo è first-come-first-served (FCFS).

Dimostrazione. Se la parte doorway di A precede quella di B, $D_A \rightarrow D_B$, allora l'etichetta di A è più piccola siccome:

$write_A(label[A]) \rightarrow read_B(label[A]) \rightarrow write_B(label[B]) \rightarrow read_B(flag[A])$

così B è bloccato finché $flag[A]$ è true.

Lemma. L'algoritmo soddisfa la mutua esclusione (Gode della ME).

Dimostrazione. Per assurdo siano A e B due thread concorrenti nella sezione critica. Prendiamo $labeling_A$ e $labeling_B$ che rappresentano l'ultima rispettiva sequenza di acquisizione della nuova etichetta di priorità prima di entrare nella sezione critica.

Supponiamo che $(label[A], A) << (label[B], B)$.

Quando B completa in modo corretto il test nella sua sezione waiting, deve aver letto che $flag[A]$ fosse falso oppure che $(label[B], B) << (label[A], A)$. Comunque, per un dato thread, i rispettivi valori di id e di $label[]$ sono sempre incrementati, così B deve aver dovuto vedere che $flag[A]$ era falsa. Questo significa che:

$labeling_B \rightarrow read_B(flag[A]) \rightarrow write_B(flag[A]) \rightarrow labeling_A$

che contraddice l'assunzione che $(label[A], A) << (label[B], B)$.

3.5 Bounded Timestamps

I timestamp sono limitati dalla dimensione dei contatori utilizzati (16, 32, 64 bit). Il limite, quindi, c'è e questo è un problema.

Le operazioni compiute dall'algoritmo del panettiere sono due:

- **Operazione 1:** Scan. Lettura dei timestamp degli altri;
- **Operazione 2:** Label. Assegnazione a sé stesso di un timestamp. Quindi legge e calcola il massimo di tutti i partecipanti, e poi si fa l'assegnazione;

Ecco un Esempio :

```
1 public interface Timestamp {
2     boolean compare(Timestamp);
3 }
4
5 public interface TimestampSystem {
6     public Timestamp[] scan();
7     public void label(Timestamp, int i);
8 }
```

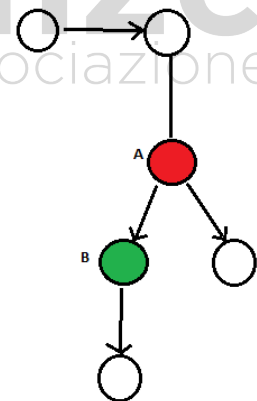
Quello che si vuole realizzare è un sistema che permette di creare un insieme di timestamp che siano limitati, cioè che non abbiano il problema di arrivare a saturazione del contatore. Esiste un sistema concorrente wait-free, abbastanza complesso e che è possibile dimostrare. Si fa la dimostrazione con la stessa idea di Timestamping sequenziale, dove però si assume che le operazioni di scan-and-label vengono fatte una dopo l'altra, senza interleaving, ovvero si sta dicendo che le operazioni non vengono fatte in maniera concorrente.

Si realizza un grafo di precedenza, cioè vengono rappresentati i timestamp attraverso un grafo. Il vantaggio è che il grafo ha un numero finito di nodi, dove avendo un numero finito di nodi si può avere memoria finita e quindi usare infinitamente il grafo per poter determinare la precedenza.

- Un arco $a \rightarrow b$ indica che a ha un timestamp "preso" dopo di b ;
- Un ordine di timestamp è *irriflessivo*: Non ci sono archi da a verso a , cioè il timestamp deve indicare una chiara precedenza;
- Un ordine di timestamp è *antisimmetrico*: Se $a \rightarrow b$ allora non c'è l'arco $b \rightarrow a$;
- Non viene richiesta la *transitività*: Se $a \rightarrow b$ e $b \rightarrow c$ non necessariamente implica $a \rightarrow c$, nel senso che non è automatico che se a ha preso il timestamp dopo b e b l'ha preso dopo c , questo significa che a prenda il timestamp dopo c . Può esserci per costruzione ma non è necessaria;
- L'assegnazione di un timestamp ad un thread corrisponde a piazzare un token del thread sul nodo del grafo.

Che cosa significa che un thread prende un timestamp?

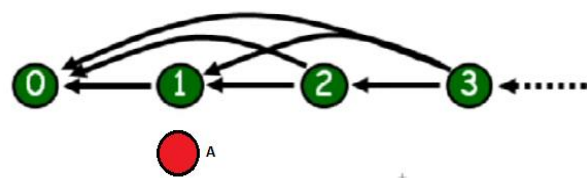
Essenzialmente significa che avendo il grafo, che è direzionato, ad un certo punto si ha un certo thread che piazza un suo token su un certo nodo. Questo token A (rosso) viene prima del thread B (verde) che aveva piazzato il suo token, perché c'è un arco che specifica la precedenza. In tal caso B viene prima di A.

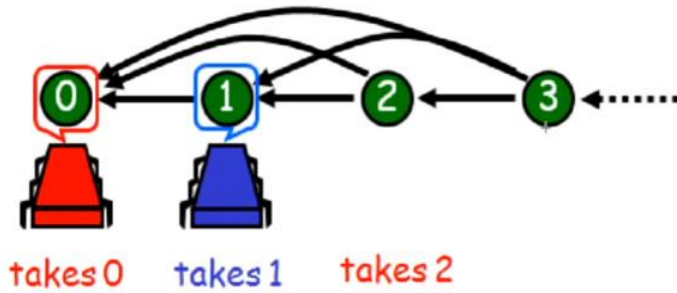


- **Scan**: Semplice localizzazione dei token degli altri;
- **Label**: Muove il proprio token su un nodo a tale che c'è un arco da a a tutti gli altri nodi con un token di un thread;

Un Esempio di Grafo di Precedenza:

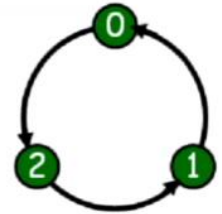
Il thread A mette il proprio token sul nodo 1. Se un thread vuole entrare, vede che c'è il thread A sul nodo 1 e si mette sul nodo 2 oppure 3. In tal caso non c'è nessuna ottimizzazione che viene effettuata sul grafo.





Quindi il nodo 0 prende 0, il nodo 1 prende 1 e il nodo 0 a tal punto, se il rosso entra dopo, può prendere il nodo successivo.

Il punto di tale meccanismo è come si fa a costruire tale grafo. L'idea della costruzione del grafo è una costruzione per induzione, cioè si usa questo grafo come punto di partenza: 0 viene prima di 1, 1 viene prima di 2 e 2 viene prima di 0. Viene prima significa che c'è l'arco da 1 verso 0.

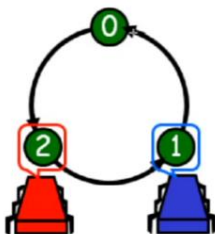
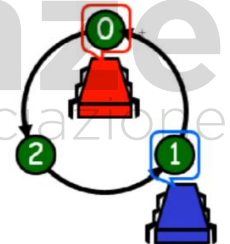


- **Invariante:** Se due thread con i loro token sono su nodi adiacenti, allora l'ordine viene indicato dall'arco. Se ad esempio avessimo il thread A sul nodo 0 e il thread B su 2, allora risulta che A viene dopo B mentre se avessimo il thread B sul nodo 1 risulta che A viene prima di B. L'arco quindi ci dice la precedenza;

Grafo T²: Nessun Nodo

La costruzione di questo grafo è fatta per il principio di induzione. Si parte dal grafo che permette di gestire due thread che entrano, fanno scan-and-label con un numero finito di memoria. In tale grafico si possono continuare a piazzare due thread senza usare un contatore che cresce all'infinito.

Supponiamo che il thread rosso prende il token 0. Ha fatto lo scan, ha visto che non c'era nessuno, prende il token 0 e si etichetta con il token 0. Arriva il thread blu, nella fase di scan ha verificato quali sono i nodi dove c'è un token, quindi il nodo 0. Deve piazzare il suo token su uno dei nodi che copre il nodo 0, cioè usa l'arco e si piazza in 1.



Nel momento in cui il thread rosso esce e prova a rientrare e il thread blu non è ancora entrato, il thread deve prendere una posizione che copre il nodo 1, quindi si mette in 2. Adesso sarà il thread blu ad entrare prima di quello rosso. In maniera circolare si riesce a gestire le entrate di questi thread, usando memoria finita.

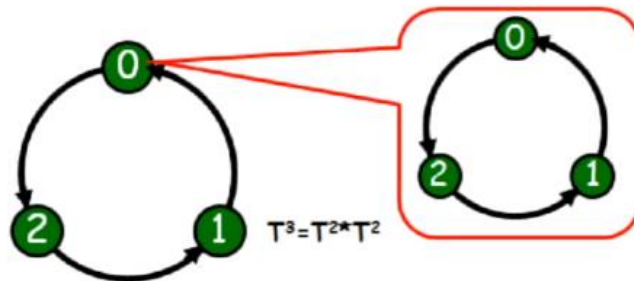
Ampliamo il Grafo

Volendo cercare di trovare un altro grafo che ci permetta di lavorare con 3, 4, 5 thread. Un ciclo a quattro nodi non basta. Quello che bisogna fare è usare uno strumento che usa la moltiplicazione tra grafi. La moltiplicazione tra grafi è uno strumento che prende due grafi e ne ottiene un terzo, che è il prodotto dei due.

In G l'insieme di nodi A domina B se ogni nodo di A ha archi diretti verso ogni nodo in B, cioè che se ogni nodo di A ha un arco verso tutti i nodi di B, in questo caso A domina B.

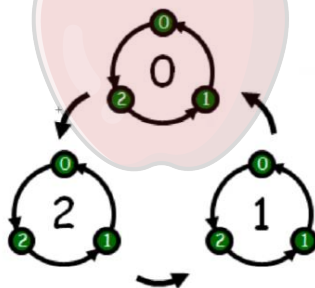
Dati due grafi G e H , il prodotto $G * H$ si ottiene:

- Sostituendo ogni nodo v di G con una copia H_v ;
- H_v domina H_u se v domina u in G ;
- T^k definito induttivamente:
 - T^1 è un nodo singolo;
 - T^2 è il ciclo di tre nodi;
 - $T^k = T^2 * T^{k-1}$, per $k > 2$



Per quanto riguarda la costruzione quindi si rimpiazza ogni vertice con una coppia del grafo.

T^3 è definito come $T^2 * T^2$, abbiamo che G è il nodo 0 del grafo a sinistra e H è il nodo 0 di quello a destra. Si sostituisce ogni nodo con un ciclo e quello che si ottiene è un grafo del genere:

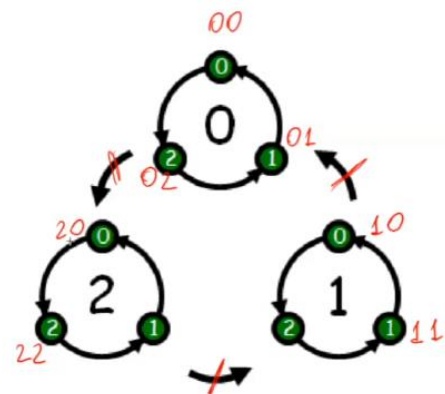


T^3 è definito come il grafo che ha 0, che corrisponde al nodo 0 di $T^2(G)$. il grafo 1 domina il grafo 0, cioè che ogni arco di ogni nodo del grafo 1 connette tutti i nodi del grafo 0 e così vale per gli altri grafi.

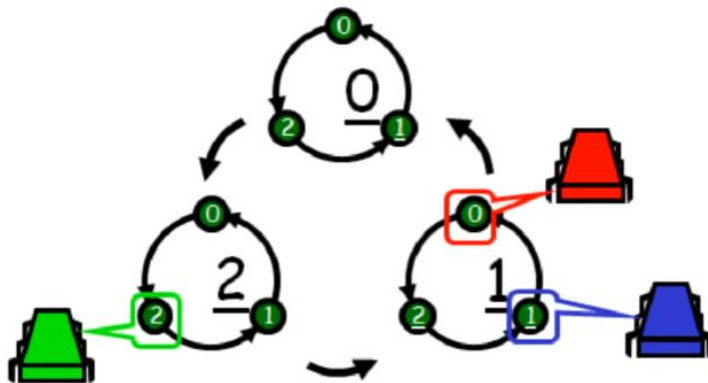
Come funziona?

Usiamo come notazione quella ternaria (00, 01, ..., 21, 22), cioè il primo indice ci dice qual è il grafo mentre il secondo ci dice qual è il nodo all'interno del grafo. Quando si deve piazzare un terzo thread, se i due altri thread sono su due grafi diversi, viene piazzato sul grafo dell'ultimo, nella posizione successiva.

Altrimenti, se tutti e due i thread presenti sono nello stesso grado G_i , si usa un nodo nel grafo che domina G_i .



Ecco l'esempio:



Il processore verde viene messo in tale grafico perché il rosso e blu si trovano sullo stesso grafo. A questo punto si stanno gestendo tre thread con una struttura dati che è finita, cioè l'accesso per infinite volte di tre thread viene gestito da memoria finita. È una cosa che tramuta all'infinito.

3.6 Un Lower Bound sulla memoria

3.6.1 L'Introduzione

La domanda che ci poniamo è:

Esiste un Lower Bound su memoria necessaria per riuscire a gestire la concorrenza su n thread, al di sotto della quale non si può andare? Quindi dato l'algoritmo del Panettiere che usa n locazioni per coordinare l'accesso alla mutua esclusione, esiste un algoritmo di Lock, basato su lettura e scrittura di memoria che usi meno locazioni?

La risposta è no. Anche su un solo thread può scrivere e gli altri possono solo leggere (Multiple Read Single Writer registers). Quello che si sta cercando di fare è trovare un Lower bound, dimostrare che per n thread sono necessarie almeno n locazioni di memoria per poter gestire la mutua esclusione.

Si lavora su una cosa molto indistinta, tutti i possibili algoritmi e su tutti questi indistinti algoritmi, vogliamo provare che tutti quelli basati sulla lettura e scrittura della memoria, che risolvono tutte le istanze del problema, devono necessariamente usare una certa quantità di risorse (tempo, spazio, accessi I/O, rete ecc..).

Il principio che useremo è che ogni informazione scritta in una locazione di memoria cancella ogni traccia presente (questa è l'unica informazione su cui possiamo basarci, solamente lettura e scrittura, se io scrivo qualcosa, quello che c'è scritto lì non c'è più). Questo è un principio importante perché se dico che la scrittura in una locazione di memoria cancella tutto quello che c'è e se la informazione che era lì era l'unica informazione su qualcosa di importante significa che da quel momento in poi nessuna decisione può essere presa sulla base di quella informazione che prima c'era lì. Quindi questa informazione che è stata cancellata, non è più disponibile.

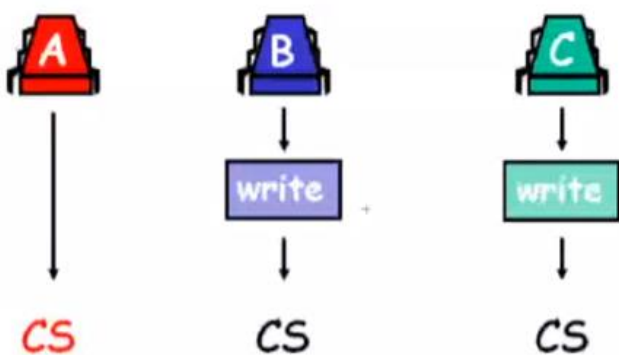
Non solo questo è vero, ma è vero che la cancellazione avviene (in memoria) senza che il thread che aveva scritto in precedenza ne sia informato, cioè se questa locazione è stata scritta da qualcun

altro, non è che c'è un evento che mi dice "attenzione, è stata modificata" e quindi io posso sapere che è stata modificata, che non è più quella di prima. Questo non è un meccanismo di base delle nostre architetture. Quello che vogliamo dimostrare, è che non esiste nessun algoritmo che usa meno di n locazioni per regolare l'accesso alla mutua esclusione a n thread, il Lower Bound è proprio su n , non ordine di n , quindi n locazioni sono necessarie ad arbitrare n thread.

Si assuma per contraddizione che esiste, e si mostra che una esecuzione particolare viola la proprietà di deadlock; quindi, si assume che questo algoritmo esiste, cioè esiste un algoritmo che usa meno di n locazioni per arbitrare qualsiasi accesso a mutua esclusione a n thread e si mostra che questo algoritmo in una certa maniera riusciamo a portarlo in deadlock. Non conosciamo l'algoritmo, sappiamo solamente che è un algoritmo che usa meno di n locazioni, assumeremo che siano $n-1$, e quello che facciamo è fare in modo di guidare tale algoritmo che non conosciamo, ripeto non c'è l'algoritmo `if for i` che va da 0 a $n-1$ ecc. Non sappiamo niente di questo algoritmo, sappiamo solamente che legge e scrive, e non più di $n-1$ locazioni, quello che cercheremo di fare è di guidare questo algoritmo, come facciamo a fare questo? Noi esibiamo un interleaving di thread che porta alla contraddizione, perché la contraddizione è che esiste un algoritmo che regola la mutua esclusione di n thread con meno di n locazioni. Alla fine, noi dimostriamo che non è vero che regolava l'accesso in mutua esclusione perché genera deadlock.

Nella dimostrazione, si ricopre il ruolo dello scheduler avversario. Quello che noi faremo, in qualità di oracolo durante la dimostrazione per assurdo (durante la dimostrazione per assurdo è possibile gestire in maniera totalmente libera l'esecuzione dell'algoritmo), sarà far partire un thread, far partire un altro thread, l'algoritmo appena succede una certa cosa altra cosa ferma il primo e fa partire il secondo, così appena succede questa altra cosa ferma il secondo e fa ripartire il primo, questo interleaving di thread, quindi lo scheduler ci permetterà di eseguire il deadlock.

Il Principio: Un Esempio



Il principio fondamentale: abbiamo 3 thread e 2 locazioni, quindi $n-1$. L'idea è che voglio far vedere che, per assurdo, ogni algoritmo che deve gestire 3 thread in mutua esclusione deve richiedere almeno 3 locazioni, se per assurdo ne esistesse uno che usa solo 2 locazioni allora io sono in grado di portarlo all'errore, quindi far vedere che non è un algoritmo di ME. Quello che faremo è essenzialmente usare i 2 registri, dove vengono scritti i valori e chiaramente

useremo il termine critical section (Critical Section), questi qua sono i termini. Usiamo alcuni nomi, infatti parleremo di stato dell'algoritmo, cioè in che condizione è in questo momento l'algoritmo, principalmente è la memoria che sta gestendo l'algoritmo, perché non possiamo sapere in che linea di codice è, perché l'algoritmo non lo conosciamo. Stiamo lavorando con tutti gli algoritmi passati e futuri, quindi è una dimostrazione che prescinde dalla struttura. Uno stato viene detto inconsistente se qualche thread è in critical section ma la situazione lock (stato del lock) è compatibile con uno

stato globale in cui nessun thread è in critical section (oppure vuole entrare). Uno stato inconsistente è un momento in cui può succedere il disastro, cioè essenzialmente problemi di mutua esclusione o deadlock, e questo è il problema critico. Il nostro obiettivo nella nostra dimostrazione sarà trovare uno stato inconsistente. Nessun algoritmo Deadlock-free può entrare in uno stato inconsistente: se entrasse in uno stato s potremmo forzare un deadlock, non potendo riconoscere se un thread è o no dentro, perché in quel momento non sappiamo se c'è qualcuno dentro o meno e allora o violiamo la mutua esclusione o violiamo il deadlock, perché la mutua esclusione significa che entriamo, mentre se non entriamo e dentro non c'era nessuno vuol dire che siamo in deadlock, non stiamo usando risorse.

Covering State: Uno stato in cui un thread sta per scrivere (istruzione prima di quella della scrittura) una locazione locale del Lock, ma lo stato del Lock fa apparire che:

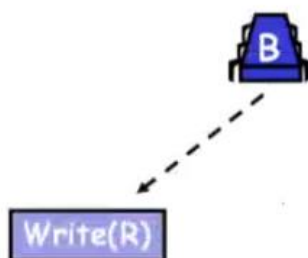
- La critical section sia (ancora) vuota;
- Nessun thread stia tentando di entrare;

Il Covering State è il momento in cui stiamo per poter scrivere qualcosa, ma quello che appare all'esterno, agli altri thread, è assolutamente indistinguibile dalla situazione di quiete, cioè la critical section è vuota e nessuno vuole entrare.

L'Intuizione per 2 Thread

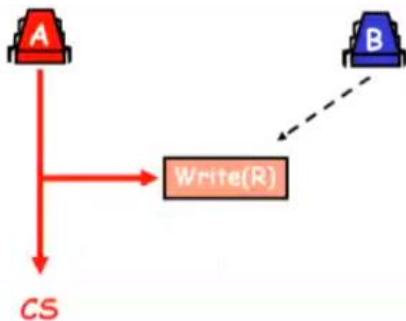


Abbiamo un algoritmo che usa una sola locazione, (Il registro R in figura), e vogliamo arbitrare due thread A e B che entrino nella critical section, quindi che usano una sola variabile per determinare l'accesso in critical section. Per assurdo supponiamo che l'algoritmo di mutua esclusione, che usi una sola variabile, è in grado di regolare l'accesso a due thread senza creare deadlock e che sia deadlock-free, quindi supponiamo che esista questo algoritmo (almeno uno di essi ha successo nell'accedere).



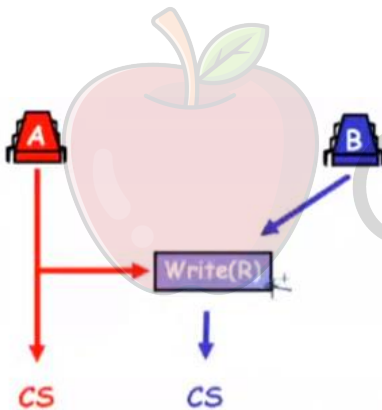
Facciamo in modo di guidare B appena entra in un covering state, cioè siamo l'oracolo (lo scheduler, quindi siamo in grado di poter decidere se questo thread si ferma o riparte, quindi abbiamo questo potere ma non abbiamo il potere di poter vedere cosa fa l'algoritmo cioè quali operazioni fa, ma possiamo solamente vedere quando scrive o quando legge, quindi vedere le operazioni che va a fare sulla memoria, non sappiamo in che maniera le fa, in che maniera sta eseguendo). Fermiamo B appena entra in un covering state, cioè B sta lì per scrivere ma il lock indica che la critical section è vuota e non

c'è nessuna maniera di sapere che nessuno voglia entrare. Questa cosa è critica, perché per sapere che qualcuno vuole entrare avremmo dovuto avere altre locazioni di memoria e questo è il punto, non solo la critical section indica che è vuota, ma non si conosce se l'altro stia in Covering state, perché non è che si aveva una variabile che B ha potuto scrivere. Non abbiamo a disposizione questa struttura.



Ora facciamo eseguire il thread A fino a quando A entra in critical section. Quando entra in critical section può avere o non avere scritto sul registro, ma non sappiamo se avesse una sola variabile in memoria, quindi o ha fatto questo o non ha fatto niente e quindi se non ha fatto niente o se ha fatto qualcosa non è che ci interessa. Quello che ci interessa è che potenzialmente può avere anche scritto qualche cosa, però a questo punto A sta in critical section, ha scritto qualcosa qua sopra << indica Write(R) >> però

attenzione B stava in Covering state, cioè stava sul momento di scrivere qualcosa, quindi sta per eseguire una istruzione e non può leggere qualsiasi cosa che sta scritta in Write(R). Non ha la possibilità di leggerla, è stato bloccato prima dell'esecuzione dell'istruzione e quindi ha un certo valore che vuole andare a scrivere in questa locazione di memoria <<Write(R)>>.



A questo punto facciamo ripartire B che era stato fermato un istante prima di scrivere, B scrive (cancellando quello che aveva potuto scrivere A) qualcosa e scrive "sono entrato" ed entra in critical section. La contraddizione è che questo algoritmo non è un algoritmo di mutua esclusione, cioè questo algoritmo è un algoritmo che non gestisce la mutua esclusione di 2 thread, (mutua esclusione violata!). L'idea è che, fermiamo B un attimo prima di quando sta per scrivere, facciamo partire A, facciamo in

modo di farlo entrare, eventualmente può avere scritto qualcosa, e a questo punto facciamo ripartire B, perché B stava per scrivere, scrive ed entra, violando la mutua esclusione.

3.6.2 La Dimostrazione per 3 Thread

Teorema: Ogni algoritmo di Lock che, utilizzando letture/scritture in memoria, risolva la mutua esclusione deadlock-free per 3 thread deve usare almeno 3 locazioni distinte di memoria.

Per assurdo si assuma che esiste tale algoritmo con due locazioni e lo si schedula in maniera da arrivare ad una contraddizione. Schedulare significa che si fa in modo di esibire un interleaving di thread che porta alla contraddizione del fatto che questo algoritmo risolve la mutua esclusione deadlock-free.

Osservazione: in uno stato s , se un thread viene eseguito da solo, deve scrivere almeno in una locazione prima di entrare in critical section, altrimenti sarebbe inconsistente.

L'osservazione è che la memoria che abbiamo a disposizione non può essere ignorata, perché se fosse ignorata, cioè se qualcuno entra e non scrive sulla memoria, è chiaro che questo porta lo stato ad essere inconsistente, perché significa che nessuno è in grado di riconoscere se dentro c'è qualcuno oppure no e questo significa che l'algoritmo o viola la mutua esclusione o viola il deadlock, viola la mutua esclusione perché potrebbe essere un algoritmo che ignora, passa avanti ed entra e dentro c'è qualcuno oppure un algoritmo che assume che dentro c'è qualcuno in critical section e invece non c'è nessuno e quindi causa un deadlock. Si deve necessariamente scrivere, e questo è uno dei perché che mette in crisi, quindi se non scrivesse avremmo stati inconsistenti ed uno stato inconsistente ci prova la contraddizione.

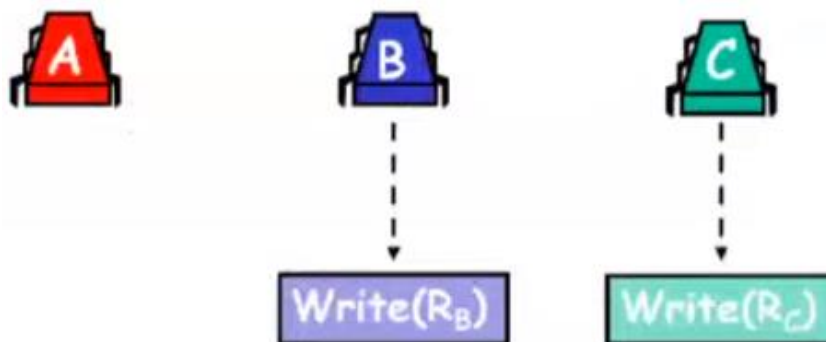
Se i registri sono Single Writer, ovviamente da questo deriva il teorema → Multiple Writer

Se ogni thread può scrivere solo in un registro significa che ovviamente dobbiamo andare da n thread a n registri, quindi 3 thread a 3 registri, e quindi dobbiamo assumere che tutti i thread possono scrivere sui registri.

La struttura della dimostrazione: si supponga di avere uno stato di covering, da parte di due thread di due locazioni distinte

A questo punto facciamo delle operazioni. Supponiamo di aver trovato uno stato di covering di 2 thread su due locazioni (lo stato di covering è quello in cui i thread stanno per scrivere sulle locazioni ma niente indica che loro stanno/hanno la volontà di farlo e niente indica che la critical section è occupata).

Dimostrazione:

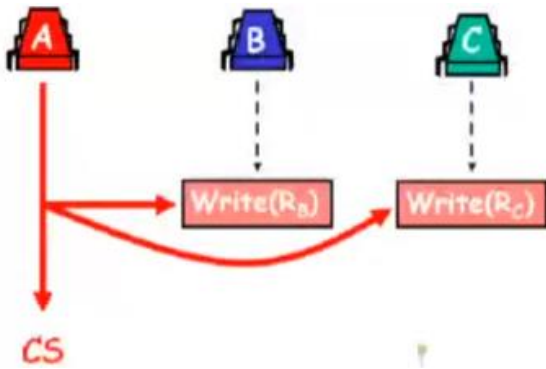


Partiamo da uno stato s in cui c'è un cover, quindi abbiamo B che sta per scrivere qualcosa nel registro $\text{Write}(R_B)$ e abbiamo C che sta per scrivere qualcosa in nel registro $\text{Write}(R_C)$. Se riusciamo a manovrare in questo stato di cover, che cosa possiamo fare?

Entrambi i thread ricoprono le due uniche locazioni, qui è cruciale che locazioni siano due perché il fatto che non ci siano altre locazioni di memoria impedisce in questo stato di covering a B di poter dire: ho scritto prima che ero interessato (quindi queste cose non ci sono), sono solo queste due. Ed il lock indica che la critical section è vuota e che nessuno vuole entrare in critical section, questo è

indicato dal fatto che non ci sta altra memoria. A questo punto si fermano i thread B e C in questo momento, quindi abbiamo questo stato di cover.

A questo punto cosa facciamo?



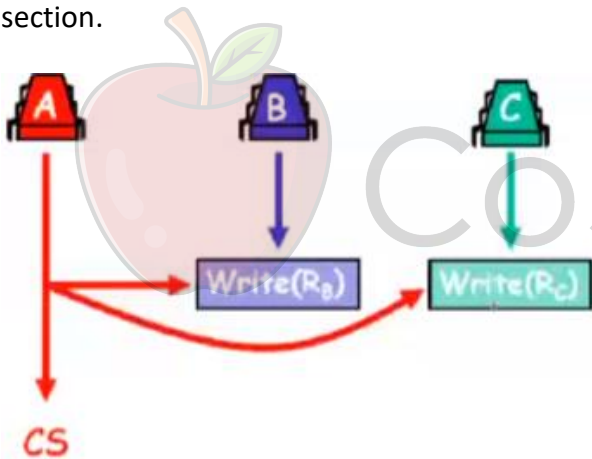
A questo punto si fa entrare in critical section A che scrive una (o entrambe) le locazioni, questo è un bel perché che tipicamente chiede Scarano all'orale.

Perché scrivere entrambe le locazioni?

Per l'osservazione che avevamo fatto prima, ossia: In uno stato s , se un thread viene eseguito da solo, deve scrivere almeno in una locazione prima di entrare in critical section, altrimenti sarebbe inconsistente.

L'algoritmo non può permettere ad un thread di entrare senza scrivere in una locazione.

Ora cosa significa che può scriverne uno o entrambe, noi non lo sappiamo, però almeno una la scrive (nell'esempio si fa vedere che se ne scrivono due) ma almeno una la scrive ed entra in critical section.



A questo punto, facciamo ripartire B e C, che stavano in un covering, che cancellano le tracce della entrata in critical section di A ed entrano in critical section. A questo punto uno dei 2 thread B o C, avendo cancellato le tracce può entrare in critical section dove già c'è A e questa è una contraddizione. Perché vuol dire che l'algoritmo non era di mutua esclusione.

Contraddizione. Questo è un concetto che deve essere chiaro, la contraddizione può uscire in due maniere, non assicura la mutua esclusione oppure non è deadlock-free. Perché noi stiamo assumendo entrambe le cose.

Il Lower Bound per tre è dimostrato, a patto che si è in grado di dimostrare come manovrare i thread in un covering state. Il covering state di prima era facile da trovare per due thread (ti fermo quando stai per scrivere), sei un solo thread, uno dei thread che scrive sull'unica locazione e ti fermo, qua i thread sono 2, come funziona:

Manovriamo in uno stato di covering

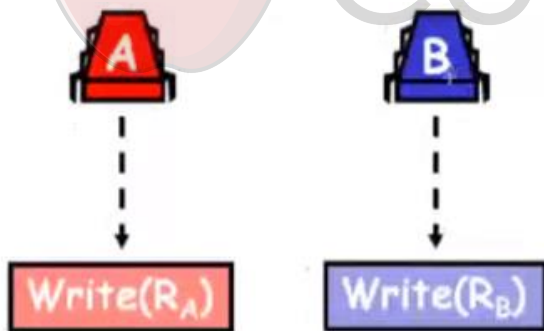
Facciamo eseguire in modo che B entri in critical section 3 volte, (il potere dello scheduler). Quindi B viene eseguito, entra in critical section, esce, poi viene eseguito, entra in critical section, esce, poi viene eseguito, entra in critical section ed esce. Chiaramente prima di entrare in critical section deve scrivere (il punto chiave) qualcosa, cioè deve scrivere qualcosa su una locazione di memoria, prima della critical section, perché altrimenti potremmo avere uno stato inconsistente, cioè in qualche maniera deve lasciare il segno che è in critical section, deve esibire verso l'esterno questa cosa.

Dovrà scrivere due volte la stessa locazione (sono solo due), sia essa R_B , nell'esempio precedente: Quindi B viene eseguito (Stato 1), scrive qualcosa nel registro R_1 , entra in critical section, esce, poi viene eseguito (Stato 2), scrive qualcosa nel registro R_2 , entra in critical section, esce, poi viene eseguito (Stato 3), scrive qualcosa nel registro R ? << deve essere per forza o R_1 oppure R_2 >>, entra in critical section ed esce. Assumiamo che sia R_1 e che chiamiamo quindi R_B , quindi l'idea è che R_B è la locazione che è stata scritta due volte dal thread B, adesso fermiamo tutto e poi ripartiamo:

Facciamo eseguire fino a quando B vuole scrivere R_B per la prima volta, quindi stiamo dicendo che, tornando all'esempio precedente, ci fermiamo al primo stop (appena prima di scrivere nel registro R_1), adesso: A entrerebbe in critical section, visto che non vede niente di B, ricordiamo che questo è un covering state e che non ci sono altre informazioni.

Ora A deve scrivere l'altro registro R_A per entrare in critical section. Se A scrivesse su R_B ed entrasse in critical section, a questo punto B non distinguerebbe che A è entrato (stato inconsistente), poiché A deve scrivere l'altro registro, facciamo eseguire fino a quando A vuole scrivere R_A .

La Situazione

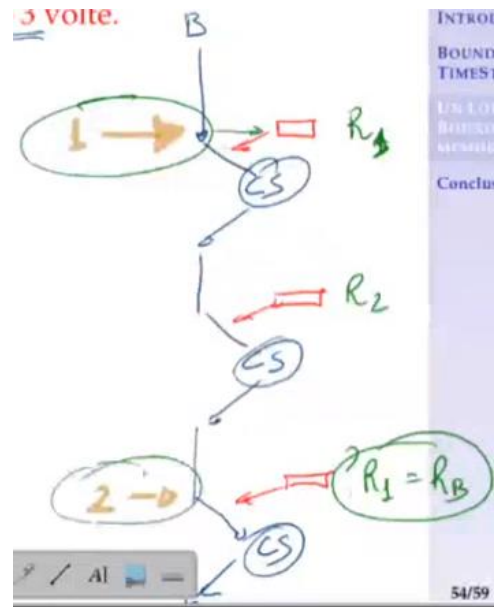


B si trova nello stato 1, tornando all'esempio precedente, vuole scrivere R_B , perché $R_1 = R_B$, poi abbiamo A che è stato fermato prima di scrivere in R_A , però attenzione: Questa non è ancora una cover. A potrebbe aver scritto qualcosa in R_B per indicare a C. Il terzo thread, ricordiamo cosa vogliamo fare, i due thread devono essere bloccati nel momento in cui stanno per scrivere, ma niente

deve indicare che loro stanno entrando, per B abbiamo detto che è stato fatto perché quello che ha scritto A ha sovrascritto qualsiasi cosa. B poteva aver scritto prima, se B prima dello Stato 1 ha scritto qualcosa in A è stato sovrascritto da A qui $Write(R_A)$, però A potrebbe dopo che B è stato bloccato aver scritto qualcosa in $Write(R_B)$, indicando che sono io che sto cercando di entrare e sotto in qualche maniera qualche valore intelligente che indichi questo interesse, quindi questo non è ancora un covering state che vuole entrare in critical section.

Com'è che diventa un Covering state?

A questo punto, facciamo ripartire B, scrive R_B (facendo arrivare alla seconda scrittura <<Stato 2>> che permette di cancellare quello che A aveva scritto, cioè quello che stiamo dicendo è che, tornando all'esempio di prima, l'esecuzione Stato 1 poi è seguito da Stato 2, cioè dal momento in cui sta per scrivere di nuovo però ha cancellato << Quando esegue lo Stato 1, cancella quello che aveva scritto A eventualmente e poi si ripresenta a fare il covering state allo Stato 2. Questo è quello che succede quando diciamo *scrive R_B* , è stato cancellato qualsiasi cosa abbia scritto A ed è pronto per scrivere in R_B) e ritorna pronto per scrivere in R_B . Questo è un covering state, ed è un covering state corretto, niente indica all'esterno che qualcosa sta per cambiare ma noi abbiamo due thread che stanno cercando di entrare e il covering state è quello da cui partiamo per la dimostrazione della parte successiva.



Esempio Grafico Stato 1 , 2 ecc

Un Riassunto del Problema

1. Si supponga che ci sia uno stato di Covering da 2 thread

- Si fa entrare in critical section il terzo thread, le cui tracce vengono cancellate appena i due thread vengono fatti ripartire;
- Uno dei due thread riesce ad entrare;
- Niente mutua esclusione! (Si viola la condizione di mutua esclusione);

Tutte le altre scelte portano o ad annullare la mutua esclusione (quindi trovare la contraddizione, perché l'algoritmo non è di mutua esclusione) oppure a negare il fatto che è deadlock-free.

2. Per ottenere uno stato di Covering:

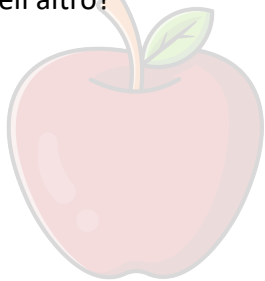
- Manovriamo il thread B a voler scrivere una locazione (che sovrascriverà dopo, la famosa esecuzione di 3 volte, l'entrata di 3 volte all'interno della critical section);
- Manovriamo A, a voler scrivere l'altra locazione;
- Facciamo eseguire B in modo da cancellare ogni traccia di A nel registro "non suo" ;
- → stato di covering (questo stato di covering ci permette di completare 1.);

3. Il covering può essere raggiunto induttivamente per k registri. Quando:

$k = N - 1$ si ottiene il teorema generale (che non facciamo, però ha la stessa struttura per passare da 2 a 3, quindi induttivamente possiamo dimostrare per k, quindi supponiamo che sia vero per k e facciamo vedere che per $k = N - 1$ si raggiunge il teorema generale, questo è il punto cruciale).

Domande di Riepilogo

- Cosa è un ordine totale? e un ordine parziale?
- Perché è importante che gli eventi (nella nostra definizione) esibisca un ordine totale?
- Perché gli intervalli di tempo (come definiti) definiscono un ordine parziale?
- Cosa è un thread well-formed per la mutua esclusione?
- Definire e descrivere le conseguenze di ciascuna delle seguenti proprietà di un lock: mutua esclusione, deadlock-free, starvation-free, first-come-first-served
- Perché la starvation-freedom implica la deadlock-freedom?
- Perché si rende necessario introdurre la proprietà first-come-first-served?
- Descrivere (e criticare) l'algoritmo LockOne
- Dimostrare la proprietà di Mutua Esclusione per il lock definito da LockOne
- Perché LockOne va in deadlock se i due thread sono interfogliati? Esibire la sequenza di interleave che causa il deadlock
- Descrivere (e criticare) l'algoritmo LockTwo
- Dimostrare la proprietà di Mutua Esclusione per il lock definito da LockTwo
- Perché LockTwo va in deadlock se i due thread sono eseguiti uno completamente prima dell'altro?



CoScienze
Associazione

4. Oggetti Concorrenti

4.1 Introduzione

In che maniera vengono trattati gli oggetti concorrenti e come descriviamo il loro comportamento? Tra le proprietà che interessano abbiamo:

- **Safety:** É la proprietà che assicura che alcune condizioni negative non si verifichino mai, molto più difficile da dimostrare in quanto bisogna provare tutti i vari stati in cui un thread può trovarsi;
- **Liveness:** É la proprietà che assicura che una particolare condizione positiva si presenterà, cioè che il programma farà progressi verso una soluzione;

Se ci si focalizza solo sulla Safety, l'oggetto Safe è un oggetto che per definizione non fa niente, cioè non entra in nessuna sezione critica, non fa nessun progresso e quindi non collabora con gli altri. Questi due diversi tipi di comportamento vengono tradotti nelle condizioni di correttezza e progresso.

- La correttezza è legata in qualche maniera all'equivalenza al comportamento sequenziale.
- Le condizioni di progresso sono definite dalle implementazioni dei metodi (bloccanti/non-bloccanti).

Quello che si fa è verificare le condizioni di correttezza e le condizioni di progresso.

4.1.1 Il Singleton

Noto design pattern della programmazione ad oggetti, il Singleton ha lo scopo di garantire che per una determinata classe venga creata una e una sola istanza, e di fornire un punto di accesso globale a tale istanza. L'implementazione più semplice prevede che la classe singleton abbia un unico costruttore privato, in modo da impedire l'istanziatura diretta della classe. La classe fornisce inoltre un metodo "getter" statico che restituisce un'istanza della classe (sempre la stessa), creandola preventivamente o alla prima chiamata del metodo, e memorizzandone il riferimento in un attributo privato anch'esso statico. Il Singleton viene costruito in maniera tale da esibire la *Lazy Allocation*, cioè si fa in modo di allocarlo solo quando serve la prima volta.

```
1  class Singleton {
2      private static Singleton instance;
3      private Singleton() {
4          //...
5      }
6
7      public static synchronized Singleton getInstance() {
8          if(instance == null) {
9              instance = new Singleton();
10         }
11         return instance;
12     }
```


Dalla classe `Singleton` è presente un metodo `getInstance()`, una variabile di classe `instance`, un costruttore privato e il metodo `static`, che restituisce un singleton. Se la `instance` è `null` (Riga 9), si crea l'istanza e a tal punto la si restituisce. A partire dalla prima volta che l'`if` viene eseguito, esso non viene più eseguito e restituisce così l'istanza creata. Ci possono essere problemi perché più thread possono eseguire contemporaneamente tale parte di codice per la prima volta, quando ancora non è stato creato il singleton. Questo può creare problemi perché più thread possono creare più singleton. Una possibile soluzione è quella di sincronizzare il metodo `getInstance()` (Riga 7). In ogni momento un solo thread può stare in esecuzione di tale metodo. Il problema è chiaramente la sincronizzazione, perché serve solo la prima volta e mai più dopo, si sta inutilmente prendendo delle istruzioni rendendole eseguibili in maniera sequenziale, ovvero aumentando la porzione sequenziale del codice a scapito della legge di Amdahl.

Una delle soluzioni che veniva proposta al problema era la seguente:

```
1 class Singleton {
2     private static Singleton instance;
3     private Singleton() {
4         //...
5     }
6
7     public static synchronized Singleton getInstance() {
8         if(instance == null) {
9             synchronized (Singleton.class){
10                 instance = new Singleton();
11             }
12         }
13         return instance;
14     }
```

Tale soluzione non funziona perché nel momento in cui ci sono più thread, che sono in Riga 8, una volta che hanno già verificato che `instance` è `null` entrano tutti e si mettono in attesa sul lock. Solo uno dei thread entra, crea l'istanza, esce rilasciando il lock, ma l'altro thread entra e ricrea una nuova istanza.

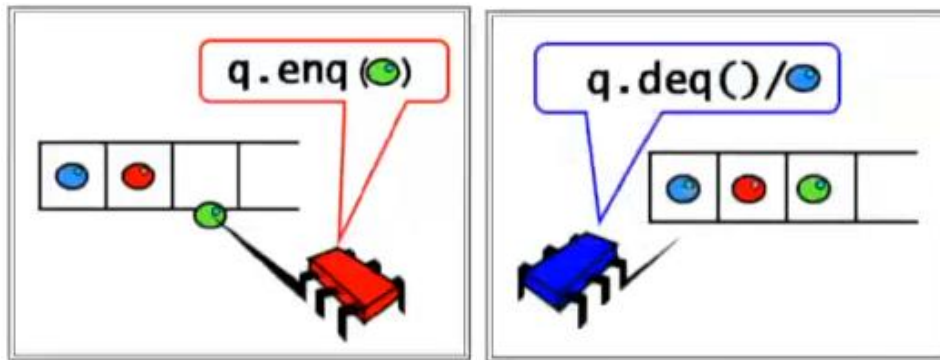
In letteratura era presente il Double-Checked Locking:

```
1 public static Singleton getInstance() {
2     if(instance == null) {
3         synchronized (Singleton.class){
4             if(instance == null)
5                 instance = new Singleton();
6             }
7     }
8     return instance;
9 }
```

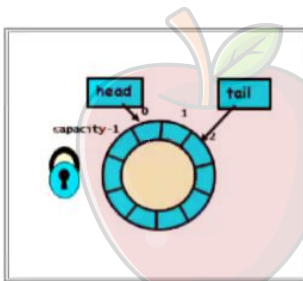
Esso prevede l'entrata in critical section del thread, viene fatto un nuovo controllo (Riga 4) in maniera tale da essere sicuri che la condizione non sia cambiata mentre si era in waiting. Questa soluzione non è garantito che funzioni rispetto al modello di memoria che viene utilizzato.

4.1.2 Concorrenza e Correttezza

Parlando di un semplice algoritmo di enqueue e dequeue all'interno di una coda:



Abbiamo il thread rosso che fa la enqueue dell'oggetto verde, mentre il thread blu che fa la dequeue gli viene restituito il primo oggetto nella end della coda. Tipica implementazione è quella della coda circolare, in cui abbiamo un array di dimensione capacity, dove l'aritmetica degli indici viene condotta con il modulo di capacity.



Ci sono due puntatori che non sono altro che degli interi che contengono la posizione della testa e della coda. La testa è la posizione dove c'è il primo elemento presente, mentre la coda indica dove potrebbe essere inserito un nuovo elemento, inoltre la coda vuota viene denotata da valore di $head = tail$. La head punta alla stessa posizione in cui tail farebbe inserire il primo elemento.

Coda Circolare con Lock

```
1 class LockBasedQueue<T> {
2     int head, tail;
3     T[] items;
4     Lock lock;
5     public LockBasedQueue(int capacity) {
6         head=0;tail=0;
7         lock = new ReentrantLock();
8         items = (T[])new Object[capacity];
9     }
10    public void enq(T x) throws FullException {
11        lock.lock();
12        try {
13            if (tail - head == items.length)
14                throw new FullException();
15            items[tail % items.length] = x;
16            tail++;
17        } finally {
18            lock.unlock();
19        }
20    }
21    public T deq() throws EmptyException {
22        lock.lock();
```

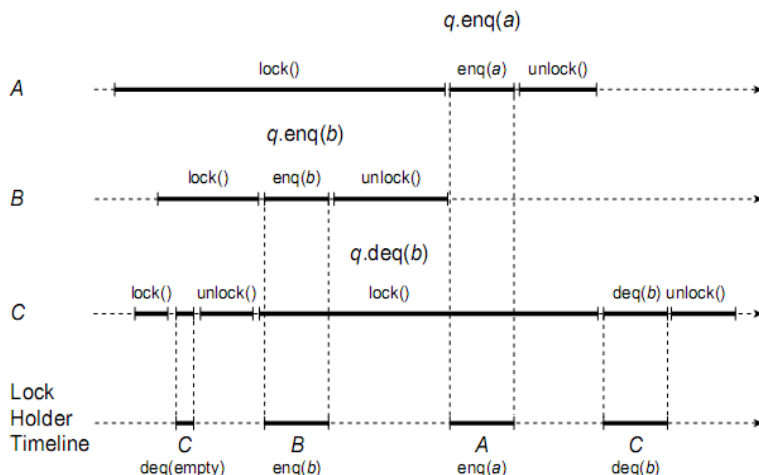
```

23         try {
24             if (tail == head)
25                 throw new EmptyException();
26             T x = items[head % items.length];
27             head++;
28             return x;
29         } finally {
30             lock.unlock();
31         }
32     }
33 }

```

Abbiamo una coda tipizzata a seconda del T e ci sono due interi `head` e `tail`, che mantengono i valori. Il lock permetterà l'ingresso in mutua esclusione nelle sezioni dove c'è necessità di eseguire in modalità isolata le istruzioni. Il costruttore prende la capacity come dimensione, setta `head` e `tail` a 0, crea un lock di tipo `ReentrantLock()` che permette di ricreare/accedere a un lock di cui si è già acquisito all'ingresso, senza creare deadlock. Per prima cosa viene fatto l'accesso in critical section in maniera isolata e dopo l'esecuzione del metodo `lock()` della variabile `lock` (Riga 11), che è bloccante. Se questo ritorna al metodo chiamante, cioè `enqueue`, allora significa che è stato acquisito il lock. A tal punto si possono fare le operazioni sulle variabili. Nel caso in cui `tail-head` è uguale alla lunghezza dell'array, si lancia un'eccezione che segnala che la coda è piena e non si può più fare `enqueue`. Se tale eccezione non viene segnalata allora si fa l'inserimento (Riga 15), si incrementa `tail` e si esce dalla critical section. Tali operazioni sono all'interno di un blocco `try finally`, che permette di fare l'`unlock` (Riga 18) alla fine dell'esecuzione del blocco in qualsiasi maniera esso venga eseguito.

La `dequeue` è molto simile ad `enqueue`. Viene utilizzato il lock per l'accesso in critical section. In tal caso viene controllato se `tail` è uguale ad `head` (Riga 24) viene lanciata un'eccezione, altrimenti si può prelevare l'elemento `x` puntato dalla `head`, si incrementa la `head` e restituisce la `x`. Anche in questo caso l'`unlock` viene chiamato prima di qualsiasi uscita da tale blocco.



Ci sono tre thread A, B, C.

Ognuno di essi esegue un lock, un'operazione che può essere una `enqueue` o `dequeue` ed esegue un `unlock`.

Le operazioni da fare sono le seguenti:

- A vuole inserire l'elemento a;
- B vuole fare `enqueue` di b;
- C vuole fare due `dequeue`;

Il fatto che del lock si ignora la maniera in cui si sovrappongono le esecuzioni è ovvio, perché A prova ad acquisire il lock ma non è detto che lo acquisisce subito. B cerca di acquisirlo dopo e lo acquisisce prima di A, quindi non è un First-Come-First-Served. La prima operazione che prova a fare C è la dequeue, cioè C acquisisce il lock e prova a fare la dequeue restituendo un'eccezione, cioè `EmptyException()`, e fa l'`unlock`. Intanto sia A che B si sono accodati sul lock, per qualche motivo il lock fornisce accesso a B che è in grado di fare la enqueue di B. In questo momento si ha che la `head` punta alla locazione dove è stato inserito B e la enqueue di B avviene successivamente. Viene fatto l'`unlock` e a questo punto l'accesso viene fornito ad A, che è in grado di fare l'enqueue di A. L'ingresso dei tre thread non avviene in First-Come-First-Served.

Coda wait-free Senza Lock

```
1 class WaitFreeQueue<T> {
2     volatile int head=0,tail=0;
3     T[] items;
4     public WaitFreeQueue(int capacity) {
5         items = (T[])new Object[capacity];
6         head=0;tail=0;
7     }
8     public void enq(T x) throws FullException {
9         if (tail - head == items.length)
10            throw new FullException();
11         items[tail % items.length] = x;
12         tail++;
13     }
14     public T deq() throws EmptyException {
15         if (tail - head == 0)
16            throw new EmptyException();
17         T x = items[head % items.length];
18         head++;
19         return x;
20     }
21 }
```

Una coda wait-free significa che non ha lock, non c'è nessuna attesa di nessun tipo. La cosa interessante è che questa coda, che ignora la concorrenza, è un'implementazione che funziona correttamente quando abbiamo un singolo enqueueer e un singolo dequeueer, cioè questa coda non sta gestendo multiple enqueueer e dequeueer, ma gestisce la concorrenza di un singolo enqueueer e dequeueer. Tale coda è un perfetto buffer, perché si può avere un processo che è un enqueueer che accoda elementi, e un processo dequeueer che estrae elementi. Entrambi lavorano in maniera perfettamente concorrente, in assenza di lock. L'algoritmo ha come caratteristica il fatto che gli accessi sono sequenziali, separatamente a `head` e `tail`. La `head` viene letta esclusivamente dalle enqueuee e lo stesso capita alla `tail`, mentre la dequeue legge.

Lo specificatore di accesso *volatile* è fondamentale, perché fa in modo che qualsiasi modifica venga resa immediatamente visibile agli altri thread. In tale situazione è critico il funzionamento, per il fatto che le variabili sono volatili, perché una scrive e l'altro legge. In maniera incrociata l'enqueueer scrive su `tail` e legge `head`, mentre il dequeueer scrive su `head` e legge `tail`. Questa coda risulta interessante perché il meccanismo di avere un buffer con un processo che produce a una certa

velocità e un altro processo consuma a un'altra velocità, se il processo è uno quello che produce e uno quello che consuma, allora non c'è bisogno di lock.

La legge di Amdahl suggerisce che, per avere efficienza, dovremmo rinunciare alle critical section o renderle almeno più piccole possibili. Possiamo verificare la correttezza facendo un mapping con l'esecuzione sequenziale, per poi analizzarla.

4.1.3 Oggetti Sequenziali

Un oggetto è un contenitore per dati, con metodi per manipolare l'oggetto stesso. La classe definisce i metodi dell'oggetto. La documentazione delle API è suddivisa in precondizioni e post condizioni. Proponiamo un esempio sulla coda FIFO: nel caso di una invocazione di $\text{enq}(x)$, se la coda è in uno stato q (precondizione) lascia la coda in stato $q \cdot x$, dove $\cdot$ indica la concatenazione. Questo stile di documentazione è chiamato *Specifica Sequenziale*, cioè lineare nel numero di metodi. La documentazione dell'oggetto descrive gli stati dell'oggetto prima e dopo ogni chiamata, quindi possiamo ignorare ogni stato intermedio. Questa specifica, che fallisce quando si parla di oggetti condivisi da più thread, non ha senso in un mondo concorrente. L'invocazione del metodo è un intervallo, non un evento. Dal momento in cui si invoca un metodo al momento in cui risponde, può passare del tempo. Questo è un intervallo di tempo che c'è tra l'invocazione e la risposta. Si possono avere delle invocazioni che si sovrappongono.

4.2 Condizioni di Correttezza

Le specifiche di correttezza degli oggetti concorrenti sono sempre legate ad una certa equivalenza con il comportamento sequenziale, ma con alcune differenze, appropriate in diversi contesti.

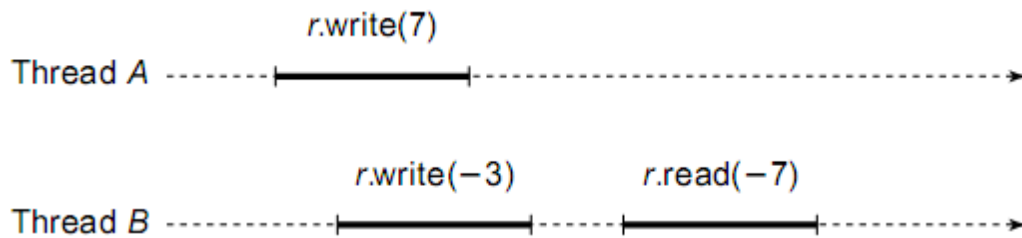
Esamineremo tre tipi di correttezza:

- **Consistenza quiescente:** È appropriata per applicazioni che richiedono alte prestazioni, al costo di piazzare vincoli deboli sul comportamento degli oggetti;
- **Consistenza sequenziale:** Permette di esprimere condizioni più stringenti;
- **Linearizzabilità:** Una condizione ancora più stringente per sistemi ad alto livello;

4.2.1 Consistenza Quiescente

Verifichiamo dove gli oggetti concorrenti seguono la nostra intuizione. Una invocazione ad un metodo inizia con l'invocazione e termina con la risposta. Una invocazione è pendente se la sua invocazione è stata fatta ma non c'è ancora la risposta.

Un esempio non accettabile:



In questo caso, due thread scrivono rispettivamente -3 e 7. Ci aspettiamo che il thread B legga -3 o 7, non -7!

Consistenza Quiescente

Principio 1: Esecuzione istantanea. Le invocazioni dovrebbero apparire come se avvenissero in un ordine sequenziale. Un oggetto è quiescente se non ha invocazioni pendenti. È quiescente se in quel momento tutte le invocazioni, che erano state fatte su di esso, sono terminate.

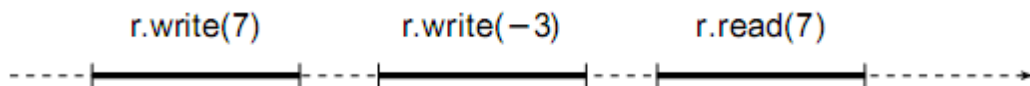
Principio 2: Quiescenza. Invocazioni separate da un periodo di quiescenza dovrebbero apparire come se fossero avvenute nel loro ordine in real-time. In un certo senso, quello che stiamo dicendo è che alla fine quando un oggetto è diventato quiescente, l'esecuzione è equivalente a una qualche esecuzione di invocazioni completate. Per esempio, supponiamo che A e B concorrentemente inseriscano nella coda FIFO x e y. La coda diventa quiescente, e successivamente C inserisce z. Non siamo capaci di predire il relativo ordine di x e y ma possiamo assicurare che sono poste prima di z.

I principi 1 e 2 insieme definiscono una proprietà di correttezza chiamata consistenza quiescente. Quando un oggetto diventa quiescente, l'esecuzione è equivalente a quella sequenziale. Un esempio di oggetto consistente quiescente è il contatore condiviso visto nel Capitolo 1. Il metodo `getAndIncrement()` restituisce un numero senza duplicati o salti di numero, anche senza ordine.

La consistenza quiescente non limita la concorrenza in quanto non blocca l'invocazione di un metodo in attesa di una altra invocazione. In ogni esecuzione concorrente, per ogni invocazione pendente di un metodo totale esiste una risposta compatibile con la consistenza quiescente. Un metodo totale è un metodo definito per ogni stato dell'oggetto (niente eccezioni).

Una proprietà è composizionale se, laddove vale per ogni oggetto in un sistema, vale nell'intero sistema. La consistenza quiescente è composizionale.

4.2.2 Consistenza Sequenziale



Un singolo thread scrive prima 7 e poi -3. Quando legge, il valore che restituisce è 7. Per molte applicazioni il risultato non è accettabile perché non è l'ultimo valore scritto nel registro. Le invocazioni fatte da un singolo thread sono fatte in ordine del programma.

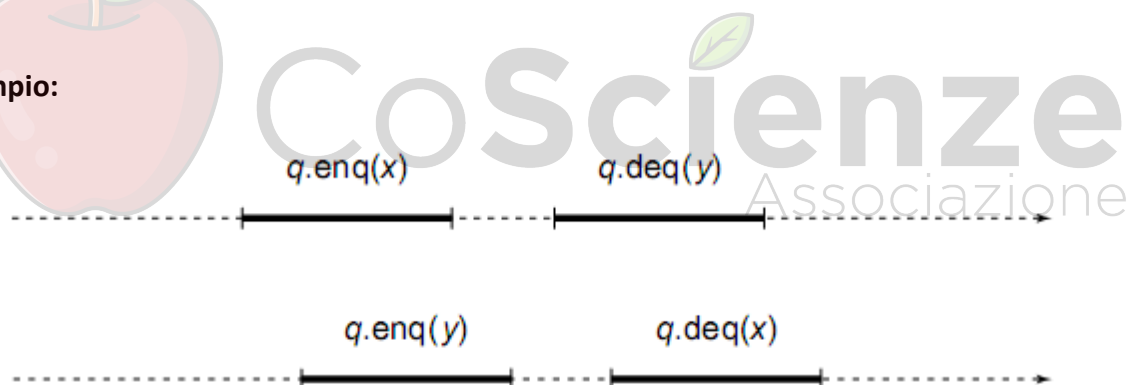
Principio 3: Ordine del programma. Le invocazioni dei metodi dovrebbero apparire come se avessero effetto nell'ordine del programma. Questo principio ci assicura che la computazione sequenziale segue la via che ci aspettiamo.

Una computazione concorrente che rispetta sia il principio di esecuzione istantanea che quello di ordine del programma osserva la consistenza sequenziale.

In ogni esecuzione concorrente si possono ordinare le invocazioni sequenzialmente in modo che:

- Le invocazioni sono consistenti con l'ordine del programma
- Risponde alle specifiche sequenziali degli oggetti

Un Esempio:



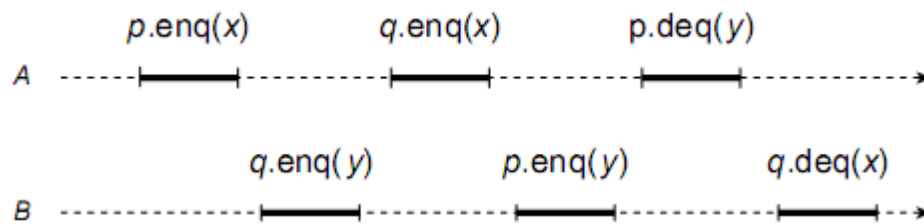
Nell'esempio sopra ci sono due possibili ordini sequenziali che giustificano l'esecuzione:

- A inserisce x, poi B inserisce y e poi B estrae x e A estrae y;
- B inserisce y, poi A inserisce x e poi A estrae y e B estrae x;

Entrambi sono consistenti con l'ordine del programma e solo uno è sufficiente per mostrare che l'esecuzione è sequenzialmente consistente. Le due consistenze sono inconfondibili se esistono esecuzioni che obbediscono alla consistenza sequenziale e non a quella quiescente e viceversa. L'intuizione è che la consistenza quiescente non necessariamente preserva l'ordine del programma e la consistenza sequenziale non viene influenzata da periodi di quiescenza. Nella maggior parte delle moderne architetture multiprocessore, le operazioni di lettura e scrittura della memoria non sono sequenziali: possono essere tipicamente riordinate in modo complesso.

Teorema. La consistenza sequenziale non è composizionale.

Dimostrazione. Si considerino queste due esecuzioni consistenti con l'ordine del programma.



- B deve avere inserito y in p prima che A accodi x (FIFO);
- A deve avere inserito x in q prima che B accodi y (FIFO);
- Dall'ordine del programma si verifica un ciclo;

Questa è una contraddizione. Non è possibile ordinare in maniera tale da poter avere qualcosa che è coerente con lo stato del programma, quindi abbiamo preso due oggetti sequenziali, li abbiamo messi insieme e siamo riusciti a trovare un sistema che non corrisponde alla consistenza sequenziale. Questa è una grossa pietra tombale sulla consistenza sequenziale, perché non ci permette di ottenere sistemi che mantengono le condizioni di correttezza delle proprie componenti.

4.2.3 Linearizzabilità

È un nuovo tipo di consistenza. Introdotta perché la consistenza sequenziale non è composizionale.

Principio 4: Linearizzabilità. Le invocazioni devono apparire come se avessero luogo istantaneamente in un qualche istante tra invocazione e risposta. In questo istante di tempo, questa invocazione avviene, cioè il risultato è presente. Il risultato viene calcolato, il risultato c'è, prima non c'era e adesso c'è. Questo punto, ossia di linearizzare, esiste un punto in questa invocazione, che rendere evidente quello che facciamo è un principio che dice che:

Una computazione concorrente che rispetta il principio di linearizzabilità si dice linearizzabile.

Questo principio dichiara che questo comportamento real-time delle chiamate a metodi deve essere preservato. Quando ci sono sezioni critiche all'interno di metodi, queste vengono usate per la linearizzazione. Se non c'è sezione critica, vengono identificati dei punti in cui i cambiamenti sono visibili. Ad esempio, nella coda circolare wait-free (singolo writer/reader senza sezione critica) i punti di linearizzazione dipendono dall'esecuzione. Se si restituisce un oggetto, il metodo `deq` ha un punto di linearizzazione quando il campo è modificato. Se la coda è vuota, con il metodo `deq` si ha un punto di linearizzazione sull'invocazione dell'eccezione. La linearizzabilità include la consistenza sequenziale (più strettamente). La linearizzabilità è composizionale e non bloccante.

4.3 Condizioni di Progresso

Esistono delle implementazioni bloccanti dove un ritardo inaspettato da parte di un thread può impedire ad altri di fare progressi nella computazione, tra cui:

- **Cache Miss:** Centinaia di cicli macchina;
- **Page Fault:** Milioni di cicli;
- **Prelazione da parte del Sistema Operativo:** Centinaia di milioni di cicli;

Una condizione di progresso non bloccante significa che un ritardo inaspettato ed arbitrario da parte di un thread non necessariamente impedisce agli altri thread di compiere progressi, un esempio è la proprietà wait-free.

- In un metodo wait-free ogni chiamata termina l'esecuzione in un numero finito di passi;
- In un metodo bounded wait-free ogni chiamata termina l'esecuzione in un numero finito e limitato di passi;

Wait-free è una proprietà attraente ma potenzialmente difficile da realizzare in maniera efficiente.

Un metodo è lock-free se garantisce che infinitamente spesso qualche chiamata di un metodo termini in un numero finito di passi, cioè tra tutti quelli che provano ad accedere ed eseguire tale metodo, c'è sempre qualcuno che riesce in maniera finita a terminarlo. Se un thread viene sacrificato, lo si fa perché gli altri fanno progresso. Ogni implementazione wait-free è anche lock-free ma non viceversa. In alcuni casi la possibilità di starvation di qualche thread (ammessa da lock-free) è accettabile (se bassa) in cambio dell'efficienza dell'implementazione.

4.3.1 Proprietà di un Lock

Le condizioni di progresso wait-free e lock-free garantiscono che la computazione effettui un progresso, indipendentemente da come il sistema schedula i thread. Deadlock-free e Starvation-free sono condizioni di progresso dipendente, i progressi sono possibili solo se il sistema operativo fornisce alcune garanzie (ad esempio "Ogni thread alla fine lascia la sezione critica"). Le classi che si basano su implementazioni con lock possono garantire solamente proprietà di progresso dipendente. Questa non è una limitazione fortissima se la preemption è rara o se il costo di assicurare progresso è basso (niente deadlock in caso di preemption in CS). Esistono anche condizioni di progresso dipendenti che sono non bloccanti.

	Non-bloccante	Bloccante
Ognuno fa progresso	Wait-free	Starvation-free
Qualcuno fa progresso	Lock-free	Deadlock-free

Questa tabella riassume le condizioni di progresso. Bloccante significa che usa il lock. Quando ognuno dei thread fa progresso, quando usa il lock significa starvation-free. Un numero infinito di thread che fa progresso significa deadlock-free. Lo stesso diagramma può essere fatto per le condizioni non bloccanti, cioè esecuzioni di metodi che terminano per tutti in un tempo finito,

oppure che possono anche non terminare in un tempo infinito se contemporaneamente un numero infinito di thread riesce ad eseguire quel metodo.

4.3.2 Commenti alle Condizioni di Progresso

La scelta di una condizione di progresso dipende dall'applicazione e dalle garanzie offerte dal sistema operativo. Le proprietà wait-free e lock-free teoricamente lavorano su ogni piattaforma e possono essere usate per applicazioni come musica, videogiochi ed altre applicazioni interattive. Le proprietà dipendenti si affidano alla piattaforma sottostante, ma hanno implementazioni più semplici ed efficienti.

4.4 Il Modello di Memoria di Java

Il modello di memoria di Java non garantisce né linearizzabilità né consistenza sequenziale. Aderire alla consistenza sequenziale eviterebbe di poter usare tecniche di ottimizzazione dei compilatori (allocazione di registri, eliminazioni di sotto espressioni comuni, eliminazioni di letture inutili). Le ottimizzazioni non fanno differenza in una esecuzione sequenziale, ma sicuramente fanno differenza in un ambiente multi-threaded. Il modello di memoria di java è un modello di memoria rilassato. Se l'esecuzione sequenzialmente del programma segue certe regole, allora l'esecuzione in un modello di memoria rilassata è sequenzialmente consistente. Se si seguono le regole, questo permette al modello di memoria di java di offrire la consistenza sequenziale.

Un esempio era il Double-Checked Locking:

```

1 public static Singleton getInstance() {
2     if(instance == null) {
3         synchronized (Singleton.class){
4             if(instance == null)
5                 instance = new Singleton();
6             }
7     }
8     return instance;
9 }

```

Si effettuava un ingresso in critical section, effettuando il lock sull'oggetto del Singleton. Se l'istanza era null si entrava in sezione critica e appena si entrava si verificava che nel momento in cui si

faceva waiting non fosse cambiato nulla. Se non veniva cambiato nulla veniva creato il nuovo oggetto, e poi si usciva dalla critical section. Questa è un'azione che sembra istantanea ma non lo è, dato che prevede la costruzione di un oggetto. La chiamata al costruttore `new Singleton()` sembra essere effettuata prima di quella che è l'assegnazione alla variabile `instance`. Poiché ci troviamo all'interno di un blocco sincronizzato e la consistenza viene mantenuta solo all'uscita della sezione critica, ma non è detto che ciò accada. Potrebbe succedere che in memoria vi sia un oggetto A, un oggetto senza nome che è stato creato dal nuovo Singleton. Questo oggetto, che ha diversi campi e valori, non è detto che sia stato completamente inizializzato, ciò nonostante, la variabile `instance` può contenere il riferimento a questo oggetto. Quindi `instance` non è null, ma non si può restituire. Gli oggetti risiedono in memoria (condivisa) e i thread usano una memoria di lavoro (privata) dove hanno copie di lavoro dei campi letti/scritti. Senza un'esplicita sincronizzazione, una scrittura può non essere subito propagata dalla memoria di lavoro alla memoria condivisa. La JVM mantiene le copie consistenti "spesso", ma non è garantito che lo faccia in ogni istante. L'unica garanzia che offre la JVM è che le read/write di un thread appaiono nell'ordine in cui quel thread le ha istanziate (ordine del programma) e che ogni valore di un campo letto da un thread era stato in quel campo. La soluzione è la sincronizzazione, usata per la mutua esclusione ma anche per riconciliare cache e memoria. Altro aspetto importante è che gli eventi di sincronizzazione sono linearizzabili. Essi rappresentano il momento all'interno di un metodo in cui un risultato istantaneamente avviene.

I Lock `synchronized` sono basati sul Lock `java.util.concurrent.locks.ReentrantLock` oppure con un blocco `synchronized`. Le implementazioni di lock più primitive prevedevano che un lock andasse in deadlock da solo. Tutte le read/write a un campo protetto dallo stesso lock sono linearizzabili quando:

- Un lock è rilasciato, i campi sono modificati. sono scritti in memoria condivisa;
- Un lock è acquisito, i campi in cache sono resi invalidi;

Altre soluzioni sono i campi `volatile`, che sono linearizzabili. In tal caso letture/scritture non sono atomiche. Leggere un campo volatile è come acquisire un lock, in quanto la memoria di lavoro viene invalidata ed il valore corrente del campo volatile viene riletto dalla memoria. Quindi `x++` può essere interfogliato, anche se `x` è volatile. Tipicamente un pattern che si usa è una variabile volatile che viene scritta da un thread e letta da molti.

Esistono anche implementazioni specifiche di memoria linearizzabile, cioè memoria le cui operazioni sono istantaneamente rappresentate sottoforma di `AtomicReference<T>` o `AtomicInteger`. Un'altra tecnica è quella dei campi `final`, campi che non possono essere modificati quando vengono inizializzati nel costruttore, ma la cosa positiva è che non hanno bisogno di sincronizzazione. Non ci sono quindi problemi se non c'è il fenomeno, chiamato *Escape del Riferimento*. Se il costruttore non rilascia `this` prima che esso termini, i campi `final` appaiono corretti ad ogni thread, senza bisogno di sincronizzazione.

Un Esempio di Uso Corretto di final:

```
1  class FinalFieldExample() {
2  final int x; int y;
3  static FinalFieldExample f;
4      public FinalFieldExample() {
5          x=3;
6          y=4;
7      }
8      static void writer() {
9          f= new FinalFieldExample();
10     }
11     static void reader() {
12         if (f != null){
13             int i = f.x; int j = f.y;
14         }
15     }
16 }
17
```

L'esecuzione del metodo `writer()` (Riga 8) assegna i campi con il costruttore, mentre il metodo `reader()` (Riga 11) non fa altro che leggere e se `f` non è `null`, allora legge i valori di `x` e `y` nelle variabili di `i` e `j` (Riga 13). Se non c'è nessun lock, se in contemporanea vengono chiamati un metodo `writer` o metodo `reader`, il metodo `writer` usa il costruttore, il costruttore sta settando un campo `final`. Il campo `final` è sicuro essere a 3 mentre potrebbe non vedere `y` a 4.

```
1  public class EventListener() {
2  final int x;
3  public EventListener(EventSource eventSource) {
4      x = 1;
5      eventSource.registerListener(this); //register with event source
6  }
7  public onEvent(Event e) {
8      .. // handle the event
9  }
10 }
```

`EventListener()` è un tipico pattern di Observer. L'Escape di `this` può avvenire ad esempio iscrivendo `this` a un listener, cioè `this` viene utilizzato prima che il metodo di istruzione ha terminato. Il valore di `x` non sarà reso disponibile immediatamente durante l'esecuzione del costruttore.

Che tipo di condizione di progresso è corretta per le applicazioni?

Un metodo invocato spesso dovrebbe essere wait-free. Il wait-free è l'obiettivo principale, ma è anche complesso ottenerlo. Un metodo invocato poco frequentemente può essere implementato con la mutua esclusione. La consistenza quiescente ha un suo significato. Ad esempio, un server di una banca dovrebbe usare code sequenzialmente consistenti per gli eventi (mettere 100 € su un conto vuoto e poi ritirarne 50 non deve farti andare in rosso). Come programmatori, sarebbe ideale avere hardware linearizzabile, strutture dati linearizzabili e buone prestazioni.

Sfortunatamente la tecnologia è imperfetta e l'hardware con buone prestazioni non soddisfa neanche la consistenza sequenziale. Le strutture dati potrebbero essere linearizzabili con buone prestazioni.

Domande di Riepilogo

- Descrivere il problema del singleton in Java e come alcune soluzioni, come il double-checked locking, non sono corrette
- Cosa significa che un oggetto concorrente è corretto?
- Descrivere l'algoritmo LockBasedQueue
- Descrivere l'algoritmo WaitFreeQueue
- Verificare come WaitFreeQueue è una soluzione per un single-enqueuer/single-dequeuer e che non è una soluzione in caso di multipli enqueueer/dequeueer
- Confrontare la LockBasedQueue con la WaitFreeQueue
- Perché è critico cercare di limitare al massimo l'utilizzo di lock negli oggetti concorrenti
- Cosa è la consistenza quiescente?
- Cosa è la composability di una consistenza?
- Perché la consistenza quiescente non limita la concorrenza?
- Cosa è la consistenza sequenziale?
- Quali sono i rapporti tra consistenza sequenziale e consistenza quiescente?
- Dimostrare che la consistenza sequenziale non è composizionale
- Cosa è la linearizzabilità e perché è necessario introdurla?
- Quali sono le condizioni di progresso non bloccanti?
- Quali sono le condizioni di progresso dipendenti?
- Descrivere il modello di memoria di Java
- Descrivere i lock e i blocchi synchronized in Java
- Descrivere i campi volatile in Java
- Descrivere i campi final in Java
- Quali sono le condizioni di progresso giuste in diverse situazioni?
- Quali sono le condizioni di correttezza giuste in diverse situazioni?

5. Spinlocks

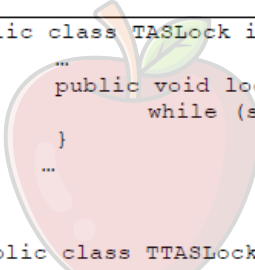
5.1 Il Mondo Reale

Il programmatore sequenziale deve essere a conoscenza dei dettagli architetturali che spesso sono necessari per il programmatore concorrente, sia per quanto riguarda la correttezza ma anche per l'efficienza. Quando si usa il lock ci sono due alternative: Spinning o Blocking.

- **Spinning:** In attesa che il lock sia disponibile, con letture ripetute;
- **Blocking:** Il thread si blocca e chiede al sistema operativo di schedarne un altro e di essere notificato quando il lock risulta essere disponibile;

Lo Spinning è meno costoso del Blocking in termini di tempo, cioè si tiene il thread sempre in esecuzione, anche se ciò consuma CPU. Rendere efficiente uno spin lock richiede conoscenza dell'architettura.

5.1.1 Test-and-Set



```
public class TASLock implements Lock {
    ...
    public void lock() {
        while (state.getAndSet(true)) {}
    }
    ...
}

public class TTASLock implements Lock {
    ...
    public void lock() {
        while (true) {
            while (state.get()) {}
            if (!state.getAndSet(true))
                return;
        }
    }
    ...
}
```

CoScienze
Associazione

L'operazione `testAndSet()` è stata la principale istruzione di sincronizzazione fornita da molte architetture multiprocessore. Questa istruzione opera su una singola parola di memoria (o byte). Quella parola contiene un valore binario, vero o falso. L'istruzione, atomicamente, memorizza `true` nella word e restituisce il valore precedente di word, scambiando il valore `true` con il valore corrente della parola. A prima vista, questa istruzione sembra ideale per l'attuazione di uno spin lock. Il lock è libero quando il valore della parola è falsa, e occupato quando il lock è `true`. Il metodo `lock()` applica ripetutamente `testAndSet()` fino a quando non restituisce `false` (vale a dire, fino a quando il lock diventa libero). Il metodo `unlock()` scrive semplicemente il valore `false`. Il package `java.util.concurrent` include una classe `AtomicBoolean` che memorizza un valore booleano. Essa fornisce un metodo `set(b)` che sostituisce il valore memorizzato con il valore di `b`, e un `getAndSet(b)` che sostituisce atomicamente il valore corrente con `b`, e restituisce il valore precedente. La vecchia `testAndSet()` è simile alla chiamata a `getAndSet(true)`. Usiamo il

termine test-and-set per rimanere compatibile con l'uso comune, ma nei nostri esempi verrà utilizzata l'espressione `getAndSet(true)` per essere compatibili con Java. La classe `TASLock` mostra un algoritmo di blocco basato sull'istruzione `testAndSet()`.

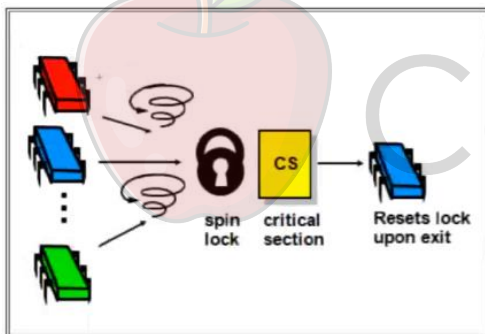
Con `TTASLock` invece in `while(true)` si leggeva solo la memoria e quando `state.get()` risultava essere `false`, si interrompeva lo Spinning, si faceva il controllo una volta per vedere se si poteva acquisire il lock. In tal caso si entrava, altrimenti si entrava nuovamente in attesa. `TASLock` abbiamo visto che funziona peggio di `TTASLock` e il motivo è relativo all'operazione della cache.

Ora il comportamento bizzarro di `TASLock` e `TTASLock` si spiega. Ad ogni `getAndSet()` di `TASLock` viene generato traffico sulla interconnessione. Lo Spinning di `TTASLock` legge da una copia in cache e non produce carico sull'interconnessione. Comunque, `TTASLock` non risultava ideale:

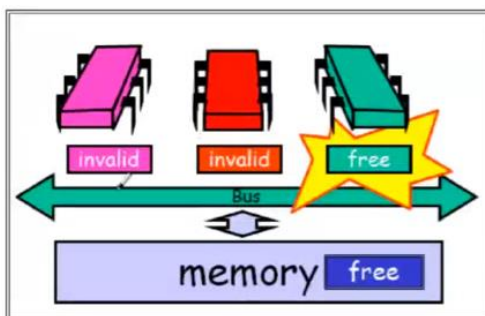
- Quando un blocco viene rilasciato, la variabile viene messa a `false`;
- Tutte le copie in cache sono invalidate;
- Tutti i processori in attesa leggono da memoria il nuovo valore;
- Tutti i processori in attesa chiamano `getAndSet(true)` causando un certo traffico;

5.1.2 TTAS con Backoff

Una cosa interessante è cercare di esprimere un algoritmo TTAS con Backoff:



Quando si fa spin lock, succede che i thread fanno spinning. Quando uno dei thread riesce ad entrare, entra in critical section e resetta il lock quando esce. Comunque, il thread rosso e verde sono in lock. Quando esce il thread blu, c'è una forte contesa tra i due. Questo è poco utile perché solo uno potrà vincere, quindi questo rallenta solamente l'uso del bus per far entrare un solo thread.



Il thread verde dice di essere free, invalida i valori che sono presenti in cache. Il valore del lock intende che la critical section è busy. Tutti i thread stanno ciclando senza usare il bus all'interno della propria cache. Quando il processore verde esce, scrive free e il protocollo MESI invalida tutti i valori degli altri, i quali cercano di accedere facendo cache miss sulla memoria per leggere il nuovo valore, che è free.

Se la `get()` ha dato lettura `false` e subito dopo la `getAndSet()` ha dato lettura `true` vuol dire che c'è *High Contention*, ovvero ci sono altri thread che sono in attesa. Risulta inutile fare spin lock su un lock che è soggetto a molti accessi (overhead sul traffico del bus processori-memoria). Per evitare contese si fa un backoff, cioè si aspetta un tempo random. Questo disperde tutti i thread

che fanno contesa nel tempo. La tecnica che viene usata è un Backoff esponenziale, cioè un backoff che ogni volta che non si riesce ad accedere, si aumenta esponenzialmente l'intervallo all'interno del quale si seleziona a caso un tempo. Se c'è molta contesa e subito dopo continua ad esserci, allora significa che l'intervallo all'interno del quale abbiamo selezionato un valore random non è abbastanza ampio da garantire una sufficiente dispersione dei thread. Al passo successivo si aumenterà, con tutti gli altri thread che fanno la stessa decisione, l'intervallo in maniera tale da raggiungere il valore in cui i thread sono abbastanza dispersi, in modo da entrare senza fare contesa.

5.1.3 La Classe BackoffClass

```
1 public class Backoff {
2     final int minDelay, maxDelay;
3     int limit;
4     final Random random;
5     public Backoff(int min, int max) {
6         minDelay = min;
7         maxDelay = max;
8         limit = minDelay;
9         random = new Random();
10    }
11    public void backoff() throws InterruptedException {
12        int delay = random.nextInt(limit);
13        limit = Math.min(maxDelay, 2 * limit);
14        Thread.sleep(delay);
15    }
16 }
```

La classe `Backoff` ha alcuni parametri, come `minDelay` e `maxDelay`, i quali vengono settati al momento della costruzione, e c'è una variabile di `limit` e una di `random`. `Backoff` è un metodo (Riga 11), il quale prende un intero a caso nel limite che all'inizio viene settato da `minDelay`. Successivamente, aumenta il limite (Riga 13) e lo raddoppia, poi rimane fermo per un certo numero di secondi pari a `limit` (Riga 14).

Come funziona il BackoffLock?

```
1 public class BackoffLock implements Lock {
2     private AtomicBoolean state = new AtomicBoolean(false);
3     private static final int MIN_DELAY = ...;
4     private static final int MAX_DELAY = ...;
5     public void lock() {
6         Backoff backoff = new Backoff(MIN_DELAY, MAX_DELAY);
7         while (true) {
8             while (state.get()) {}
9             if (!state.getAndSet(true)) {
10                 return;
11             } else {
12                 backoff.backoff();
13             }
14         }
15     }
16     public void unlock() {
17         state.set(false);
18     }
19     ...
20 }
```

Figure 7.6 The Exponential Backoff lock. Whenever the thread fails to acquire a lock that became free, it backs off before retrying.

Come sempre si implementa la stessa interfaccia `Lock`. Abbiamo una variabile `state`, usata come variabile `AtomicBoolean`, cioè le operazioni sono atomiche. Con `MIN_DELAY` e `MAX_DELAY` viene scelto un valore dall'architettura. Nel `lock` quello che si fa è far partire un backoff, cioè si crea un generatore di ritardi esponenziale. Un generatore di ritardi è fatto in maniera tale che ogni volta che si chiama il

metodo `backoff()`, il `limit` viene aumentato, quindi la volta successiva che si chiama `backoff()`, l'intervallo all'interno del quale si sceglie il valore si usa un intervallo più ampio, che è il doppio del precedente valore. A questo punto si fa il TTAS, cioè si rimane fermi nel fare il `get()`

(Riga 8). Se si riesce ad entrare (Riga 9) si fa `return` e tutto va bene, altrimenti nel caso in cui facendo `getAndSet()` ed esso restituisce `true`, questo significa che c'è contesa e quindi si chiama il metodo `backoff()` (Riga 12), il quale fa stare fermo il thread per un certo intervallo di tempo casualmente scelto e facendo in modo che se si volesse richiamare il metodo `backoff()` sullo stesso oggetto, l'intervallo all'interno del quale viene selezionato il tempo che è raddoppiato. In questo momento non si fa spin lock, ma si è fermi in esecuzione (Riga 12). Si torna nel `while(true)` solo quando il thread ha terminato lo sleep e torna a fare l'operazione di TTAS. Il metodo `unlock()` è semplice e funziona senza problemi, settando il valore a `false` (Riga 16).

Questa ovviamente non è la soluzione migliore. Si vorrebbe poter dare priorità ai thread che sono in attesa da più tempo.

Cosa manca ai BackoffLock?

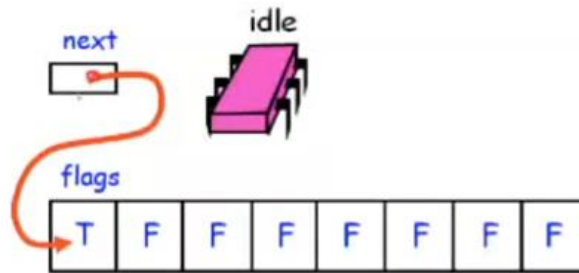
Quello che manca ai BackoffLock sono:

- **Efficienza:** tutti i thread fanno spin sulla stessa locazione condivisa. Questo da un lato non occupa memoria, ma allo stesso tempo genera traffico;
- **Sottoutilizzazione:** il ritardo impedisce un accesso continuo alla risorsa critica, anche in caso di High Contention;

L'idea è quella di far formare ai thread una coda, cioè gestire l'attesa in maniera che ci sia una sorta di priorità acquisita dal fatto che si è in coda, quindi fare spin lock sul thread precedente. Il valore aggiunto è che, se si forma una coda, la politica è First-Come-First-Served. First-Come-First-Served va benissimo perché assicura la starvation-free, cioè assicura che ogni thread viene servito in un tempo finito, che in tal caso è la dimensione della coda (numero di thread in coda) prima che il thread si mettesse in coda.

5.2 Queue Locks

Ora esploriamo un approccio diverso per l'attuazione di spin lock scalabile, leggermente più complicato di lock con backoff, ma molto più portabile. Come abbiamo visto, il lock basato su Backoff comporta problemi. Si può ovviare a questi inconvenienti facendo uso di thread a forma di coda. Ogni thread fa spinning su un booleano che si trova in un array di booleani. L'idea è che i thread accedono a questo array come se fosse una coda circolare e ogni thread fa spinning su uno dei booleani. La tail è acceduta in mutua esclusione. Ogni thread preleva un proprio slot e fa spin lock su di esso. Il thread A entra, sa che la sua posizione è la numero 3, così fa spinning sulla locazione 3. Il thread aspetterà che la locazione 3 diventi true per poter entrare in critical section. Quando esce dalla critical section, setta a false il proprio slot e a true quello successivo, cioè sbloccando il thread in attesa. Esiste un puntatore all'interno della quale posizionarci.



Dalla figura è possibile vedere che lo stato della coda è vuota. Ci sono tutte false tranne la prima, che è quella della posizione puntata da next. Il thread rosa accede alla locazione, effettua l'operazione di `getAndIncrement()`, cioè un'operazione per garantire la mutua esclusione e un'operazione equivalente a `getAndSet()`. Il thread acquisisce la sua posizione, e `next` passa a puntare al thread successivo. Il thread ha così acquisito la sezione critica. Di seguito vediamo che:

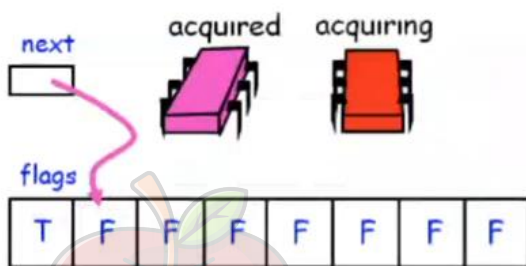


Figura 1

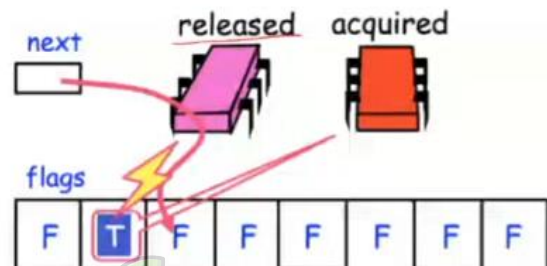


Figura 2

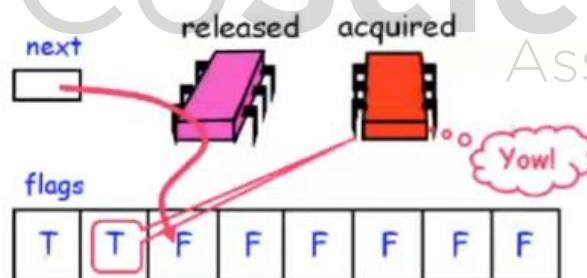


Figura 3

Il thread rosso prova ad entrare nella sezione critica, fa la `getAndIncrement()` e prende la posizione 2. Con il `next` incrementato, a questo punto il thread rosso fa lo spinning sulla locazione dopo T, cioè la seconda. A rilascio del lock, il thread rosa mette a false il proprio lock e mette a true quello successivo, cioè fa in modo che se ci fosse qualcuno il thread rosso riesce ad entrare perché vede che la sua posizione è true. Questo meccanismo permette al thread rosso di entrare in sezione critica.

È stato distribuito tutto il carico su un array di booleani e ora è su tante locazioni.

5.2.1 Lock di Anderson

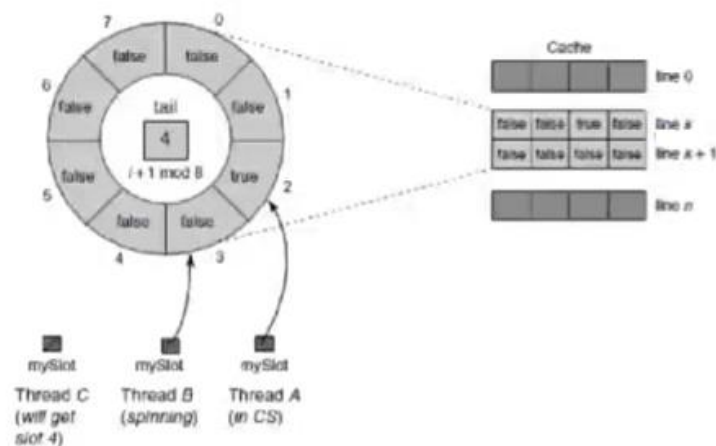
```

1 public class ALock implements Lock {
2     ThreadLocal<Integer> mySlotIndex = new ThreadLocal<Integer> () {
3         protected Integer initialValue() {
4             return 0;
5         }
6     };
7     AtomicInteger tail;
8     boolean[] flag;
9     int size;
10    public ALock(int capacity) {
11        size = capacity;
12        tail = new AtomicInteger(0);
13        flag = new boolean[capacity];
14        flag[0] = true;
15    }
16    public void lock() {
17        int slot = tail.getAndIncrement() % size;
18        mySlotIndex.set(slot);
19        while (!flag[slot]) {};
20    }
21    public void unlock() {
22        int slot = mySlotIndex.get();
23        flag[slot] = false;
24        flag[(slot + 1) % size] = true;
25    }
26 }

```

Figure 7.7 Array-based Queue Lock.

La figura mostra ALock, un semplice array basato su lock a coda. Ogni thread ha una sua variabile ThreadLocal chiamata mySlotIndex, una variabile non accessibile a nessun altro thread. La tail è un AtomicInteger (Riga 12), poi è presente un array di booleani e il metodo costruttore ALock(int capacity), il quale setta la capacity di una coda, inizializza una tail a puntare a 0, crea un array di flag e mette il primo elemento a true. Il primo elemento indica che qualora un thread si mettesse in attesa, questo userebbe lo slot 0, e trovando true entrerebbe in sezione critica. Viene effettuata l'operazione di getAndIncrement() di tail (Riga 17), cioè si preleva il valore di tail, si fa il modulo size e la variabile slot viene messa nel metodo set() (Riga 18). Successivamente si fa spin lock, cioè si fa in modo di rimanere in attesa sul flag al quale si sta puntando. Quando si deve fare unlock() (Riga 21), si setta semplicemente flag[slot] a false e si libera quello successivo. Un problema di ALock è che il False sharing su array creano High Contention.



Nella linea k si possono avere diverse posizioni e quindi generare più traffico di quello che c'è. Una soluzione è inserire del padding, cioè fare in modo che l'array di booleani è sparso di valori fittizi. Si moltiplica l'indice per la dimensione della cache line, in modo che ogni posizione dell'array sta su una cache line propria e si evitano problemi. I thread A e B stanno uno dopo l'altro ed essendo A in sezione critica mentre B fa spinning, entrambi non stanno generando problemi.

ALock migliora BackoffLock poiché riduce le invalidazioni al minimo delle copie in cache, minimizzando l'intervallo tra il momento in cui il lock diventa libero da un thread e quando viene acquisito da un altro. A differenza del TASLock e BackoffLock, questo algoritmo garantisce che non si verifichi mai la starvation: il "primo arrivato è il primo servito". Purtroppo, il lock ALock non è efficiente in base allo spazio di memoria $O(L \cdot n)$, inoltre si deve gestire n come dimensione massima della coda.

5.2.2 CLH Locks

```

1 public class CLHLock implements Lock {
2     AtomicReference<QNode> tail = new AtomicReference<QNode>(new QNode());
3     ThreadLocal<QNode> myPred;
4     ThreadLocal<QNode> myNode;
5     public CLHLock() {
6         tail = new AtomicReference<QNode>(new QNode());
7         myNode = new ThreadLocal<QNode>() {
8             protected QNode initialValue() {
9                 return new QNode();
10            }
11        };
12         myPred = new ThreadLocal<QNode>() {
13             protected QNode initialValue() {
14                 return null;
15            }
16        };
17     }
18     ...
19 }

```

Figure 7.9 The CLHLock class: fields and constructor.

```

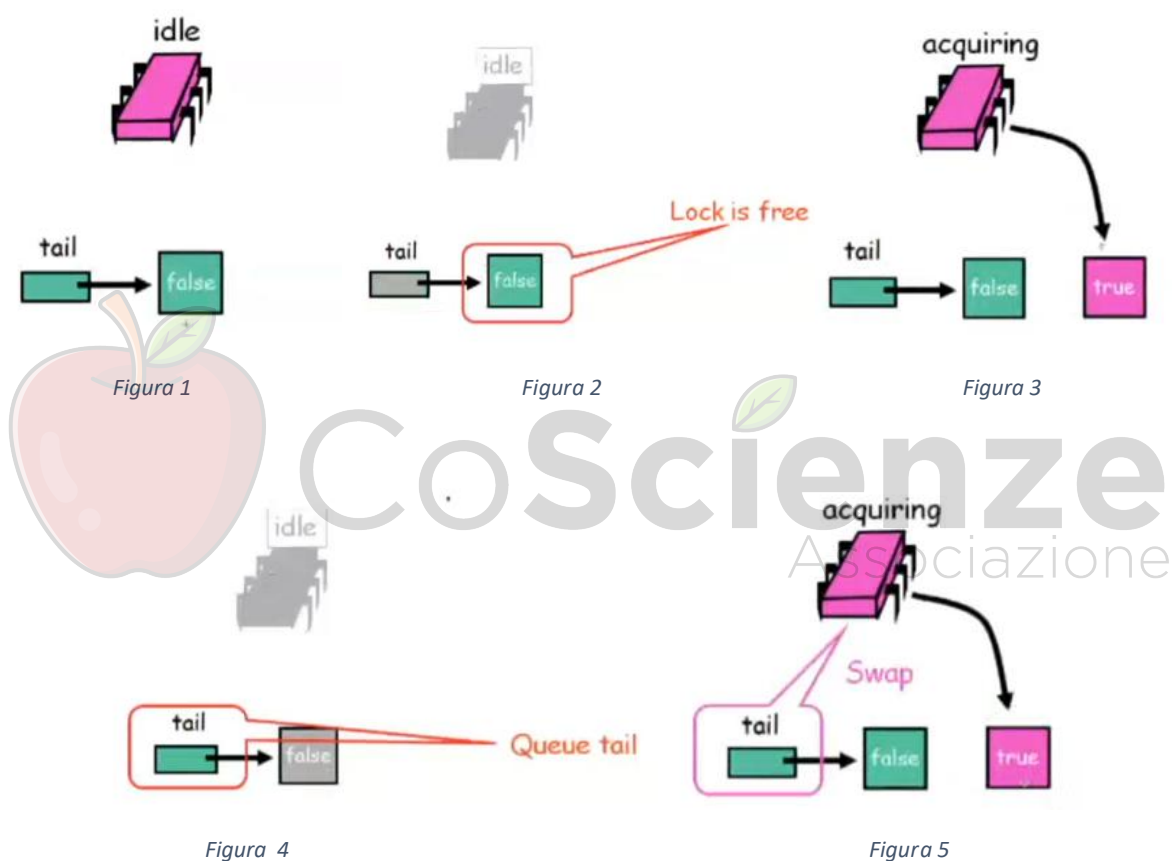
20 public void lock() {
21     QNode qnode = myNode.get();
22     qnode.locked = true;
23     QNode pred = tail.getAndSet(qnode);
24     myPred.set(pred);
25     while (pred.locked) {}
26 }
27 public void unlock() {
28     QNode qnode = myNode.get();
29     qnode.locked = false;
30     myNode.set(myPred.get());
31 }
32 }

```

Figure 7.10 The CLHLock class: lock() and unlock() methods.

Questa classe registra ogni status di thread in un oggetto `QNode`, che ha un campo Boolean `locked`. Se tale campo è vero, allora il thread corrispondente o ha acquisito il lock oppure è in attesa del lock. Se tale campo è falso, allora il thread ha rilasciato il lock. Il lock stesso è rappresentato come una lista linkata di oggetti virtuali `QNode`. Usiamo il termine "virtuale" perché l'elenco è implicito: ogni thread si riferisce al suo predecessore con una variabile locale di thread `pred`.

Il campo pubblico `tail` è un `AtomicReference<QNode>` che punta al nodo più recentemente aggiunto alla coda. Quando si acquisisce un lock, un thread setta il campo `locked` della sua `QNode` a `true`, che indica che il thread non è pronto a rilasciare il lock. Il thread applica `getAndSet()` al campo `tail` per fare del proprio nodo la coda della coda e contemporaneamente acquisisce un riferimento al `QNode` del suo predecessore. Il thread poi gira (spin) sul campo `locked` del predecessore, fino a quando il predecessore rilascia il lock. Quando si rilascia un lock, il thread setta il campo `locked` su `false`. Riutilizza allora il `myPred` come nuovo nodo per gli accessi futuri. È possibile riciclare i nodi in modo che se ci sono lock di L e ogni thread (ce ne sono n) accede al massimo un lock alla volta, allora la classe `CLHLock` rispetto allo spazio è $O(L + n)$, a differenza di $O(Ln)$ di `ALock`. Le prossime figure mostrano una esecuzione `CLHLock` tipica:



`tail` punta all'oggetto `QNode` che ha come status `false`. Il thread rosa vuole tentare di accedere: Accede alla `tail` della coda, vede qual è l'oggetto riferito dalla `tail`, vede che l'oggetto è `false` e quindi cerca di fare un lock creando un oggetto con `true`, fa lo swap atomico di `false` con `true`. Il thread rosa ha acquisito il lock `false` e ora la `tail` punta a `true`.

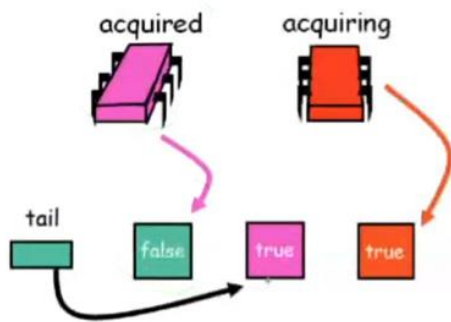


Figura 6

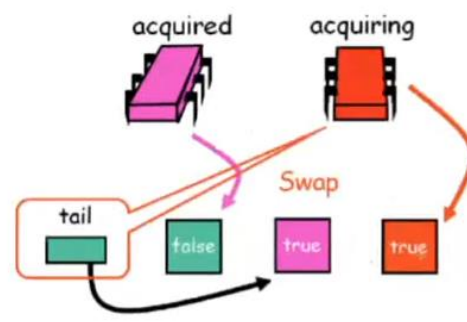


Figura 7

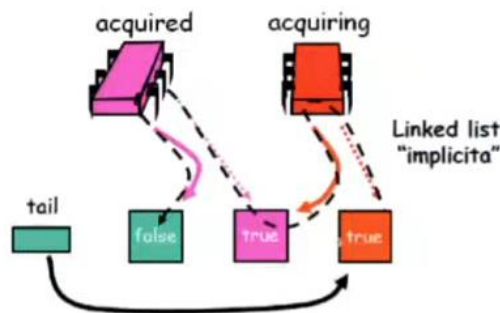


Figura 8

Il thread rosso fa lo stesso processo fatto dal thread precedente: Viene creato un nuovo oggetto a true e fa lo swap della tail con il nuovo oggetto. A questo punto lo swap atomico gli dà modo di avere accesso alla variabile, ma questa è true e quindi sa di non poter accedere. La lista a puntatori è costruita dai riferimenti, dal fatto che ognuno di essi ha in memoria locale il puntatore al predecessore.

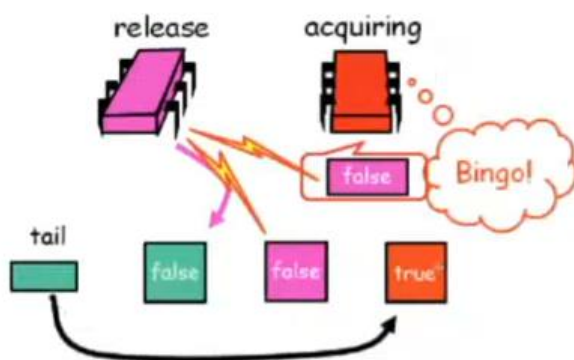


Figure 1

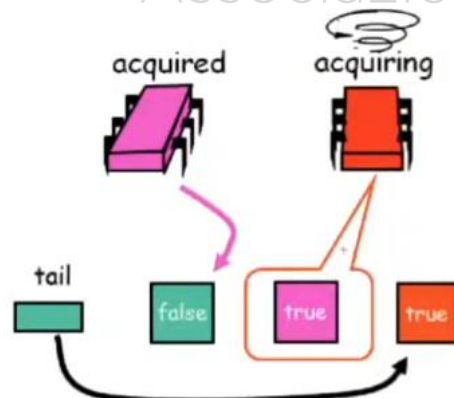


Figure 2

Adesso il thread rosso ha preso il QNode con valore true ed è su questo oggetto che deve fare spinning, perché questo oggetto true (rosa) è un oggetto che conosce anche il thread rosa. Lo spinning fa in modo di avvisare il thread rosso quando il thread rosa rilascia la sezione critica. Il rilascio della sezione critica fa in modo che l'oggetto viene reso invalido nella cache del thread rosso. Il thread rosso accede alla memoria, legge il valore, legge false e sa quindi che può entrare. Alla fine,

il thread rosa è entrato e uscito, il thread rosso ha acquisito la sezione critica e la tail punta verso true.

Quindi ogni thread fa spin su una locazione distinta. Si ha molto meno spazio del lock di Anderson. Non è necessario definire n (numero di thread), inoltre è first-come-first-served. Lo svantaggio è dato da basse prestazioni su NUMA senza cache, cioè ogni thread fa spin sulla copia del predecessore.

5.2.3 MCS Locks

Il MCS Lock è dovuto a Mellor-Crummery e Scott. Il lock viene rappresentato come una lista concatenata di oggetti `QNode`, dove ogni `QNode` rappresenta sia un titolare di lock, sia un thread che è in attesa di acquisire il blocco. A differenza del `CLHLock`, la lista è esplicita e non virtuale: invece di incarnare la lista nella variabile locale del thread, è incarnata nell'oggetto `QNode` (accessibile a livello globale). Questo algoritmo è più adatto alla cache-less delle architetture NUMA perché ogni thread controlla la posizione su cui fa spin. Come `CLHLock`, i nodi possono essere riciclati in modo che questo lock ha una complessità di spazio $O(L + n)$. Uno svantaggio dell'algoritmo `MCSLock` è che il rilascio di un lock richiede spinning.

```
1 public class MCSLock implements Lock {
2     AtomicReference<QNode> tail;
3     ThreadLocal<QNode> myNode;
4     public MCSLock() {
5         tail = new AtomicReference<QNode>(null);
6         myNode = new ThreadLocal<QNode>() {
7             protected QNode initialValue() {
8                 return new QNode();
9             }
10        };
11    }
12    ...
13    class QNode {
14        boolean locked = false;
15        QNode next = null;
16    }
17 }
```

Figure 7.12 MCSLock class: fields, constructor and QNode class.

Innanzitutto, viene implementata un'interfaccia `Lock` (Riga 1) e una variabile atomica che punta alla tail (Riga 2), inoltre abbiamo un nodo `myNode` (Riga 3) che è una variabile `ThreadLocal`. La costruzione del `MCSLock()` prevede la creazione di un nodo che è un `AtomicReference` con tipologia `QNode` inizializzato a null. Ogni nodo crea nella propria memoria locale un nuovo `QNode` che è locale, dentro `myNode`. La classe `QNode` (Riga 13) è banale, infatti oltre ad avere il campo `locked` ha anche un campo `next`, inizializzato a null.


```

1  //...
2  public void lock() {
3      QNode qnode = myNode.get();
4      QNode pred = tail.getAndSet(qnode);
5      if (pred != null){
6          qnode.locked = true;
7          pred.next = qnode;
8          while(qnode.locked) {}
9      }
10 }
11 public void unlock() {
12     QNode qnode = myNode.get();
13     if (qnode.next == null){
14         if(tail.compareAndSet(qnode, null))
15             return;
16         while(qnode.next == null) {}
17     }
18     qnode.next.locked = false;
19     qnode.next = null;
20 }

```

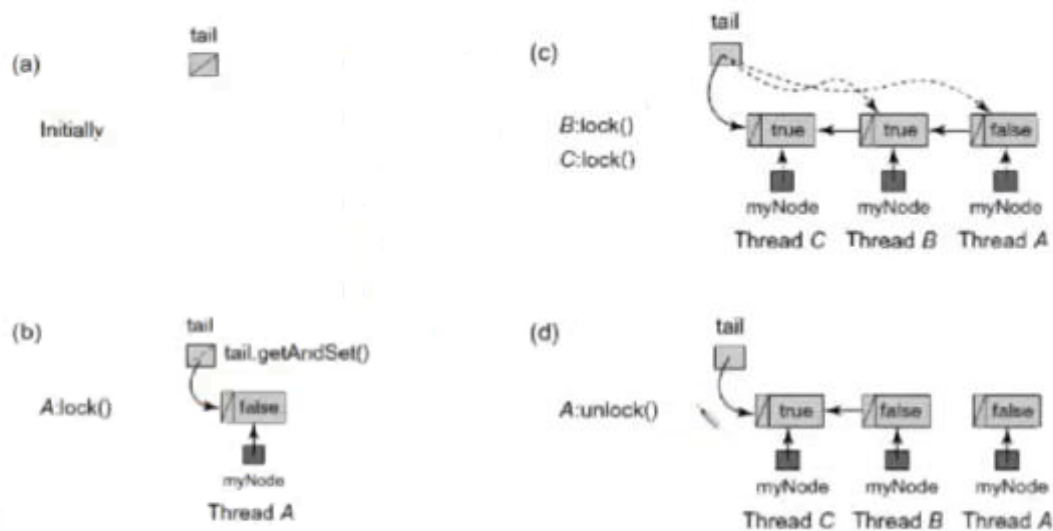
Come funziona il metodo lock ()?

La prima cosa da fare è prendere il proprio nodo (Riga 2), viene così riferito da una variabile non ThreadLocal, e viene messo in coda, cioè la tail punta a questo nodo. Abbiamo la tail, un thread A che prende il proprio QNode e fa in modo che la tail punti a un certo valore. Contemporaneamente preleva il pred (Riga 3), il quale se esiste, è il valore a cui prima puntava la tail. Se il pred non è null, si mette il campo locked del proprio QNode a true (Riga 5) e si fa puntare il next del pred a QNode (Riga 6). A questo punto si fa spinning sul proprio QNode, dove questo viene eseguito se pred non è uguale a null. Se pred è uguale a null significa che non c'era nessuno in coda. Alla fine, while(qnode.locked) informa il pred sulla sua entrata e lo attende in spinning.

Come funziona il metodo unlock ()?

Nel momento in cui si legge dal nodo, abbiamo la tail, il qnode, e si ha il campo del next che può essere null oppure non null. Se non è null non ci sono dubbi, cioè si va avanti perché c'è qualcun altro che è in coda. In tal caso nel campo locked (Riga 18) del campo next si deve mettere locked a false e next a null. La complicazione si presenta quando il valore qnode.next.locked = null, perché si è portati a dire che se questo è null significa che si è ultimi in coda, quindi non bisogna avvisare nessuno. Purtroppo, non è così perché potrebbe essere che c'è qualche nodo che ha appena scambiato la tail. Se c'è un nodo che ha fatto la prima operazione (Riga 14) ma non ha fatto la seconda operazione, si ha null. Si deve vedere se la tail punta ancora a me, allora non c'è nessuna situazione strana. Se si pensa di essere la tail viene fatta una tail.compareAndSet(qnode, null) e si verifica se si è ultimi, in tal caso effettivamente si mette null. Se c'è la situazione problematica, cioè che compareAndSet() fallisce, allora in tail non è qnode. Se succede questo significa che c'è qualcuno sta entrando e si aspetta fino a quando non cambierà il next, perché quando cambia il next non si è ultimi in coda e si può effettuare l'operazione finale (Riga 18-19). Non c'è rischio di deadlock perché il thread entra e aspetta che il thread esca dalla sezione critica. Il thread che deve entrare deve mettere il next, il thread presente in sezione critica aspetta che l'altro thread metta il next e quando lo fa esce, liberando la sezione critica per il thread successivo.

Se il pred è null non ha nessuno da avvisare/sbloccare, però ha cambiato la tail. Avendo cambiato la tail, chiunque viene dopo prenderà il nodo che è andato a mettere in tail e inizierà a costruire la coda.



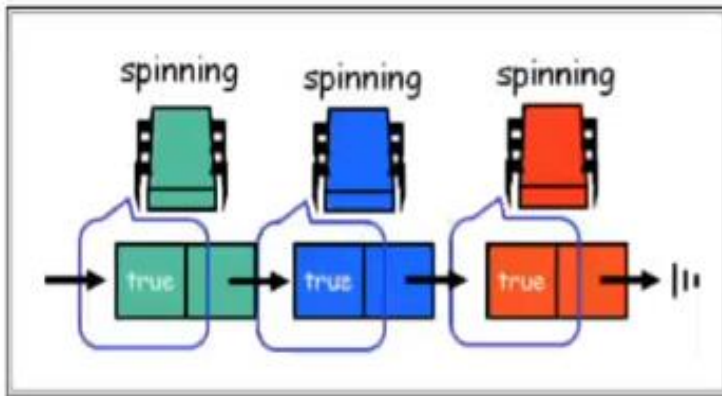
Dall'esempio vediamo che ci sono tre operazioni di lock: Thread A, Thread B, Thread C. Ognuno punta a quello successivo e nel momento in cui il thread A fa l'`unlock()`, sblocca il thread B che può entrare. Quindi è stato risolto il problema di NUMA, perché ogni nodo ha il controllo della posizione dove fa spinning (se è locale è più efficiente nelle cache-less NUMA). Si riciclano i nodi, cioè il nodo `qnode` rimane sempre lo stesso. Inoltre, si fa spinning per fare l'`unlock()`, il quale può essere ritardato da un altro thread che stava entrando mentre l'altro thread stava uscendo dalla sezione critica. Richiede più operazioni del CLHLock.

5.3 Altri Locks

5.3.1 Locks con Timeout

L'interfaccia `Lock` in Java comprende un metodo `tryLock()`, che consente al chiamante di specificare un timeout: la durata massima che il chiamante è disposto ad aspettare per l'acquisizione del lock. Se il timeout scade prima che il chiamante acquisisca il lock, il tentativo viene abbandonato. Un valore booleano viene restituito per indicare se il tentativo è andato a buon fine o meno. Il timeout è wait-free, richiede solo un numero costante di passi. Come nel CLHLock, il lock è una coda virtuale di nodi, e ogni thread gira sul suo predecessore in attesa che il blocco venga rilasciato. Come è noto, quando ad un thread scade, il timeout non può semplicemente abbandonare la coda, altrimenti il suo successore non sarà mai avvisato quando il lock verrà rilasciato.

Un Esempio:



Abbiamo una coda di lock. I tre thread stanno facendo spinning, dove il thread verde acquisisce il lock, il blu acquisisce il lock, lo stesso fa anche il thread rosso. Il problema sorge quando il thread blu esce. A tal punto quando il thread verde esce e segnala, non c'è più il thread blu che entra e il thread rosso non ha nessuna informazione che il thread blu è uscito, quindi può rimanere bloccato.

Quando un thread lascia la coda non

può semplicemente abbandonare una coda. Non si hanno gli strumenti per gestire la concorrenza.

Prendiamo un approccio pigro (Lazy): quando ad un thread scade il timeout, segna il suo nodo come abbandonato. Il suo successore in coda si accorge che il nodo su cui fa spin è stato abbandonato, e sposta la sua attenzione sul predecessore del nodo che ha abbandonato la coda. Questo approccio ha il vantaggio aggiunto che il successore può riciclare il nodo abbandonato.

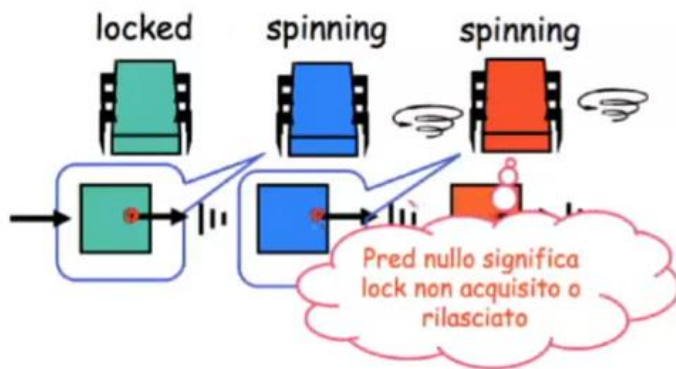
Si utilizza una struttura nodo `QNode` con un campo `pred`. Il campo `pred` deve indicare tre cose:

- Quando `pred` è null allora il thread non ha acquisito il lock, oppure lo ha rilasciato;
- Quando `pred` punta ad un nodo AVAILABLE, allora il thread ha rilasciato il lock;
- Quando `pred` si riferisce ad un nodo `QNode` che non sia AVAILABLE, allora il thread ha abbandonato e il successore dovrebbe seguire il `pred` per fare spinning sul predecessore;

In pratica, a parte casi speciali, il `pred` indica che il nodo è da "saltare" (il suo thread ha abortito). La lista linkata che si crea serve solamente per permettere il risalire ai thread che hanno abortito la richiesta di accesso. La `tail` accoda i nodi che non sono linkati in lista, lo sono solo se il thread ha abbandonato.

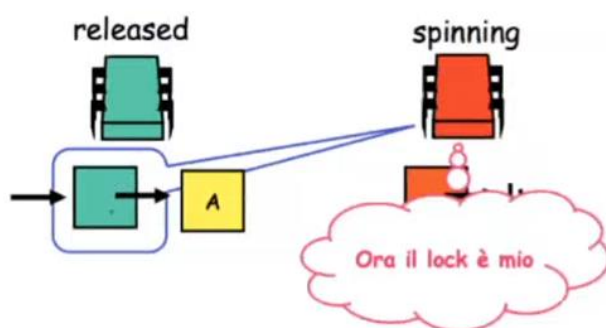
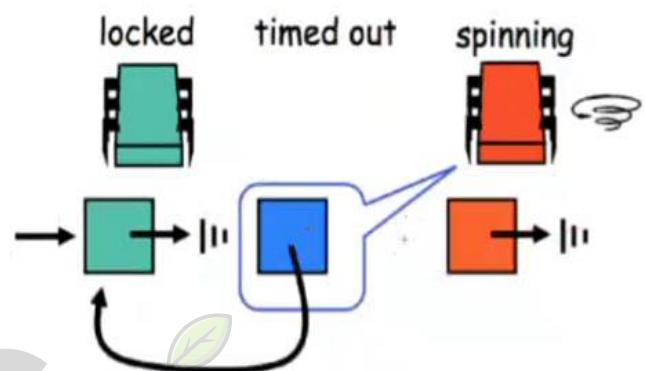


La `tail` punta a un nodo, e questo ha come campo `pred` AVAILABLE, cioè significa che il lock è disponibile. Il thread viola è idle. A tal punto il thread viola vuole entrare, lo fa creando un nodo in cui il campo `pred` viene messo a null, effettua lo swap atomico. Il thread viola adesso punta al campo verde, il cui campo `pred` punta ad AVAILABLE, cioè significa che la sezione critica è libera. La `tail` punta a un nodo il cui campo `pred` è null. A questo punto il thread viola sa che il lock può entrare, e così entra.



Riguardo al problema del thread blu che usciva, essenzialmente ci permetterà di usare il `pred` in maniera intelligente. In questo momento ogni `pred` è null. Il thread fa spinning sul nodo blu e vede che lock non è stato acquisito o rilasciato.

Il thread blu a questo punto esce, settando il proprio thread al nodo che lo seguiva. Il thread rosso, che stava facendo spinning, vede che il `pred` non è null e neanche AVAILABLE. Questo significa che il thread predecessore su cui stava facendo spinning risale la catena e va a fare spinning sul campo verde. La cosa importante è questo comportamento innaturale, cioè la lista linkata delle priorità è fatta implicitamente. La lista linkata non serve mai, tranne per risalire la catena dei nodi. Per questo viene considerata innaturale.



A questo punto il nodo, quando il thread verde rilascia esso, punta ad AVAILABLE e lo spinning sa che il lock è il suo, quindi può entrare nella sezione critica.

Come funziona TOLock?

```
1 public class TOLock implements Lock{
2     static QNode AVAILABLE = new QNode();
3     AtomicReference<QNode> tail;
4     ThreadLocal<QNode> myNode;
5     public TOLock() {
6         tail = new AtomicReference<QNode>(null);
7         myNode = new ThreadLocal<QNode>() {
8             protected QNode initialValue() {
9                 return new QNode();
10            }
11        };
12    }
13    ...
14    static class QNode {
15        public QNode pred = null;
16    }
17 }
```

Figure 7.15 TOLock class: fields, constructor, and QNode class.

Innanzitutto, come al solito implementa l'interfaccia Lock. Si crea un nodo AVAILABLE, che è statico, cioè uguale per tutti gli altri. È presente la variabile atomica tail e la variabile ThreadLocal myNode. Il costruttore QNode setta la tail e crea un nuovo QNode(), che mette all'interno myNode.

Come funziona tryLock?

```
1 public boolean tryLock(long time, TimeUnit unit)
2     throws InterruptedException {
3     long startTime = System.currentTimeMillis();
4     long patience = TimeUnit.MILLISECONDS.convert(time, unit);
5     QNode qnode = new QNode();
6     myNode.set(qnode);
7     qnode.pred = null;
8     QNode myPred = tail.getAndSet(qnode);
9     if (myPred == null || myPred.pred == AVAILABLE) {
10         return true;
11     }
12     while (System.currentTimeMillis() - startTime < patience) {
13         QNode predPred = myPred.pred;
14         if (predPred == AVAILABLE) {
15             return true;
16         } else if (predPred != null) {
17             myPred = predPred;
18         }
19     }
20     if (!tail.compareAndSet(qnode, myPred))
21         qnode.pred = myPred;
22     return false;
23 }
24 public void unlock() {
25     QNode qnode = myNode.get();
26     if (!tail.compareAndSet(qnode, null))
27         qnode.pred = AVAILABLE;
28 }
29 }
```

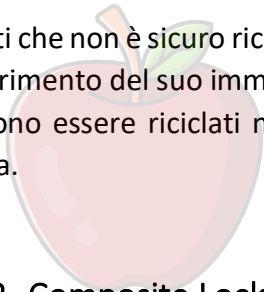
Figure 7.16 TOLock class: tryLock() and unlock() methods.

La `tryLock` ha un parametro `time` che indica la quantità di tempo e la `TimeUnit`. Setta lo `start time`, cioè il momento in cui si è partiti per fare `tryLock` e quanto tempo è in grado di aspettare. Una volta creato il nodo, lo si mette in `ThreadLocal`, viene settato `pred` a `null` (Riga 7) e si mette in coda. Se il lock è libero (Riga 9), il thread entra nella sezione critica. In caso contrario, fa spin in attesa che il campo del suo predecessore cambi (Riga 12-19). Se il timeout del thread predecessore scade, imposta il campo `pred` al suo predecessore e il thread fa spin sul nuovo predecessore. Infine, se il thread fa spin su sé stesso (Riga 20), tenta di rimuovere il suo `QNode` dalla lista applicando `compareAndSet()` al campo `tail`. Se la `compareAndSet()` ha esito negativo, indicando che il thread ha un successore, il thread setta il campo `pred`, precedentemente a `null`, al `QNode` del suo predecessore, indicando così che ha abbandonato la coda.

Nel metodo `unlock()`, il thread controlla, usando `compareAndSet()`, se ha un successore (Linea 26), e se così setta il suo campo `pred` ad `AVAILABLE`. Ci sono alcuni aspetti positivi:

- Spinning in locale su cache;
- Veloce riconoscimento che il lock è libero;
- Allocazione di un nuovo nodo per ogni thread;
- Si deve risalire la catena di thread usciti prima di poter accedere alla sezione critica;
- Si fa spinning su memoria di qualcun altro;

Si noti che non è sicuro riciclare un vecchio thread in questo punto, visto che il nodo potrebbe essere il riferimento del suo immediato successore, o di una catena di riferimenti. I nodi in una tale catena possono essere riciclati non appena un thread scavalca i nodi in timeout ed entra nella sezione critica.



Coscienze
Associazione

5.3.2 Composite Locks

Lo Spin lock quindi impone dei trade-off, tra i quali:

- **Queue lock:** FCFS, fast lock release, bassa contesa ma complessità nel riciclare i nodi;
- **Backoff lock:** semplici, ma non scalabili e con problemi di rilascio se il timeout non è ben settato;

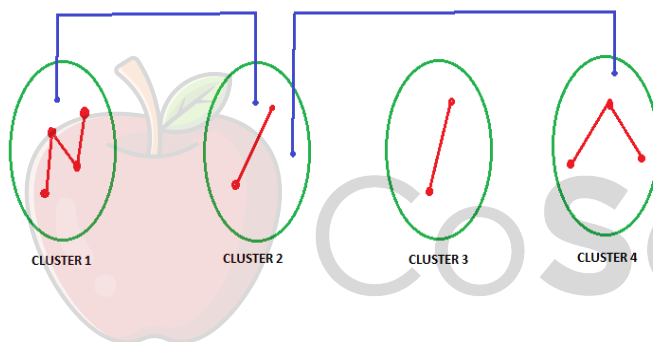
I composite lock combinano le considerazioni precedenti. L'idea è che, in una coda di lock, solo quelli che sono in testa alla coda devono accedere al lock. Il concetto è che solo un piccolo numero di nodi è in coda, mentre il resto usa backoff esponenziale per accedere a questa coda di accesso. Una parte di quelli che sono in testa formano una coda, mentre gli altri cercano di entrare in questa coda facendo il lavoro di selezionare un ritardo a caso, aspettare in sleep per un certo tempo e poi provare ad accedere alla coda. È presente un piccolo array, che serve per la coda, con inserimento in coda selezionando a caso uno slot. Se è occupato si fa backoff e si riprova. La gestione dei nodi liberati non è semplice, cioè il concetto è che esiste un nodo che è liberato e bisogna avere delle etichette per dire che un nodo è libero e non occupabile. Il problema dei Composite Locks è che con un solo thread i passaggi del composite sono troppo lenti.

La soluzione è fare in modo che un thread provi prima un fast-path. Se trova il lock libero entra, altrimenti significa che c'è contesa ed esegue il composite tradizionale. I composite sono una miscela di due algoritmi dove il secondo, con le random backoff, serve per entrare nella coda rappresentata dal primo e dove l'aggiunta del fast-path permette di poter provare prima ad accedere prima di entrare. Con il fast-path quindi si aggiunge uno stato al lock che distingue se è usato da un thread normale o uno con fast-path

5.3.3 Hierarchical Locks

Le gerarchie di memoria sono raramente piatte, ma tipicamente organizzate in maniera gerarchica. Oggi molte architetture cache-coherent organizzano i processori in cluster, quindi ci sono comunicazioni (intra-cluster) che sono più veloci di altre (inter-cluster).

Esempio:



Ci sono vari cluster di nodi, dove la comunicazione interna è particolarmente veloce, mentre la comunicazione intra-cluster risulta più lenta. La comunicazione blu rende più tempo. Importante è progettare locks che rispettano il principio di località, cioè fare operazioni su thread locali in maniera efficiente.

Vorremmo progettare sistemi di lock sensibili alla locality. Questi dispositivi di lock sono chiamati gerarchici perché prendono in considerazione la gerarchia dell'architettura della memoria ed i costi di accesso.

5.3.3.1 Hierarchical Backoff Lock

I lock basati su test and set sono facilmente gerarchizzabili. HBOLock è un esempio poiché usa diversi backoff time per i thread locali da quelli remoti. Fare in modo che ci sia una priorità per i thread locali, con il pericolo di una starvation dei thread remoti.

5.3.3.2 CLH Hierarchical

Un'altra possibilità è rendere Hierarchical l'algoritmo delle code di CLH. In questo modo si gestisce un insieme di code locali ed una coda globale. Tutti i thread presenti all'interno dello stesso cluster

useranno la stessa coda per i thread interni sulla coda locale, cioè l'inserimento di nodi della coda locale viene fatta insieme per motivi di efficienza (ogni nodo farà spin lock su nodi locali). Le code vengono svuotate periodicamente da un nodo master che si occupa di inserire batch di richieste di lock dallo stesso cluster nella coda globale. Una parte dello spinning viene fatta in locale rispetto che in maniera globale.



Domande di Riepilogo

- Cosa è il meccanismo dello spinning?
- Cosa è il meccanismo del Blocking?
- Quale è la differenza tra spinning e Blocking? In quali situazioni una soluzione è migliore dell'altra?
- Quali sono i due problemi che sconsigliano l'utilizzo di algoritmi di lock come Filter, Bakery o Peterson?
- IN che maniera la dimostrazione di correttezza del lock di Peterson viene inficiata dalla mancanza di consistenza sequenziale?
- A cosa può essere addebitato la mancanza di consistenza sequenziale delle moderne architetture hardware/software?
- Perché le Memory Barrier sono costose in termini di tempo?
- Descrivere il Test-and-Set lock
- Descrivere il Test-and-Test-and-Set lock
- Perché TTAS risulta sperimentalmente più efficiente di TAS?
- Quali sono le motivazioni per introdurre il backoff nei lock, a seconda di high o low contention?
- Descrivere l'algoritmo di BackoffLock
- Perché si fa backoff solamente all'interno del ciclo while (cioè se il Test-and-Set non va a buon fine) e non all'esterno (cioè durante il Test)? Cosa succederebbe se facessimo il backoff fuori dal ciclo (anche magari rendendo più lenta la crescita del ritardo (passandolo da esponenziale a lineare, ad esempio))?
- Quali sono i problemi di BackoffLock che portano alla definizione delle code di lock?
- Quali sono le caratteristiche principali di queue lock che le rendono più efficienti rispetto agli algoritmi precedentemente visti?
- Cosa offre in più come funzionalità una qualsiasi queue lock rispetto agli altri tipi di lock?
- Descrivere l'algoritmo di Lock di Anderson (ALock)
- Perché sono critiche le variabili thread-local in ALock?
- Perché flag [] deve essere condiviso (e non thread-local) e, ciò nonostante, non crea problemi di efficienza?
- Descrivere il meccanismo di false sharing che può capitare con ALock e come può essere risolto
- Descrivere le motivazioni a CLH Lock
- Descrivere l'algoritmo di CLH Lock
- In che maniera in CLH si mantiene una lista "virtuale" di nodi per rappresentare la coda?
- Quali sono i vantaggi e gli svantaggi di CLH Lock? E rispetto ad ALock?
- Quali sono le motivazioni a MCSLock? E le differenze strutturali con CLH Lock?
- Descrivere l'algoritmo di MCSLock
- Perché MCSLock funziona meglio su architetture NUMA cache-less?
- In che cosa si differenzia la coda "gestita" da MCS con la coda "gestita" da CLH?

- Perché servono le code con timeout?
- Descrivere un meccanismo di BackoffLock con timeout
- Quali sono i problemi che l'introduzione di un timeout può provocare su una implementazione classica (A, CHL, MCS)
- Descrivere l'algoritmo TOLock
- Quale motivazione hanno i Composite Locks?
- Quale è l'idea di un Composite Lock?
- Perché serve il Fast Path?
- Perché sono necessari gli Hierarchical Locks? In che architettura parallela sono particolarmente utili?
- Descrivere gli Hierarchical Backoff lock
- Descrivere l'idea dei Hierarchical CLH lock



6. Monitor

6.1 Introduzione

I monitor sono una maniera strutturata per combinare dati e sincronizzare. Come una classe incapsula dati e metodi, un monitor incapsula dati, metodi e sincronizzazione in un singolo modulo. Esso sposta il carico della sincronizzazione dal codice che usa una struttura dati all'interno della struttura stessa. Risulta di facile utilizzo ma è sempre più cruciale la sua efficienza, proprio perché essendo usato da tanti è importante che sia anche realizzato con la massima efficienza.

Perché i Monitor?

Proviamo ad immaginare un'applicazione con due thread, un produttore e un consumatore, che comunicano attraverso una coda FIFO condivisa. I thread potrebbero condividere due oggetti: una coda non sincronizzata ed un blocco per proteggere la coda. Il produttore sembra essere qualcosa del genere:

```
1  mutex.lock();  
2  try {  
3      queue.enq(x);  
4  } finally {  
5      mutex.unlock();  
6  }
```

Abbiamo la possibilità di usare un oggetto che implementa l'interfaccia lock, quindi i metodi `lock()` e `unlock()`. Sapendo che si devono accodare gli oggetti in questa coda, con un metodo `enq(x)`. La cosa naturale è pesare che c'è un lock di guardia (Riga 1), che permette di poter eseguire in mutua esclusione le operazioni, perché appunto `enq(x)` è un'operazione che va fatta in mutua esclusione. Inizia la sessione critica, si fa l'operazione di `enq(x)` e in uscita si rilascia il lock. Sembra un'operazione positiva, ma in realtà ci sono una serie di problemi perché non si è in grado riconoscere cosa sta succedendo alla coda prima di effettuare quest'operazione. Tra i problemi abbiamo che:

- La coda è limitata, quindi il produttore non può aggiungere in una coda piena, inoltre lo stato della coda è interno e reso inaccessibile al chiamante;
- Se si aggiungono produttori/consumatori si deve essere certi che usino lo stesso lock e la stessa procedura. Ogni thread deve tenere traccia sia del blocco che degli oggetti della coda, e l'applicazione sarà corretta solo se ogni thread seguirà lo stesso blocco.

Un approccio corretto è quello di permettere ad ogni coda di gestire la sua sincronizzazione. La coda di per sé ha il proprio lock interno, acquisito da ciascun metodo quando viene chiamato e rilasciato quando termina. Se un thread tenta di accodare un elemento in una coda che è già piena, il metodo `enq()` di per sé è in grado di rilevare il problema, sospendere il chiamante, e riprendere il chiamante quando la coda ha spazio.

6.2 Monitor Lock e Condition

6.2.1 Definizione

Il lock è il meccanismo di base per garantire la mutua esclusione, solo un thread alla volta può avere il lock. Un monitor è una classe che esporta metodi che sono sincronizzati, in maniera tale che ogni accesso usa lo stesso lock per assicurare la mutua esclusione.

Che cosa succede se il lock non è accessibile?

In caso non sia possibile accedere al lock, il metodo può fare spin, può bloccarsi (permettendo ad un altro thread di essere eseguito) o fare una combinazione delle due azioni.

- **Spinning:** Efficiente per attese di breve durata;
- **Blocking:** Efficiente per attese di lunga durata (costo in tempo dello switch tra thread da ammortizzare). Quando le attese sono di lunga durata, l'overhead implicito nei passaggi di stato ha un costo;

Esempio di Interfaccia Lock:

```
1 public interface Lock {
2     void lock();
3     void lockInterruptibly() throws InterruptedException;
4     boolean tryLock();
5     boolean tryLock(long time, TimeUnit unit)
        Condition newCondition();
7     void unlock();
8 }
```

- Il metodo `lock()` blocca il chiamante fino a quando non acquisisce il lock;
- Il `lockInterruptibly()` agisce come metodo `lock()`, ma crea un'eccezione se il thread è interrotto mentre è in attesa. In qualche maniera c'è possibilità di buttare fuori dallo spinning;
- Il metodo `tryLock()` acquisisce il lock se è libero, e subito restituisce un valore booleano (`true`) che indica se ha acquisito il lock. Questo metodo può anche essere chiamato con un timeout;
- Il `newCondition()` è una factory che crea e restituisce un oggetto `Condition` associato al lock;
- Il metodo `unlock()` rilascia il lock;

Mentre un thread è in attesa che accada qualcosa, per esempio che un altro thread sistemi un elemento in una coda, è una buona idea rilasciare il lock sulla coda, perché altrimenti l'altro thread non sarà mai in grado di accodare l'elemento. Il thread ha però bisogno di un metodo che ha il compito di informarlo quando deve riacquisire il lock e riprovare. Nel pacchetto di concorrenza di Java, la capacità di rilasciare un lock temporaneo è fornito da un oggetto `Condition` associato al lock.

6.2.2 Condition

Quando un thread è in attesa che qualcosa accada, può essere utile rilasciare il lock per dare la possibilità ad altri thread di far accadere quel qualcosa. Il fatto che un thread che provi ad entrare sia bloccato, può limitare la capacità dell'intero programma di fare progresso perché l'entrata di quel thread specifico avrebbe potuto sbloccare una situazione. Ad esempio, nella coda potrebbe esserci un enqueue, cioè un thread che sta provando a mettere degli elementi quando altri elementi stanno provando ad estrarre. Se tali elementi, che stanno provando ad estrarre, sono all'interno della sezione critica, prevengono chi potrebbe farli uscire dall'attesa di un elemento inserito. Un thread dovrebbe essere in grado di ricevere notifiche, ed è questo quello che fa la Condition, cioè crea un oggetto a partire dal quale è possibile mettere un thread in attesa di ricevere delle notifiche. Questo è un semplice meccanismo di comunicazione ad eventi, specializzato nelle Condition legate al lock. È presente in `java.util.concurrent.locks`. Ogni Condition è associata ad un lock tramite la factory `newCondition()`. Una chiamata ad un metodo `await()` rilascia il lock e sospende il thread. Quando un thread acquisisce un lock, vede che vuole fare un'operazione che non è possibile farla, allora vuole aspettare chiamando un `await()`. La `await()` rilascia il lock e poi sospende il thread. Il thread viene prima risvegliato, restituendo eventualmente un'eccezione in caso di `interrupt()`, poi riacquisisce il lock (in competizione con gli altri).

Come si usano le Condition:

```
1 Condition condition = mutex.newCondition();
2 //...
3 mutex.lock()
4 while(!property){
5     try {
6         condition.await(); // wait for property
7     } catch (InterruptedException e) {
8         //...
9     }
10 }
11 //...
```

Abbiamo il lock. È dal lock che si generano le Condition. A questo punto si acquisisce il lock e mentre la property non vale, si continua a fare `await()` ed eventualmente ci fosse stata un'interruzione da un `await()` con un `interrupt` e non con una signal, nella cache si può scrivere tutto quello che si può fare in tali condizioni. A questo punto la proprietà vale e il thread è in sezione critica. Se il thread fosse sospeso senza rilasciare il lock, nessun altro potrebbe acquisire il lock. Quindi si rilascia il lock, si va in stato blocked, quando si riceve un segnale si risveglia il thread e si prova a riacquisire il lock.

Cosa si può fare con la Condition:

```
1 public interface Condition {
2     void await() throws InterruptedException;
3     boolean await(long time, TimeUnit unit)
4         throws InterruptedException;
5     boolean awaitUntil(Date deadline)
6         throws InterruptedException;
7     long awaitNanos(long nanosTimeout)
8         throws InterruptedException;
9     void awaitUninterruptibly();
10    void signal();
11    void signalAll();
12 }
```

- L' `await()` , cioè attende fino ad una interruzione (`signal`, `interrupt`);
- Chiedere di fare una `await` per un certo tempo o fino a una certa data oppure per un tempo molto breve;
- Fare una `awaitUninterruptibly()` significa senza possibilità di essere interrotta;
- `signal()` significa risveglia uno dei thread che erano in `blocked`, mentre `signalAll()` significa risveglia tutti i thread che sono su tale `Condition`;

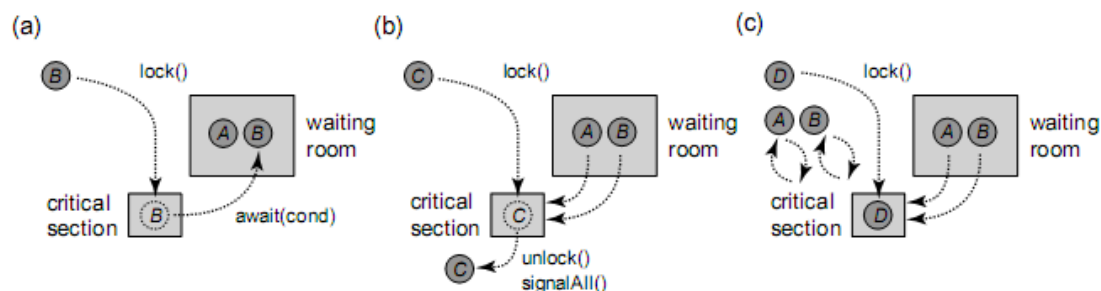
Quando si invoca un metodo `await()` il lock associato alla `Condition` viene rilasciato in maniera atomica e il thread viene sospeso fino a quando capita una delle seguenti cose:

- Qualche thread invoca `signal()` ed il thread viene scelto per il risveglio;
- Qualche thread invoca `signalAll()`;
- Qualche thread manda un `interrupt()` al thread;
- Il tempo specificato è passato;
- Occorre uno spurious wakeup;

Cos'è un Wakeup Spurio?

È un qualcosa che viene data come libertà alla piattaforma sottostante alla JVM. Uno spurious wakeup può occorrere come concessione alla semantica della piattaforma sottostante (sistema operativo e hardware sottostanti alla Virtual Machine). Significa che il wakeup non è generato da niente. Si sveglia, nonostante non c'è stato né un `signal()`, `signalAll()` o un `interrupt()`. Il programmatore attende i wakeup in un loop, in questa maniera gli implementatori della macchina virtuale permettono dei wakeup che possono occorrere anche in presenza di `signal()` o `interrupt()`.

Esempio di Monitor:



Nell'esempio (a) entrano A e B e si mettono in `await()`, in attesa di `signal()`. Nell'esempio (b) entra C ed esce segnalando con `signalAll()`. A questo punto i thread vengono risvegliati. Nell'esempio (c) A e B vengono risvegliati e cercano di acquisire il lock, a intanto lo acquisisce D e A e B fanno spinning.

Una delle cose più classiche che servono è la gestione di una coda FIFO in un monitor. Ogni metodo lo acquisisce all'inizio e lo rilascia al termine. Ci sono due condizioni da gestire:

- **notFull**: Una condizione che serve a segnalare che un elemento è stato estratto dalla coda, quindi qualche thread che era in attesa di inserire in una coda piena può essere avvisato;
- **notEmpty**: Una condizione che serve a segnalare che un elemento è stato inserito in coda, e quindi qualche thread che era in attesa di estrarre un elemento può essere avvisato;

Coda FIFO:

```
1  class LockedQueue<T> {
2      final Lock lock = new ReentrantLock();
3      final Condition notFull = lock.newCondition();
4      final Condition notEmpty = lock.newCondition();
5      final T[] items;
6      int tail, head, count;
7      public LockedQueue(int capacity) {
8          items = (T[])new Object[capacity];
9      }
10     public void enq(T x) {
11         lock.lock();
12         try {
13             while (count == items.length)
14                 notFull.await();
15             items[tail] = x;
16             if (++tail == items.length)
17                 tail = 0;
18             ++count;
19             notEmpty.signal(); //notifica che la coda non è vuota
20         } finally {
21             lock.unlock();
22         }
23     }
24     public T deq() {
25         lock.lock();
26         try {
27             while (count == 0)
28                 notEmpty.await();
29             T x = items[head];
30             if (++head == items.length)
31                 head = 0;
32             --count;
33             notFull.signal();
34             return x;
35         } finally {
36             lock.unlock();
37         }
38     }
39 }
```

Il lock è all'interno della classe, quindi strettamente all'interno della classe. Su questo lock, tramite le factory, vengono create due Condition: `notFull` e `notEmpty`. C'è un array di tipo `T` e `tail`, `head` e `count` che sono i valori all'interno dell'array circolare. La costruzione è semplice, perché si deve

solo creare un array di capacity di oggetti di tipo T. Riguardo il metodo `enq(T x)` (Riga 10), la prima cosa che fa è acquisire il metodo `lock()`. Acquisito il lock, esso verifica se può accodare l'elemento. Per farlo verifica se il numero di elementi attualmente che è all'interno (`count`) è uguale alla lunghezza dell'item (Riga 13). Questo sta in un `while` perché si mette in attesa sulla Condition `notFull` e ciò significa che rilascia il lock e poi si blocca. Questo è cruciale perché, rilasciando il lock, permette a qualcuno che fa `dequeue` di estrarre gli elementi. Quando viene risvegliato, riacquisisce il lock e quando lo riacquisisce controlla nuovamente il `while`: se la condizione è ancora vera torna in `await()` (Riga 14), altrimenti prosegue. A questo punto ci si trova con il thread che potrebbe essere stato in sala di attesa, ma quando ha acquisito il lock è sicuro che la property è verificata (Riga 15). Questo significa che può operare, cioè mettere l'elemento `x` in `tail`, aggiornare `tail` in aritmetica modulo `items.length`, incrementare `count` e avendo inserito un elemento, può segnalare che la coda non è più vuota (Riga 19). Alla fine, in ogni caso si rilascia il lock (Riga 21).

Il metodo `deq()` (Riga 24) non fa altro che estrarre un elemento e restituirlo, cerca di acquisire (Riga 25) e deve controllare se sta facendo una `dequeue` da una coda vuota (Riga 28). In tal caso si mette in `await()` e quando viene risvegliato, esso prova facendo `spinning` ad acquisire il lock. Quando acquisisce il lock, controlla la condizione e successivamente se essa non è più vera allora è in grado di estrarre, così si fa una segnalazione (Riga 33). Alla fine, si restituisce `x` (Riga 36) e in ogni caso si rilascia il lock.

6.2.3 Lost Wakeup

Il concetto è simile alla relazione che c'è tra il lock e deadlock. Lo stallo è generato dal fatto che si può imporre la possibilità di poter bloccare un thread. il thread A blocca una risorsa `x` e cerca di accedere a `y`, il thread B contemporaneamente blocca la risorsa `y` e cerca di accedere alla risorsa `x`. Entrambi sono bloccati e non si ha successo. Le Condition, se usate male, possono essere soggette al problema dei `lost wakeup`.

Per `Lost Wakeup` si intende thread che erano in attesa di una condizione che si è verificata ma di cui hanno perso la notifica. Si tratta di un problema algoritmico e non di implementazione.

```
1  //...
2  public void enq(T x) {
3      lock.lock();
4      try {
5          while (count == items.length)
6              notFull.await();
7          items[tail] = x;
8          if (++tail == items.length)
9              tail = 0;
10         ++count;
11         if (count == 1) { // Wrong!
12             notEmpty.signal();
13         }
14     } finally {
15         lock.unlock();
16     }
17 }
18 //...
```


È stata apportata una modifica al metodo `enqueue(T x)`. Il metodo è identico, tranne per il fatto che se la coda è appena passata da vuota a non vuota (con l'elemento che ha inserito il thread) allora segnala `notEmpty`. La concorrenza avviene tra produttori e consumatori multipli. Ci sono diversi produttori e consumatori che stanno accedendo alla coda.

Consideriamo il seguente scenario: vi sono due consumatori A e B che cercano di fare `dequeue` di un elemento da una coda vuota. Appena visto che la coda è vuota, si bloccano in attesa della condizione `notEmpty`. Ora il produttore C accoda un elemento nel buffer e segnala `notEmpty`, svegliando A. Prima che A acquisisca il lock, un altro produttore D mette una seconda voce nella coda, e poiché la coda non è vuota, il segnale `notEmpty` raggiunge A. Allora A acquisisce il lock, rimuove il primo elemento, ma B, vittima del risveglio perso, attende sempre, anche se vi è un elemento nel buffer che può essere consumato. Questa modifica apportata, che sembra ragionevole, presenta quindi problemi. Per risolvere in parte questo problema si può:

- Inviare a tutti i processi una signal che sono in attesa di una condizione, non ad uno solo;
- Specificare un timeout nell'attesa, cioè fare una `await()` con un timeout per verificare se la property è cambiata;

6.2.4 I Semafori

Un semaforo è una generalizzazione di un lock con mutua esclusione. Ogni semaforo ha una *capacity*, indicata con *c*. Invece di lasciare un solo thread alla volta nella sezione critica, un semaforo consente al massimo *c* thread. Tale capacità è determinata quando il semaforo è inizializzato. La classe `Semaphore` fornisce due metodi:

- `acquire()` permette di entrare in sezione critica se ci sono già thread in sezione critica, allora il thread è sospeso fino a quando non c'è spazio, cioè rimane a fare spinning;
- `release()` permette di far uscire dalla sezione critica;

Classe `Semaphore`:

```
1  public class Semaphore {
2      final int capacity;
3      int state;
4      Lock lock;
5      Condition condition;
6      public Semaphore(int c) {
7          capacity = c;
8          state = 0;
9          lock = new ReentrantLock();
10         condition = lock.newCondition();
11     }
12     public void acquire() {
13         lock.lock();
14         try {
15             while (state == capacity) {
16                 condition.await();
17             }
18             state++;
19         } finally {
20             lock.unlock();
21         }
22     }
23 }
```

```

23
24     public void release() {
25         lock.lock();
26         try {
27             state--;
28             condition.signalAll();
29         } finally {
30             lock.unlock();
31         }
32     }
33 }

```

Abbiamo la capacity del codice, il numero di thread che sono attualmente in critical section (state), il lock che viene utilizzato, oltre alla condition la quale permette di verificare i thread in attesa. Il costruttore prende c e lo mette in capacity, poi crea un lock e una condition dal lock. Il metodo `acquire()` permette di entrare in un lock solo se ce ne sono c all'interno. La prima cosa che fa è operare sulle variabili in mutua esclusione. Il lock (Riga 13) ci permette di dire che si possono usare le variabili senza problemi di accesso concorrente. Si attende fino a quando la proprietà non sia stabilita, cioè se ci sono meno thread della capacità. Se state è uguale a capacity bisogna andare in `await()`, cioè significa che si rilascia il lock e si va nello stato blocked. Una uscita dalla `await()` significa che:

- Qualcuno ha mandato una `signal()` o una `signalAll()`;
- Si è nello stato Runnable ed è stato selezionato per l'esecuzione, e si è riusciti ad acquisire il lock;
- Si è ricontrollata la condizione del while e si è verificato che non è più vera;

A questo punto si è entrati e si può rilasciare il lock (Riga 20). Il metodo `release()` è più semplice, perché per uscire non bisogna testare nessuna condition. L'unica cosa da fare è quando si esce, fa sapere ai thread che sono in attesa del fatto che ce n'è uno in meno. In uscita poi viene fatto l'`unlock()`.

Qual è la motivazione per cui è stata aggiunta la Wakeup spurious?

Quando bisogna svegliare qualche thread, a volte è più veloce ed efficiente svegliare un gruppo di thread piuttosto che uno. Se l'implementatore vuole avere la possibilità di risvegliare gruppi di thread, può risvegliare anche un thread che non era da risvegliare, ma si trovava semplicemente vicino ad un thread che doveva essere risvegliato. È quindi la libertà che viene data al programmatore. Non è un bug, ma la possibilità di usare tecniche per poter risvegliare thread in gruppo.

6.3 Readers-Writers Lock

6.3.1 I Readers Writers

Il problema che si vuole risolvere con i monitor è quello dei readers-writers. Ci sono oggetti condivisi dove un insieme di metodi restituiscono solo informazioni mentre un altro insieme di

metodi ne modificano lo stato. Non c'è bisogno di sincronizzare i readers, mentre i writers devono accedere in mutua esclusione con readers e writers.

```
1 public interface ReadWriteLock {
2     Lock readLock();
3     Lock writeLock();
4 }
```

Il Lock si compone di due lock: `readLock()` e `writeLock()`. Da `ReadWriteLock()` è possibile prelevare il `readLock()` e `writeLock()`, usati per reader e writer. Il concetto è che nessun thread può acquisire il write lock se un thread ha il read lock o write lock, cioè un writer deve entrare da solo. Nessun thread può acquisire il read lock se un thread ha il write lock, cioè un reader può entrare se c'è un altro reader che ha acquisito il read lock.

6.3.2 Una Semplice Soluzione

```
public class SimpleReadWriteLock implements ReadWriteLock {
    int readers;
    boolean writer;
    Lock lock;
    Condition condition;
    Lock readLock, writeLock;

    public SimpleReadWriteLock() {
        writer = false;
        readers = 0;
        lock = new ReentrantLock();
        readLock = new ReadLock();
        writeLock = new WriteLock();
        condition = lock.newCondition();
    }

    public Lock readLock() {
        return readLock;
    }

    public Lock writeLock() {
        return writeLock;
    }

    // inner class ReadLock e WriteLock
}
```

Questa è l'implementazione dei due lock read e write. Con tutti i lock presenti è necessario un altro lock, perché si ha bisogno di poter accedere in mutua esclusione sia ai read lock che ai write lock. La presenza di un writer viene indicata con un booleano, mentre la presenza di readers viene indicata con un intero, specificando quanti sono i reader all'interno. L'inizializzazione fa in modo che viene inizializzato vuoto, con

readers a 0, poi vengono creati i `ReadLock()` e si estrae dal lock globale la condition, che si utilizzerà per accedere in mutua esclusione a tutto.

```

class ReadLock implements Lock {

    public void lock() {
        lock.lock();
        try {
            while (writer) {
                condition.await();
            }
            readers++;
        } finally {
            lock.unlock();
        }
    }

    public void unlock() {
        lock.lock();
        try {
            readers--;
            if (readers == 0)
                condition.signalAll();
        } finally {
            lock.unlock();
        }
    }
}

```

ReadLock è un lock tradizionale, il quale per arbitrare il metodo lock usa l'oggetto lock, di cui chiama il metodo lock(). La condizione per acquisire ReadLock è solamente sul writer, se c'è un writer all'interno bisogna aspettare facendo la await(). Se ci fossero dei reader, ciò non interessa e si prosegue. Si prosegue anche se writer non è più vero, perché in tal caso qualcuno segnala sulla condition, quindi si esce dallo stato blocked, si entra in uno

stato Runnable, si acquisisce il lock e si riesce così a verificare se writer non è più true. Se writer non è più true significa che c'è un reader in più. A questo punto si esce, rilasciando il lock che controlla gli accessi a read e write lock. Entrati nel blocco try viene decrementato readers e se i reader sono a 0 significa che i readers sono fuori. Se per caso ci fosse un writer in attesa, allora si segnala perché writer si metterà in attesa se readers era maggiore di 0. Questo serve per i writers e non per i readers perché i readers sono entrati. Alla fine, viene fatto l'unlock().

```

protected class WriteLock implements Lock {

    public void lock() {
        lock.lock();
        try {
            while (readers > 0 || writer) {
                condition.await();
            }
            writer = true;
        } finally {
            lock.unlock();
        }
    }

    public void unlock() {
        writer = false;
        condition.signalAll();
    }
}

```

Il lock del WriteLock controlla sia se c'è un writer, ma anche se il numero di readers è maggiore di 0. In tal caso va in attesa. Nel momento in cui entra, mette writer a true. Da questo momento nessun reader potrà entrare.

Il metodo unlock() funziona in maniera molto semplice. Il writer viene messo a false e

viene inviata una signalAll() perché si segnala a tutti indistintamente. Per primi non entrano i reader o writer, bensì dipende. Lo scheduler decide di mettere in esecuzione uno dei thread, senza decidere se siano readers o writers.

Il problema è che se i reader accedono più frequentemente dei writer, allora i writer potrebbero essere bloccati fuori dal lock. Ad esempio, se i reader possono accedere e il writer rimane fuori, prima o poi i dati finiranno, quindi i reader non leggeranno più, quindi il writer potrà scrivere. Da questo argomento ci sono due errori concettuali:

- Questo non è un produttore-consumatore, cioè produttore-consumatore significa che i dati vengono estratti. Essendo un reader-writer, un reader legge i dati ma non è detto che li estrae. Quindi non c'è il concetto di estrazione di dati, per cui significa che non è vero che prima o poi i dati finiscono e i reader non leggeranno più.
- I reader possono continuare a leggere, ma non hanno nessuna maniera per sapere che c'è un writer in attesa.

Una soluzione equa potrebbe essere che quando un writer chiama il `lock()`, nessun reader può acquisire il read lock, fino a quando il writer non ha acceduto.

6.3.3 Una Soluzione Equa tra Readers-Writers

```
public class FifoReadWriteLock implements ReadWriteLock {
    int readAcquires, readReleases;
    boolean writer;
    Lock lock;
    Condition condition;
    Lock readLock, writeLock;

    public FifoReadWriteLock() {
        readAcquires = readReleases = 0;
        writer = false;
        lock = new ReentrantLock();
        condition = lock.newCondition();
        readLock = new ReadLock();
        writeLock = new WriteLock();
    }

    public Lock readLock() {
        return readLock;
    }

    public Lock writeLock() {
        return writeLock;
    }

    // inner class ReadLock e WriteLock
}
```

Una soluzione viene vista attraverso `FifoReadWriteLock`. Vengono utilizzate due variabili: `readAcquires` e `readReleases`. Non si hanno più i readers, bensì il numero di readers che hanno acquisito il lock e il numero di readers che hanno rilasciato il lock. Quando `readAcquires` è uguale a `readReleases`, allora significa che non c'è nessun thread reader che ha il read lock. La costruzione è semplice.

Come funziona il ReadLock?

```
private class ReadLock implements Lock {  
  
    public void lock() {  
        lock.lock();  
        try {  
            while (writer) {  
                condition.await();  
            }  
            readAcquires++;  
        } finally {  
            lock.unlock();  
        }  
    }  
  
    public void unlock() {  
        lock.lock();  
        try {  
            readReleases++;  
            if (readAcquires == readReleases)  
                condition.signalAll();  
        } finally {  
            lock.unlock();  
        }  
    }  
}
```

Innanzitutto, si acquisisce il lock (`lock.lock()`). Il punto cruciale è che un writer, che forse è in coda, ferma i reader. Il meccanismo è lo stesso, con la solita `condition.await()`, in attesa che ci siano writer all'interno. La differenza è che si incrementa la variabile `readAcquires` anziché `readers`. Nel metodo `unlock()` si aumentano i rilasci (`readReleases`) e la condizione ci segnala se ci sono altri reader che la condition è libera.

Come funziona il WriteLock?

```
private class WriteLock implements Lock {  
  
    public void lock() {  
        lock.lock();  
        try {  
            while (writer) {  
                condition.await();  
            }  
            writer = true;  
            while (readAcquires != readReleases) {  
                condition.await();  
            }  
        } finally {  
            lock.unlock();  
        }  
    }  
  
    public void unlock() {  
        writer = false;  
        condition.signalAll();  
    }  
}
```

il WriteLock acquisisce il lock e se ci sono writer attende. La cosa differente è che quando non ci sono più writer, cioè se il writer non è più uscito, prosegue e mette a `true` la propria variabile, ciò significa il manifestare interesse ad entrare e altri thread reader che provano a entrare rimarrebbero bloccati non solo perché c'è un writer ma anche perché c'è un writer che intende entrare. Dopo la `writer = true`, si rimette in attesa, ma lo fa

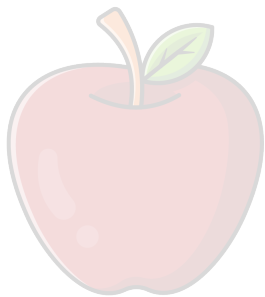
sull'uscita dei reader che stanno dentro e non più sui writer. Il primo writer che entrerà è quello che è in coda e ha acquisito il lock, mettendosi in attesa che escono i reader. Altri reader non entrano

perché si bloccano al `while(writer)` . I reader si accodano, non possono entrare, e ci si aspetta che escano i reader che stavano dentro, che aumentano `readAcquires` fino a raggiungere i `readReleases` . Quando `readAcquires` è uguale a `readReleases` viene lanciata una signal che risveglia il writer che era in attesa e che a questo punto può acquire il lock, ed è così dentro la critical section. Il metodo `unlock()` è molto semplice, in quanto segnala a tutti che è uscito un writer se per caso dovesse essere di aiuto sia per altri writer e reader in attesa.



Domande di Riepilogo

- Quale è il parallelo tra monitor e programmazione a oggetti e procedurale?
- Perché un monitor facilita la vita del programmatore, ma, se male implementato, può mettere a rischio la efficienza del programma?
- Quando è più appropriato per un thread fare spinning o blocking? E perché?
- Cosa è uno spurious wakeup? E quali strumenti abbiamo, come sviluppatori, per evitare che creino problemi alla nostra applicazione?
- Quale è la differenza tra lost wakeup e spurious wakeup?
- Perché segnalare a tutti i thread in attesa (signalAll) è utile per evitare i lost wakeup? Quale è il costo in cui incorriamo, invece di usare semplicemente signal ()?
- Quale è la differenza tra lock e semafori?
- Quali sono le condizioni di Readers-Writers che rendono necessaria una implementazione ad-hoc?
- È possibile realizzare Readers-Writers con una semplice sezione critica guardata da un lock? Se no, perché? Se sì, perché allora non usiamo quella che è più semplice?
- Perché FifoReadWriteLock per i Readers-Writers è più equa?



CoScienze
Associazione

7. Liste

7.1 Introduzione

Gli spin lock e i monitor suggeriscono come strategia una delle modalità di sincronizzazione.

7.1.1 Tipi di Sincronizzazione

- ❖ **Coarse-grained Synchronization:** Usare una struttura dati sequenziale e aggiungere lock ai metodi. È un approccio estremamente Safe. Si ottiene così una struttura dati concorrente e scalabile (per le qualità del lock). Oltre alle problematiche di concorrenza, sono presenti altri problemi:
 - Un singolo lock da condividere a tutti i metodi può non scalare quando il livello di concorrenza è alto;
 - Necessario introdurre diversi tipi di sincronizzazione;
- ❖ **Fine-grained Synchronization:** Aniché usare un singolo lock per tutte le attività di sincronizzazione, l'idea è cercare di rendere il meccanismo di locking più scalabile, aumentando la quantità di lock sulla quale si effettua la sincronizzazione. Si divide l'oggetto in componenti diverse, in modo che esse siano sincronizzate al proprio interno con un lock sincronizzato. In questa maniera operazioni su componenti diverse possono essere condotte concorrentemente e senza overhead (no lock). Identificare però le componenti può non essere agevole, perché bisogna conoscere la struttura.
- ❖ **Optimistic Synchronization:** È una tecnica che si applica alle strutture dati che usano puntatori (liste, code, alberi). Queste strutture hanno come caratteristica il fatto che si prevede un metodo di ricerca dati per effettuare operazioni, cioè per trovare la posizione dove deve essere fatta un'operazione; quindi, si fa una ricerca nella struttura dati. Se in una lista linkata o coda si vuole inserire o cancellare un certo elemento, bisogna prima trovarlo. L'accesso può essere previsto in maniera ottimistica, cioè si fa un accesso senza fare lock. Quando viene trovata la posizione, si effettua il lock del nodo. Se il metodo trova il componente ricercato, lo blocca e controlla che il componente non sia cambiato durante l'intervallo di tempo da quando è stato ispezionato a quando è stato bloccato. Risulta utile se si ha un tasso di successo alto.
- ❖ **Lazy Synchronization:** Il lavoro di correzione della struttura dati viene postposto in caso di conflitto, cioè si prova a prevedere una rimozione logica che viene seguita da una rimozione fisica. Si prevedono due livelli in cui l'operazione viene effettuata. Questo permette di gestire situazioni di conflitto.
- ❖ **Nonblocking Synchronization:** A volte è possibile eliminare tutti i lock, basandosi su operazioni atomiche come `compareAndSet()` per la sincronizzazione.

7.1.2 Struttura Dati per Insiemi

Un'interfaccia di un insieme è molto semplice:

```
1 public interface Set<T> {  
2     boolean add(T x);  
3     boolean remove(T x);  
4     boolean contains(T x);  
5 }
```

Un insieme di oggetti di un certo tipo prevede diverse funzionalità:

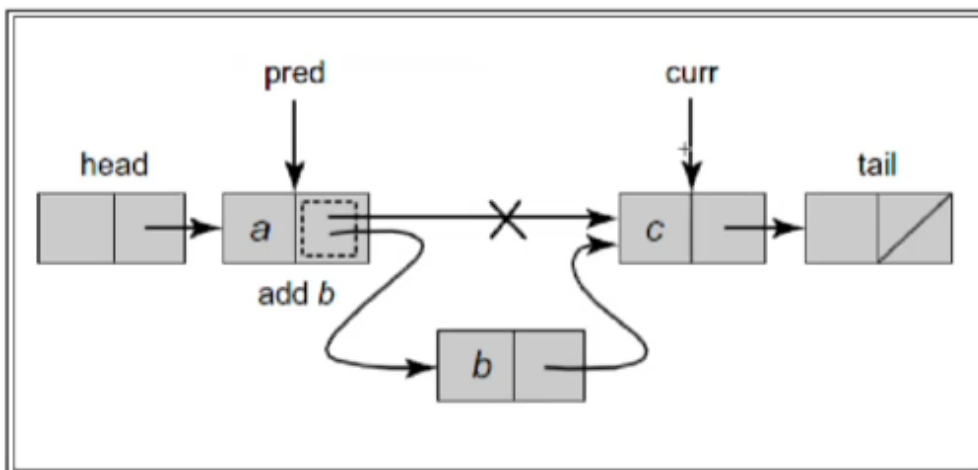
- `add(x)` : inserisce l'elemento `x` solo se non è già presente restituendo `true` in caso positivo, `false` altrimenti. La `add(x)` può fallire perché non si può inserire lo stesso elemento più volte;
- `remove(x)` : rimuove `x` dall'insieme e restituisce l'esito dell'operazione;
- `contains(x)` : restituisce `true` se l'elemento `x` è contenuto nell'insieme, `false` altrimenti.

Tutti e tre i metodi restituiscono un boolean, in quanto è previsto che l'operazione abbia o meno un risultato. Questo è rappresentato da una classe `Node`:

```
1 private class Node {  
2     T item;  
3     int key;  
4     Node next;  
5 }
```

Abbiamo un `item`, un intero che rappresenta la chiave. Ogni nodo ha la sua chiave e gli elementi vengono ordinati attraverso la `key`. `Next` è un puntatore al prossimo elemento all'interno della lista che rappresenta la struttura dati.

Inserimento in Lista

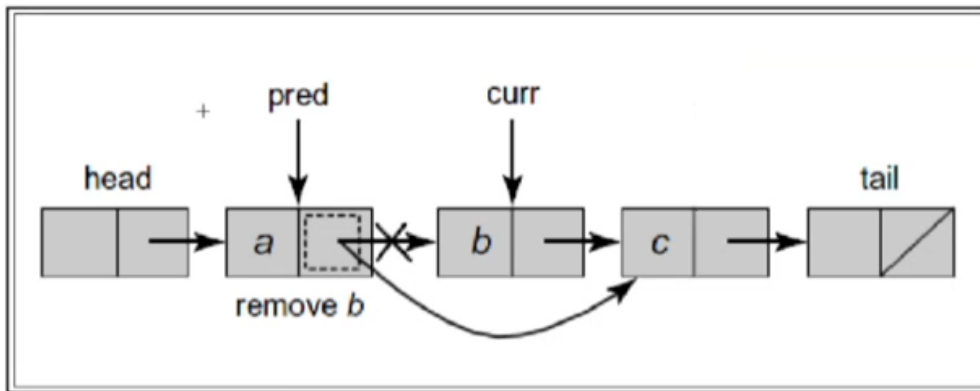


Abbiamo due tipi di nodi: quelli normali e due nodi sentinella che sono `head` e `tail`, rispettivamente il primo e l'ultimo elemento. Con l'inserimento in lista si identifica la posizione prima della quale `b` va inserito l'elemento. L'idea è che `curr` contiene il più piccolo elemento con chiave più grande dell'elemento da inserire. A questo punto il `pred`, cioè il nodo che punta al nodo `curr` in un certo

momento, fa l'aggiunta di b. l'inserimento consiste di due operazioni (operazioni che si possono interfogliare):

- Cancellare il puntatore;
- Far puntare il nodo b a c;

Cancellazione in Lista



Anche in questo caso si identifica lo stesso problema. Si cerca di identificare il nodo (curr) da eliminare. La cancellazione viene effettuata facendo in modo che, identificando il pred e curr, cioè il nodo che in quel momento punta a curr, viene fatta un'operazione di cancellazione, modificando il puntatore in modo da escludere il nodo b dalla lista.

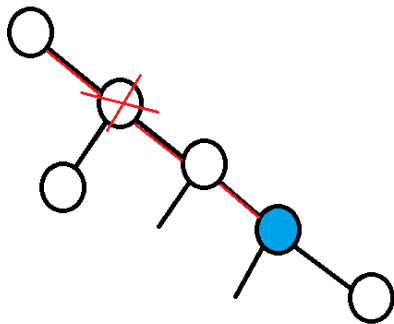
7.1.3 Problematiche della Concorrenza

Una volta anticipati i vari problemi, essi vengono formalizzati per utilizzarli successivamente all'interno della correttezza degli algoritmi.

Invarianti: Una proprietà P che vale per una struttura dati concorrente e che:

1. P vale quando l'oggetto è creato;
2. Nessun thread può rendere non vera P;

L'invariante è qualcosa di particolarmente importante per una struttura dati, perché è qualcosa su cui qualsiasi algoritmo può contare. È invariante una proprietà che serve all'algoritmo per operare correttamente. Nel Set si dimostrano gli invarianti preservati da `add()`, `remove()` e `contains()`. Tutto questo deve essere realizzato senza che ci sia interferenza dei thread, chiamata anche *freedom from interference*. Le invarianti devono valere anche per nodi cancellati, cioè i nodi cancellati che vengono attraversati da altri thread sono in grado di effettuare il riposizionamento in maniera corretta.



Supponendo di lavorare sul nodo blu, e si sta attraversando il percorso, il concetto è che se uno dei nodi viene cancellato, la cosa importante è che rimanga a disposizione di chi lo sta attraversando. Quindi non si fa riuso di nodi e ci si affida al Garbage Collector che prima o poi eliminerà il nodo, ma bisogna essere sicuri che il nodo che si sta attraversando non venga riusato dalla struttura dati perché potenzialmente viene usato per navigare all'interno.

Dimostrare, data l'implementazione, che i nodi sono ordinati per chiave.

Si hanno tutte le etichette ordinate. Se si suppone che siano etichette ordinate, le operazioni di add e remove mantengono le etichette ordinate perché la add identifica la più piccola chiave più grande della chiave che si sta inserendo. Se era ordinato prima, rimane ordinato anche ora e lo stesso meccanismo vale per la remove. Riguardo le invarianti del Set, abbiamo:

1. Sentinelle mai aggiunte o rimosse al set;
2. I nodi sono ordinati per chiavi;
3. Chiavi uniche;

Le invarianti vengono viste come contratti dei metodi: Ogni metodo si impegna a mantenerle e si basa sul fatto che gli altri metodi facciano lo stesso. Queste caratteristiche di definire le precondizioni e post condizioni servono per eliminare i side effect non voluti. In questa maniera, ogni metodo può considerarsi eseguito in isolamento.

7.1.3.1 Linearizzabilità

CoScienze
Associazione

Principio di Linearizzabilità: Il principio di linearizzabilità afferma che le invocazioni devono apparire come se avessero luogo istantaneamente in un qualche istante tra invocazione e risposta. Questo significa che l'applicazione, l'esecuzione, la resa evidente, la resa pubblica di una modifica all'interno della struttura dati deve apparire in un certo momento.

Linearizzabile: Una computazione concorrente che rispetta il principio di linearizzabilità si dice Linearizzabile.

Come vengono trovati i punti di linearizzabilità?

Quando ci sono sezioni critiche risulta facile usare la linearizzazione. Se si usano i lock per entrare in una certa operazione, quando si esce dal lock, al rilascio del lock quello è il punto in cui l'operazione è resa visibile. Se non ci sono sezioni critiche, c'è un momento preciso in cui il cambiamento è visibile. Ad esempio, nella coda circolare wait-free, quando head e tail sono modificati da `enq()` e `deq()`, il concetto era che la linearizzabilità era quando head e tail venivano modificate. La cosa utile è che la linearizzabilità include la consistenza sequenziale (più strettamente). Se si prova che i nostri algoritmi rispettano la linearizzabilità, allora si è dimostrato la consistenza sequenziale del nostro programma. Lavorando sulla linearizzazione si è in grado di definire la correttezza dell'algoritmo. Si vorrebbe utilizzare la linearizzabilità per definire anche la Safety.

7.1.3.2 Safety

Un elemento è nell'insieme se è raggiungibile partendo dalla head. Questa è chiamata *Abstraction Map*, cioè i nodi che sono raggiungibili partendo dalla head sono nella Abstraction Map. In un certo momento, se c'è un nodo che è stato cancellato, esso non fa parte dell'Abstraction map e quindi non c'è. Un thread che sta attraversando, se l'operazione è fatta in maniera concorrente, è in grado di ritornare sulla struttura. La Abstraction map usa i punti di linearizzazione per definire una rappresentazione delle operazioni sull'insieme. Ricordiamo che:

- **Wait-free:** ogni invocazione termina in un numero finito di passi.
- **Lock-free:** qualche invocazione sicuramente termina in un numero finito di passi.

Iniziamo ora a considerare una serie di algoritmi che trattano gli insiemi. Si comincia con algoritmi grossolani che utilizzano una sincronizzazione a grana fine, e successivamente affinarli per ridurre granularità di lock. In ognuno di questi algoritmi, i metodi effettuano la scansione utilizzando due variabili locali: curr è il nodo corrente e pred è il suo predecessore. Queste sono variabili locali del thread, e denoteremo currA e predA per indicare le istanze del thread A.

7.2 Coarse-Grain Synchronization

Si comincia con un semplice algoritmo utilizzando la sincronizzazione a grana grossa. La lista ha un unico lock che ogni metodo deve acquisire. Il maggior vantaggio di questo algoritmo che non dovrebbe essere scontato è che è corretto.

```
1 public class CoarseList<T> {
2     private Node head;
3     private Lock lock = new ReentrantLock();
4     public CoarseList() {
5         head = new Node(Integer.MIN_VALUE);
6         head.next = new Node(Integer.MAX_VALUE);
7     }
8     public boolean add(T item) {
9         Node pred, curr;
10        int key = item.hashCode();
11        lock.lock();
12        try {
13            pred = head;
14            curr = pred.next;
15            while (curr.key < key) {
16                pred = curr;
17                curr = curr.next;
18            }
19            if (key == curr.key) return false
20            else {
21                Node node = new Node(item);
22                node.next = curr;
23                pred.next = node;
24                return true;
25            }
26        } finally { lock.unlock(); }
27    }
28    public boolean remove(T item) {
29        Node pred, curr;
30    }
```

```

31         int key = item.hashCode();
32         lock.lock();
33         try {
34             pred = head;
35             curr = pred.next;
36             while (curr.key < key) {
37                 pred = curr;
38                 curr = curr.next;
39             }
40             if (key == curr.key) {
41                 pred.next = curr.next;
42                 return true;
43             } else {
44                 return false;
45             }
46         } finally {
47             lock.unlock();
48         }
49     }
50
51     public boolean remove(T item) {
52         //...
53     }
54 }

```

Nella classe `CoarseList` viene usato un lock (Riga 2), viene definito head e viene messo come chiave il minimo valore degli interi (Riga 5) e tail viene creato, facendo in modo che next punti a tail.

La prima operazione è quella della `add(T item)`. Si sta quindi cercando di inserire un certo elemento con una chiave `key`. Quello che si vuole è identificare una posizione (`pred` e `curr`) in maniera tale da poter fare le operazioni di modifica. Innanzitutto, viene fatto il lock, viene effettuato poi lo scorrimento della lista partendo da `pred` e `curr` che puntano a head e al nodo successivo. Fin quando `curr.key` è minore di `key` (Riga 15). Per poter scorrere si fa in modo che il `pred` diventa `curr`, e `curr` diventa `curr.next` (Riga 16). Una volta usciti dal `while`, si è identificato un valore `curr` tale che la chiave è maggiore o uguale di `key`. Adesso ci sono due possibilità: Se la chiave è uguale alla chiave dell'elemento da inserire. Ma in tal caso l'elemento già c'è e quindi ritorna a `false`. Altrimenti significa che la chiave è maggiore della chiave di `key` ed è la chiave più piccola perché è la prima a cui ci si è fermati. Per questo viene creato un nodo (Riga 21), mettendo la `key` e l'hashcode, e vengono fatte due operazioni. La prima operazione è quella di mettere in `node.next = curr`. La seconda operazione è `pred.next = node`. Le operazioni vengono fatte in mutua esclusione. Se non venissero fatte in mutua esclusione sarebbe critico fare la prima operazione e poi la seconda. Anche fare una operazione prima di un'altra, quando c'è concorrenza questo può far sballare il funzionamento.

La seconda operazione è la `remove(T item)`. In questo caso si deve cancellare un elemento, quindi si preleva l'hashcode dell'elemento perché è quello da ricercare, poi si effettua lo stesso meccanismo dell'operazione precedente (`add`). Quando si esce dal `while`, si ha un nodo `curr` tale che la chiave è maggiore o uguale di `key`. Ancora una volta le possibilità sono due, però in tal caso è speculare rispetto alla `add`, cioè l'operazione ha successo se effettivamente `key == curr.key` (Riga 40). In tal caso `pred.next == curr.next`. Questa operazione esclude dall'Abstraction map il nodo con la chiave `curr.key`. Questo nodo non viene riutilizzato perché deve essere disponibile per riportare sulla strada giusta nodi che sono rimasti fuori. Altrimenti, se `key > curr.key` significa che viene dato da cancellare un elemento che non c'è, quindi viene dato `false`.

L'operazione `contains(T x)` è simile alla `remove`, ma senza effettuare operazioni. Viene fatto il lock, poi si controlla se la chiave matcha e se è così restituisce `true`.

La classe `CoarseList` soddisfa la condizione di:

- **Progresso:** Se il Lock è starvation-free, lo è anche la nostra implementazione. Se la contesa è molto bassa, questo algoritmo è un ottimo modo per realizzare una lista. Se, tuttavia, vi è contesa, quindi anche se il lock si comporta bene, i thread continueranno ad essere ritardati in attesa l'uno dell'altro;
- **Safety:** É semplice perché i punti di linearizzazione corrispondono al momento in cui si acquisisce il lock;
- **Correttezza:** É ovvia perché è sequenziale;

7.3 Fine-Grain Synchronization

Cosa è la sincronizzazione a grana fine?

Invece di usare un singolo lock per le attività di sincronizzazione si divide l'oggetto in componenti diverse, che sono sincronizzate al proprio interno con un lock dedicato.

Obiettivo: operazioni su componenti diverse possono essere condotte concorrentemente e senza overhead (no lock). Siamo in grado di migliorare la concorrenza bloccando i singoli nodi, piuttosto che un lock complessivo della lista. Invece di piazzare un lock su tutta la lista, cerchiamo di aggiungere un lock ad ogni nodo. Quando un thread attraversa la lista, fa il lock di ogni nodo prima che venga visto, e dopo un pò di tempo lo rilascia. In questo modo permette a thread concorrenti di attraversare la lista tutti insieme in pipeline.

7.3.1 La Classe `Finelist`

Si usa un lock per ogni nodo, questo è necessario perché è la struttura dati che si ha a disposizione, però la cosa è un po' strana, dato che fare il lock di un solo nodo non basta, nel senso che bisogna avere due posizioni `pred` e `curr` per inserire un elemento. Per poter cancellare un elemento, quindi non lavoriamo con un solo nodo ma con più nodi, ed è chiaro che non si può fare altro che mettere un lock per ogni nodo, perché è la struttura dati che si ha a disposizione, non abbiamo una struttura dati che esprime le coppie di nodi. Ogni operazione sui nodi usa il lock sui vari nodi incontrati, in questa maniera più operazioni (indipendenti) possono essere compiute concorrentemente sulla lista, e questo porta sì maggiore efficienza, ma anche maggiore attenzione nella implementazione.

Lock—Coupling

Il meccanismo fondamentale che viene utilizzato è il *Lock-Coupling*, ossia il meccanismo che prevede di acquisire il lock di un nodo solo quando si è acquisito quello del suo predecessore. Prendiamo in

considerazione due nodi a e b dove a punta a b. Non è sicuro unlockare a prima di lockare b perché un altro thread potrebbe rimuovere b dalla lista durante l'intervallo tra la fase di unlock di a e lock di b. Invece, il thread A deve acquisire i lock in una sorta di ordine "hand-over-hand": tranne che per il nodo iniziale, bisogna acquisire il lock per CurrA solo mentre si tiene il lock per PredA.

Questo protocollo di lock è chiamato *Lock Coupling*, e si è sicuri che le operazioni dipendenti vengono serializzate.

Metodo add():

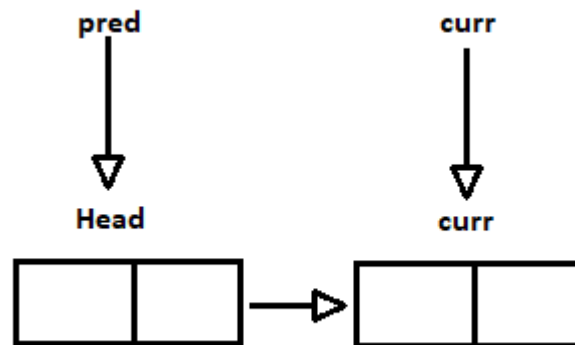
```
1  public boolean add(T item) {
2      int key = item.hashCode();
3      head.lock();
4      Node pred = head;
5      try {
6          Node curr = pred.next;
7          curr.lock();
8          try {
9              while (curr.key < key) {
10                 pred.unlock();
11                 pred = curr;
12                 curr = curr.next;
13                 curr.lock();
14             }
15             if (curr.key == key) {
16                 return false;
17             }
18             Node newNode = new Node(item);
19             newNode.next = curr;
20             pred.next = newNode;
21             return true;
22         } finally {
23             curr.unlock();
24         }
25     } finally {
26         pred.unlock();
27     }
28 }
```

Ricordiamoci che ogni nodo ha un suo lock, quindi acquisiamo il lock del nodo chiamando il metodo lock sul nodo stesso, partendo dal lock sulla head (Riga 3). Questa istruzione permette di dire che si acquisisce il nodo sulla testa, il lock del primo nodo. Questo in un certo modo significa garantire l'accesso alla lista, ma anche qui le caratteristiche del lock sono fondamentali, perché se ci sono tanti thread che stanno iniziando a fare operazioni, la maniera in cui il lock gestisce, per esempio FCFS, garantisce il progresso dell'applicazione. Se per esempio il lock non è starvation-free ma è solamente deadlock-free, ci potrebbe essere un thread che cerca di fare un'operazione e non riesce mai a farla, perché il lock della head viene sempre dato a qualcun altro. Nel momento in cui ha acquisito la head, in un certo senso, il thread si mette in coda, ed è a questo punto, in questa catena di thread che devono fare l'operazione, è quello che inizia a cercare di fare il suo lavoro.

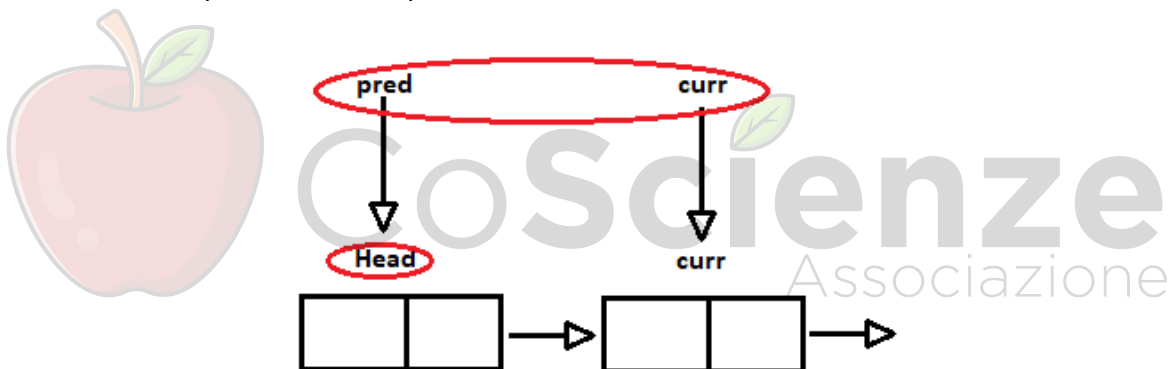
Come fa a fare il suo lavoro?

Ricordiamoci che in questo momento il thread ha preso pred e vuole cercare di acquisire curr, quindi calcola current qual è (Riga 6) e acquisisce il curr, quindi in questo momento abbiamo una situazione

di questo tipo: abbiamo il nodo di head e abbiamo il primo nodo vero della lista (curr) e il nostro thread è in questa posizione:



Quando si trova in questa posizione, ha acquisito il lock (Riga 7), quindi se c'era un thread prima di lui che stava occupando il primo nodo della lista sarebbe stato fermo, bloccato, fino a quando non sarebbe rimasto sbloccato. A questo punto la situazione di partenza è questa (figura inferiore), dove il thread rosso ha acquisito head e il primo nodo della lista.



A questo punto si scorre la lista nella stessa maniera in cui si faceva prima, cioè si fa in modo di prendere il curr attuale, che diventa il pred e il curr diventa il curr.next; quindi, l'avanzamento dei puntatori viene fatto nella stessa maniera, però lo si fa in questa maniera, cioè si rilascia il precedente (il lock del pred, Riga 10). Questo al primo nodo la prima volta significa che si permette a qualcun altro di acquisire la head, poi si sposta la pred su curr (Riga 11), si sposta curr su curr.next (Riga 12). Questo è l'aggiornamento logico. Prima di proseguire si deve acquisire il lock del nuovo nodo (Riga 13). Il blocco del `while` è l'avanzamento, quindi si sta cercando il curr tale che possa inserire il mio elemento prima del curr, cioè la più piccola chiave più grande della chiave che sto inserendo. Se la chiave ci sta già restituisce false (Riga 16), semplicemente, dato che evidentemente il nodo è già presente, ricordando che questo è un set, altrimenti posso fare le classiche operazioni (Riga 18 a 21) e restituire true. In uscita, le uscite sono due (Riga 16 e 21). Si deve rilasciare il current (Riga 23) perché il pred è stato rilasciato quando si stava avanzando il curr, quindi è quello che devo andare a rilasciare.

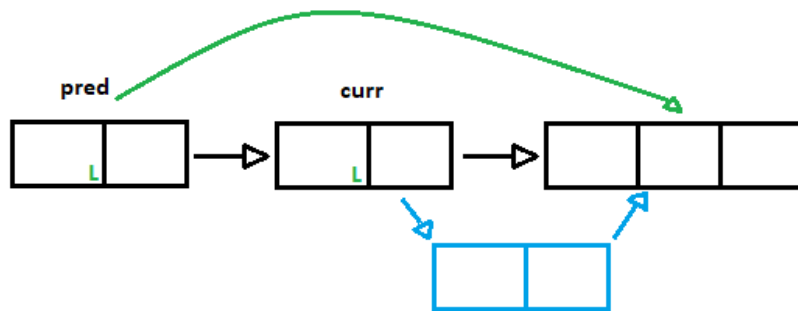
Metodo remove():

```
29  public boolean remove(T item) {
30      Node pred = null, curr = null;
31      int key = item.hashCode();
32      head.lock();
33      try {
34          pred = head;
35          curr = pred.next;
36          curr.lock();
37          try {
38              while (curr.key < key) {
39                  pred.unlock();
40                  pred = curr;
41                  curr = curr.next;
42                  curr.lock();
43              }
44              if (curr.key == key) {
45                  pred.next = curr.next;
46                  return true;
47              }
48              return false;
49          } finally {
50              curr.unlock();
51          }
52      } finally {
53          pred.unlock();
54      }
55  }
```

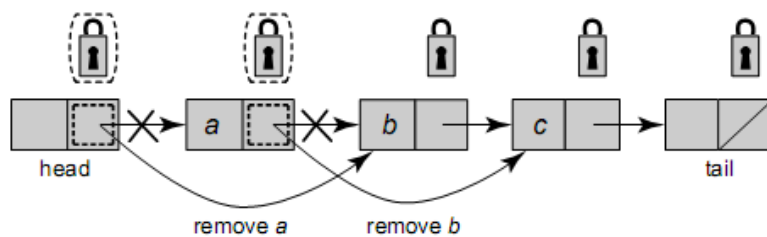
La parte di `remove` è praticamente identica, nel senso che mettiamo insieme la logica della `remove` e la logica dell'avanzamento dei puntatori gemelli. Partendo dal lock sulla `head` (Riga 32), si fa lock sul nodo corrente (Riga 36), si scorre la lista (Riga 39), ogni volta si avanza rilasciando il lock precedente (Riga 40), spostandosi e provando ad acquisire il successivo (Riga 42). La differenza tra `add()` e `remove()` sta chiaramente nel comportamento dell'operazione, quindi qui se l'elemento c'è, abbiamo trovato l'elemento da cancellare (Riga 45), lo cancelliamo facendo in modo di aver identificato `pred` e `curr`, spostiamo `pred` su `curr` e il `next` di `pred` a puntare su `curr.next` e restituiamo `true` (Riga 44-46), altrimenti vuol dire che stavamo cercando di cancellare un elemento che non era presente e possiamo semplicemente restituire `false` (Riga 48).

Perché si fa lock anche di `curr` se non si modifica?

Il lock su `curr` è necessario, perché noi stiamo cancellando l'elemento selezionato in rosso, ma dobbiamo essere sicuri che non è il supporto per nessun'altra operazione che viene effettuata.



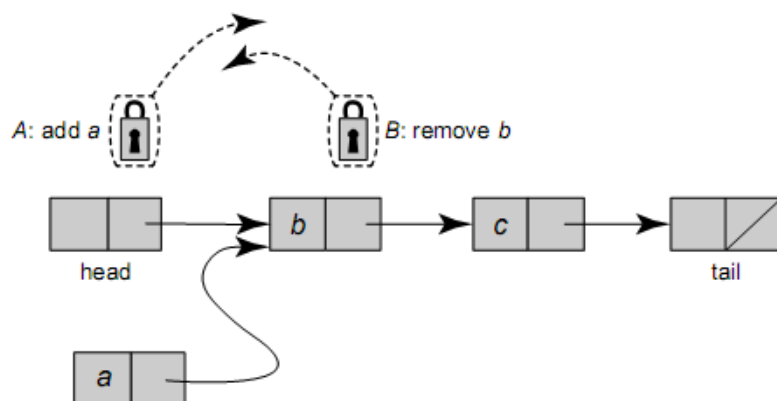
Si può fare l'esempio anche con la remove, per capire il perché, dobbiamo considerare il seguente scenario:



7.3.2 Analisi della Soluzione

Safety: Deadlock

Lock acquisiti nella stessa direzione: da head verso tail. Nessun deadlock, che sarebbe possibile se avvenisse l'accesso in direzione diverse.



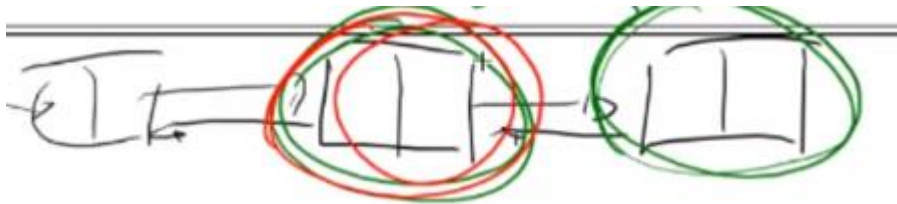
Se si cerca di ottimizzare Fine-grain, immaginate una lista molto grande, una lista con 1000 elementi, allora una cosa ragionevole sarebbe dire: le liste doppiamente puntate, quindi la possibilità di poter scorrere in avanti e indietro, sinistra verso destra e viceversa. Questo aggiornamento, di questa struttura dati sequenziale, alla concorrenza questo algoritmo non funziona bene, perché è chiaro che si potrebbe fare in modo che in maniera probabilistica far accedere metà dei thread su head e

metà su tail e poi farli navigare per trovare il loro posto per la loro operazione. Quello che può succedere chiaramente è che a questo punto si incontrano due thread su una struttura dati di questo tipo, ad esempio:

Se noi abbiamo una lista doppiamente linkata, se ho un thread verde che ha acquisito questo nodo



E vuole acquisire il lock di questo, e un thread rosso che invece ha acquisito questo nodo e vuole acquisire il lock di questo nodo è chiaro vanno in deadlock.



Una lista doppiamente puntata non funziona, sembra essere un'efficienza, che migliora l'efficienza sequenziale, e in concorrenza si apre completamente a problemi di Safety.

Come al solito la Safety è indipendente dalla natura del lock, cioè se il lock non è Safe o non è deadlock-free o starvation-free, il nostro algoritmo riceve la stessa caratteristica di Safety che offre il lock.

Safety: Invarianti e linearizzabilità

- I nodi sentinella non sono mai aggiunti/eliminati, nel senso che `add` è sempre l'ultimo e le operazioni vengono fatte sempre su `curr`;
- Inserimento in ordine, senza duplicati, quindi viene mantenuto l'invariante dell'ordine della lista;
- Abstraction map: cioè la maniera in cui noi definiamo come la struttura dati rappresenta l'insieme, un elemento è nell'insieme se e solo se esso è raggiungibile dalla `add()` ;
- Linearization point per la `add(a)` : a seconda del successo/insuccesso (significa quando posso o non posso inserire l'elemento) quando si fa il lock del nodo con chiave maggiore di `a` più piccola;

Progresso: Starvation-free

- Assumiamo che tutti i lock siano starvation-free;
- Lock starvation-free: ogni thread che cerca di acquisire il lock, ha successo alla fine;
- Tutti i metodi di accesso acquisiscono i lock, dalla testa alla coda della lista (no deadlock);

- Un thread che è in attesa di fare il lock di head eventualmente avrà successo;
- Da quel momento, nessun altro thread potrà entrare e gli altri thread che sono in numero finito che sono nella lista eventualmente finiranno il loro lavoro;

Perché sono un numero finito thread?

La lista consiste in un numero finito di nodi, se non altro dato che le chiavi vanno da min int a max int, quindi all'interno di una lista per il numero finito di nodi c'è un numero finito di lock e un thread non può entrare se non acquisisce il lock di almeno uno di questi nodi e quindi il numero è finito.

7.4 Optimistic Synchronization

L'idea di fondo è che si stanno usando:

- Strutture dati a puntatori, componenti linkati (liste, code, alberi etc.);
- Metodi che prevedono una ricerca di dati per effettuare operazioni;
- Una prima "passata" (una prima lettura), senza fare lock (in solo lettura);
- Se viene trovato, si effettua il lock del "nodo" (per le liste non è un nodo ma sono due);
- Allora si verifica se la struttura è cambiata tra la lettura ed il lock;
- Se non cambia, si procede;
- Utile se ha una rate di successo alto (Optimistic);

Un modo per ridurre i costi di sincronizzazione è di cercare un nodo senza acquisire lock, lockare i nodi trovati, e quindi confermare che i nodi bloccati siano corretti. Se vi è un conflitto di sincronizzazione a causa di lock di nodi sbagliati, si rilascia il lock e si ricomincia la ricerca. Normalmente, questo tipo di conflitto è raro ed è per questo che questa tecnica viene chiamata *Sincronizzazione Ottimistica*.

Come l'ottimismo migliora sulla grana fine

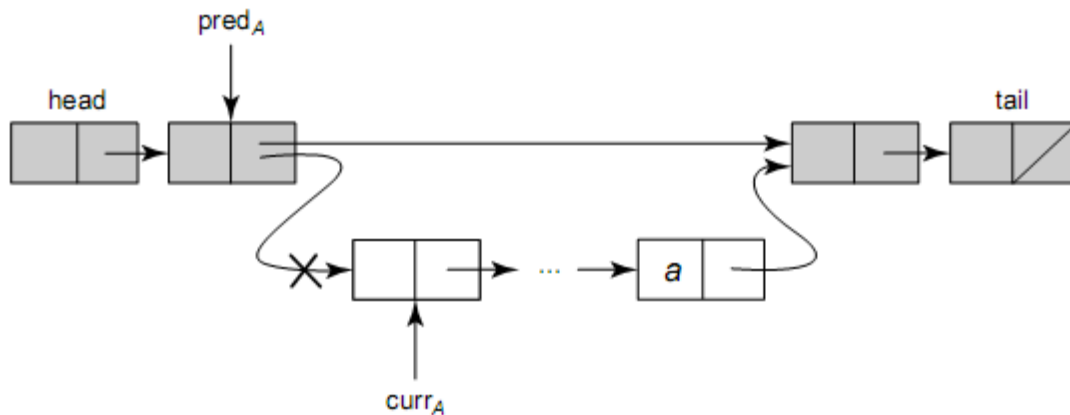
Fine-grain Synchronization:

- Più efficiente del Coarse-grain (accessi concorrenti in posti separati della lista);
- Lunga sequenza di lock;
- Thread possono comunque bloccare altri (anche se dovrebbero lavorare su parti diverse di lista);

L'idea consiste nel fare la ricerca senza acquisire il lock. Se si trova l'elemento si fa il lock e se la lista non è cambiata in maniera significativa (nodi raggiungibili e struttura pred/curr invariata), allora si fa l'operazione.

Perché serve la validazione?

Se il thread cerca di inserire a, che intanto viene cancellato:



Il problema è che, mentre si stava facendo il lock, ad esempio ci potrebbe essere stato qualcuno che ha cancellato il current che si stava portando, quindi il controllare che il pred punti a curr, garantisce che non c'è stata un'operazione che ha cambiato la struttura, cancellando ad esempio a, oppure inserendo un nodo tra pred e curr, queste sono le due possibilità.

Metodo Optimistic add():

```
1 public boolean add(T item) {
2     int key = item.hashCode();
3     while (true){
4         Node pred = head;
5         Node curr = pred.next;
6         while (curr.key <= key) {
7             pred = curr; curr = curr.next;
8         }
9         pred.lock(); curr.lock();
10        try {
11            if (validate(pred, curr)) {
12                if (curr.key == key) {
13                    return false;
14                } else {
15                    Node node = new Node(item);
16                    node.next = curr;
17                    pred.next = node;
18                    return true;
19                }
20            }
21        } finally {
22            pred.unlock(); curr.unlock();
23        }
24    }
25 }
```

The OptimisticList class: the add() method traverses the list ignoring locks, acquires locks, and validates before adding the new node.

(Riga 3), ciclo infinito, perché l'idea è che si scorre tornando alla validazione. Si fa il lock e se la lista non è cambiata in maniera significativa allora si fa l'operazione, altrimenti ti manda a monte tutto e si riparte da capo, cioè si è identificata l'operazione, qualcuno ha modificato in maniera significativa la zona dove stiamo andando a lavorare e bisogna ripartire da capo, e per questo serve il while(true).

(Riga 4) si parte da head e curr (Riga 5) e lo scorrimento è banale (Riga 4-5-6) non si fanno lock, non si fa niente.

(Riga 9) Identifico il posto e si fa il lock dei due nodi, a questo punto bisogna cercare di capire se si può fare l'operazione, questa cosa è ri-fattorizzata in una funzione, che si chiama `validate()`.

(Riga 11), cioè prende pred e curr e restituisce true o false se la struttura di pred e curr è cambiata o meno, quindi se validate conferma si effettuano le operazioni di add, altrimenti si torna nel ciclo, se si è fatta la validazione e questa non ha funzionato si ricomincia da capo. Il rilascio dei lock avviene a Riga 22.

Metodo Optimistic remove():

```
26 public boolean remove(T item) {
27     int key = item.hashCode();
28     while (true){
29         Node pred = head;
30         Node curr = pred.next;
31         while (curr.key < key) {
32             pred = curr; curr = curr.next;
33         }
34         pred.lock(); curr.lock();
35         try {
36             if (validate(pred, curr)) {
37                 if (curr.key == key) {
38                     pred.next = curr.next;
39                     return true;
40                 } else {
41                     return false;
42                 }
43             }
44         } finally {
45             pred.unlock(); curr.unlock();
46         }
47     }
48 }
```

La `remove()` è identica, nel senso che si fa sempre `while(true)` (Riga 28), si fa l'avanzamento senza fare il lock (Riga 31), si fa il lock dei due nodi (Riga 34), se la validazione va a buon fine si fa l'operazione (Riga 36) altrimenti si torna in ciclo.

Metodo Optimistic contains():

The OptimisticList class: the remove() method traverses ignoring locks, acquires locks, and validates before removing the node.

```
49  public boolean contains(T item) {
50      int key = item.hashCode();
51      while (true){
52          Node pred = this.head; // sentinel node;
53          Node curr = pred.next;
54          while (curr.key < key) {
55              pred = curr; curr = curr.next;
56          }
57          try {
58              pred.lock(); curr.lock();
59              if (validate(pred, curr)) {
60                  return (curr.key == key);
61              }
62          }finally { // always unlock
63              pred.unlock(); curr.unlock();
64          }
65      } // end while
66  }
```

Il problema è che si sta scorrendo una lista che è soggetta a modifiche, e quindi bisogna utilizzare il lock, ma per utilizzare il lock si deve verificare, avendo scorso la lista senza fare il lock, che la struttura è ancora valida (Riga 51), quindi scorre (Riga 52-53), identicamente a tutti gli altri, si fa il lock (Riga 58). Se a questo punto la validazione va a buon fine (Riga 59), si restituisce il valore del test (Riga 60), se è uguale restituisce true, se non è uguale restituisce false, altrimenti si torna in ciclo nel while.

Metodo Optimistic validate():

```
67  private boolean validate(Node pred, Node curr) {
68      Node node = head;
69      while (node.key <= pred.key) {
70          if (node == pred)
71              return pred.next == curr;
72          node = node.next;
73      }
74      return false;
75  }
```

Ricordiamo che la validate() viene chiamata su pred e curr che sono lockati (Riga 67), quindi, qualsiasi operazione, add(), remove() o contains(), effettua il lock di pred e curr e poi chiama la validate(). La validate() scorre la lista (Riga 69), e va avanti e essenzialmente fa node = node.next (Riga 72), quindi scorre in questa maniera fino a quando non trova il pred (Riga 70), quando ha trovato il pred, significa che:

- Riga 70, questa è la condizione raggiungibile;
- Riga 71, a questo punto restituisce il valore del test `pred.next == curr` che significa che `pred` punta a `next`;
- Si possono attraversare nodi che sono stati rimossi dal sistema;
- L'assenza di interferenza (i nodi non vengono riutilizzati e mantengono i loro valori) permette di ritornare sulla lista;
- Interazione con il Garbage Collector del sistema;
- `OptimisticList` non è starvation-free: Un thread A può essere ritardato infinitamente da thread (che arrivano dopo) e che modificano la lista in posizione precedente alla posizione della operazione di A. Ad esempio: aggiungono e cancellano un nodo;
- Efficienza si poggia sul comportamento nella pratica (molto efficiente par bassi regimi di carico);

7.5 Lazy Synchronization

L'obiettivo è sempre cruciale, aumentare il parallelismo sui lock, noi cerchiamo di non avere un solo lock, ma cerchiamo anche di limitare al massimo il numero di operazioni sui lock.

Una visione critica di `OptimisticList`:

- `OptimisticList` ha alcuni svantaggi, ma anche vantaggi;
- Il costo di scorrere la lista due volte deve essere più basso, si scorrere la lista con i lock;
- `contains()` richiede lock, poco efficiente se è un metodo chiamato spesso;

L'idea consiste in:

- Raffinamento in modo che `contains` è wait-free (non usa lock) e `add()` e `remove()` usano solo uno scorrimento di lista (se non c'è contesa);
- L'idea: aggiungiamo ad ogni nodo un campo booleano `marked` che indica se il nodo è o non è nell'insieme;
- Invariante: ogni nodo non marcato raggiungibile nella lista è nell'insieme;
- `Lazy remove()` : si fanno due operazioni, si marca il nodo da cancellare (1. cancellazione logica) e poi lo si elimina fisicamente (2. set del campo `next` del predecessore);

Come si scorre la lista?

- Tutti i metodi scorrono la lista ignorando i lock;
- I metodi `add()` e `remove()` fanno lock su `pred` e `next`;

Ma la fase di validazione non scorre l'intera lista, dovendo controllare:

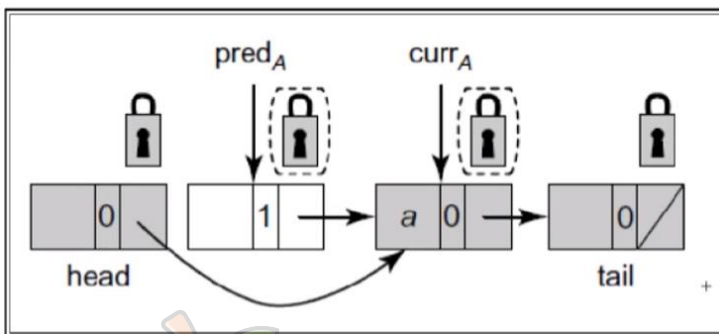
```

1 private boolean validate(Node pred, Node curr) {
2     return !curr.marked &&
3           !pred.marked &&
4           pred.next == curr;
5 }

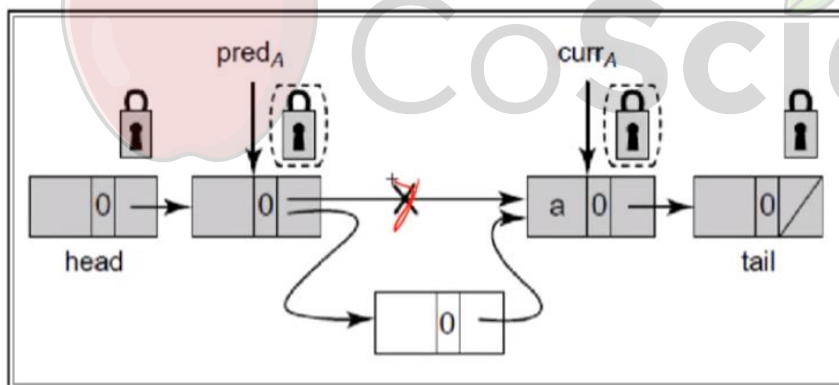
```

1. curr non è stato marcato (cancellazione logica)
2. pred non è stato marcato (cancellazione pred logica)
3. pred punta ancora verso curr (inserimento di un altro nodo tra pred e curr)

Perché serve la validazione?



pred potrebbe essere cancellato e avere il curr a 1, oppure:



pred non punta più, che significa è stato inserito un nuovo nodo all'interno.

7.5.1 La Classe LazyList

Metodo LazyList add():

```
1  public boolean add(T item) {
2      int key = item.hashCode();
3      while (true){
4          Node pred = head;
5          Node curr = head.next;
6          while (curr.key < key) {
7              pred = curr; curr = curr.next;
8          }
9          pred.lock();
10         try {
11             curr.lock();
12             try {
13                 if (validate(pred, curr)) {
14                     if (curr.key == key) {
15                         return false;
16                     } else {
17                         Node node = new Node(item);
18                         node.next = curr;
19                         pred.next = node;
20                         return true;
21                     }
22                 }
23             } finally { curr.unlock();}
24         } finally { pred.unlock();}
25     }
26 }
```

Le operazioni sono simili a quelle dell'Optimistic, in effetti cambia la `validate()`, quindi apriamo il `while(true)` (Riga 3), abbiamo lo scorrimento della lista senza fare il lock (Riga 6), si fa il lock del pred (Riga 9), si fa il lock del curr (Riga 11), si fa la validazione (Riga 13). Se la validazione ha restituito true allora possiamo fare l'operazione, controlliamo se la chiave dell'elemento current è uguale alla chiave dell'elemento che vogliamo inserire, quindi non possiamo inserirlo e restituiamo false (Riga 14), oppure possiamo inserirlo e si fa la solita operazione su pred e su curr (Riga 18-19), su pred in maniera tale da inserirlo prima di curr. Se non è valida si riprova e si comincia (Riga 3).

Metodo LazyList remove():

```
1  public boolean remove(T item) {
2      int key = item.hashCode();
3      while (true){
4          Node pred = head;
5          Node curr = head.next;
6          while (curr.key < key) {
7              pred = curr; curr = curr.next;
8          }
9          pred.lock();
10         try {
11             curr.lock();
12             try {
13                 if (validate(pred, curr)) {
14                     if (curr.key == key) {
15                         return false;
16                     } else {
17                         curr.marked = true;
18                         pred.next = curr.next;
19                         return true;
20                     }
21                 }
22             } finally { curr.unlock();}
23         } finally { pred.unlock();}
24     }
25 }
```

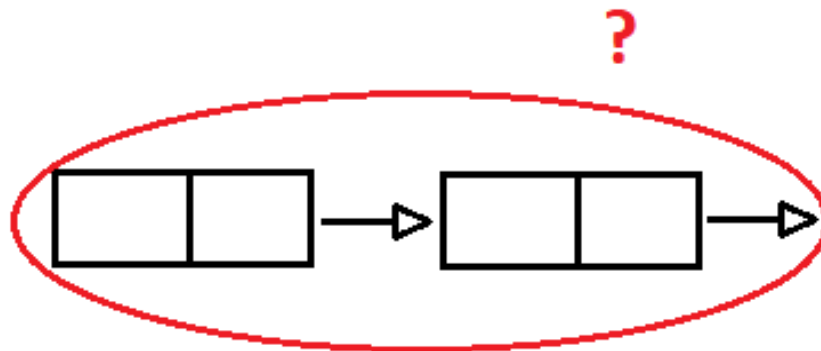
La remove è pressoché identica all'add(), cioè manteniamo sempre il ciclo infinito, il solito controllo della validazione, e se la validazione va a buon fine allora si effettua l'operazione di remove, mentre se non va bene si rilasciano i due lock e si riprova a fare lo scorrimento.

Metodo LazyList contains():

```
1  public boolean contains(T item) {
2      int key = item.hashCode();
3      Node curr = head;
4      while (curr.key < key) {
5          curr = curr.next;
6          return curr.key == key && !curr.marked;
7      }
8  }
```

La contains() a questo punto è uno dei miglioramenti che abbiamo, perché la contains() prima, nell'Optimistic aveva la necessità di fare la validazione dei lock, e la validazione quindi era un problema, ed era chiaramente una inefficienza. È importante rendere efficiente il caso delle operazioni più probabili. La contains() non fa altro che ignorare completamente i lock, perché la contains() deve solamente verificare la presenza di un elemento. Avendo inserito il punto di linearizzazione sull'operazione di marked, questo ci permette di fare questo controllo (Riga 6), cioè l'elemento che si è trovato, che è uno solo (curr.key==key) è cancellato logicamente o anche fisicamente dalla lista oppure no. Ci interessa solo logicamente perché tutti gli elementi logicamente cancellati poi vengono anche fisicamente cancellati, quindi la cancellazione logica, usata come

Linearization point ci permette di poter controllare in un unico posto. Prima per dover controllare la cancellazione bisognava controllare se la domanda era: questo elemento è cancellato oppure no? Noi dovevamo controllare la struttura, ossia controllare due cose:



Ciò significava bloccare due elementi, quindi fare il lock dell'Optimistic, scorrere e fare cose che in questo momento non sono necessarie perché abbiamo qui questo flag, che è quello che per primo viene inserito quando viene cambiato, ed è il punto in cui la cancellazione appare.

Metodo LazyList validate():

È identica alle altre.

```
1 private boolean validate(Node pred, Node curr) {
2     return !curr.marked && // (1)
3         !pred.marked && // (2)
4         pred.next == curr; // (3)
5 }
```

7.6 Non-Blocking Synchronization

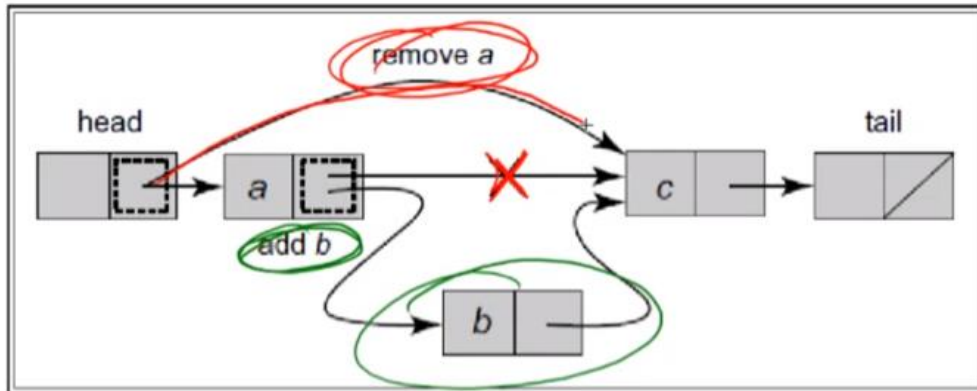
Questo è chiaramente l'obiettivo, cioè non usare del tutto i lock. Abbiamo fatto in modo che ci sia un solo lock usato da tutti, altamente inefficiente, tantissimi lock usati da tutti in maniera ordinata. Questo migliora la situazione, ma non è il meglio. Abbiamo fatto in modo che ci siano tanti lock ma vengono usati solamente per l'ordine di un lock, che sono necessari per fare le operazioni (Optimistic). Abbiamo migliorato l'Optimistic dicendo la `contains()` riesce a funzionare senza avere lock, con altre caratteristiche che abbiamo visto e inserendo un altro campo, tutti questi vantaggi hanno un costo in termini di complessità dell'algoritmo, poiché stiamo aggiungendo memoria, quindi non è una cosa indolore.

L'idea:

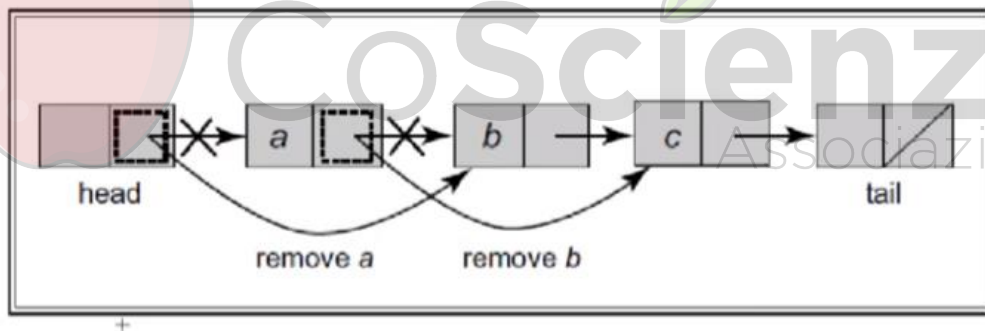
- Eliminare del tutto i lock (a qualsiasi granularità);
- Tornare alle operazioni atomiche come `compareAndSet()`;

- Eliminazione basata su mark;
- Osservazione: non basta fare operazione atomiche sul campo next;

Se si fa l'inserimento di b (thread verde) e contemporaneamente si ha un thread rosso che sta facendo questo (remove a) e uno invece che sta cambiando il campo cerchiato in verde. Se lavorano insieme abbiamo dei problemi, nel senso che il nodo b è stato inserito non compare nella lista, perché contemporaneamente ci stava il thread rosso che stava cancellando il nodo a, che era il thread della posizione in cui b è stato inserito.



Questo vale anche per due rimozioni in cascata.



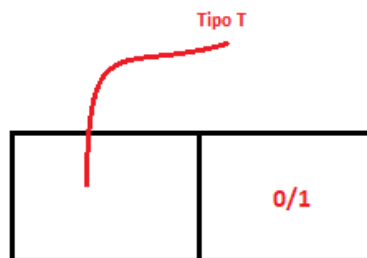
7.6.1 Il tipo AtomicMarkableReference<T>

Da `java.util.concurrent.atomic` incapsula due cose insieme: un riferimento ad un oggetto di tipo `T` ed un mark booleano. Essenzialmente riesce ad operare in maniera atomica su queste due grandezze, atomica significa non interrompibile, cioè non è possibile che faccia l'operazione sulla prima e non sulla seconda perché intanto è stato schedato un altro thread, quindi riesce a fare entrambe le operazioni senza essere interrotto. Gli update sono atomici, sia congiunti che dei campi singoli. Con il `compareAndSet()` viene chiesto di aggiornare atomicamente il valore. Se il valore che si trova in quel momento è uguale ad uno che viene passato, cioè una operazione del tipo "si immagina che non sia ancora successo niente, quindi il valore era quello che si è letto in precedenza". Se quello non è stato mantenuto allora si cambia con un altro valore in maniera

atomica. Questo è il `compareAndSet()`, se il valore è quello che ci si aspetta si fa la modifica altrimenti restituisce false, restituisci che questa cosa non è stata fatta, quindi segnala all'esterno che c'è qualche situazione strana di interfogliamento che ha fatto sì che questo valore non è quello che ci si aspettava. Il `compareAndSet()` in questo contesto viene fatto a coppie, nel senso che sono due le grandezze che si vuole cambiare.

Esempio:

In questa `AtomicMarkableReference` abbiamo sia il riferimento ad un oggetto di tipo T, sia un bit 0/1, che è il mark che vogliamo andare a mettere (true, false).



Il concetto dovrà essere: ci si aspetta che in questa variabile (si intendono i due blocchi dell'immagine superiore), il valore di T sia questo (il valore del riferimento dell'oggetto è questo, punta a questa locazione di memoria) e contemporaneamente il valore del marker sia true o false. Se questa coppia è rispettata, cioè questa coppia di richieste ovvero se l'oggetto puntato è uno in particolare e il valore di mark è quello che è stato passato, allora effettua questa operazione, altrimenti fallisce.

La interfaccia di `AtomicMarkableReference<T>`:

Pragma 9.8.1. `AtomicMarkableReference<T>` è un oggetto del package `java.util.concurrent.atomic` che comprende sia un riferimento a un oggetto di tipo T, sia ad un Boolean mark. Questi campi possono essere aggiornati atomicamente, insieme o singolarmente. Ad esempio, il metodo `compareAndSet()` testa il riferimento atteso e il mark, e se entrambi i test hanno successo, li sostituisce con il riferimento ed il mark aggiornato. Il metodo `attemptMark()` fa un test di un riferimento atteso e se il test ha successo, lo sostituisce con un nuovo valore mark. Il metodo `get()` ha una insolita interfaccia: restituisce il riferimento dell'oggetto T e memorizza il valore marked in un array booleano.

```
1 public boolean compareAndSet(T expectedReference,  
2     T newReference,  
3     boolean expectedMark,  
4     boolean newMark);  
5  
6 public boolean attemptMark(T expectedReference, boolean newMark);  
7  
8 public T get(boolean[] marked);
```

La `compareAndSet()` prende 4 parametri, perché deve dire lo stato che si aspetta di trovare, quindi il valore atteso del riferimento (Riga 1), il nuovo riferimento (Riga 2), il valore atteso del mark (Riga 3) e il nuovo mark (Riga 4). Ci si aspetta che l'`AtomicMarkableReference` punti lì con questo marker e se questo è vero allora cambia il valore e viene messo il nuovo marker. Ci sono metodi più semplici per scrivere il mark, cioè:

Set atomico del mark (Riga 6), dove ci si aspetta un determinato tipo di reference e quindi si vuole andare a mettere un nuovo mark. Atomicamente si vuole andare a scrivere il marker. Poi c'è la possibilità di poter fare la get. La get (Riga 8) è un problema del linguaggio java (un linguaggio procedurale, quindi non funzionale). La restituzione è vincolata ad essere un tipo, non una coppia (nei linguaggio più moderni si fa agevolmente). In tal caso bisogna fare un "trucchetto", ossia restituire due valori: un valore è restituito attraverso il tipo di ritorno (La T dopo public), l'altro valore viene restituito all'interno di un array di booleani che viene passato per riferimento, cioè essendo questo parametro un array di booleani e non un booleano, se venisse passato un booleano a questo metodo sarebbe un parametro che viene passato per valore, diciamo all'interno del metodo viene passato questo valore, poiché si passa un array di booleani. Questo array viene passato per riferimento, quindi il metodo get accede all'array che è stato passato. A questo punto ci può scrivere dentro, e scrivendo il marker all'uscita di get, in pratica si può andare a prendere il valore restituito che è il riferimento T, e si può andare a vedere in questo array di booleani nella posizione 0 perché lì ci trovo il marker booleano, che è stato messo dal metodo. L'Array, quindi, non viene copiato ma passato per riferimento.

7.6.2 La Classe LockFreeList

```
1 class Window {
2     public Node pred, curr;
3     Window(Node myPred, Node myCurr) {
4         pred = myPred; curr = myCurr;
5     }
6 }
7 public Window find(Node head, int key) {
8     Node pred = null, curr = null, succ = null;
9     boolean[] marked = {false};
10    boolean snip;
11    retry: while (true){
12        pred = head;
13        curr = pred.next.getReference();
14        while (true){
15            succ = curr.next.get(marked);
16            while (marked[0]) {
17                snip = pred.next.compareAndSet(curr, succ, false, false);
18                if (!snip) continue retry;
19                curr = succ;
20                succ = curr.next.get(marked);
21            }
22            if (curr.key >= key)
23                return new Window(pred, curr);
24            pred = curr;
25            curr = succ;
26        } // fine while(true) interno
27    } // fine while(true) esterno
28 }
```

Dobbiamo introdurre prima delle classi aggiuntive, che ci serviranno all'interno dell'algoritmo, la classe Window. La classe Window mette insieme predecessore e current (Riga 2), quindi una coppia di nodi che sono in relazione, poi definiamo il metodo find() (Riga 7). find() è un metodo che scorre la lista, cerca qualcosa, ma ogni volta mentre sta lavorando, ogni volta che incontra un nodo

che è stato cancellato logicamente (markato) ma non fisicamente, prova ad eliminarlo. L'idea è che nel momento in cui si trova a passare, trova dei nodi, li vede come se fossero spazzatura. Un nodo che è stato cancellato logicamente ma non fisicamente lo cancella. La classe Window (Riga 1) è un wrapper.

Il metodo `find()` (Riga 7) parte dalla testa della lista e ha una chiave. Essenzialmente `find()` sostituisce la modalità con cui scorriamo la lista, quindi si parte con varie inizializzazioni (Riga 8), e abbiamo l'array di `marked` con una sola posizione (Riga 9). Questo è un aspetto importante, abbiamo un booleano (Riga 10) e poi andiamo in un `while(true)` (Riga 11). Questo `while(true)` è anticipato da una etichetta che ci servirà per fare un goto (un salto con etichetta). A questo punto (Riga 14) è il momento in cui si parte a scorrere la lista, si setta `pred` e `curr` (Riga 12-13) come predecessori. `next` è un `AtomicMarkableReference`, quindi è un qualcosa da cui andiamo a prendere il riferimento. Ora scorrendo la lista, andiamo a prelevare il prossimo elemento e settiamo anche questo valore `successor`, quindi abbiamo il predecessore `current` e `successor`. Quando ci troviamo in questa situazione, possiamo incontrare un nodo `successor` che è stato cancellato logicamente, e potrebbe essere uno o più di uno. Quindi è stato prelevato il `successor`, ci si è resi conto che `successor` è un nodo. Il concetto è che si dovrebbe spostare predecessore e `current` avanti; quindi, il `current` dovrebbe andare sul `successor`. Questa è l'idea, ma non si vuole spostare su un `successor` che è stato cancellato logicamente. Ora si prende il `successor` e si saltano tutti i nodi che sono eventualmente logicamente cancellati, quindi si salta, cioè si eliminano dalla lista, si fa la cancellazione fisica quando si incontrano una serie di nodi marcati. Questo è lo scopo del `while` (Riga 16), cioè si legge il riferimento del `next` (Riga 15) e contemporaneamente questo taglia l'array `marked`, cioè in `marked[0]` si trova se quel nodo è marcato. Se quel nodo non è marcato si salta tutto (Riga 14-21). Se non si trovano i nodi cancellati logicamente, allora si va avanti e si continua a fare l'incremento (Riga 15) e se non è arrivato (Riga 22) alla chiave giusta si va avanti, e il `pred` (Riga 24) diventa il vecchio `curr` e il `curr` (Riga 25) diventa il `successor`.

Cosa succede se invece si incontrano i nodi che sono stati cancellati logicamente?

Si prova ad eliminarli fisicamente, cioè il `next` punta ad un nodo cancellato logicamente, quello che facciamo è eseguire la `compareAndSet()` (Riga 17) dicendo che guarda il valore che c'era (`curr`). Il valore che si deve prendere è `succ`, si è quindi trovato un nodo che deve essere cancellato e ci si aspetta che il `mark` sia `false` e lo si vuole tenere a `false`, ovvero è quello che si sta facendo essenzialmente in questa maniera. Trovato che il `successor` del `current` è in questo momento un nodo che è cancellato logicamente, lo si vuole cancellare fisicamente. Per cancellarlo fisicamente si vuole provare a cambiare il riferimento. Si vuole saltare, ma saltando quello che succede è che si prova a fare la `compareAndSet()` ed essa può avere successo o non avere successo. Se ha successo, quindi se `snip` è `true` (Riga 18), questo `if` non viene fatto e a questo punto si salta e si va dal `curr` che diventa il `successor` (Riga 19). Il `successor` diventa il `next` del `current` (Riga 20), quindi l'idea è che ci si sposta in avanti di questa lista, cioè si è cancellato il nodo e ci si è spostati avanti. Se questo `snip` non ha successo (Riga 18), la `compareAndSet()` non ha avuto successo. Se non riesce a fare questa operazione significa che stava facendo un'operazione immaginando che in quella posizione ci fosse il `curr` e invece non c'è, evidentemente perché ci sta lavorando qualcun altro. Questo significa che c'è qualcun altro con cui si è interfogliati, ha avuto prima gioco facile

l'altro thread a fare l'operazione, ma adesso non sa esattamente qual è la situazione. L'unica soluzione è quella di ripartire da capo, cioè significa uscire sia dal `while` (Riga 14) sia da questo `while` (Riga 11) perché non sa esattamente se quel nodo è raggiungibile, deve veramente ricominciare da capo dalla head e uscire da due `while` innestati. La cosa più semplice da fare è attraverso il goto. Il concetto essenzialmente è questo, cioè ci si sposta sulla lista e si fa in modo di controllare ogni volta che ci si sposta. Se il nodo `successor` è un nodo che è cancellato logicamente, evidentemente bisogna anche cancellarlo fisicamente, cioè si deve terminare la cancellazione dell'azione. Il motivo di tutto questo è che ci sono delle operazioni che qualcuno fa, che qualche altro thread ha fatto e che non ha potuto completare, quindi qualche altro thread si troverà a scorrere e troverà il nodo cancellato logicamente e si trova a cancellare fisicamente il nodo che è stato cancellato solamente logicamente. Se una volta che questo è stato fatto (Riga 16-21), cioè una volta che abbiamo scorso e saltato tutti i nodi relativi, che sono stati logicamente cancellati, è possibile a questo punto andare a fare il controllo, cioè abbiamo `current`, verifichiamo la chiave (Riga 22) rispetto alla chiave che si sta cercando (Riga 7) e se questa chiave è più grande o maggiore o uguale di quella che si sta cercando, deve restituire la coppia `pred curr` (Riga 23) quindi una `window` (wrapper). L'idea è che questa sarà usata per fare le operazioni o di `add()` o di `remove()`. Se invece il `>=` (Riga 22) non è vero significa che si deve continuare a scorrere e quindi si aggiorna il `pred` (Riga 24) e `curr` (Riga 25) e si torna su a fare il `while` (Riga 14), che è quello che ci permette di scorrere.

Metodo `add()`:

```
1 public boolean add(T item) {
2     int key = item.hashCode();
3     while (true){
4         Window window = find(head, key);
5         Node pred = window.pred, curr = window.curr;
6         if (curr.key == key) {
7             return false;
8         } else {
9             Node node = new Node(item);
10            node.next = new AtomicMarkableReference(curr, false);
11            if (pred.next.compareAndSet(curr, node, false, false)) {
12                return true;
13            }
14        }
15    } //fine while(true)
16 }
```

Alla `add()`, gli passiamo un `item` di tipo `T` (Riga 1) e abbiamo anche qui operazioni all'interno di un `while (true)` (Riga 3), perché anche in questo caso facciamo operazioni fino a quando pensiamo di avere un'idea chiara di quello che sta succedendo. Se ci sono dei problemi evidentemente torniamo a rieseguire. Il lavoro fatto prima si riassume in questo (Riga 4), cioè dare con questa chiave, partendo dalla head della testa, una coppia di nodi, cioè `window pred` e `window curr` (Riga 5) che identificano la posizione dove si dovrebbe andare a fare l'inserimento. Ora che questo `find` ha anche ripulito dai nodi cancellati logicamente, a questo punto si deve fare la `add()`, quindi se il nodo già c'è si deve restituire `false` (Riga 6-7), altrimenti si crea un nodo (Riga 9), si fa puntare il `next`

del nodo creando una nuovo AtomicMarkableReference (Riga 10), poi prendendo il campo next di pred bisogna farlo puntare a node e lo si fa con un `compareAndSet()` (Riga 11), dove quello che ci si aspetta è il valore. Quello che c'era prima era curr, cioè il valore che è stato trovato e che ci ha fatto fermare quando è stata fatta la `find()`, era che predecessor puntasse al curr. Ora, se pred punta ancora a curr, al suo posto viene messo node, non solo ci si aspetta che il nodo pred non sia stato cancellato logicamente, perché è chiaro ci potrebbe essere qualcuno che l'ha cancellato, se non è stato cancellato ci si aspetta che sia false, allora si rimette false. Quindi se il nodo pred non è stato cancellato, il marker è rimasto false e punta ancora a curr. A questo punto si fa questo cambiamento, ovvero che nessuno aveva cambiato pred, e con questo `compareAndSet()` in maniera atomica si riesce, con una sola operazione non interrompibile, non interfogliabile, a inserire il nodo node all'interno della lista e a questo punto si può restituire true (Riga 12). Se il set atomico (Riga 11) va bene restituisce true, mentre se non va bene significa che è cambiato qualcosa, significa che si stava inserendo un nodo dove poi questo è stato cancellato, qualcun altro l'ha inserito, ma non si sa cosa sia successo. L'unica soluzione che si ha è che se non si fa questo si ricomincia da capo, quindi ricomincia la ricerca e si prova a inserire di nuovo questo elemento.

Metodo remove():

```
17 public boolean remove(T item) {
18     int key = item.hashCode();
19     boolean snip;
20     while (true){
21         Window window = find(head, key);
22         Node pred = window.pred, curr = window.curr;
23         if (curr.key != key) {
24             return false;
25         } else {
26             Node succ = curr.next.getReference();
27             snip = curr.next.attemptMark(succ, true);
28             if (!snip)
29                 continue;
30             pred.next.compareAndSet(curr, succ, false, false);
31             return true;
32         }
33     } //fine while(true)
34 }
```

Anche in questo caso la rimozione fa ampio uso dell'utilizzo delle operazioni atomiche, quindi l'idea è che si prova una volta identificato a cancellarlo logicamente, perché questo garantisce che in quel nodo non esiste più Abstraction map, poi se si riesce e la situazione è stabile, cioè che non ci sono conflitti, se nessuno ha cambiato le carte in tavola si fa anche la cancellazione fisica, la remove. Si prova a fare la cancellazione fisica, ma se per caso, dopo aver fatto la cancellazione logica la situazione invece è cambiata, perché si è interfogliata con degli altri thread che hanno cambiato dei valori, la remove non sa cosa è successo. Intanto ha marcato logicamente, quindi l'idea essenzialmente è che remove cancella logicamente, se ci riesce prova anche a fare la cancellazione fisica, se non ci riesce lascia le cose così. Viene fatta la `find()` (Riga 21), troviamo pred e curr, poi se la chiave del current non è uguale (Riga 23) alla chiave che stiamo cancellando ritorna false (Riga 24), altrimenti vuol dire che si deve cancellare il nodo current. Per prima cosa si prende la reference

(Riga 26) e si prova a cancellare il nodo (Riga 17) quindi si fa l'attemptMark mettendo il valore a true, ossia il valore della reference del curr (si sta cambiando il valore dell'AtomicMarkReference del curr, che sta dentro next) quindi questa cosa si riferisce a curr, sapendo che dentro c'era il valore successor. Se questo valore è rimasto tale, cioè non si è interfogliato con qualcun altro, perché magari c'è qualcun altro che sta cancellando il successor, allora mette true. Questo significa che da qualche parte l'AtomicMarkReference compare un bit a 1, compare una T, compare qualcosa che dice che da questo momento questo nodo è cancellato logicamente. Questa operazione è un'operazione che viene effettuata in maniera atomica (Riga 17), che può riuscire o non riuscire, e questo entra nel booleano che viene restituito, quindi snip se non è riuscita (Riga 18), cioè non si è riusciti a marcare logicamente perché evidentemente c'è un thread che si è interfogliato e sta facendo delle operazioni, non si sa esattamente cosa si deve fare, si deve continuare e si torna a fare il ciclo while (Riga 20), cioè si riparte da 0, altrimenti si vada sul predecessor next (Riga 30).

Cosa c'è di strano a questo punto (Riga 30)?

Non vediamo il valore di ritorno. Non si prende il valore di ritorno perché non ci interessa, cioè il concetto è che si prova a fare questa operazione, si prova a cancellarlo fisicamente. Il next di pred puntava a curr e non era cancellato, se queste cose sono vere si fa anche la cancellazione fisica, ma se non sono vere non ci interessa, perché tanto è stato cancellato logicamente. Quindi si fa la prova del set atomico (Riga 30), la rimozione fisica e poi la else (Riga 31) restituisce true. Nel caso in cui facciamo le continue (Riga 29) oppure non si riesce a fare la cancellazione logica si ritorna da capo e questo è lo scopo del while(true).

Metodo contains():

```
35 public boolean contains(T item) {
36     boolean[] marked = false{};
37     int key = item.hashCode();
38     Node curr = head;
39     while (curr.key < key) {
40         curr = curr.next;
41         Node succ = curr.next.get(marked);
42     }
43     return (curr.key == key && !marked[0])
44 }
```

Con il while (Riga 39) si ricerca l'elemento, poi si va man mano avanti e legge se il prossimo curr è marcato (Riga 41). Il get ogni volta contemporaneamente setta l'array marked con il mark dell'ultimo nodo che è stato acceduto. Quando si esce da questo while si restituisce l'insieme del test (Riga 43), current key è la chiave di cui si voleva verificare l'esistenza (curr.key) e il nodo non è cancellato logicamente (key), quindi se è stato trovato il nodo e non è stato cancellato logicamente dall'ultima get (Riga 41) allora si può restituire true. In tutti gli altri casi potrei trovare l'elemento ma è stato cancellato logicamente e restituisce false, quindi si potrebbe non aver trovato l'elemento e trovato un'altra chiave più grande della chiave che si stava cercando, quindi restituisce false.

In conclusione, riguardo le prestazioni, lo scopo è aumentare la concorrenza, aumentare il parallelismo, in maniera tale da utilizzare le risorse a disposizione. Se si ha la possibilità di utilizzare un algoritmo che è estremamente efficiente dato che i thread non si fermano mai ad aspettare gli altri, non c'è mai un thread che cambia stato (nessuno va in blocked). Questo è un vantaggio, perché cambiare stato a un thread è notevole, stare sempre ready per essere eseguiti e quello che succede è al massimo decidere che c'è stato troppo conflitto, quindi devono ripetere l'operazione, ma sono sempre in esecuzione. Quindi:

- Progressione di implementazioni di lock;
- Da Coarse grain;
- A Fine Grain;
- A Optimistic Synchronization;
- A Lazy Synchronization;
- E a Non Blocking;
- Questa progressione è in pratica un paradigma di transizione di algoritmi per problemi concorrenti, non vale solo per le liste, ma vale anche per altre cose. Specialmente da Optimistic a Lazy, poter fare operazioni logiche che poi vengono perfezionate da operazioni fisiche.

La migliore? Non esiste il migliore algoritmo.

7.6.3 Optimistic vs. Lazy Synchronization

LockFreeList

- Fornisce progresso (lock-free/lock-wait);
- Costa:
 - Supporto hardware per le modifiche atomiche di riferimenti con mark;
 - Pulizie di nodi di `add()` e `remove()` con possibilità di contesa e di necessità di dover ripartire con lo scorrimento (dal punto dell'efficienza ha un impatto, ripartire da capo è sicuramente una cosa costosa);

LazyList

- Non richiede il costo di aggiungere mark;
- Non richiede "pulizia" dei nodi durante `add()` e `remove()`, la cancellazione fisica viene fatta in un'altra posizione;

Qual è il miglior algoritmo concorrente per le liste?

Questo dipende da:

- Cosa rischiamo di fare il blocking dell'applicazione, cioè quali sono i rischi reali del far usare operazioni che possono portare al deadlock;

- Frequenza relativa di `add()` e `remove()` rispetto al totale;
- Overhead necessario sulla piattaforma per introdurre riferimenti con mark, cioè se io ho una macchina virtuale che per poter fare questi riferimenti con mark, si comporta in maniera molto peggiore, sono molto più lente le operazioni AtomicMarkable, quindi in questo caso può essere accettabile usare i lock, magari usando Optimistic o Lazy piuttosto che fare altro;

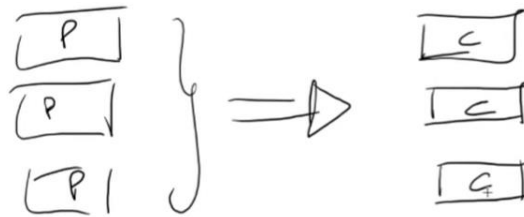


8. Code

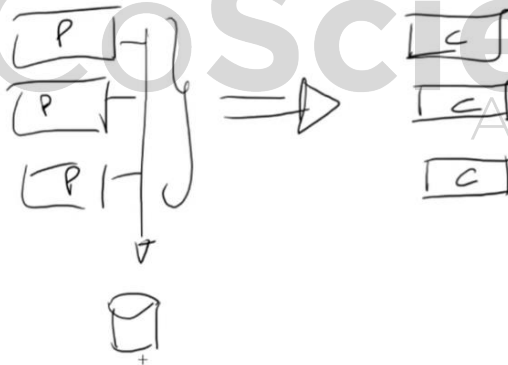
8.1 Introduzione alle Code

Si tratta di strutture dati simili ai set visti in precedenza, ma con due differenze:

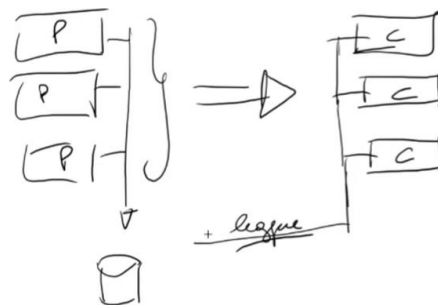
- Senza contains e con la possibilità di inserire duplicati, (quando inseriamo un elemento, l'inserimento andrà sempre a buon fine);
- Utili come buffer per produttore/consumatore, più precisamente: Molto spesso ci sono, essenzialmente dei processi produttore che si occupano di produrre dei dati per i consumatori, quindi c'è una sorta di invio di dati che fa in modo di legare i dati che vengono prodotti da una parte del nostro sistema ad un'altra parte che invece usa questi dati.



Ora è chiaro che una estremizzazione, diciamo naturale ed è una prima soluzione è fare in modo che tutti questi vadano a scrivere ad esempio all'interno di un database.



Database dal quale ovviamente, tutti i consumatori vanno poi a leggere.

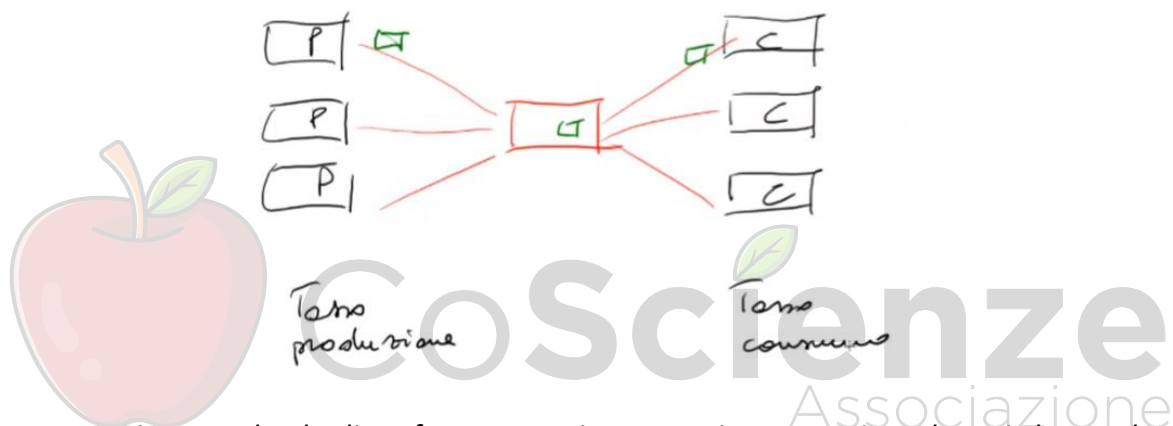


È chiaro che l'utilizzo di memoria di massa è una maniera, a volte non eliminabile, ma è una maniera estremamente pulita di risolvere il problema, noi ci stiamo assicurando anche che i dati prodotti dai

produttori siano assolutamente permanenti, stiamo assicurando che l'affidabilità del nostro sistema di storage garantisce la memorizzazione di questi dati per tempi lunghissimi, e in un certo senso completamente effettua quello che è un disaccoppiamento tra quello che viene realizzato all'interno dell'insieme di produttori da quello che viene consumato dall'insieme dei consumatori, cioè produttori e consumatori sono completamente disaccoppiati, addirittura in maniera completamente asincrona, potrei avere i produttori che producono qualcosa in una settimana e i consumatori che lavorano su questi dati a fine mese. Questa cosa ha degli svantaggi, cioè andare a usare dello storage in maniera così massiccia inficia sui tempi di accesso, e accedere alla memoria di massa è chiaramente meno efficiente.

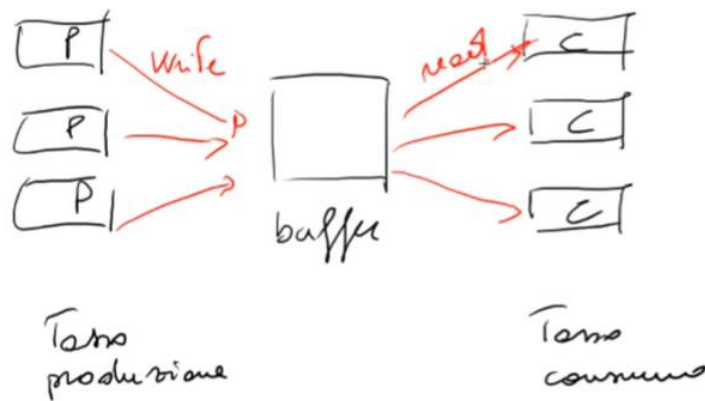
Qual è la maniera più efficiente?

Potremmo fare in modo di far inviare direttamente tutto quello che viene prodotto dai produttori direttamente ai consumatori, quindi basta in un certo senso, piazzare un processo che non fa altro che smistare questi dati in questa maniera e questo è sicuramente molto più efficiente, cioè si produce un certo dato e poi questo dato viene smistato a un consumatore.

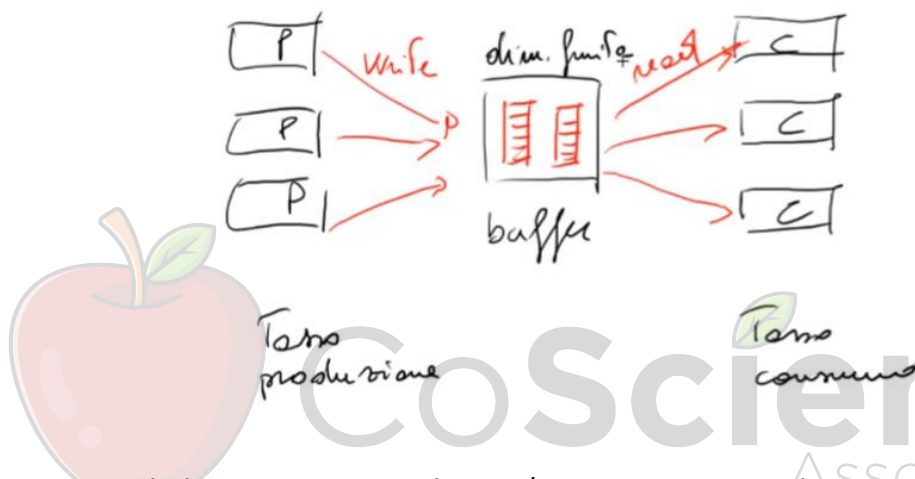


Chiaramente stiamo parlando di un forte accoppiamento, ci sono tanti produttori che producono dati e tanti consumatori che consumano dati. Qual è il tasso di produzione a cui i dati vengono prodotti? Quanti dati per unità di tempo vengono prodotti dall'insieme dei produttori? Il corrispettivo tasso di consumo è commisurato al tasso di produzione? E se il tasso di produzione varia, quindi ci sono momenti in cui ci sono un sacco di dati prodotti, e i consumatori sono pochi cosa succede?

Questi problemi sono problemi che pongono come soluzione naturale, la soluzione di usare una struttura dati interna, quindi non più un processo senza memoria, ma una struttura dati interna che funziona da buffer. Essenzialmente i produttori scrivono in questo buffer e i consumatori leggono da questo buffer.



Ora il vantaggio è che questa è una struttura dati, quindi dentro (nel buffer), c'è lo spazio necessario per poter produrre questi dati:



All'interno quindi di un processo produttore/consumatore, tra i due estremi, quello dell'usare la memoria di massa oppure usare un processo senza memoria, che non fa altro che prendere e redirigere i dati, quindi si riceve un dato e lo si manda subito, si apre un socket e lo manda al consumatore, riceve dal produttore e lo manda al consumatore. Tra i due estremi c'è la possibilità di avere una struttura dati in memoria, di dimensione finita, quindi chiaramente questo è un problema. Chiaramente la memoria di massa può arrivare a dimensioni estremamente più ampie rispetto a quelle della memoria centrale. In questa situazione ereditiamo i vantaggi delle due soluzioni precedenti, cioè l'efficienza del lavorare in memoria, quindi lavorare con un certo ordine di grandezza di tempo per quanto riguarda le operazioni che almeno due o tre volte o più basso o più veloce delle operazioni che vengono fatte quando si lavora sulla memoria di massa. D'altro canto, siamo in grado, come accadeva nella memoria di massa, di poter essere abbastanza disaccoppiati. Questo significa che i processi produttori possono produrre dati più o meno alla velocità che vogliono e i consumatori possono consumare/trattare i dati prodotti alla velocità e alla possibilità che hanno di farlo. Il punto sta in questa dimensione, nel senso che i produttori possono produrre per un certo periodo di tempo limitato, alla velocità che vogliono, la velocità e il tempo limitato, la velocità di produzione cioè il numero, il tasso, il numero di elementi prodotti per unità di tempo e il tempo in cui loro possono lavorare alla massima velocità detta. In pratica la dimensione della memoria che stiamo utilizzando come buffer, cioè quando la memoria buffer si riempie i

produttori devono necessariamente fare qualcosa, quindi in un certo senso la dimensione del buffer rappresenta un interruttore, un potenziometro, in cui si può usare per dare più spazio alla velocità di produzione da parte dei produttori o meno spazio, meno libertà ai produttori se il buffer è più piccolo, in maniera tale poi da permettere ai consumatori di consumare al loro tasso di esecuzione.

8.1.1 Diversi tipi di Pool

Sono presenti diversi tipi di pool, tra cui:

- **Bounded:** con capacità fissata, cioè siamo in grado, avendo fissato la velocità, di tenere a bada sincronizzati i produttori e consumatori. Per sincronizzati si intende che in questo caso la produzione non può avvenire in maniera totalmente libera, ma quando si ferma, quando si riempie il buffer, i produttori vengono avvisati e si regolano di conseguenza, quindi rallentano o possono bloccare la propria produzione;
- **Unbounded:** abbiamo modo di poter riversare parte del nostro buffer su memoria di massa, quindi la struttura dati in mezzo è parzialmente in memoria e parzialmente in memoria di massa;
- **Metodi totali:** metodi che non attendono che certe condizioni diventino vere e restituiscono errori/eccezioni se queste condizioni non sono vere. Ad esempio: c'è una coda piena, si prova ad inserire, la coda è piena e viene restituito un errore, in maniera tale che da produttore, prendo in considerazione questa cosa e si prende una decisione.
- **Metodi Parziali:** l'esecuzione attende, fa la wait della condizione necessaria per la realizzazione dell'operazione. Ad esempio: si vuole inserire un elemento, la coda è piena, il metodo rimane bloccato fino a quando la cosa non ci permette l'inserimento, fino a quando la coda non verrà svuotata. In questo caso la gestione è molto più facilitata dal produttore, nel senso che non deve gestire l'errore, ma deve gestire il fatto che quest'operazione lato produttore deve essere fatta in un thread perché questa operazione si può bloccare per un tempo indefinito. Si sta aspettando che la coda si svuoti un poco per permettere effettuare l'inserimento.
- **Metodi Sincroni:** attesa di un altro metodo in overlap sulla propria chiamata (metodi che fanno in modo di sincronizzarsi con la corrispondente chiamata di qualcuno che consuma, cioè un produttore produce un elemento e resta in attesa fin quando questo elemento non è stato consumato da un consumatore, questo offre il vantaggio che non solo il dato viene passato, ma sappiamo anche esattamente quando è stato passato;
- **Fairness:** FIFO, LIFO ecc;

La Struttura Coda come Pool:

```
public interface Pool<T> {  
    void put(T item);  
    T get();  
}
```

La Pool ha due metodi: `put()` e `get()`.

Il `put()` non restituisce un booleano. Come si diceva in precedenza, nei set potrebbe non essere possibile inserire un elemento, dato che potrebbe già essere stato inserito, in quel caso è un errore, bisogna restituire boolean `false`, in questo caso è sempre possibile inserire un elemento.

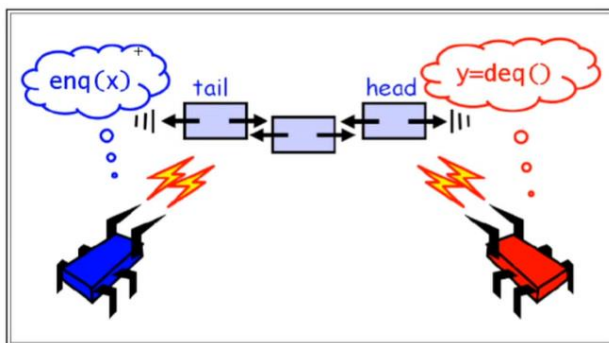
- Una cosa sequenziale è una sequenza di elementi di tipo `T`;
- Con operazioni di enqueue e dequeue;
- Una coda implementa naturalmente un pool (con il mapping della parte di `put()` / `get()` rispettivamente a `enq()` / `deq()`);

8.2 Implementazioni Concorrenti di Code

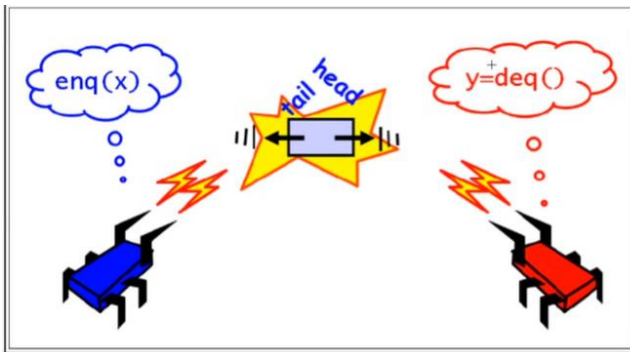
8.2.1 Coda Bounded Partial

Bounded significa c'è un limite alla dimensione della coda. Partial significa che il metodo può anche attendere che la condizione si verifichi. L'idea dell'algoritmo:

- In generale, `enq()` e `deq()` lavorano alle parti opposte della coda, dovrebbero in principio poter lavorare concorrentemente, in principio perché è ovvio che ci sono delle situazioni di contorno che sono particolari, cioè enqueue e dequeue quando la coda è completamente vuota interagiscono insieme, quando la coda è sufficientemente ampia, enqueue e dequeue avvengono agli estremi diversi e quindi possono avvenire in maniera concorrente.
- Implementazione con lista linkata (ma si può ottenere una implementazione simile con array).



Abbiamo due thread, il blu è un produttore a coda, quindi va sulla tail e mette un certo elemento, mentre il rosso è un consumatore che consuma gli elementi e quindi legge dalla dequeue e mette in una variabile che poi utilizzerà per accedere.



Ovviamente il problema capita nel caso in cui abbiamo la cosa vuota, cioè dove sia tail che head sono praticamente insieme a puntare allo stesso elemento, e questo è chiaramente una situazione al contorno.

Classe BoundedQueue<T>

```

1  public class BoundedQueue<T>{
2      ReentrantLock enqLock, deqLock;
3      Condition notEmptyCondition, notFullCondition;
4      AtomicInteger size;
5      Node head, tail;
6      int capacity;
7
8      public BoundedQueue (int _capacity){
9          capacity = _capacity;
10         head = new Node (null);
11         tail = head;
12         size = new AtomicInteger(0);
13         enqLock = new ReentrantLock();
14         notFullCondition = enqLock.newCondition();
15         deqLock = new ReentrantLock();
16         notEmptyCondition = deqLock.newCondition();
17     }
18     public void enq (T x){
19         boolean mustWakeDequeuers = false;
20         enqLock.lock();
21
22         try{
23             while (size.get () == capacity)
24                 notFullCondition.await();
25
26             Node e = new Node(x);
27             tail.next = tail = e; // inserimento
28
29             if(size.getAndIncrement()==0)
30                 mustWakeDequeuers=true;
31         }finally{
32             enqLock.unlock();
33         }
34
35         if(mustWakeDequeuers){
36             deqLock.lock();
37             try{
38                 notEmptyCondition.signalAll();
39             }finally{
40                 deqLock.unlock();
41             }
42         }
43     }
44 }
45     public T deq (){
46         boolean mustWakeEqueuers = false;
47         deqLock.lock();
48
49         try{
50             while (size.get () == 0)

```

```

51         notEmptyCondition.await();
52         result = head.next.value();
53         head = head.next;
54         if (size.getAndIncrement() == capacity)
55             mustWakeEqueuers = true;
56     } finally {
57         deqLock.unlock();
58     }
59 }
60
61     if (mustWakeEqueuers) {
62         enqLock.lock();
63         try {
64             notFullCondition.signalAll();
65         } finally {
66             enqLock.unlock();
67         }
68     }
69     return result;
70 }
71

```

Definiamo alcune grandezze:

Definiamo due lock, il concetto che in principio enqueue e dequeue possono lavorare da soli, significa che tutti gli enqueueer si accodano sull'enqueue lock e tutti i dequeueer si accodano sul dequeue lock (Riga 2). Abbiamo due condizioni (Riga 3) che si chiamano notEmptyCondition e notFullCondition. Da notare un'ambiguità, ci si aspettava Condition Empty e la Condition Full, qui invece è concettualmente al contrario, quindi la notEmpty significa non è più Empty, la notFull significa non è più full quindi è possibile accodare. Usiamo la dimensione della coda (Riga 4) come un integer atomico, che andremo ad incrementare o decrementare sulla base delle condizioni, ma questi incrementi e decrementi verranno fatti in maniera atomica non interfogliabile. Abbiamo due nodi sentinella (Riga 5) head e tail. Abbiamo la capacity (Riga 6) cioè la capacità massima di questa coda. L'inizializzazione (Riga 8) è abbastanza banale, il parametro viene messo nella capacity, viene creata la head (Riga 10) e la tail punta alla head (Riga 11), quindi abbiamo le due variabili head e tail che puntano lo stesso nodo all'inizio, head = tail è la condizione che ci dice che la coda è vuota. La dimensione viene inizializzata a 0 (Riga 12) e ci sono due condizioni che vengono create dai due lock che vengono creati da enqLock (Riga 13) e da deqLock (Riga 15). Dall'enqLock si ricava la notFullCondition (Riga 14), dalla deqLock si ricava la notEmptyCondition (Riga 16). La Enqueueer usa un flag che si chiama "mustWakeDequeueers" (Riga 19) che è un flag che viene inizializzato a false e che alla fine farà decidere se si devono svegliare, se bisogna fare delle signal su possibili dequeueer che sono in attesa oppure no. La prima cosa che si fa è essere sicuri di accedere ME con altri enqueueers (Riga 20), è una cosa importante. È chiaro che gli enqueueers si accodano, ma i dequeueer possono lavorare senza nessun problema in maniera concorrente.

Una volta acquisito il lock (Riga 23) siamo sicuri all'interno di essere l'unico enqueueer tra gli enqueueer che sta lavorando, quindi ci rendiamo conto se la coda è piena, ricordiamo che stiamo facendo la enqueueer, vogliamo verificare che possiamo effettivamente fare la enqueueer, se la size.get è uguale alla capacity, quindi la dimensione della size (questo intero atomico) è uguale alla capacità massima, andiamo in attesa (Riga 24), aspettiamo segnali (da deq()), gli enqueueer si mettono in attesa sulla condizione che verrà segnalata dai dequeueer, perché ovviamente gli enqueueer stanno aspettando per accodare un elemento, però si vuole che questa cosa viene fatta

e si trova in una coda piena, quindi si è in attesa che qualcuno estragga elementi, chi li estrae. I dequeuer saranno i dequeuer che segneranno la `notFullCondition`. In ogni caso, ogni volta che veniamo svegliati controlliamo se la `size` (Riga 23-24) è effettivamente diminuita. Se la `size` non è effettivamente diminuita torniamo a fare la `await`, se siamo arrivati (Riga 26) è perché siamo sicuri che:

1. Abbiamo acquisito di nuovo il lock, siamo di nuovo l'unica enqueueer che sta lavorando;
2. La dimensione è tale che non è uguale a `capacity`, quindi minore di `capacity`;

Facciamo l'inserimento all'interno della coda, nel senso che mettiamo `tail.next` (Riga 27) e poi mettiamo l'elemento e all'interno della `tail` (Riga 27), quindi avanziamo la `tail` e facciamo in modo di poter inserire l'elemento. Se abbiamo inserito in una cosa vuota, lo capiamo perché usiamo `getAndIncrement` (Riga 29) che significa, dammi il valore vecchio, intanto però metti incrementa di 1. Se il valore vecchio era 0 significa che la coda era vuota, e quindi dobbiamo svegliare i dequeuer, settiamo quindi questo flag a `true` (Riga 30). Intanto usciamo dalla parte di ME, questa parte (Riga 32) non è fatta in ME perché noi stiamo liberando degli enqueueer che stanno andando ad accodare altri elementi, se devo andare a svegliare dequeuer acquisisco il lock dei dequeuer (Riga 37) e questo è purtroppo necessario perché noi vogliamo fare la `signalAll` (Riga 39) su `notEmptyCondition`, quindi per poter fare la `signal` su una `condition` dobbiamo acquisire il lock da cui la `condition` è stata creata e questo significa, una cosa contro-intuitiva, cioè che gli enqueueer acquisiscono anche il lock dei dequeuer, lo fanno perché li devono poter risvegliare. Nel caso in cui c'è la `notEmptyCondition` non è più una coda vuota, perché siamo passati da 0 a 1. Intanto altri enqueueer stanno lavorando, quindi potenzialmente possono esserci più di 1 elemento e intanto i dequeuer vanno svegliati.

Si crea un deadlock sul lock con altri dequeuers? Sapendo che la dequeue essenzialmente è speculare a questa, cioè la dequeue è speculare dove si legge enqueue si scrive dequeue, dove legge la `notFull` scrive `notEmpty` ecc...

Non si crea, perché non acquisisco il lock all'interno dell'altro lock, cioè si acquisisce il lock dell'enqueue, lo rilascio e dopo di che acquisisco quello per la dequeue e lo rilascio. Questo è il motivo per il quale posticipiamo (Riga 30) questa operazione a dopo che abbiamo fatto l'`unlock`. Quindi la possibilità di deadlock è evitata facendo le due operazioni di lock una dopo l'altra, quindi ogni dequeuer quando nella parte speculare di questa, ogni dequeuer quando si è andati in `await` (Riga 51) sulla `notEmpty` ha rilasciato il lock, e non siamo in grado di essere sicuri che non c'è nessun problema. La dequeuer è estremamente simile, è veramente speculare, c'è `mustWakeEnqueueer` (Riga 46) poi si acquisisce il lock con gli altri dequeuer (Riga 47). Questa volta il ciclo in cui si va sempre in `await`, mentre la dimensione è 0 (Riga 50), dopo di che si prende il risultato come `head.next.value` (Riga 52) si avanza la `head` (Riga 53). La `head`, cioè il nodo che stavamo usando, il nodo che abbiamo appena estratto diventa la sentinella, ovvero stiamo posticipando il suo passaggio alla Garbage Collection di uno step, quindi il nodo che noi abbiamo estratto non viene cancellato dalla coda, ma verrà cancellato alla prossima estrazione, perché è un nodo che ci serve come `head`.

(Riga 54) Anche qui totalmente speculare, invece di fare la `getIncrement` su `size` e controllare se è 0 facciamo `getAndDecrement` e controlliamo se è la `capacity`, allora se è `capacity` significa che

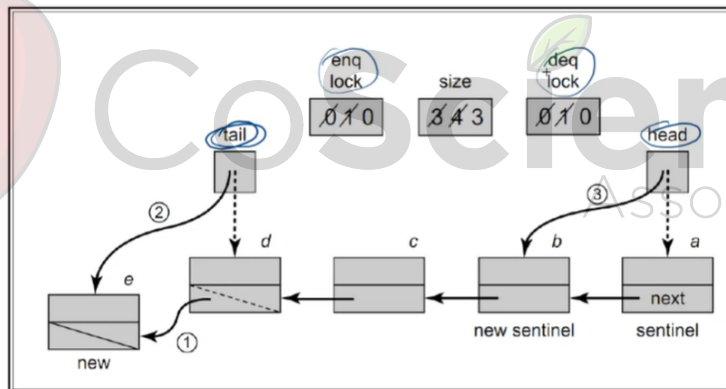
dobbiamo svegliare gli enqueueer perché il not Any More Full non è più piena come coda ed è esattamente quello che facciamo (Riga 64).

```
1  protected class Node{
2      public T value;
3      public Node Next;
4
5      public Node(T x){
6          value = x;
7          next = null;
8      }
9  } // end inner class Node
```

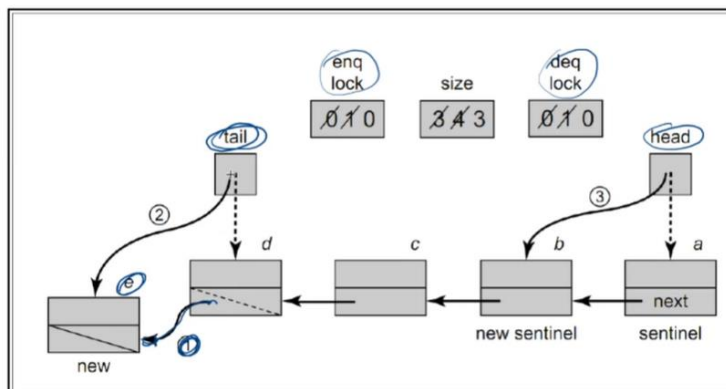
Questa è la definizione del nodo interno, (Riga 5) costruttore. (Riga 6) link a null.

Vantaggi: il consumatore che ha chiamato dequeue, non deve gestire nessuna situazione complicata se non la possibilità che questo metodo si blocchi in attesa che la coda venga riempita, quindi se volete basta che lui fa in modo che questa invocazione di metodi venga fatta in un thread separato la sua computazione può proseguire senza problemi, in certi casi può essere estremamente utile che sia bloccante, perché il consumatore potrebbe non dover fare niente fino a quando non ha ricevuto qualcosa. In questo caso è proprio naturale per chi scrive il dequeuer che sta fermo, chiama il dequeue e si blocca, non fa niente fino a quando non è stato inserito l'elemento all'interno.

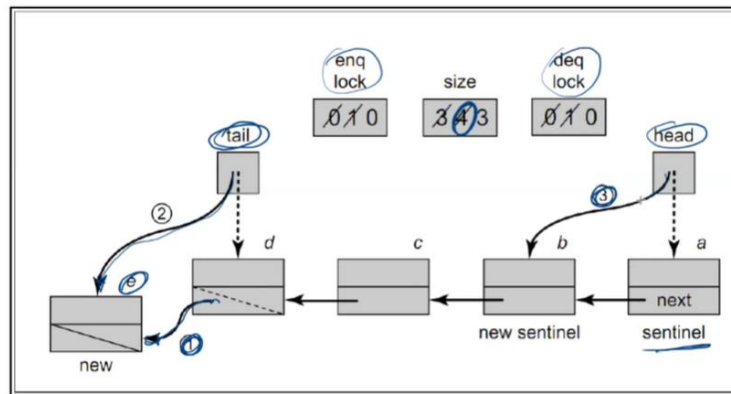
Un Esempio:



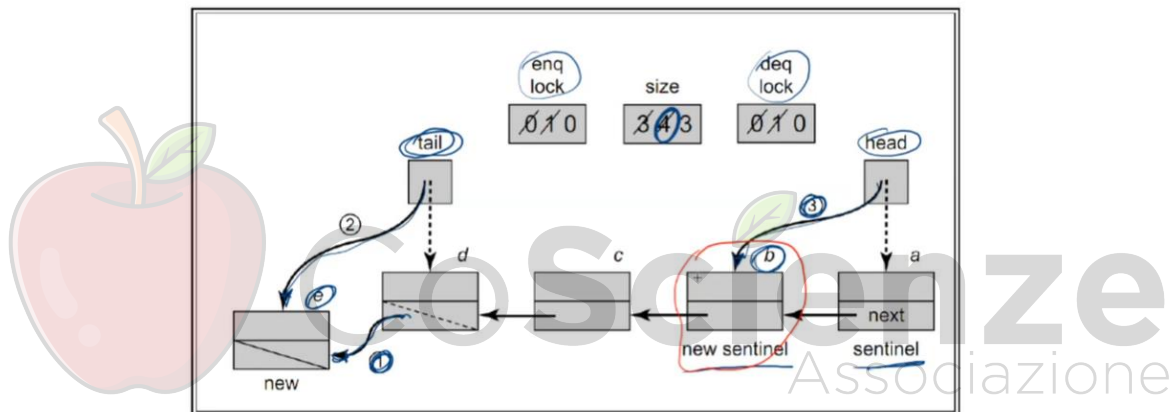
Abbiamo head e tail, e i lock (enq e deq) che risultano essere acquisiti o rilasciati e c'è come prima operazione (dove sta 1) l'accodamento dell'elemento e, quindi si fa puntare questo all'elemento e, poi si fa avanzare la tail al nuovo elemento. (operazione di estrazione):



Si aumenta la dimensione a 4, prima era 3, ora è diventata 4, mentre come operazione di cancellazione, di estrazione, questa era la sentinella:



Si avanza il puntatore (da head a b), quindi l'elemento b viene estratto, però rimane ad essere come nuova sentinella, quindi l'idea è che questo nodo rosso, nodo che contiene ancora tutt'ora il dato b è un nodo che non fa parte dell'insieme:



Tant'è vero che ci sono tre elementi, i tre elementi sono: c - d - e , mentre b è un nodo che serve solamente come sentinella e verrà rilasciato al Garbage Collector alla prossima estrazione, ovvero il prossimo avanzamento della head.

8.2.2 Coda Unbounded Total

Non ci preoccupiamo del bound, quindi essendo una lista linkata non ci si preoccupa della memoria, si va avanti fino a quando possiamo, mentre per quanto riguarda i metodi, non abbiamo metodi che si bloccano (nella coda precedente il metodo deq andava in await ad un certo momento, ossia quando si cerca di fare la deq da una coda vuota ed il metodo si bloccava). Ogni metodo ritorna/restituisce qualcosa al metodo chiamante:

- La complicazione principale sorgerà dalla limitazione in dimensione;
- Quando si inseriva in una coda piena, l'enqueuer si bloccava in attesa e doveva essere svegliato da dequeuers;

- Se si elimina il problema della dimensione, la soluzione è molto più semplice;
- Uso di due lock per sincronizzare tra di loro (a gruppi) gli enqueueers e tra di loro i dequeuers. Puntiamo ad ottenere l'indipendenza tra l'insieme degli enqueueer e l'insieme dei dequeuer, quindi noi possiamo avere un enqueueer e un dequeuer che insieme lavorano concorrentemente all'interno della lista;
- Unica condizione di boundary da verificare: dequeue in coda vuota che restituisce un'eccezione (quindi metodi totali). Stiamo parlando di qualcosa di unbounded, quindi l'inserimento è sempre possibile, perché non c'è una dimensione della coda, l'unica cosa che non possiamo eliminare è chiaramente quando la coda è vuota. Se si va ad estrarre un elemento dalla coda vuota, in qualche maniera devo mappare questo ad una restituzione di informazioni, la scelta è di lanciare un'eccezione, quindi metodi totali che lanciano un'eccezione.

Unbounded Total Queue:

```

1  public void enq(T x){
2      enqLock.lock();
3      try {
4          Node e = new Node(x);
5          tail.next = e;
6          tail=e;
7      }finally{
8          enqLock.unlock();
9      }
10 }
11 .
12 public T deq() throw EmptyException {
13     T result;
14     deqLock.lock();
15     try{
16         if(head.next == null){
17             throw new EmptyException();
18         }
19         result = head.next.value;
20         head = head.next;
21     } finally {
22         deqLock.unLock();
23     }
24     return result;
25 }

```

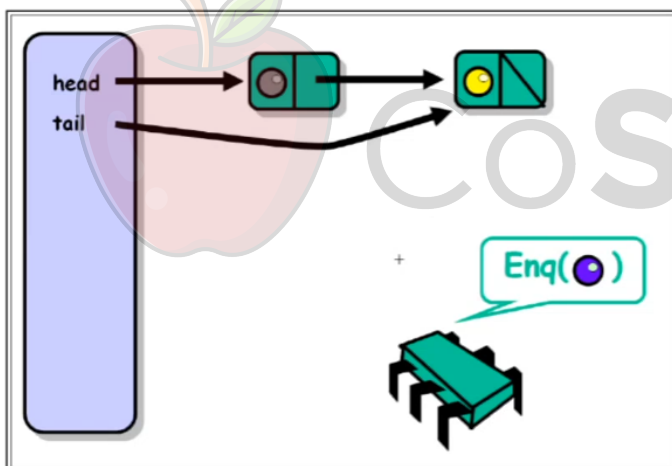
Gli enqueueer non devo fare altro che controllare la dimensione, possono sempre aggiungere (Riga 5). Nel momento in cui hanno acquisito il lock (Riga 2) sono sicuri che sono gli unici ad essere all'interno a lavorare sulla lista linkata e fanno l'inserimento e aggiornano la tail (Riga 6), dopo di che rilasciano il lock (Riga 8). La dequeuer, che stavolta ha un qualcosa di particolare (Riga 12 l'eccezione), acquisisce il lock (Riga 14) e l'unico controllo che fa è questo (Riga 15 a 18). Nel momento in cui andiamo a verificare che `head.next` è null (Riga 16), head non ha nessun elemento all'interno della lista. Questa asimmetria, cioè tail (Riga 6) punta sempre all'ultimo elemento della lista, head punta prima del primo elemento della lista, head ci sta sempre, tail al massimo punta a head nella lista vuota, se `head.next` è null (Riga 16) significa che la lista è vuota, che è esattamente quello che verifichiamo e lanciamo l'eccezione. Se non è stata lanciata l'eccezione, a questo punto si può fare l'inserimento, quindi prendiamo il value dal next (Riga 19) di head, questo non è il value di head, ma è il value del next di head e facciamo avanzare la sentinella (Riga 20). Ancora stessa

tecnica, riutilizziamo il nodo appena estratto come sentinella e rilasciamo il lock di dequeue (Riga 22) prima di aver fatto il result del risultato (Riga 24).

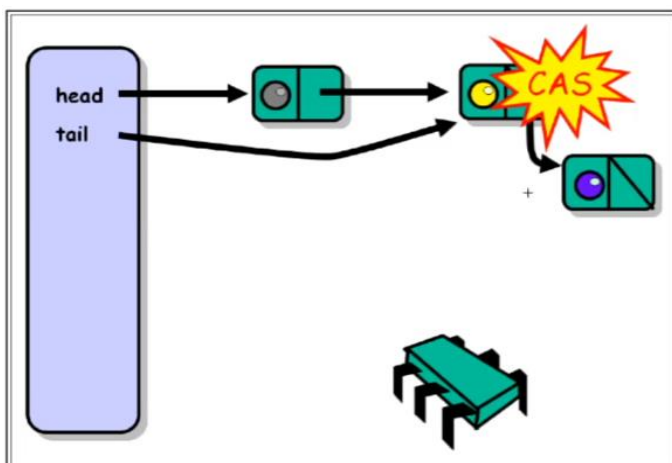
8.2.3 Coda Unbounded Lock-Free

È una naturale estensione dell'algoritmo precedente. Basata su una lista linkata di nodi, dove ogni campo `next` + una `AtomicReference<Node>`. Ci sono dei problemi. Modificare la lista linkata significa che ogni campo `next` è un `AtomicReference<Node>`, cioè le modifiche a `next` sono di tipo atomico, ovvero non sono interfogliabili tra thread. C'è solo un thread in un dato istante che può modificare questo riferimento. Gli altri vedranno subito dopo il risultato della sua operazione. La `enq()` viene fatta in maniera Lazy, principalmente in due passi: l'inserimento consiste nell'apprendere il nuovo nodo in coda e nel cambiare il campo `tail`. Deve essere chiaro che queste due operazioni di per sé possono interfogliarsi, e quello che si cerca di fare in maniera Lazy quanto più possibile l'inserimento effettuato, senza dar fastidio ma anzi segnalando che non si è potuto completare l'operazione per gli altri enqueueer. Ciascuna delle operazioni è singolarmente atomica, ma possono essere interleaved. Quindi una nuova `enq()` può trovarsi a dover aiutare a completare una `enq()` non ancora completata. In tal caso si ha una `compareAndSet()`.

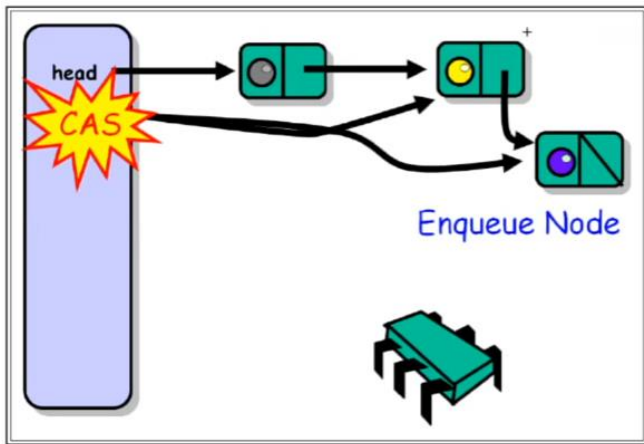
LockFree:



Abbiamo la variabile `head` e `tail`, che puntano ai due nodi. In tal caso la lista, prima che venga fatta la `enq()`, è 1. Questo perché la `head` non punta a un vero nodo, ma il primo vero nodo è puntato dal `next` della `head`. Il nodo grigio è l'elemento che è stato estratto al passo precedente, e alla prima estrazione succederà che verrà eliminato il nodo giallo. C'è un solo elemento nella lista. Il nodo verde vuole fare la `enq()`.



La condizione che si sta verificando è che la `tail` punti a un nodo il cui campo `next` è null. A questo punto il nodo verde fa `compareAndSet()` e in maniera atomica prende il campo `next`. Il passo successivo è quello di andare sulla `tail`, effettuare una `compareAndSet()` che fa puntare al nodo viola. Tra le due `compareAndSet()` si possono inserire anche altri. Questa situazione è però una situazione distinguibile. Nel caso di un thread enqueueer, questo si rende conto che la `tail` punta a un nodo il cui campo `next` non è null,



perché punta a un altro nodo. È successo che se il thread vede questa situazione, cioè che il campo tail punta a un nodo che non ha il nodo con il campo next a null, allora significa che c'è ancora qualcuno che sta facendo una `enq()` e non ha ancora terminato.

L'idea è che se il thread si interfaccia con un altro thread, esso è in grado di riconoscerlo e addirittura in grado di aiutarlo, cioè fare in modo che la tail punti al nuovo nodo (viola).

Il problema del LockFree sta nel fatto che le due operazioni di `CompareAndSet()` sono atomiche, mentre insieme non lo sono. La tail si può riferire quindi al vero ultimo nodo della lista oppure si può riferire al penultimo nodo, perché c'è qualcuno che sta facendo `enq()`.

Come si riconosce un nodo che punta al penultimo nodo?

Il campo `tail.next` non è null.

Cosa si fa se si incontra un campo tail che punta prima della coda vera?

Si aggiusta la situazione, perché si cerca di inserire nella tail il riferimento all'ultimo nodo.

Classe Node:

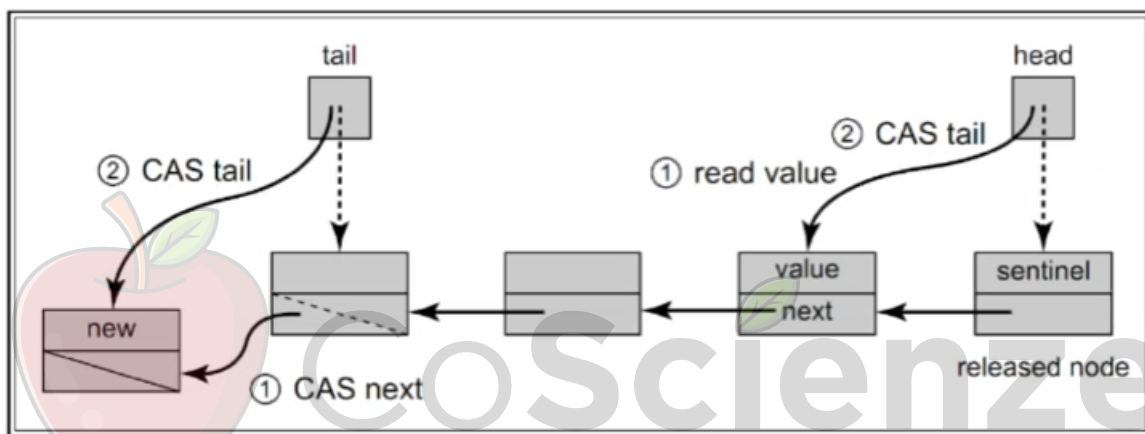
```
1 public class Node {
2     public T value;
3     public AtomicReference<Node> next;
4     public Node(T value) {
5         value = value;
6         next = new AtomicReference<Node>(null);
7     }
8 }
```

Classe Node (Riga 1) definisce il nodo della lista linkata. Il campo `AtomicReference<Node>` (Riga 6) permette modifiche atomiche ed è inizializzato a null.

LockFreeQueue:

```
1 public void enq(T value) {
2     Node node = new Node(value);
3     while (true) {
4         Node last = tail.get();
5         Node next = last.next.get();
6         if (last == tail.get()) {
7             if (next == null) {
8                 if (last.next.compareAndSet(next, node)) {
9                     tail.compareAndSet(last, node);
10                    return;
11                }
12            } else {
13                tail.compareAndSet(last, next);
14            }
15        }
16    }
17 }
```

Quello che si fa è prelevare il nodo dalla tail (Riga 4). Il nodo last è il nodo che la tail punta, inoltre viene prelevato il campo next (Riga 5). Se il nodo non ha successore (Riga 6) e se next è null, allora la situazione è statica per cui devono fare le operazioni di inserimento. Si prova a fare l'inserimento del nodo. Ci si aspetta che nel campo next di last (Riga 8) ci sia next, se è così allora atomicamente cerca di inserire il nodo come successore dell'ultimo. Se la `compareAndSet()` restituisce true viene fatta `tail.compareAndSet(last, node)`, cioè si prova a mettere nella tail che doveva esserci last, in tal caso viene messo node. In tal caso non si controlla il risultato perché non interessa questo, in quanto l'inserimento logico avviene in `last.next.compareAndSet()` (Riga 8), quindi questo è l'inserimento che garantisce che l'elemento è stato inserito, perché è stato inserito dopo l'ultimo. Si prova anche a fare l'inserimento fisico, ma se last è cambiato allora ritorna al metodo chiamante. Questo è il meccanismo di chi ha trovato la situazione staticamente corretta, eventualmente può essere stato disturbato da qualcuno che entra, il quale esegue la `else` (Riga 13) e trova che secondo lui è l'ultimo nodo ma in realtà non lo è, rendendosi conto che deve effettuare lo spostamento. Alla fine, si torna nuovamente al `while`.



Il problema in questo caso è che abbiamo la tail e si cerca di inserire un elemento. La prima operazione che viene fatta è cambiare il next, da null passa a puntare al nodo successivo, e si sposta la tail. Quello che si fa per estrarre un elemento consiste nel leggere il valore e si avanza la head. Come al solito questo elemento diventa la nuova sentinella, anche se contiene il valore appena cancellato. L'idea è che esiste una fase di inserimento logico (quando si inserisce il nodo come successore dell'ultimo) e una fase di inserimento fisico (quando la tail viene cambiata a puntare all'ultimo). La linearizzazione avviene sull'inserimento fisico, cioè quando avviene l'ultima `compareAndSet()`. In particolare:

- Se esiste un conflitto durante l'inserimento logico, si riparte con il ciclo `while`;
- Se esiste un conflitto durante l'inserimento fisico, qualcuno ha avanzato il puntatore al posto nostro, quindi ha reso la soluzione stabile;

LockFreeQueue:

```

1 public T deq() throws EmptyException {
2     while (true) {
3         Node first = head.get();
4         Node last = tail.get();
5         Node next = first.next.get();
6         if (first == head.get()) {
7             if (first == last) {

```

```

8             if(next == null){
9                 throw new EmptyException();
10            }
11            tail.compareAndSet(last, next);
12        } else {
13            T value = next.value();
14            if(head.compareAndSet(first, next))
15                return value;
16        }
17    }
18 }
19 }

```

La `deq()` è abbastanza simile. Anche qui c'è un ciclo `while(true)`. In questo caso bisogna controllare anche l'ultimo nodo della lista. Se non cambia nulla (Riga 6) ci possono essere dei problemi: se `first` è uguale a `last` (Riga 7) e il campo `next` di `first` è `null`, allora si lancia un'eccezione. Se `first` è uguale a `last` ma il campo `next` non è `null`, bensì punta a qualcosa. Questo significa che c'è stato un conflitto tra una `deq()` e una `enq()`, perchè la `deq()` sta facendo mentre proprio quella `enq()` sta inserendo un elemento ma non ha ancora completato perché `tail` punta ancora allo stesso valore di `head`. A questo punto quello che fa è `compareAndSet()` e lo fa puntare a `next`. Se ci sono nodi nella lista, la situazione è abbastanza semplice. Si preleva il valore del campo `next` (Riga 13) e se la `head` punta ancora a `first`, viene messo il campo `next`. Se il controllo non va a buon fine (Riga 14) va a monte tutto, si ritorna al `while` e si ritorna ad eseguire.

Una situazione di Border:

```

1  public T deq() throws EmptyException {
2      while (true){
3          // ...
4          if(first == head.get()) {
5              if(first == last){
6                  if(next == null){
7                      throw new EmptyException();
8                  }
9                  tail.compareAndSet(last, next);
10             } else {
11                 // ...
12             } // end while
13 } // method deq()

```

Un thread sta accodando `b` ma non ha ancora completato (`tail` punta ancora ad `a`) perché la `head` punta ad `a`. Se un thread inizia la `deq(a)` sposta la `head` su `b`, mentre `tail` non è stato ancora aggiornato. A questo punto la soluzione è un aiuto nello spostare la `tail`.

Dove vengono linearizzate le operazioni?

Se l'operazione non lancia l'eccezione, si linearizza quando viene fatta la `compareAndSet()` prima di restituire il valore. Questo è il momento in cui accade la `deq()` di un elemento, altrimenti si lancia l'eccezione e si rende conto che la coda è effettivamente vuota.

8.2.4 Code con Rendezvous

L'idea è di accoppiare insieme produttori e consumatori anche dal punto di vista del tempo. Si vuole una sincronizzazione tra quelli che consumano rispetto a quelli che producono. La sincronizzazione richiesta può essere più stretta: Un nodo che fa una enqueue deve attendere (bloccato) fino a quando il suo elemento viene rimosso da un altro thread. Si tratta di un'implementazione abbastanza semplice, ma costosa in termini di sincronizzazione. È possibile migliorarla in termini di condition e altre tecniche.

SynchronousQueue:

```
1  public class SynchronousQueue<T>{
2      T item = null;
3      boolean enqueueing;
4      Lock lock;
5      Condition condition;
6      // ...
7      public void enq(T value) {
8          lock.lock();
9          try {
10             while (enqueueing){
11                 condition.await();
12                 enqueueing = true;
13                 item = value;
14                 condition.signalAll();
15                 while (item != null){
16                     condition.await();
17                     enqueueing = false;
18                     condition.signalAll();
19                 } finally {
20                     lock.lock();
21                 }
22             } // end enq
```

Item è l'elemento su cui si fa Rendezvous (Riga 2). C'è un booleano e un lock che è unico per tutti i metodi, infine c'è la condition. Una volta acquisito il lock per la enqueueer (Riga 8), si attende che un enqueueer faccia rendezvous. Un enqueueer acquisito prima ha bloccato gli altri, che stanno facendo la condition. In seguito, viene messo il valore dentro item (Riga 13) e sveglia tutti (incluso deq). Gli enqueueer tornano subito in `await()`, si svegliano, trovano `enqueueing` a true e tornano a fare `await()`. A questo punto si aspetta che l'elemento venga eliminato. Se c'è un dequeuer e lo ha letto, ha messo il value a null. Se value non è a null si va in `await()`, avendo lasciato l'`enqueueing`. Quando l'item è diventato null, si fa `enqueueing = false` e segnala tutti, compresi gli enqueueer. A questo punto trovano che `enqueueing` è a false, c'è qualcuno di loro che acquisisce il lock quando esce dalla `await()`, vede che `enqueueing` è a false, quindi procede e mette `enqueueing` a false.

```
72 public T deq(){
73     lock.lock();
74     try {
75         while (item == null){
76             condition.await();
77             T t = item;
78             item = null;
```

```

79         condition.signalAll();
80         return t;
81     } finally {
82         lock.unlock();
83     }
84 } // end class

```

La `deq()` è più semplice, questo perché il dequeuer deve semplicemente aspettare che in `item` ci sia qualcosa diverso da `null`. Se c'è è perché ha piazzato un enqueueer, e quando questo è vero (Riga4), allora viene estratto e mette `item` a `null`, perché l'elemento è stato estratto, segnalando a tutti che si è liberto spazio per un altro rendezvous. Il rendezvous avviene quando il dequeuer riesce a prelevare l'elemento e mette `item` a `null` e lo restituisce.

8.3 Gestione della Memoria e Problema ABA

Molte implementazioni gestiscono la memoria indipendentemente senza affidarsi al Garbage Collector del linguaggio. Questo porta ad alcuni vantaggi, tra cui:

- Efficienza (un'operazione di `new` richiede più tempo);
- Alcuni linguaggi non hanno un Garbage Collector, quindi l'implementazione dell'algoritmo non è semplice;
- Il Garbage Collector potrebbe non essere lock-free, creando ostacoli a concorrenza ed efficienza dell'esecuzione.

A questo punto risulta naturale pensare al riciclo dei nodi eliminati, cioè la possibilità di poter eliminare dei nodi per poi farli usare per l'aggiunta all'interno di un'altra operazione. L'idea è mettere tutti i nodi liberati all'interno di una lista di nodi free che sia `ThreadLocal`. Ogni thread ha al proprio interno una lista privata dei nodi che ha consumato, e qualora debba andare ad inserire può utilizzare per inserirli all'interno del pool.

Una Lista di Nodi Free:

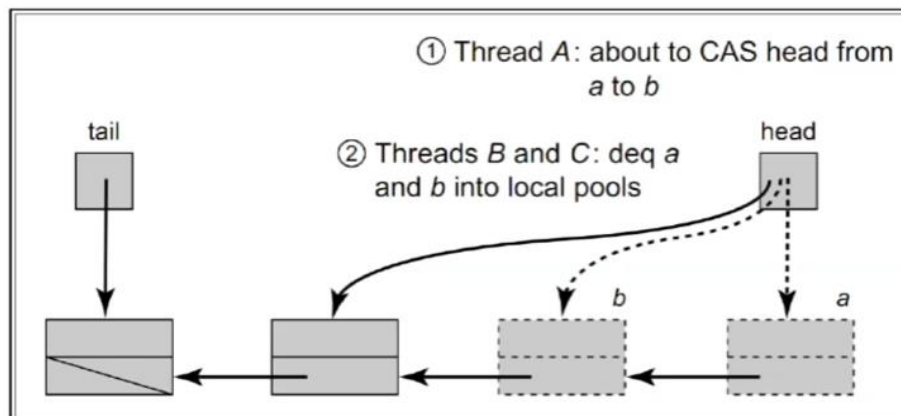
```

1 ThreadLocal<Node> freeList = new ThreadLocal<Node>{
2     protected Node initialValue() {
3         return t;
4     }
5 };

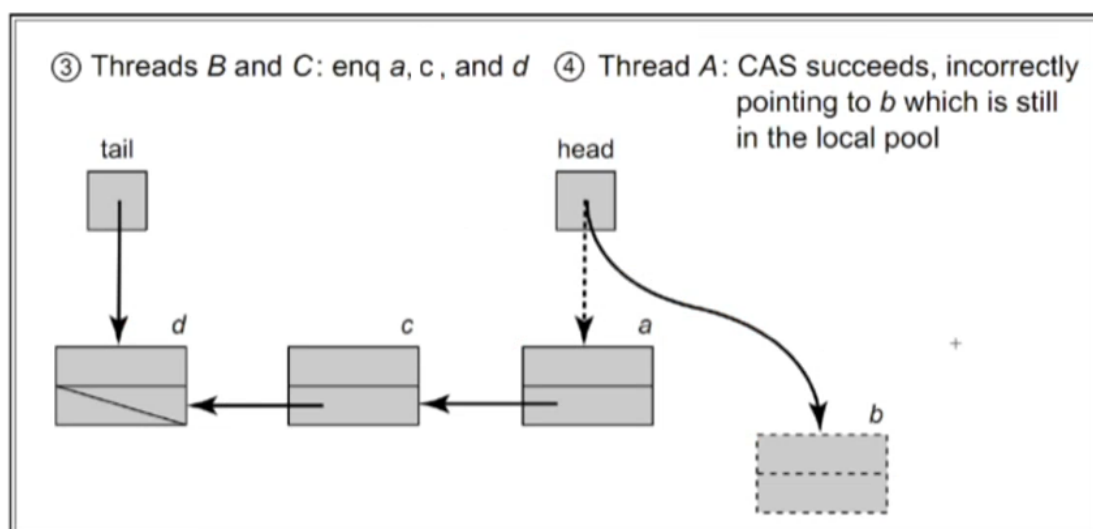
```

Quando una `enq()` ha bisogno di un nodo, cerca prima nella sua lista locale. L'accesso è privato ed efficiente, senza alcuna precauzione di alcun tipo. Se non ha nodi, allora esegue una `new`. L'efficienza è davvero ragionevole perché si auto-tara, cioè se un thread più o meno effettua lo stesso numero di `enq` e `deq` ugualmente disperse all'interno dell'asse temporale, quello che succede è che chiamerà la `new` molto raramente. Quando accade ciò, questo amplia l'insieme dei nodi free, andando a chiamarla molto meno in futuro.

8.3.1 Il Problema di ABA



Supponiamo che si effettui il riutilizzo dei nodi. Abbiamo tre thread: A, B e C. Supponiamo che il thread A sta per fare il `compareAndSet()` per spostare la head dal nodo *a* al *b*. La `compareAndSet()` ha la caratteristica di confrontare il valore presente all'interno, se questo corrisponde si effettua il set. Il concetto precedente di compare del valore consisteva nel fatto che quello che si era letto prima in tale momento è ancora presente, quindi si può andare a fare il set. Adesso mentre il thread A è prossimo a fare il `compareAndSet()`, i thread B e C fanno la `deq()` di *a* e di *b* all'interno del proprio local pool, cioè estraggono dei nodi. Estragendo dei nodi, il nodo *a* va nel local pool di *b*, e il nodo *b* va nel local pool di *c*. Abbiamo così congelato il thread A, mentre i thread B e C hanno le due operazioni di `deq()` e hanno estratto i nodi *a* e *b*. A questo punto, i thread B e C fanno la `enq()` di nuovi valori di *a*, *b* e *c*. Quello che succede è che la head punta al nodo *a*. Per quanto riguarda il thread A, esso sapeva che head puntava al nodo *a*.

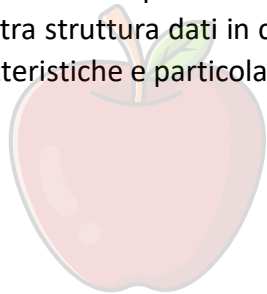


Quando viene eseguito, il `compareAndSet()` ha successo, cioè il `compareAndSet()` che stava facendo *a* in precedenza consisteva nello spostare la head da *a* verso *b*. Ora il valore di head punta effettivamente a un nodo *a*, che intanto è stato riciclato dal thread B, il quale lo ha estratto e

reinserito prima che il thread A facesse l'operazione di `compareAndSet()`. Il thread A si ritrova con la head che punta verso il nodo a, ma questo contiene informazioni diverse rispetto da quelle precedenti (lista, coda). L'operazione che fa il thread A è un'operazione errata, perché il nodo non fa parte della lista, ma anzi si trova da qualche parte nella free list di qualcuno dei thread. Il punto cruciale deriva dal fatto che la head punti ad a, e se si riciclano nodi, allora questo non garantisce che non ci sia stato interfogliamento di thread. Con il riciclo di nodi c'è il problema di ABA, cioè di un valore che è portato da a a b e poi ritorna ad essere a.

Il problema ABA si verifica durante algoritmi che usano la sincronizzazione come compare-And-set che fanno riuso della memoria, quando capita che un riferimento che sta per essere modificato, viene modificato in b e poi in a. Il thread che stava per modificarlo lo considera non cambiato e la chiamata ha successo. Anche se la struttura dati nel tempo è cambiata, si dovrebbero prendere contromisure adeguate. Una soluzione semplice è quella dell'`AtomicStampedReference<T>`, cioè un riferimento che è atomico ma che ha anche un intero, con un contatore che può essere incrementato ad ogni operazione. In questa maniera la variabile head punta al nodo a, però con un stamp di valore 121. A ogni operazione, lo stamp aumenta e se torna ad essere a, torna con uno stamp diverso. A questo punto la `compareAndSet()` fallisce non perché è tornato ad a, bensì perché lo stamp è cambiato.

La modifica all'operazione di `deq()` è immediata. In conclusione, Le code hanno permesso di vedere un'altra struttura dati in cui la concorrenza viene gestita in una certa maniera, ognuna con proprie caratteristiche e particolarità.



CoScienze
Associazione

9. Scheduling

9.1 Pool di Thread

Alcune applicazioni possono essere naturalmente divise in thread paralleli, ad esempio web server o applicazioni strutturate come produttori e consumatori. In seguito, analizzeremo applicazioni che hanno un parallelismo innato ma di cui non è chiaro come ottenere un vantaggio dall'esecuzione parallela.

9.1.1 Operazioni Concorrenti tra Matrici

Iniziamo dal problema di dover moltiplicare due matrici in parallelo. Ricordiamo che se a_{ij} è il valore in posizione (i,j) della matrice A , il prodotto C di due matrici A e B $n \times n$ è dato dall'elemento:

$$C_{ij} = \sum_{k=0}^{n-1} a_{ki} * b_{jk}$$

Partiamo con una soluzione che calcola in parallelo ogni singolo elemento: Il thread Worker in posizione i, j calcola C_{ij} .

Moltiplicazione con Thread:

```
1  class MMThread {
2      double[][] a, b, c;
3      int n;
4      public MMThread(double[][] myA, double[][] myB) {
5          n = myA.length;
6          a = myA;
7          b = myB;
8          c = new double[n][n];
9      }
10     void multiply() {
11         Worker[][] worker = new Worker[n][n];
12         for (int row = 0; row < n; row++)
13             for (int col = 0; col < n; col++)
14                 worker[row][col] = new Worker(row, col);
15         for (int row = 0; row < n; row++)
16             for (int col = 0; col < n; col++)
17                 worker[row][col].start();
18         for (int row = 0; row < n; row++)
19             for (int col = 0; col < n; col++)
20                 worker[row][col].join();
21     }
22     class Worker extends Thread {
23         int row, col;
24         Worker(int myRow, int myCol) {
25             row = myRow; col = myCol;
26         }
27         public void run() {
28             double dotProduct = 0.0;
29             for (int i = 0; i < n; i++)
30                 dotProduct += a[row][i] * b[i][col];
31             c[row][col] = dotProduct;
32         }
33     }
34 }
```

```
32         }  
33     } // fine inner class worker  
34 }
```

Vengono dichiarate tre matrici di double, il costruttore inizializza le due matrici in input a e b e dichiara c una matrice nxn di double. Con `multiply()` si crea una matrice di worker, cioè un array bidimensionale di thread. viene inizializzato un worker per ciascuna row col (Riga 14), e si fa partire per ciascuna [row] [col] i thread e si fa la join, cioè si attende che tutti i thread terminino, restituendo il controllo. La classe Worker viene inizializzata con `myRow` e `myCol`. Ogni worker sa esattamente la posizione i, j nella quale deve lavorare. Quando deve svolgere l'operazione, non fa altro che mettere in `dotProduct` la moltiplicazione dell'i-esimo elemento della riga di a con l'i-esimo elemento della colonna di b. Alla fine scrive dentro elemento `c[row][col]` e lo memorizza. Questa è una soluzione altamente parallela, però è una soluzione che per matrici di grandi dimensioni richiede milioni di thread. In questo c'è qualcosa che è probabilmente non ideale. Il codice è semplice da scrivere, inoltre l'efficienza sembra essere alta ma il problema è che ogni thread porta overhead, inficiando sull'efficienza. In pratica, questo approccio funziona solo con matrici piccole mentre le sue prestazioni sono molto scarse con matrici abbastanza grandi. Il motivo è che i thread richiedono memoria per gli stack ed altre informazioni. Creare, schedulare e distruggere thread richiede un notevole sforzo computazionale. Creare molti thread a vita breve è quindi un modo inefficiente di organizzare una computazione multi-thread. Una soluzione migliore consiste nel creare un pool di thread (di lunga vita) a cui vengono assegnati task di breve durata.

La soluzione naturale è passare dai thread, che hanno vita **lunga**, alla struttura in task, cioè dei thread che eseguono dei task. Si fanno eseguire più task allo stesso thread, arrivando ad una granularità più bassa. La dimensione dei task può essere plasmata attorno alla natura del problema, in maniera indipendente dall'hardware. La dimensione del pool di thread è commisurata alla piattaforma hardware. In aggiunta ai benefici sulle prestazioni, i pool di thread hanno un altro vantaggio: isolano il programmatore dai dettagli specifici della piattaforma come il numero di thread concorrenti che possono essere schedulati efficientemente. Con i pool di thread è possibile scrivere un singolo programma in grado di girare bene su un sistema monoprocesso e su un multiprocesso su larga scala. Forniscono una semplice interfaccia che nasconde i complessi trade-off dovuti alla piattaforma.

9.1.2 Thread pool per Operazioni su Matrici

In java un pool di thread è chiamato *Executor Service* (`java.util.Executor-Service`). Fornisce la possibilità di sottomettere task, aspettare fin quando un insieme di task sottomessi venga completato e cancellare task incompleti. Un task che non restituisce un risultato è di solito rappresentato come un oggetto *Runnable*, nel quale il lavoro è eseguito dal metodo `run()`. Un task che restituisce un valore di tipo T è rappresentato invece come un oggetto `Callable<T>`, in cui il risultato è restituito dal metodo `call()`. Quando un oggetto `Callable<T>` viene passato al pool, restituisce un oggetto che implementa l'interfaccia `Future<T>`. Questa non fa altro che permettere di accedere al risultato in maniera bloccante, di cancellare l'esecuzione `cancel()`, di

testare la situazione `isDone()`, `isCanceled()`. Ci sono una serie di `executor` che possono essere creati:

- **CachedThreadPool**: la possibilità di creare nuovi thread se servono, e vengono riutilizzati se idle;
- **FixedThreadPool**: si usa un set fissato di thread, che accedono ad una coda unbounded di task in attesa di essere eseguiti;
- **ScheduledThreadPool**: set di thread che possono essere eseguiti dopo un certo tempo, oppure con un certo intervallo;
- Possibile specificare una factory per i thread, in modo da generare i thread in qualche maniera specifica e dipendente dalla natura del problema;

Esempio di Future:

```
1 Callable<T> task = ...;
2 //...
3 Future<T> future = executor.submit(task);
4 //...
5 T value = future.get();
```

Viene dichiarato il task con il metodo `T call()`, viene sottomesso all'executor il task e il future (Riga 5) permette successivamente di accedere, rimanendo in attesa del risultato.

```
1 Runnable task = ...;
2 ...
3 Future<?> future = executor.submit(task);
4 ...
5 T value = future.get();
```

Se non ci sono dei valori restituiti, è possibile eseguire un `Runnable`, cioè un `Runnable` può essere visto come un thread tradizionale, mentre `future.get()` non fa altro che rimanere bloccato fino a quando il task non ha terminato, perché non c'è nessun valore da restituire. Costruiamo una classe `Matrix` che serve a suddividere una matrice, ma solamente in maniera logica, cioè le modifiche vengono effettuate sulla matrice originale (nessuna copia). Si usano i puntatori. L'idea è che il metodo che si usa è il metodo `split()`, il quale suddivide una matrice $n \times n$ in quattro sottomatrici $n/2 \times n/2$.

La Classe Matrix:

```
1 public class Matrix {
2     int dim;
3     double[][] data;
4     int rowDisplace, colDisplace;
5     public Matrix(int d) {
6         dim = d;
7         rowDisplace = colDisplace = 0;
8         data = new double[d][d];
9     }
10    private Matrix(double[][] matrix, int x, int y, int d) {
11        data = matrix;
12        rowDisplace = x;
13        colDisplace = y;
14        dim = d;
15    }
16 }
```

```

17     public double get(int row, int col) {
18         return data[row+rowDisplace][col+colDisplace];
19     }
20     public void set(int row, int col, double value) {
21         data[row+rowDisplace][col+colDisplace] = value;
22     }
23     public int getDim() {
24         return dim;
25     }
26     Matrix[][] split() {
27         Matrix[][] result = new Matrix[2][2];
28         int newDim = dim / 2;
29         result[0][0] =
30         new Matrix(data, rowDisplace, colDisplace, newDim);
31         result[0][1] =
32         new Matrix(data, rowDisplace, colDisplace + newDim, newDim);
33         result[1][0] =
34         new Matrix(data, rowDisplace + newDim, colDisplace, newDim);
35         result[1][1] =
36         new Matrix(data, rowDisplace + newDim, colDisplace + newDim, newDim);
37         return result;
38     }

```

Abbiamo il contenitore per la matrice, un displacement, cioè il calcolo della riga e colonna viene spazzato. L'elemento `ij` è riferito a un elemento della matrice originaria, che è spazzato in una certa quantità per le righe e colonne, indicate rispettivamente da `rowDisplace()` e `colDisplace()`. Il costruttore non fa altro che costruire la matrice mettendo displacement a 0. Per prendere l'elemento, si chiede l'elemento `[row][col]` (Riga 16) e viene restituito l'elemento in posizione `[row+rowDisplace][col+colDisplace]`.

Il metodo `split()` (Riga 25) è un metodo che restituisce un array 2x2 di matrici. Quando si splitta una matrice, viene creata una matrice di matrici 2x2, dove la dimensione è esattamente la metà (Riga 27). La matrice `result[0][0]` (Riga 28) è la matrice con gli stessi dati, `rowDisplace` e `colDisplace` è lo stesso della matrice originale. La matrice `result[0][1]` ha lo stesso `rowDisplace` della matrice originale ma a questo punto ha un `colDisplace` che parte da `n/2`. Lo stesso accade per la matrice `result[1][0]` dove il displacement avviene solo sulle righe. Alla fine, viene fatta la matrice `result[1][1]` che corrisponde al displacement sia sulla riga che sulla colonna. Quindi sono state create 4 matrici logiche, dove ognuna accede alla matrice `data`, e quello che fa è ricalcolare gli indici con il displacement che viene realizzato.

Come si fa l'operazione di somma tra matrici?

È possibile suddividere ricorsivamente la matrice in sottomatrici, creando task di dimensione sempre più piccoli, fino ad arrivare alla somma di sottomatrici di dimensione uno (si suppone n potenza di 2).

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} + \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix}$$

$$= \begin{pmatrix} A_{00} + B_{00} & A_{01} + B_{01} \\ A_{10} + B_{10} & A_{11} + B_{11} \end{pmatrix}$$

Si vuole realizzare la somma di $C = A+B$ e quello che si vuole fare è suddividere la matrice C in 4 sottomatrici, in maniera tale che le matrici A e B vengano anch'esse suddivise in 4 sottomatrici. Alla fine, il risultato che si ottiene è che $C_{00} = A_{00} + B_{00}$... fino ad arrivare alle sottomatrici di dimensioni 1.

MatrixTask Addizione:

```

1  public class MatrixTask {
2      static ExecutorService exec = Executors.newCachedThreadPool();
3      //...
4      static Matrix add(Matrix a, Matrix b) throws ExecutionException {
5          int n = a.getDim();
6          Matrix c = new Matrix(n);
7          Future<?> future = exec.submit(new AddTask(a, b, c));
8          future.get();
9          return c;
10     }
11     static class AddTask implements Runnable {
12         Matrix a, b, c;
13         public AddTask(Matrix myA, Matrix myB, Matrix myC) {
14             a = myA; b = myB; c = myC;
15         }
16         public void run() {
17             try {
18                 int n = a.getDim();
19                 if (n == 1) {
20                     c.set(0, 0, a.get(0,0) + b.get(0,0));
21                 } else {
22                     Matrix[][] aa = a.split(), bb = b.split(), cc = c.split();
23                     Future<?>[][] future = (Future<?>[][][]) new Future[2][2];
24                     for (int i = 0; i < 2; i++)
25                         for (int j = 0; j < 2; j++)
26                             future[i][j] =
27                                 exec.submit(new AddTask(aa[i][j], bb[i][j], cc[i][j]));
28                     for (int i = 0; i < 2; i++)
29                         for (int j = 0; j < 2; j++)
30                             future[i][j].get();
31                 }
32             } catch (Exception ex) {
33                 ex.printStackTrace();
34             } // fine metodo run()
35         }
36     }

```

Viene creato un pool (Riga 2). Il metodo `add()` permette di far partire il lavoro. Il metodo `add()` riceve due matrici `a` e `b`, su cui andare a effettuare la somma, e restituisce una matrice. Viene presa la dimensione `n` e crea la matrice risultante, cioè `c`. Con il `future` (Riga 7) viene sottomesso allo `executor` un task di addizione, cioè deve addizionare `a` e `b` e il risultato va in `c`. Il `submit` è una feature, e si attende quindi il risultato. Tutto il lavoro è realizzato dal fatto che si sta sottomettendo il task all'interno della coda.

La classe `AddTask` (Riga 11) implementa `Runnable` e non restituisce niente. La costruzione non fa altro che assegnare ad `a`, `b` e `c` le tre matrici passate e poi ha il metodo `run()`. Con il metodo `run()` non viene restituito alcun valore. Si vede la dimensione (Riga 18). Se `n == 1` possiamo settare l'elemento (0, 0) di `c` con l'elemento (0, 0) di `a` più l'elemento (0, 0) di `b`. l'elemento (0, 0) di `c`, `a` e `b` viene acceduto usando `set()` e `get()`, cioè usando il displacement. Ognuna di tali matrici ha costruito, riducendo la sua porzione fino ad arrivare a identificare una matrice 1x1, cioè la somma di tutti i displacement di tutte le matrici, fino ad arrivare alla posizione giusta della matrice originaria da cui si è partiti. Questo avviene se `n == 1` (Riga 19), altrimenti si deve fare la suddivisione ricorsiva, quindi si splitta `a`, `b` e `c`. Splittare non è un'operazione costosa, ma semplicemente si aggiunge il displacement corretto, per fare in modo che la matrice venga acceduta in maniera indipendente dal displacement come se fosse una radice. A questo punto si sottomettono i quattro subtask (Riga 27), ciascuno per la sottomatrice `aa[i][j]`, `bb[i][j]` e `cc[i][j]` e si attendono i risultati (Riga 30). Il metodo `run()` termina, e quindi termina il task.

9.2 Scheduling

Bisogna chiedersi quanto costa fare lo scheduling, perché lo scheduling è un overhead. Tutto il vantaggio della concorrenza viene annullato da uno scheduling molto complesso, quindi tutti i vantaggi vengono annullati.

9.2.1 Analisi del Parallelismo

Pensiamo alla computazione multithread come un grafo direzionato aciclico (DAG), dove ogni nodo rappresenta un task ed ogni arco direzionato collega un task predecessore ad un task successore, dove il successore dipende dal risultato del predecessore. Vengono rappresentati anche archi per trasmettere il risultato dal figlio al padre, cioè l'invocazione e la restituzione del parametro.

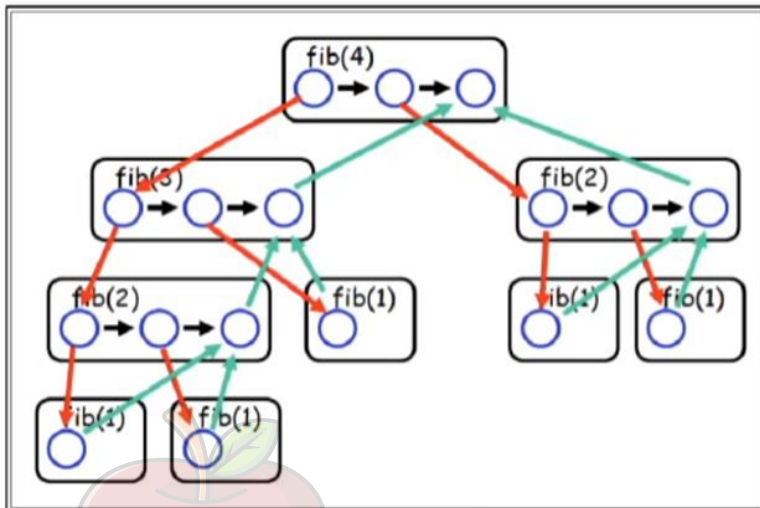
Il terzo problema più popolare è quello di Fibonacci.

Fibonacci:

```
1  class FibTask implements Callable<Integer> {
2      static ExecutorService exec = Executors.newCachedThreadPool();
3      int arg;
4      public FibTask(int n) {
5          arg = n;
6      }
7      public Integer call() {
8          if (arg > 2)
9              Future<Integer> left = exec.submit(new FibTask(arg-1));
10             Future<Integer> right = exec.submit(new FibTask(arg-2));
11             return left.get() + right.get();
12         } else {
13             return 1;
14         }
15     }
16 }
```

FibTask è un task che restituisce un intero. Si parte con un thread pool di tipo `CachedThreadPool()`. Il costruttore prende l'ennesimo task per calcolare l'ennesimo numero di Fibonacci. Se `arg` è maggiore di 2 viene sottomesso il calcolo di Fibonacci `arg-1` e `arg-2` nelle variabili `left` e `right`. Quando i due task `left.get()` e `right.get()` hanno terminato, si restituisce il valore appena calcolato come una somma di `left` e `right`. Se l'argomento non è maggiore di 2, si restituisce uno per indicare che è terminata la ricorsione. Questa soluzione non è ottimizzata.

Le invocazioni su Fibonacci:



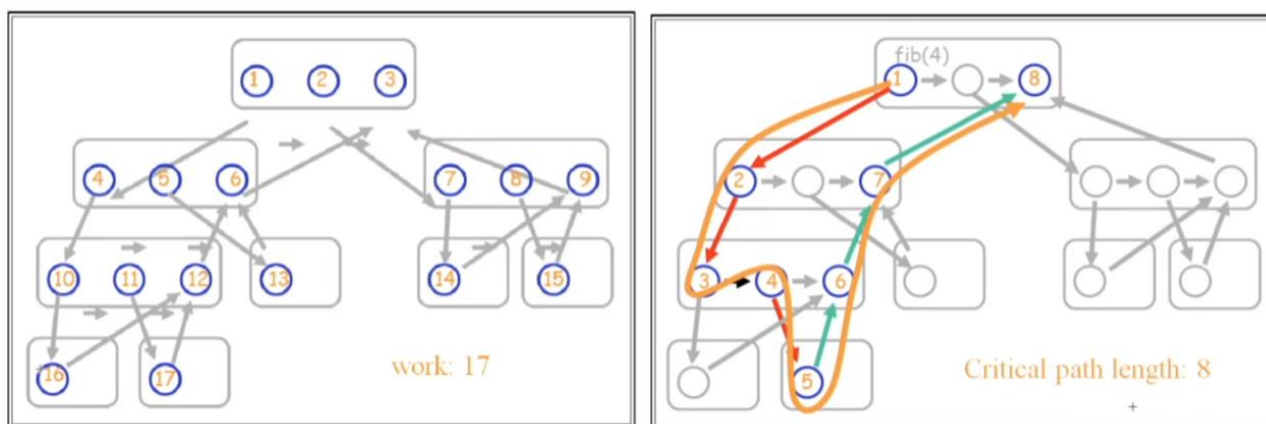
In rosso ci sono le invocazioni e in verde ci sono le trasmissioni dei risultati. Ad esempio, il calcolo di Fibonacci di 2 comporta il calcolo di Fibonacci di 1 e di 2. Il valore può essere restituito a chi lo ha invocato, cioè a Fibonacci di 3, che a sua volta restituisce il valore a Fibonacci di 4. L'inefficienza è che l'intero sottoalbero `fib(2)` è calcolato due volte, quando potrebbe essere calcolato solo una volta.

Come si misura il parallelismo di una soluzione?

Sia T_P il tempo minimo (misurato in passi) necessario per eseguire un programma multithread su un sistema con P processori dedicati. T_P è quindi la latenza del programma, il tempo necessario per eseguirlo dall'inizio alla fine, misurato da un osservatore esterno. T_P è una misura ideale in quanto non è sempre possibile per ogni processore trovare step da eseguire e una computazione reale può essere limitata da altri fattori, come l'uso della memoria. Consideriamo T_1 il numero di step necessario per eseguire il programma su un singolo processore, chiamato *work* e rappresenta anche il numero totale di step dell'intera computazione. In uno step, P processori possono eseguire al massimo P step, quindi $T_P \geq T_1/P$.

T_∞ è il tempo per l'eseguire il programma su un numero illimitato di processori, chiamato *Critical-path*: $T_P \geq T_\infty$. Qualsiasi sia la dimensione del programma, questo è comunque limitato da T , il quale esprime la dipendenza dei task l'una dall'altra. Non importa se si hanno infiniti processori per eseguire il path. Il Critical path length vincola il tempo di esecuzione, in termini di numero di task, del nostro programma parallelo, indipendentemente dalle risorse possedute.

Lo speedup su P processori è il rapporto T_1/T_P . Diciamo che una computazione ha uno speedup lineare se $T_1/T_P = \theta P$. Infine, il parallelismo di una computazione è il massimo speedup possibile T_1/T_∞ , anche l'ammontare medio di lavoro disponibile ad ogni step lungo il critical path. Fornisce quindi una buona stima del numero di processori necessari ad una computazione.



In questo caso vengono lanciati i task:

- 3 lancia 10;
- 11 lancia 17;
- 17 lancia 12;
- 12 lancia 6;
- 6 lancia 3;

Il work è 17, mentre la critical path length è il percorso più lungo (uno dei due colorati). Riguardo la complessità dell'addizione tra matrici, sia $A_P(n)$ il numero di passi necessari per fare la somma di due matrici $n \times n$ su P processori. Il work necessario è $A_1 n = 4A_1 n^2 + \theta(1) = \theta(n^2)$. Questo è uguale al lavoro necessario per il tradizionale doppio for innestato.

Se le addizioni fossero fatte in parallelo ad ogni passo, il critical path sarebbe dato da:

$$A^\infty(n) = A^\infty(n/2) + \theta(1) = \theta(\log n)$$

Cioè una soluzione con hardware infinito fa l'esecuzione di due matrici $n \times n$ in tempo $\log n$.

Riguardo la complessità della moltiplicazione tra matrici, sia $M_P(n)$ il numero di step necessari per moltiplicare due matrici $n \times n$ su P processori:

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} * \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix} = \begin{pmatrix} A_{00} * B_{00} + A_{01} * B_{10} & A_{00} * B_{01} + A_{01} * B_{11} \\ A_{10} * B_{00} + A_{11} * B_{10} & A_{10} * B_{01} + A_{11} * B_{11} \end{pmatrix}$$

La moltiplicazione tra matrici richiede otto moltiplicazioni $(n/2) * (n/2)$ e quattro addizioni:

$$M_1(n) = 8M_1(n/2) + 4A_1(n) = 8M_1(n/2) + \theta(n^2) = \theta(n^3)$$

Risulta ancora uguale al lavoro necessario per il tradizionale triplo for innestato. La ricorsione non fa guadagnare nulla in tal caso. Le otto moltiplicazioni possono essere effettuate in parallelo, come le addizioni ma queste devono aspettare che le moltiplicazioni siano complete, quindi il critical path è:

$$M^\infty(n) = M^\infty(n/2) + A^\infty(n) = M^\infty(n/2) + \theta(\log n) = \theta(\log^2 n)$$

Il parallelismo per la moltiplicazione di matrici è dato da:

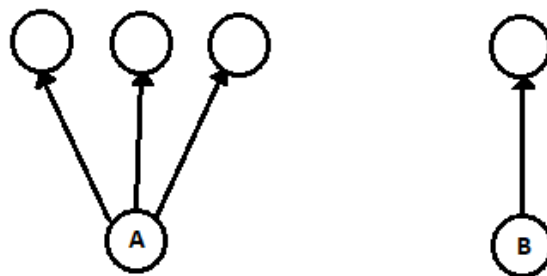
$$M_1(n) / M_\infty(n) = \theta(n^3 / \log^2 n)$$

Cioè passare da n^3 a $\log^2 n$ e quindi altamente parallelizzabile.

Il parallelismo che viene descritto da T_p è solo quello che si può sperare di ottenere (Upper bound). L'idea è che non sempre è possibile tenere occupati tutti i thread con un task (graffi di precedenza di problemi reali non hanno molti thread disponibili). Si deve cercare di avere sempre dei task che siano eseguibili, perché il problema che si può avere è che se si hanno ad esempio 16 thread e in un certo momento si hanno solo 12 task. Questo significa che i quattro thread non hanno lavoro da fare. Al passo successivo si potrebbe avere la stessa situazione. Per come è strutturato il programma bisogna tenerli sempre occupati, altrimenti si sta generando inefficienza, cioè dei ritardi dovuti all'algoritmo, in quanto si è schedulato i lavori da fare senza pensare al fatto che i thread e le risorse devono essere sempre utilizzabili. Inoltre, le caratteristiche della memoria (memoria disponibile influenza page fault) rendono difficile modellare con precisione. Ad ogni page fault si incorre in una penalità di efficienza.

9.2.2 Scheduling su Multiprocessore

Supponiamo di avere la seguente situazione:

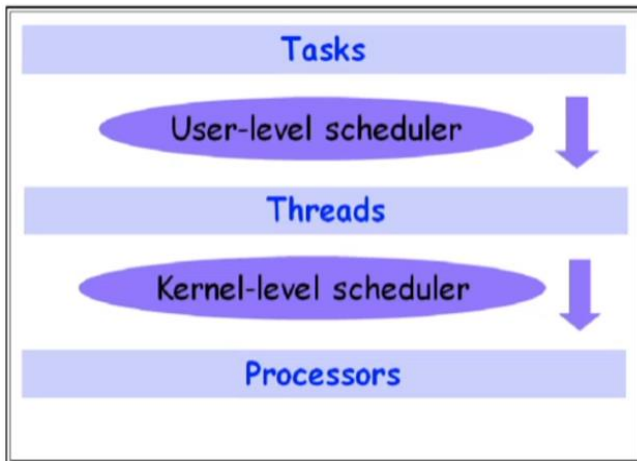


Abbiamo due task A e B. Supponiamo che ad un certo punto si ha solo un thread da eseguire.

Qual è il task che deve essere eseguito?

Se si esegue A al passo successivo possiamo eseguire 3 thread. Se si esegue B al passo successivo, possiamo eseguire un thread. A questo punto è meglio eseguire A perché A libera 3 task da eseguire. Se si esegue B si libera solo un task da eseguire dopo. La scelta di cosa schedulare è importante, come importante è tenere sempre il massimo numero di tasks da eseguire.

Modello Reale:



I task non vengono eseguiti direttamente sui processori, ma vengono eseguiti dai thread, che poi vengono schedulati sui processori. La situazione è complessa e lo scheduling è problematico. Idealmente si vorrebbe ottenere la massima velocizzazione possibile. In pratica, ad ogni passo i si vanno ad assegnare un certo numero p_i di task da eseguire. Non si possono eseguire più task di quante sono le risorse. Si vuole che p_i sia il più possibile vicino alle risorse occupate.

La Processor Average P_A è la sommatoria di p_i per ogni tempo, per i che va da 0 a $T-1$.

$$P_A = \frac{1}{T} \sum_{i=0}^{T-1} p_i$$

Si somma tutta l'esecuzione dei task in ogni passo, e li dividiamo per T . Questo ci dice la media di utilizzo del processore. Quello a cui si può puntare è uno speedup di fattore P_A .

Com'è fatto uno schedule che sia ottimale?

Un semplice schedule, è detto *Greedy* se ad ogni passo esegue il massimo numero di task ready. Questo schedule raggiunge buoni risultati, poiché esegue il minimo tra p_i processori disponibili e il numero di nodi che sono ready.

Si può dimostrare che ogni schedule greedy ottiene un tempo T con il seguente upper bound:

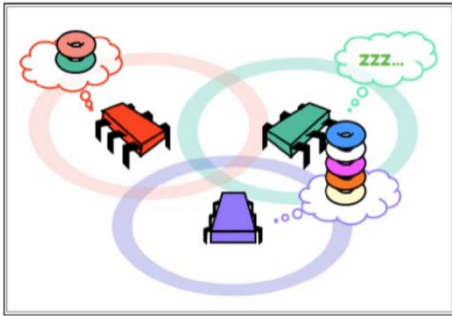
$$T \leq \frac{T_1}{P_A} + \frac{T_\infty(P-1)}{P_A}$$

Inoltre, si può dimostrare che questo è due volte l'ottimo, cioè si è a distanza due dal valore ottimo che si riesce ad ottenere.

9.3 Distribuzione del Lavoro

Dal punto di vista del software design è importante sapere la differenza tra questi due metodi di distribuire il lavoro.

9.3.1 Work Stealing

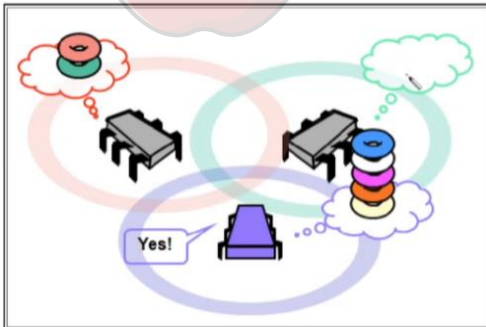


Possiamo avere tre processori, dove c'è un processore mediamente caricato (il rosso), un processore che è estremamente caricato (il blu) e c'è un processore che sta dormendo (il verde), è chiaro che questa è una situazione dove non stiamo usando al meglio le nostre risorse.

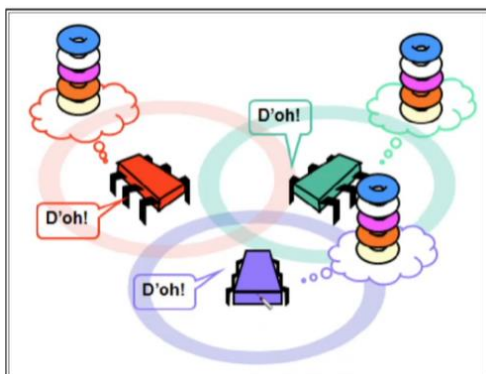
L'obiettivo è:

- Si vuole tenere i processori occupati il più possibile
 - Tenendo i thread user-level sempre con task pronti per la esecuzione. Ricordando che il thread è, come dire, il proxy per il processore dal nostro punto di vista, che si ha un thread per processore e si vorrebbe cercare di utilizzare tutti processori sempre al meglio;
- Questo però può richiedere bilanciamento di carico tra thread (più nel calcolo parallelo tradizionale che nel multiprocessore);
- Si potrebbe pensare di far distribuire il lavoro in eccesso a chi ne ha troppo (*Work Dealing*);

Funzionamento del Work Dealing:



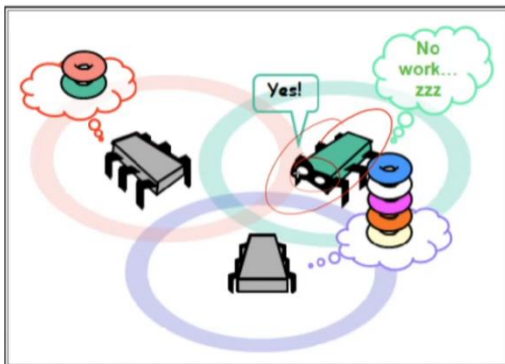
Il processore blu si accorge di aver troppo lavoro e quindi divide il suo carico sugli altri: questo sembra funzionare, cioè quello che stiamo facendo in pratica, è che il processore blu agisce e prende una parte del suo carico e lo sposta agli altri. Questa decisione è presa sulla conoscenza locale.



Il problema quando sono tutti sovraccarichi. Tutti i processori ritengono di essere sovraccarichi e quindi spostano il lavoro da qualcun altro. Quello che può succedere e che si sta spostando il carico e qualcun altro sta spostando il suo carico a me, quindi si sta facendo un qualcosa abbastanza complesso, cioè spostare i carichi, che significa comunicare, spostare dati ecc....ma in aggiunta a questo non si sta ottenendo nessun miglioramento, perché magari la situazione è già sovraccarica di per sé.

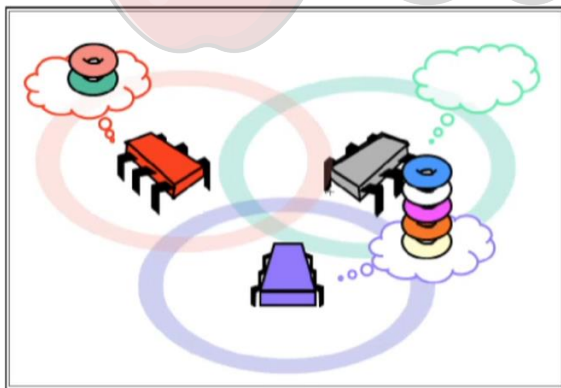
Le tecniche che funzionano meglio non sono chi ha molto lavoro che deve distribuire ma usare il *Work Stealing*, cioè chi ha poco lavoro deve andare a cercare lavoro da fare. La strategia consiste in:

- Un nodo che è idle cerca di rubare lavoro ad altri thread;
- Non c'è necessariamente bisogno di coordinare se tutti i thread sono al lavoro. Ribaltando il problema, automaticamente risolviamo il fatto che se tutti hanno lavoro da fare nessuno cerca lavoro dagli altri, nessuno va in overhead di nessun tipo;
- Thread overloaded non sprecano CPU clocks in cercare di smistare il proprio lavoro, ma vengono aiutati da thread idle (che hanno CPU a disposizione);



Quindi il thread verde si rende conto che non ha lavoro da fare e ruba il lavoro da qualcuno che ne ha. Strategie diverse possono essere utilizzate per selezionare qualcuno di quelli che sono in sovraccarico, tipicamente la maniera migliore è quella di vedere il carico tra un piccolo numero di nodi scelti a caso, in maniera tale da fare in modo che tutti gli Stealer (tutti quelli che sono inattivi) cerchino su vari processori.

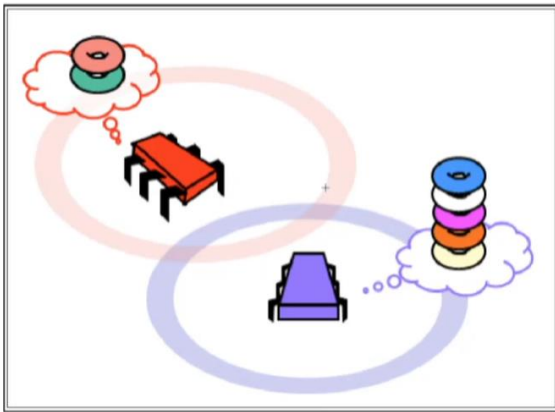
9.3.2 Work Balancing



Il Work Balancing è un qualcosa in cui ci sono meno problematiche rispetto al work stealing, si cerca di bilanciare il proprio carico con quello di qualcun altro. Non ci si aspetta di essere completamente scarico, cioè si agisce in maniera proattiva, rispetto al work stealing dove si aspetta di essere completamente scarico prima di cercare nuovo lavoro, e questo potrebbe già essere troppo tardi, si può aver perso del tempo. Con il work balancing si può, in maniera proattiva, cercare di bilanciare il carico con altri nodi,

in maniera tale da non doverci trovare nella situazione in cui uno di noi non ha lavoro da fare e a questo punto perde del tempo utile di computing e essenzialmente va a fare lo stealing.

Random Select:

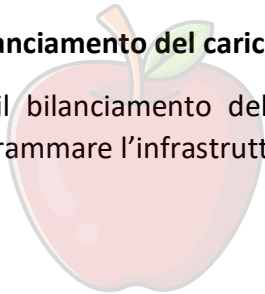


Le strategie sono quelle che riguardano la selezione a caso del nodo e si bilancia la quantità di lavoro che hanno a disposizione. In questa maniera il bilanciamento dei task all'interno dei thread funziona in maniera automatica, cercando di evitare la situazione estrema di avere dei task che sono completamente senza lavoro da svolgere. La descrizione di work stealing e work balancing è estremamente qualitativa e concettuale. Questi sono principi base particolarmente importanti,

normalmente si pensa quando c'è un sovraccarico che il nodo sovraccaricato oltre a questo si deve anche prendere cura della selezione del nodo a cui mandare, questo è chiaramente un ulteriore sovraccarico, ovviamente non positiva come strategia e le due strategie, Stealing e balancing: Stealing è quando si crea il problema di avere un nodo senza lavoro, allora va a prendere il lavoro altrove. Nel balancing si cerca invece di lavorare in maniera proattiva per evitare che si creino situazioni del genere bilanciando il carico tra nodi, ancora una volta selezionati a caso.

Il bilanciamento del carico è a cura del programmatore, quindi da codice?

No, il bilanciamento del carico è a cura della infrastruttura, però voi potreste trovarvi per programmare l'infrastruttura.



CoScienze
Associazione